



RTL Design Sherpa

RTL Common Library Module Reference — Rev 0.90

October 2026

Table of Contents

1 rtl-common Library Overview.....	64
1.1 Library Philosophy.....	64
1.1.1 Design Principles.....	64
1.1.2 Quality Standards.....	64
1.2 Architecture Overview.....	65
1.2.1 Module Hierarchy.....	65
1.3 Functional Categories.....	65
1.3.1 1. Arithmetic and Mathematical Operations.....	65
1.3.2 2. Data Integrity and Error Management.....	66
1.3.3 3. Control Logic and Arbitration.....	66
1.3.4 4. Clock and Reset Management.....	66
1.3.5 5. Counting and Sequence Generation.....	67
1.3.6 6. Data Conversion and Encoding.....	67
1.3.7 7. Bit Manipulation and Searching.....	68
1.3.8 8. Shift Operations and LFSRs.....	68
1.3.9 9. Memory and Storage.....	68
1.3.10 10. Utility and Specialized Functions.....	69
1.4 Performance and Optimization.....	69
1.4.1 Synthesis Guidelines.....	69
1.4.2 Technology-Specific Considerations.....	70
1.5 Verification and Validation.....	70
1.5.1 Testbench Coverage.....	70
1.5.2 Compliance Testing.....	70

1.6 Usage Guidelines.....	70
1.6.1 Module Selection.....	70
1.6.2 Integration Best Practices.....	70
1.7 Future Enhancements.....	72
1.7.1 Planned Additions.....	72
1.7.2 Technology Evolution.....	72
2 Deficit Round Robin Arbiter.....	73
2.1 Overview.....	73
2.2 Parameters.....	73
2.2.1 User-Settable Parameters.....	73
2.2.2 Derived Parameters.....	74
2.3 Ports.....	75
2.3.1 Inputs.....	75
2.3.2 Outputs.....	75
2.4 Functional Description.....	76
2.4.1 The Deficit Discipline.....	76
2.4.2 The Cost Pipeline.....	76
2.4.3 Share Example.....	76
2.4.4 Quantum Changes and Disable.....	76
2.5 Design Notes.....	76
2.6 Timing Characteristics.....	77
2.7 Usage Examples.....	77
2.8 Related Modules.....	78
2.9 Testing.....	78

2.10 Navigation.....	78
3 Priority Encoder Arbiter.....	79
3.1 Overview.....	79
3.2 Parameters.....	79
3.2.1 User-Settable Parameters.....	79
3.2.2 Derived Parameters.....	79
3.3 Ports.....	80
3.3.1 Inputs.....	80
3.3.2 Outputs.....	80
3.4 Functional Description.....	80
3.4.1 Priority Order.....	80
3.4.2 Request Selection.....	80
3.4.3 Combinational Operation.....	81
3.4.4 Fixed Priority vs Round-Robin.....	81
3.4.5 Priority Inversion.....	81
3.4.6 Optimized Implementations.....	81
3.4.7 Priority Selection Logic.....	82
3.5 Timing Characteristics.....	82
3.5.1 Latency and Propagation.....	82
3.5.2 Resource Utilization.....	82
3.6 Usage Examples.....	83
3.6.1 Standalone Priority Encoder.....	83
3.6.2 Dual-Input Priority Encoder.....	83
3.6.3 Integration in Round-Robin Arbiter.....	84

3.6.4 Multi-Level Priority Groups.....	84
3.6.5 Example Priority Scenarios.....	85
3.7 Design Notes.....	86
3.7.1 When to Use.....	86
3.7.2 Performance Optimization.....	86
3.7.3 Synthesis Considerations.....	86
3.7.4 Common Pitfalls.....	86
3.8 Related Modules.....	87
3.9 Testing.....	87
3.10 Navigation.....	87
4 Round Robin Arbiter.....	88
4.1 Overview.....	88
4.2 Parameters.....	88
4.3 Ports.....	88
4.3.1 Inputs.....	88
4.3.2 Outputs.....	89
4.4 Functional Description.....	89
4.4.1 Fair Round-Robin Operation.....	89
4.4.2 Block Arbitration.....	89
4.4.3 Grant Acknowledgment Support.....	90
4.4.4 Key Internal Signals.....	90
4.4.5 Priority Mechanism.....	90
4.4.6 Winner Selection.....	90
4.4.7 Mask Update Logic.....	91

4.5 Timing Characteristics.....	91
4.6 Usage Examples.....	91
4.7 Design Notes.....	92
4.7.1 Usage Notes.....	92
4.8 Related Modules.....	92
4.9 Testing.....	92
4.10 Navigation.....	93
5 Simple Round-Robin Arbiter.....	94
5.1 Overview.....	94
5.2 Parameters.....	94
5.3 Ports.....	95
5.3.1 Inputs.....	95
5.3.2 Outputs.....	95
5.4 Functional Description.....	95
5.4.1 Round-Robin Algorithm.....	95
5.4.2 Key Features.....	95
5.4.3 Reset Behavior.....	96
5.4.4 Rotation Logic.....	96
5.4.5 Priority Selection.....	96
5.4.6 Grant Encoding.....	97
5.5 Timing Characteristics.....	97
5.6 Usage Examples.....	97
5.6.1 Basic 4-Agent Arbiter.....	97
5.6.2 8-Agent System.....	97

5.6.3 Arbitration Sequence Example.....	98
5.7 Design Notes.....	98
5.7.1 Advantages.....	98
5.7.2 Limitations.....	98
5.7.3 Synthesis Considerations.....	98
5.8 Related Modules.....	99
5.9 Testing.....	99
5.10 Navigation.....	99
6 Weighted Round Robin Arbiter.....	100
6.1 Overview.....	100
6.2 Parameters.....	101
6.2.1 User-Settable Parameters.....	101
6.2.2 Derived Parameters.....	101
6.3 Ports.....	102
6.3.1 Inputs.....	102
6.3.2 Outputs.....	102
6.4 Functional Description.....	103
6.4.1 Credit-Based Weighting.....	103
6.4.2 Arbitration Stages.....	103
6.4.3 Weight Ratio Example.....	103
6.4.4 Zero-Weight Client Handling.....	104
6.4.5 ACK Protocol Support.....	104
6.4.6 Starvation Prevention.....	104
6.4.7 Dynamic Weight Changes.....	104

6.4.8 Credit Management.....	105
6.4.9 Global Replenishment Logic.....	105
6.4.10 Request Masking for Fairness.....	105
6.4.11 Sub-Module: arbiter_round_robin.....	106
6.5 Timing Characteristics.....	106
6.5.1 Latency and Throughput.....	106
6.5.2 Critical Paths.....	107
6.5.3 Resource Utilization.....	107
6.5.4 Maximum Frequency.....	107
6.6 Usage Examples.....	107
6.6.1 Basic QoS Arbiter (No-ACK Mode).....	107
6.6.2 Dynamic Weight Adjustment.....	108
6.6.3 ACK Mode for Pipelined Masters.....	109
6.7 Design Notes.....	109
6.7.1 When to Use Each Mode.....	109
6.7.2 Weight Selection Guidelines.....	109
6.7.3 Dynamic Weight Change Best Practices.....	110
6.8 Related Modules.....	110
6.9 Testing.....	110
6.10 Navigation.....	110
7 Single-Client Arbiter.....	111
7.1 Overview.....	111
7.1.1 Key Features.....	111
7.1.2 Use Cases.....	111

7.2 Parameters.....	112
7.3 Ports.....	112
7.3.1 Inputs.....	112
7.3.2 Outputs.....	113
7.4 Functional Description.....	113
7.4.1 Request Qualification.....	113
7.4.2 Grant-Eligibility Logic (ACK Mode).....	113
7.4.3 Grant Lifecycle (ACK Mode).....	113
7.4.4 No-ACK Fallback Mode.....	114
7.4.5 Output Mapping.....	114
7.5 Timing Characteristics.....	114
7.6 Usage Examples.....	115
7.7 Design Notes.....	115
7.8 Related Modules.....	115
7.9 Testing.....	116
7.10 Navigation.....	116
8 Token-Bucket Request Shaper.....	117
8.1 Overview.....	117
8.2 Parameters.....	117
8.2.1 Derived Parameters (do not override).....	118
8.3 Ports.....	118
8.4 Functional Description.....	119
8.5 Design Notes.....	119
8.6 Timing Characteristics.....	120

8.7 Usage Examples.....	120
8.8 Related Modules.....	120
8.9 Testing.....	121
8.10 Navigation.....	121
9 Binary to BCD Converter.....	122
9.1 Overview.....	122
9.2 Parameters.....	122
9.2.1 Parameter Relationship Table.....	123
9.3 Ports.....	123
9.3.1 Inputs.....	123
9.3.2 Outputs.....	123
9.4 Functional Description.....	124
9.4.1 The Double-Dabble Algorithm.....	124
9.4.2 Why Add 3?.....	124
9.4.3 Mathematical Proof.....	124
9.4.4 State Machine.....	124
9.4.5 State Transition Diagram.....	124
9.4.6 State Descriptions.....	125
9.4.7 Register Definitions.....	126
9.4.8 Core Processing Logic.....	127
9.4.9 BCD Format and Encoding.....	128
9.5 Related Modules.....	129
9.6 Timing Characteristics.....	130
9.6.1 Conversion Latency.....	130

9.6.2 Latency Breakdown.....	130
9.6.3 Latency Examples.....	130
9.6.4 Pipeline Considerations.....	130
9.6.5 Synthesis Results (Typical FPGA).....	131
9.6.6 Power Consumption.....	131
9.7 Usage Examples.....	131
9.8 Design Notes.....	131
9.8.1 Optimization Techniques.....	131
9.8.2 Common Design Patterns.....	133
9.8.3 Troubleshooting Guide.....	136
9.9 Testing.....	137
9.9.1 Test Strategy.....	137
9.9.2 Coverage Model.....	139
9.9.3 Formal Verification Properties.....	140
9.10 Navigation.....	141
10 CAM Tag.....	142
10.1 Overview.....	142
10.2 Parameters.....	142
10.3 Ports.....	142
10.3.1 Inputs.....	142
10.3.2 Outputs.....	143
10.4 Functional Description.....	143
10.4.1 Tag Lookup.....	143
10.4.2 Tag Insertion.....	143

10.4.3 Tag Invalidation.....	143
10.4.4 Enable Control.....	143
10.4.5 Allocation Strategy.....	143
10.4.6 Storage Arrays.....	144
10.4.7 Search Logic.....	144
10.4.8 State Management.....	145
10.4.9 Status Signals.....	145
10.5 Related Modules.....	145
10.6 Timing Characteristics.....	145
10.7 Usage Examples.....	146
10.8 Design Notes.....	146
10.8.1 Usage Considerations.....	146
10.9 Testing.....	146
10.10 Navigation.....	147
11 Clock Divider.....	148
11.1 Overview.....	148
11.2 Parameters.....	148
11.2.1 Parameter Relationships.....	150
11.3 Ports.....	150
11.3.1 Inputs.....	150
11.3.2 Outputs.....	150
11.4 Functional Description.....	150
11.4.1 Counter Operation.....	150
11.4.2 Pickoff Point Selection.....	151

11.4.3 Out-of-Range Handling.....	151
11.4.4 Output Registration.....	151
11.4.5 Division Ratios (100MHz Input Clock).....	151
11.4.6 Duty Cycle.....	151
11.4.7 Internal Signals.....	152
11.5 Timing Characteristics.....	152
11.5.1 Latency.....	152
11.5.2 Phase Relationships.....	152
11.5.3 Jitter.....	152
11.5.4 Maximum Frequency.....	152
11.5.5 Resource Scaling.....	152
11.6 Usage Examples.....	153
11.6.1 Four Debug Clocks.....	153
11.6.2 Baud Rate Generator (Approximate).....	153
11.6.3 Runtime-Programmable Divider.....	154
11.6.4 Test Clock Generation.....	154
11.7 Design Notes.....	155
11.7.1 Changing Pickoff Points at Runtime.....	155
11.7.2 Synthesis Considerations.....	156
11.7.3 Important Warnings.....	157
11.8 Related Modules.....	157
11.9 Testing.....	157
11.9.1 Test Scenarios.....	157
11.9.2 Coverage Points.....	158

11.10 Navigation.....	158
12 Clock Gate Controller.....	159
12.1 Overview.....	159
12.2 Module Interface.....	159
12.3 Parameters.....	159
12.3.1 Derived Parameters.....	160
12.4 Ports.....	160
12.4.1 Inputs.....	160
12.4.2 Outputs.....	161
12.5 Functional Description.....	161
12.5.1 Why Clock Gating?.....	161
12.5.2 Clock Gating Safety.....	161
12.5.3 State Machine Behavior.....	161
12.5.4 Timing Diagram.....	161
12.5.5 Idle Counter Logic.....	161
12.5.6 Gating Control Logic.....	162
12.5.7 The ICG Cell.....	162
12.5.8 Technology-Specific ICG Implementations.....	163
12.5.9 Timing and Performance.....	164
12.5.10 Resource Utilization.....	164
12.6 Timing Characteristics.....	165
12.7 Usage Examples.....	165
12.7.1 CPU Core Clock Gating.....	165
12.7.2 Peripheral Interface Clock Management.....	166

12.7.3 Memory Subsystem Clock Gating.....	168
12.7.4 Adaptive Clock Gating with Activity Monitoring.....	170
12.7.5 Hierarchical Clock Gating System.....	172
12.8 Design Notes.....	174
12.8.1 Glitch-Free Clock Switching.....	174
12.8.2 Multi-Level Clock Gating.....	175
12.8.3 Measuring Power Savings.....	176
12.8.4 Synthesis Constraints.....	177
12.8.5 Design Guidelines.....	178
12.8.6 Common Applications.....	179
12.8.7 Troubleshooting Guide.....	179
12.9 Related Modules.....	179
12.10 Testing.....	180
12.10.1 Test Bench.....	180
12.10.2 Coverage Model.....	183
12.10.3 Formal Verification Properties.....	183
12.11 Navigation.....	184
13 clock_pulse.....	185
13.1 Overview.....	185
13.2 Parameters.....	185
13.3 Ports.....	185
13.4 Functional Description.....	186
13.4.1 Internal Counter.....	186
13.4.2 Core Logic.....	186

13.4.3 Operation Principles.....	186
13.5 Related Modules.....	187
13.6 Timing Characteristics.....	187
13.6.1 Timing Characteristics.....	187
13.6.2 Basic Operation (WIDTH=4).....	187
13.6.3 Reset Behavior.....	187
13.6.4 Different WIDTH Values.....	188
13.7 Usage Examples.....	188
13.8 Design Notes.....	188
13.8.1 Resource Utilization.....	188
13.8.2 Synthesis Considerations.....	188
13.8.3 Design Considerations.....	189
13.8.4 Common Applications.....	190
13.9 Testing.....	190
13.9.1 Comprehensive Test Bench.....	190
13.9.2 Coverage Model.....	194
13.9.3 Formal Properties.....	195
13.10 Navigation.....	196
14 Binary Counter with Load and Variable Increment.....	197
14.1 Overview.....	197
14.2 Module Interface.....	197
14.3 Parameters.....	197
14.3.1 User-Settable Parameters.....	197
14.3.2 Derived Localparam (Computed Internally).....	198

14.4 Ports.....	198
14.4.1 Inputs.....	198
14.4.2 Outputs.....	199
14.5 Functional Description.....	199
14.5.1 Operation Priority.....	199
14.5.2 FIFO-Optimized Wraparound.....	199
14.5.3 Full/Empty Detection Pattern.....	200
14.5.4 Priority Logic.....	200
14.5.5 Registered Output.....	201
14.5.6 Combinational Next-Value Preview.....	201
14.6 Usage Examples.....	201
14.6.1 FIFO Read Pointer with Drop Support.....	201
14.6.2 FIFO Write Pointer.....	202
14.6.3 Full/Empty Status Logic.....	202
14.6.4 Packet Drop Controller.....	203
14.7 Timing Characteristics.....	204
14.7.1 Latency and Throughput.....	204
14.7.2 Operation Timing Diagrams.....	204
14.7.3 Resource Utilization.....	205
14.7.4 Maximum Frequency.....	205
14.8 Testing.....	205
14.9 Design Notes.....	205
14.9.1 When to Use counter_bin_load.....	205
14.9.2 When to Use Alternatives.....	206

14.9.3 Parameter Selection Guidelines.....	206
14.9.4 Critical Paths.....	206
14.9.5 Common Pitfalls.....	206
14.10 Related Modules.....	207
14.11 Navigation.....	208
15 Binary Counter for FIFOs.....	209
15.1 Overview.....	209
15.2 Module Interface.....	209
15.3 Parameters.....	209
15.4 Ports.....	210
15.4.1 Inputs.....	210
15.4.2 Outputs.....	210
15.5 Functional Description.....	210
15.5.1 Internal Signals.....	210
15.5.2 Special Wrap Logic.....	210
15.5.3 Normal Counting.....	211
15.5.4 Wrap Condition.....	211
15.5.5 Enable Control.....	211
15.5.6 FIFO Application Context.....	211
15.5.7 Address Generation.....	211
15.5.8 FIFO Pointer Comparison.....	211
15.5.9 Advanced Variants.....	211
15.5.10 Address Extraction.....	211
15.5.11 Gray Code Conversion.....	211

15.6 Timing Characteristics.....	212
15.6.1 Timing.....	212
15.6.2 Maximum Frequency.....	212
15.6.3 Power Consumption.....	212
15.7 Usage Examples.....	213
15.7.1 1. Small FIFO Buffer (8 entries).....	213
15.7.2 2. Large FIFO Buffer (1024 entries).....	213
15.7.3 3. Asymmetric FIFO.....	213
15.7.4 Counting Sequence Example.....	213
15.7.5 Configuration: WIDTH=4, MAX=6.....	213
15.7.6 Pattern Analysis.....	214
15.7.7 Test Scenarios.....	214
15.7.8 Coverage Points.....	214
15.8 Design Notes.....	214
15.8.1 Resource Utilization.....	214
15.8.2 Timing Optimization.....	214
15.8.3 Parameter Validation.....	215
15.8.4 Common Issues and Solutions.....	215
15.9 Related Modules.....	215
15.10 Navigation.....	215
16 Frequency Invariant Counter.....	216
16.1 Overview.....	216
16.2 Module Interface.....	216
16.3 Parameters.....	217

16.4 Ports.....	218
16.4.1 Inputs.....	218
16.4.2 Outputs.....	218
16.5 Functional Description.....	218
16.5.1 LUT Generation Strategies.....	218
16.5.2 Division Factor Lookup.....	219
16.5.3 Change Detection.....	220
16.5.4 Prescaler.....	220
16.5.5 Counter and Tick.....	220
16.6 Timing Characteristics.....	221
16.6.1 Timing.....	221
16.6.2 Resource Use.....	221
16.6.3 Critical Path.....	222
16.7 Usage Examples.....	222
16.7.1 1. Microsecond timebase on an unknown FPGA clock.....	222
16.7.2 2. ASIC range, 100 MHz to 1 GHz.....	222
16.7.3 3. Millisecond timeout built on the tick.....	223
16.7.4 4. Gating the timer with sync_reset_n.....	223
16.8 Testing.....	223
16.9 Design Notes.....	224
16.9.1 Choosing the LUT Range.....	224
16.10 Related Modules.....	224
16.11 Navigation.....	224
17 Counter with Load and Clear.....	225

17.1 Overview.....	225
17.2 Module Interface.....	225
17.3 Parameters.....	225
17.4 Ports.....	225
17.4.1 Inputs.....	225
17.4.2 Outputs.....	226
17.5 Functional Description.....	226
17.5.1 Internal Registers.....	226
17.5.2 Priority Logic.....	226
17.5.3 Main Control Logic.....	227
17.5.4 Done Signal Generation.....	227
17.5.5 Advanced Variants.....	227
17.5.6 1. Dual-Rate Timer.....	227
17.5.7 2. PWM Generator.....	228
17.5.8 3. Burst Generator.....	228
17.5.9 4. Communication Timeout.....	229
17.6 Timing Characteristics.....	229
17.6.1 Basic Operation.....	229
17.6.2 Load During Count.....	229
17.6.3 Maximum Frequency.....	230
17.6.4 Latency.....	230
17.7 Usage Examples.....	230
17.7.1 1. Basic Counting Mode.....	230
17.7.2 2. Configurable Delay Mode.....	230

17.7.3 3. Retriggerable Timer Mode.....	231
17.7.4 Design Patterns.....	231
17.7.5 Pattern 1: Auto-Reloading Timer.....	231
17.7.6 Pattern 2: Conditional Counting.....	231
17.7.7 Pattern 3: Dynamic Period Adjustment.....	232
17.8 Testing.....	232
17.8.1 Test Scenarios.....	232
17.8.2 Coverage Model.....	232
17.8.3 Assertions.....	233
17.9 Design Notes.....	234
17.9.1 Resource Usage.....	234
17.9.2 Timing Optimization.....	234
17.9.3 Parameter Validation.....	234
17.10 Related Modules.....	234
17.11 Navigation.....	234
18 Basic Counter.....	235
18.1 Overview.....	235
18.2 Module Interface.....	235
18.3 Parameters.....	235
18.4 Ports.....	235
18.4.1 Inputs.....	235
18.4.2 Outputs.....	235
18.5 Functional Description.....	236
18.5.1 Counter Register.....	236

18.5.2 State Machine Behavior.....	236
18.5.3 Main Counter Logic.....	236
18.5.4 Tick Generation.....	236
18.5.5 Key Features.....	236
18.6 Timing Characteristics.....	237
18.6.1 Period Calculation.....	237
18.6.2 Examples.....	237
18.6.3 Resource Usage.....	237
18.6.4 Maximum Frequency.....	237
18.6.5 Power Optimization.....	237
18.7 Usage Examples.....	237
18.7.1 1. Basic Timer (1ms tick at 100MHz).....	237
18.7.2 2. Heartbeat Generator (1Hz at 50MHz).....	238
18.7.3 3. Simple Divider (Divide by 8).....	238
18.8 Testing.....	238
18.8.1 Test Scenarios.....	238
18.8.2 Coverage Points.....	238
18.9 Design Notes.....	239
18.9.1 Parameter Constraints.....	239
18.9.2 Optimization Hints.....	239
18.9.3 Limitations.....	239
18.10 Related Modules.....	239
18.11 Navigation.....	239
19 Ring Counter.....	240

19.1 Overview.....	240
19.2 Module Interface.....	240
19.3 Parameters.....	240
19.4 Ports.....	240
19.4.1 Inputs.....	240
19.4.2 Outputs.....	241
19.5 Functional Description.....	241
19.5.1 Ring Counter Principle.....	241
19.5.2 Mathematical Representation.....	241
19.5.3 Core Logic.....	241
19.5.4 Operation Breakdown.....	241
19.5.5 State Sequences.....	242
19.5.6 4-bit Ring Counter (WIDTH=4).....	242
19.5.7 3-bit Ring Counter (WIDTH=3).....	242
19.5.8 8-bit Ring Counter Example.....	242
19.5.9 Properties and Characteristics.....	242
19.5.10 One-Hot Encoding.....	242
19.5.11 Self-Correcting Behavior.....	243
19.5.12 Comparison with Other Counters.....	243
19.5.13 Advanced Variants.....	243
19.5.14 1. Self-Correcting Ring Counter.....	243
19.5.15 2. Bidirectional Ring Counter.....	243
19.5.16 3. Programmable Ring Counter.....	244
19.6 Timing Characteristics.....	244

19.6.1 Maximum Frequency.....	244
19.6.2 Power Consumption.....	244
19.7 Usage Examples.....	245
19.7.1 1. Sequential Enable Generator.....	245
19.7.2 2. LED Chaser Display.....	245
19.7.3 3. Memory Bank Sequencer.....	246
19.7.4 4. State Machine Controller.....	246
19.7.5 5. Clock Domain Multiplexer.....	247
19.8 Testing.....	248
19.8.1 Test Scenarios.....	248
19.8.2 Coverage Model.....	248
19.8.3 Assertions.....	248
19.9 Design Notes.....	249
19.9.1 Resource Utilization.....	249
19.9.2 Optimization Tips.....	249
19.9.3 Timing Considerations.....	249
19.9.4 Common Pitfalls.....	249
19.10 Related Modules.....	250
19.11 Navigation.....	250
20 Count Leading Zeros.....	251
20.1 Overview.....	251
20.2 Module Interface.....	251
20.3 Parameters.....	251
20.4 Ports.....	252

20.4.1 Inputs.....	252
20.4.2 Outputs.....	252
20.5 Functional Description.....	252
20.5.1 Function-Based Approach.....	252
20.5.2 Bit Scanning Process.....	252
20.5.3 Why MSB-to-LSB Scanning?.....	253
20.5.4 Advanced Variants.....	253
20.5.5 1. Pipeline Version for High Speed.....	253
20.5.6 2. Hierarchical Implementation.....	254
20.5.7 3. LUT-Based Implementation (Small Widths).....	254
20.6 Usage Examples.....	255
20.6.1 1. Floating-Point Normalization.....	255
20.6.2 2. Priority Encoder.....	255
20.6.3 3. Bit Width Calculation.....	256
20.6.4 4. Log2 Calculation.....	256
20.6.5 5. Data Compression - Run Length Encoding.....	257
20.6.6 Examples and Truth Tables.....	257
20.6.7 8-bit Examples (WIDTH=8).....	257
20.6.8 32-bit Examples.....	258
20.7 Timing Characteristics.....	259
20.7.1 Resource Utilization.....	259
20.7.2 Timing Characteristics.....	259
20.8 Testing.....	259
20.8.1 Test Scenarios.....	259

20.8.2 Coverage Model.....	259
20.8.3 Assertions.....	260
20.9 Design Notes.....	260
20.9.1 Tool-Specific Attributes.....	260
20.9.2 Area vs. Speed Trade-offs.....	260
20.10 Related Modules.....	261
20.11 Navigation.....	261
21 count_trailing_zeros.....	262
21.1 Overview.....	262
21.1.1 Choosing Between CLZ and CTZ.....	262
21.1.2 Module Declaration.....	262
21.2 Parameters.....	263
21.3 Ports.....	263
21.4 Functional Description.....	263
21.4.1 Function-Based Approach.....	263
21.4.2 Bit Scanning Process.....	264
21.4.3 Why LSB-to-MSB Scanning?.....	264
21.4.4 8-bit Examples (WIDTH=8).....	264
21.4.5 32-bit Examples.....	265
21.4.6 Relationship to CLZ.....	265
21.5 Timing Characteristics.....	265
21.5.1 Resource Utilization.....	265
21.5.2 Timing Characteristics.....	266
21.6 Usage Examples.....	266

21.6.1 Address Alignment Detection.....	266
21.6.2 Maximum Burst Size for a DMA Transfer.....	266
21.6.3 Lowest-Index Request Arbitration.....	266
21.6.4 Isolating and Clearing the Lowest Set Bit.....	267
21.6.5 Power-of-Two Detection.....	267
21.7 Design Notes.....	268
21.7.1 Hierarchical Implementation.....	268
21.7.2 LUT-Based Implementation (Small Widths).....	268
21.7.3 Mask-Based Implementation.....	269
21.7.4 Synthesis Trade-offs.....	269
21.7.5 Common Applications.....	269
21.8 Related Modules.....	270
21.9 Testing.....	270
21.9.1 Test Scenarios.....	270
21.9.2 Coverage Model.....	270
21.9.3 Assertions.....	271
21.10 Navigation.....	271
22 dataint_checksum.....	272
22.1 Overview.....	272
22.1.1 Module Declaration.....	272
22.2 Parameters.....	272
22.3 Ports.....	272
22.4 Functional Description.....	273
22.4.1 Operation.....	273

22.4.2 Reset Behavior.....	273
22.4.3 Key Features.....	273
22.4.4 Special Considerations.....	273
22.4.5 State Machine.....	273
22.5 Related Modules.....	273
22.6 Timing Characteristics.....	274
22.7 Usage Examples.....	274
22.8 Design Notes.....	274
22.8.1 Common Applications.....	274
22.9 Testing.....	275
22.10 Navigation.....	275
23 dataint_crc.....	276
23.1 Overview.....	276
23.1.1 Module Declaration.....	276
23.2 Parameters.....	277
23.2.1 User-Settable Parameters.....	277
23.2.2 Calculated Parameters (Do Not Override).....	277
23.3 Ports.....	277
23.4 Functional Description.....	278
23.4.1 CRC Algorithm Implementation.....	278
23.4.2 Key Features.....	278
23.4.3 Input Data Reflection.....	279
23.4.4 Cascade Processing Architecture.....	279
23.4.5 Cascade Selection Logic.....	279

23.5 Timing Characteristics.....	280
23.5.1 Throughput.....	280
23.5.2 Latency.....	280
23.5.3 Critical Paths.....	280
23.6 Usage Examples.....	280
23.6.1 Basic CRC Calculation.....	280
23.6.2 Streaming CRC with Cascade.....	281
23.6.3 Multi-Stage CRC Pipeline.....	282
23.6.4 CRC-32 (IEEE 802.3).....	282
23.6.5 CRC-16 (CCITT).....	283
23.6.6 CRC-64 (ECMA-182).....	283
23.7 Design Notes.....	283
23.7.1 Area Optimization.....	283
23.7.2 Power Optimization.....	283
23.8 Related Modules.....	283
23.9 Testing.....	284
23.9.1 Setup Requirements.....	284
23.9.2 Verification Notes.....	284
23.10 Navigation.....	284
24 dataint_crc_xor_shift_cascade.....	285
24.1 Overview.....	285
24.1.1 Module Declaration.....	285
24.2 Parameters.....	285
24.3 Ports.....	285

24.4 Functional Description.....	286
24.4.1 Cascade Operation.....	286
24.4.2 Data Bit Ordering.....	286
24.4.3 Architecture Overview.....	286
24.4.4 Generate Block Structure.....	286
24.4.5 Intermediate Signals.....	286
24.4.6 Key Features.....	287
24.5 Timing Characteristics.....	287
24.5.1 Combinational Delay.....	287
24.5.2 Critical Path.....	287
24.6 Usage Examples.....	287
24.6.1 Basic Usage.....	287
24.6.2 Multi-Byte Processing.....	287
24.7 Design Notes.....	288
24.7.1 Performance Optimization.....	288
24.7.2 Integration with the CRC System.....	288
24.7.3 Applications.....	288
24.8 Related Modules.....	288
24.9 Testing.....	289
24.10 Navigation.....	289
25 dataint_crc_xor_shift.....	290
25.1 Overview.....	290
25.1.1 Module Declaration.....	290
25.2 Parameters.....	290

25.3 Ports.....	290
25.4 Functional Description.....	291
25.4.1 Core CRC Operation.....	291
25.4.2 Mathematical Foundation.....	291
25.4.3 Circular Dependency Solution.....	291
25.4.4 Bit-wise Operations.....	291
25.4.5 Feedback Bit Logic.....	292
25.4.6 Polynomial Application.....	292
25.4.7 No State Elements.....	292
25.4.8 Mathematical Verification.....	292
25.5 Timing Characteristics.....	292
25.6 Usage Examples.....	292
25.6.1 Single Bit Processing.....	292
25.6.2 Cascaded for Multiple Bits.....	292
25.7 Design Notes.....	293
25.7.1 Applications.....	293
25.8 Related Modules.....	293
25.9 Testing.....	293
25.10 Navigation.....	294
26 dataint_ecc_hamming_decode_secded.....	295
26.1 Overview.....	295
26.1.1 Module Declaration.....	295
26.2 Parameters.....	295
26.2.1 Calculated Parameters.....	295

26.3 Ports.....	296
26.4 Functional Description.....	296
26.4.1 Decoding Process.....	296
26.4.2 Error Detection Logic.....	296
26.4.3 Key Functions.....	297
26.4.4 Syndrome Generation.....	297
26.4.5 Overall Parity Check.....	298
26.4.6 Control States.....	298
26.4.7 Error Correction Logic.....	298
26.4.8 Data Extraction.....	299
26.4.9 Error Handling Details.....	299
26.4.10 Error Correction Capabilities.....	299
26.4.11 Debug Support.....	300
26.5 Timing Characteristics.....	300
26.6 Usage Examples.....	300
26.6.1 Basic Decoder.....	300
26.6.2 Error Handling System.....	300
26.7 Design Notes.....	301
26.7.1 Applications.....	301
26.8 Related Modules.....	301
26.9 Testing.....	301
26.10 Navigation.....	301
27 dataint_ecc_hamming_encode_secded.....	302
27.1 Overview.....	302

27.1.1 Module Declaration.....	302
27.2 Parameters.....	302
27.2.1 Calculated Parameters.....	302
27.2.2 Example Configurations.....	302
27.3 Ports.....	303
27.4 Functional Description.....	303
27.4.1 Hamming Code Structure.....	303
27.4.2 Bit Position Layout.....	303
27.4.3 Key Functions.....	303
27.4.4 Encoding Algorithm.....	304
27.4.5 Error Correction Capabilities.....	305
27.4.6 Combinational Design.....	305
27.4.7 Scalable Architecture.....	305
27.4.8 Debug Support.....	305
27.5 Timing Characteristics.....	305
27.6 Usage Examples.....	306
27.6.1 Basic Encoding.....	306
27.6.2 With Debug.....	306
27.7 Design Notes.....	306
27.7.1 Applications.....	306
27.8 Related Modules.....	306
27.9 Testing.....	307
27.10 Navigation.....	307
28 dataint_parity.....	308

28.1 Overview.....	308
28.1.1 Module Declaration.....	308
28.2 Parameters.....	308
28.2.1 Calculated Parameters.....	308
28.3 Ports.....	308
28.4 Functional Description.....	309
28.4.1 Dual Operation Modes.....	309
28.4.2 Parity Types.....	309
28.4.3 Data Chunking.....	309
28.4.4 Generate Block Architecture.....	309
28.4.5 Chunk Boundary Calculation.....	310
28.4.6 Parity Calculation Logic.....	310
28.4.7 Error Detection.....	310
28.4.8 Example Configurations.....	310
28.4.9 Key Features.....	310
28.5 Related Modules.....	311
28.6 Timing Characteristics.....	311
28.7 Usage Examples.....	311
28.7.1 Basic Parity Generation.....	311
28.7.2 Parity Checking.....	312
28.7.3 Error Correction System.....	312
28.8 Design Notes.....	312
28.8.1 Applications.....	312
28.8.2 Chunk Size Selection.....	313

28.8.3 Parity Type Selection.....	313
28.8.4 Width Considerations.....	313
28.9 Testing.....	313
28.10 Navigation.....	313
29 debounce.....	314
29.1 Overview.....	314
29.2 Parameters.....	314
29.3 Ports.....	314
29.4 Functional Description.....	314
29.4.1 Core Algorithm.....	314
29.4.2 Sampling Control.....	315
29.4.3 Button Type Support.....	315
29.4.4 Debounce Logic.....	315
29.4.5 Output Registration.....	315
29.5 Timing Characteristics.....	316
29.5.1 Debounce Delay.....	316
29.5.2 Response Time.....	316
29.6 Usage Examples.....	316
29.7 Design Notes.....	317
29.7.1 Applications.....	317
29.7.2 Design Considerations.....	317
29.8 Related Modules.....	317
29.9 Testing.....	317
29.10 Navigation.....	317

30 decoder.....	318
30.1 Overview.....	318
30.2 Parameters.....	318
30.3 Ports.....	318
30.4 Functional Description.....	318
30.4.1 Operation Principle.....	318
30.4.2 Truth Table Example (M=2, N=4).....	318
30.4.3 Key Characteristics.....	319
30.4.4 Core Logic.....	319
30.4.5 Parameterization.....	319
30.5 Timing Characteristics.....	319
30.6 Usage Examples.....	319
30.6.1 Memory Address Decoding.....	319
30.6.2 State Machine Output Decoding.....	320
30.7 Design Notes.....	320
30.7.1 Applications.....	320
30.7.2 Synthesis Considerations.....	320
30.7.3 Other Notes.....	320
30.8 Related Modules.....	320
30.9 Testing.....	320
30.10 Navigation.....	321
31 encoder.....	322
31.1 Overview.....	322
31.2 Parameters.....	322

31.3 Ports.....	322
31.4 Functional Description.....	322
31.4.1 Operation Principle.....	322
31.4.2 Truth Table Example (N=8, output width=3).....	322
31.4.3 Key Characteristics.....	323
31.4.4 Core Algorithm.....	323
31.4.5 Automatic Width Calculation.....	323
31.4.6 Priority Behavior.....	323
31.4.7 Behavioral Notes.....	323
31.5 Timing Characteristics.....	324
31.6 Usage Examples.....	324
31.6.1 Interrupt Controller.....	324
31.6.2 Arbiter Output Encoding.....	324
31.7 Design Notes.....	324
31.7.1 Applications.....	324
31.7.2 Input Validation.....	324
31.7.3 Synthesis Implications.....	324
31.8 Related Modules.....	325
31.9 Testing.....	325
31.10 Navigation.....	325
32 encoder_priority_enable.....	326
32.1 Overview.....	326
32.2 Parameters.....	326
32.3 Ports.....	326

32.4 Functional Description.....	326
32.4.1 Operation Principle.....	326
32.4.2 Priority Resolution Example (WIDTH=8).....	326
32.4.3 Key Differences from the Basic Encoder.....	327
32.4.4 Core Algorithm.....	327
32.4.5 True Priority Encoding.....	328
32.4.6 Enable Control.....	328
32.4.7 Latch Prevention.....	328
32.5 Timing Characteristics.....	328
32.6 Usage Examples.....	328
32.6.1 Interrupt Controller.....	328
32.6.2 Resource Arbitration.....	328
32.6.3 Error Reporting.....	329
32.7 Design Notes.....	329
32.7.1 Applications.....	329
32.7.2 Priority Assignment.....	329
32.7.3 Performance Scaling.....	329
32.7.4 Enable Usage Patterns.....	329
32.7.5 Synthesis Considerations.....	329
32.8 Related Modules.....	330
32.9 Testing.....	330
32.10 Navigation.....	330
33 fifo_control.....	331
33.1 Overview.....	331

33.2 Parameters.....	331
33.3 Ports.....	331
33.4 Functional Description.....	332
33.4.1 Dual-Domain Design.....	332
33.4.2 Pointer Arithmetic Foundation.....	332
33.4.3 Full Detection Logic.....	332
33.4.4 Almost Full Logic.....	333
33.4.5 Empty Detection Logic - Mode Aware.....	333
33.4.6 Almost Empty Logic.....	334
33.4.7 Count Generation.....	334
33.4.8 Flag Registration.....	335
33.4.9 Width Casting Fix.....	335
33.5 Timing Characteristics.....	336
33.6 Usage Examples.....	336
33.7 Design Notes.....	336
33.7.1 Key Design Insights.....	336
33.7.2 Applications.....	337
33.8 Related Modules.....	337
33.9 Testing.....	337
33.10 Navigation.....	337
34 fifo_sync.....	338
34.1 Overview.....	338
34.2 Parameters.....	338
34.3 Ports.....	338

34.4 Functional Description.....	339
34.4.1 Core Components.....	339
34.4.2 Memory Organization.....	339
34.4.3 Pointer Management.....	339
34.4.4 Memory Operations.....	340
34.4.5 Status Flag Generation.....	340
34.4.6 Operating Modes.....	340
34.4.7 Functional Behavior.....	341
34.5 Timing Characteristics.....	341
34.5.1 Write Sequence.....	341
34.5.2 Read Sequence - Mux Mode.....	341
34.5.3 Read Sequence - Flop Mode.....	341
34.5.4 Performance Characteristics.....	341
34.6 Usage Examples.....	341
34.7 Design Notes.....	342
34.7.1 Mode Selection Guidelines.....	342
34.7.2 Sizing Considerations.....	342
34.7.3 Error Detection.....	342
34.8 Related Modules.....	343
34.9 Testing.....	343
34.9.1 WaveDrom Visualization.....	343
34.9.2 Functional Test Suite.....	344
34.10 Navigation.....	344
35 find_first_set.....	345

35.1 Overview.....	345
35.2 Parameters.....	345
35.3 Ports.....	345
35.4 Functional Description.....	345
35.4.1 Search Strategy.....	345
35.4.2 Truth Table Examples (WIDTH=8).....	345
35.4.3 Key Characteristics.....	346
35.4.4 Core Algorithm.....	346
35.4.5 Width Parameterization.....	346
35.4.6 Found Flag Logic.....	347
35.4.7 Bit Width Handling.....	347
35.5 Timing Characteristics.....	347
35.5.1 Propagation Delay.....	347
35.5.2 Critical Path.....	347
35.6 Usage Examples.....	347
35.6.1 Interrupt Processing.....	347
35.6.2 Resource Allocation.....	347
35.6.3 Error Detection.....	348
35.6.4 Bit Manipulation.....	348
35.6.5 Round-Robin Arbitration.....	348
35.6.6 Free List Management.....	348
35.6.7 Packet Classification.....	348
35.7 Design Notes.....	349
35.7.1 Algorithm Comparison.....	349

35.7.2 Design Considerations.....	349
35.7.3 Synthesis Considerations.....	349
35.8 Related Modules.....	349
35.9 Testing.....	350
35.9.1 Test Scenarios.....	350
35.9.2 Coverage Metrics.....	350
35.10 Navigation.....	350
36 find_last_set.....	351
36.1 Overview.....	351
36.2 Parameters.....	351
36.3 Ports.....	351
36.4 Functional Description.....	351
36.4.1 Search Strategy.....	351
36.4.2 Truth Table Examples (WIDTH=8).....	351
36.4.3 Key Characteristics.....	352
36.4.4 Core Algorithm.....	352
36.4.5 Reverse Loop Implementation.....	352
36.4.6 Width Parameterization.....	353
36.4.7 Found Flag Logic.....	353
36.5 Timing Characteristics.....	353
36.5.1 Propagation Delay.....	353
36.5.2 Critical Path Analysis.....	353
36.6 Usage Examples.....	353
36.6.1 Interrupt Priority Handling.....	353

36.6.2 Leading Zero Count.....	353
36.6.3 Bit Width Calculation.....	354
36.6.4 Cache Line Selection.....	354
36.6.5 Hierarchical Priority Encoding.....	354
36.6.6 Conditional Priority Selection.....	354
36.6.7 Dynamic Width Processing.....	355
36.7 Design Notes.....	355
36.7.1 Algorithm Comparison.....	355
36.7.2 Zero Input Handling.....	355
36.7.3 Performance Scaling.....	356
36.7.4 Synthesis Optimization.....	356
36.8 Related Modules.....	356
36.9 Testing.....	356
36.9.1 Comprehensive Test Coverage.....	356
36.9.2 Random Testing.....	356
36.10 Navigation.....	357
37 hex_to_7seg.....	358
37.1 Overview.....	358
37.2 Parameters.....	358
37.3 Ports.....	358
37.4 Functional Description.....	358
37.4.1 Segment Naming Convention.....	358
37.4.2 Bit Assignment.....	358
37.4.3 Character Patterns.....	359

37.4.4 Case Statement Structure.....	360
37.5 Timing Characteristics.....	360
37.6 Usage Examples.....	360
37.6.1 Debug Displays.....	360
37.6.2 Multi-Digit Displays.....	361
37.6.3 Memory/Register Viewers.....	361
37.6.4 Error Code Display.....	361
37.7 Design Notes.....	361
37.7.1 Design Choices.....	361
37.7.2 Display Technology Integration.....	361
37.7.3 Design Variations.....	363
37.7.4 Multiplexed Display Support.....	363
37.7.5 Common Issues and Debugging.....	364
37.8 Related Modules.....	364
37.9 Testing.....	364
37.9.1 Exhaustive Testing.....	364
37.9.2 Visual Verification.....	364
37.9.3 Pattern Validation.....	365
37.10 Navigation.....	365
38 icg.....	366
38.1 Overview.....	366
38.1.1 Module Declaration.....	366
38.2 Parameters.....	366
38.3 Ports.....	366

38.4 Functional Description.....	366
38.4.1 Clock Gating Operation.....	366
38.4.2 Implementation Details.....	367
38.5 Timing Characteristics.....	367
38.5.1 Critical Timing Requirements.....	367
38.5.2 Timing Diagram.....	367
38.6 Usage Examples.....	368
38.6.1 Basic Register Bank Gating.....	368
38.6.2 CPU Pipeline Stage Gating.....	368
38.6.3 Memory Interface Gating.....	369
38.7 Design Notes.....	369
38.7.1 Power Optimization Benefits.....	369
38.7.2 Safety Requirements.....	369
38.7.3 Enable Signal Guidelines.....	369
38.7.4 Implementation Alternatives.....	370
38.7.5 Synthesis Guidelines.....	370
38.7.6 Design Integration.....	370
38.8 Related Modules.....	371
38.9 Testing.....	371
38.9.1 Functional Tests.....	371
38.9.2 Timing Tests.....	371
38.9.3 Power Analysis.....	371
38.10 Navigation.....	372
39 leading_one_trailing_one.....	373

39.1 Overview.....	373
39.2 Parameters.....	373
39.3 Ports.....	373
39.4 Functional Description.....	374
39.4.1 Submodules.....	374
39.4.2 One-Hot Vector Generation.....	374
39.4.3 Status Flag Logic.....	374
39.4.4 Special Implementation Notes.....	375
39.5 Timing Characteristics.....	375
39.6 Usage Examples.....	375
39.6.1 8-bit Example.....	375
39.6.2 Edge Cases.....	376
39.7 Design Notes.....	376
39.7.1 Applications.....	376
39.8 Related Modules.....	376
39.9 Testing.....	376
39.10 Navigation.....	377
40 pwm.....	378
40.1 Overview.....	378
40.2 Parameters.....	378
40.3 Ports.....	378
40.4 Functional Description.....	379
40.4.1 State Machine.....	379
40.4.2 State Transitions.....	379

40.4.3 Per-Channel Parameter Extraction.....	380
40.4.4 Edge Detection for Start Signal.....	380
40.4.5 PWM Output Generation Logic.....	380
40.4.6 Counter Management.....	380
40.4.7 Critical Control Signals.....	380
40.4.8 Special Implementation Notes.....	381
40.5 Timing Characteristics.....	382
40.5.1 Single Period Example (WIDTH=8, duty=3, period=8).....	382
40.5.2 Multi-Repeat Example (repeat_count=2).....	382
40.6 Usage Examples.....	382
40.6.1 Motor Speed Control.....	382
40.6.2 LED Dimming Sequence.....	382
40.6.3 Servo Motor Control.....	382
40.7 Design Notes.....	382
40.7.1 Applications.....	382
40.8 Related Modules.....	383
40.9 Testing.....	383
40.10 Navigation.....	383
41 rtl-common Quick Start Guide.....	384
41.1 Overview.....	384
41.2 Module Categories.....	384
41.3 Quick Start.....	385
41.3.1 1. Browse Available Modules.....	385
41.3.2 2. Check Module Interface.....	385

41.3.3 3. Review Test Examples.....	385
41.3.4 4. Integrate into Your Design.....	386
41.4 Common Use Cases.....	386
41.4.1 Use Case 1: Simple Event Counter.....	386
41.4.2 Use Case 2: Timeout Timer.....	386
41.4.3 Use Case 3: Multi-Master Arbitration.....	387
41.4.4 Use Case 4: CRC-32 (Ethernet).....	387
41.4.5 Use Case 5: Memory ECC (Single Error Correction).....	388
41.4.6 Use Case 6: Clock Domain Crossing (CDC).....	389
41.5 Module Selection Guide.....	390
41.5.1 “I need a counter...”.....	390
41.5.2 “I need an arbiter...”.....	390
41.5.3 “I need CRC...”.....	390
41.5.4 “I need a FIFO...”.....	391
41.6 Integration Checklist.....	391
41.7 Testing Your Integration.....	391
41.7.1 Option 1: Quick Functional Test.....	391
41.7.2 Option 2: Create Your Own Test.....	392
41.7.3 Option 3: Waveform Debug.....	392
41.8 Common Pitfalls and How to Avoid Them.....	392
41.8.1 Pitfall 1: Wrong Reset Polarity.....	392
41.8.2 Pitfall 2: Parameter Mismatch.....	393
41.8.3 Pitfall 3: Clock Domain Crossing Without Sync.....	393
41.8.4 Pitfall 4: Creating New Module Without Searching.....	394

41.9 Performance Considerations.....	394
41.9.1 Area Optimization.....	394
41.9.2 Timing Optimization.....	394
41.9.3 Power Optimization.....	394
41.10 Getting Help.....	395
41.10.1 Documentation.....	395
41.10.2 Examples.....	395
41.10.3 Commands.....	395
41.10.4 For Claude Code Users.....	396
41.11 Quick Command Reference.....	396
41.12 What's Next?.....	396
41.12.1 Explore Other Subsystems.....	396
41.12.2 Create Your Own Modules.....	397
41.12.3 Contribute.....	397
42 reset_sync.....	398
42.1 Overview.....	398
42.2 Parameters.....	398
42.3 Ports.....	399
42.3.1 Inputs.....	399
42.3.2 Outputs.....	399
42.4 Functional Description.....	400
42.4.1 The Reset Problem.....	400
42.4.2 Asynchronous Assert, Synchronous Deassert.....	400
42.4.3 Synchronization Register Chain.....	400

42.4.4 Reset Logic.....	400
42.4.5 Output Assignment.....	400
42.5 Timing Characteristics.....	401
42.5.1 Reset Assertion (Asynchronous).....	401
42.5.2 Reset Deassertion (Synchronous, N=3).....	401
42.5.3 State Sequence for N=3.....	401
42.6 Usage Examples.....	402
42.6.1 Choosing N Value.....	402
42.6.2 Global Reset Distribution.....	403
42.6.3 Clock Domain Crossing.....	403
42.6.4 Power-On Reset.....	404
42.6.5 Reset Controller Integration.....	404
42.7 Design Notes.....	404
42.7.1 1. Why the chain looks inverted.....	404
42.7.2 2. Metastability Resolution.....	405
42.7.3 3. Initialization.....	405
42.7.4 4. Vendor attributes.....	405
42.7.5 5. Parameterization Benefits.....	405
42.7.6 Reset Tree Planning.....	405
42.7.7 Clock Quality Requirements.....	405
42.7.8 Reset Source Management.....	406
42.7.9 Incorrect Sensitivity List.....	406
42.7.10 Wrong Reset Polarity.....	406
42.7.11 Clock Domain Issues.....	406

42.8 Related Modules.....	406
42.9 Testing.....	407
42.9.1 Test Coverage.....	407
42.9.2 Test Scenarios.....	407
42.9.3 Testbench Scenarios.....	407
42.9.4 Assertions.....	407
42.9.5 Coverage Points.....	407
42.9.6 Running Tests.....	407
42.9.7 Test Results.....	408
42.10 Navigation.....	408
43 reverse_vector.....	409
43.1 Overview.....	409
43.2 Parameters.....	409
43.3 Ports.....	409
43.3.1 Inputs.....	409
43.3.2 Outputs.....	409
43.4 Functional Description.....	409
43.4.1 Bit Reversal Logic.....	409
43.4.2 8-bit Vector Reversal.....	410
43.4.3 4-bit Vector Reversal.....	410
43.5 Timing Characteristics.....	410
43.6 Usage Examples.....	411
43.6.1 Endianness Conversion.....	411
43.6.2 Serial Communication.....	411

43.6.3 Cryptographic Operations.....	411
43.6.4 Signal Processing.....	411
43.6.5 Data Format Conversion.....	411
43.6.6 Mirror Pattern Generation.....	411
43.6.7 Instantiation with Different Widths.....	411
43.6.8 Pipeline Integration.....	411
43.6.9 Conditional Reversal.....	412
43.7 Design Notes.....	412
43.7.1 1. Combinational Operation.....	412
43.7.2 2. Parameterizable Width.....	412
43.7.3 3. For-Loop Synthesis.....	412
43.7.4 4. Bidirectional Property.....	412
43.7.5 Power Consumption.....	412
43.7.6 Area Utilization.....	412
43.7.7 Hardware Implementation.....	413
43.7.8 Optimization.....	413
43.7.9 Performance Metrics.....	413
43.7.10 Common Pitfalls.....	413
43.8 Related Modules.....	414
43.8.1 Byte Reversal.....	414
43.8.2 Configurable Reversal.....	414
43.8.3 Multi-Width Reversal.....	414
43.9 Testing.....	414
43.9.1 Test Cases.....	414

43.9.2 Coverage Points.....	415
43.10 Navigation.....	415
44 shifter_barrel.....	416
44.1 Overview.....	416
44.2 Parameters.....	416
44.3 Ports.....	416
44.3.1 Inputs.....	416
44.3.2 Outputs.....	416
44.4 Functional Description.....	417
44.4.1 Control Signal Encoding.....	417
44.4.2 Shift Amount Modulation.....	417
44.4.3 Wrap-Around Implementation.....	418
44.4.4 Pre-computed Rotation Arrays.....	418
44.4.5 Main Shift Logic.....	418
44.5 Timing Characteristics.....	419
44.5.1 8-bit Examples (WIDTH=8).....	419
44.6 Usage Examples.....	419
44.7 Design Notes.....	420
44.7.1 1. Combinational Design.....	420
44.7.2 2. Arithmetic Right Shift.....	420
44.7.3 4. Double-Width Concatenation.....	420
44.7.4 5. Generate Block Optimization.....	420
44.7.5 Area Considerations.....	420
44.7.6 Speed Characteristics.....	420

44.8 Related Modules.....	420
44.9 Testing.....	421
44.10 Navigation.....	421
45 shifter_beat_pack.....	422
45.1 Overview.....	422
45.2 Parameters.....	423
45.3 Ports.....	424
45.3.1 Inputs.....	424
45.3.2 Outputs.....	424
45.4 Functional Description.....	425
45.4.1 Runtime Beat Width.....	425
45.4.2 Handshake and Status.....	425
45.4.3 Next-State: Pop First, Push Second.....	425
45.4.4 Elaboration-Time Contract.....	426
45.4.5 Sizing Guidance.....	426
45.5 Timing Characteristics.....	426
45.6 Usage Examples.....	426
45.7 Design Notes.....	427
45.8 Related Modules.....	427
45.9 Testing.....	428
45.10 References.....	428
45.10.1 Source Code.....	428
45.10.2 Documentation.....	428
45.11 Navigation.....	428

46 shifter_lfsr_fibonacci.....	429
46.1 Overview.....	429
46.2 Parameters.....	429
46.3 Ports.....	429
46.3.1 Inputs.....	429
46.3.2 Outputs.....	430
46.4 Functional Description.....	430
46.4.1 Fibonacci LFSR Characteristics.....	430
46.4.2 Tap Processing (Same as Standard LFSR).....	431
46.4.3 Fibonacci Feedback Calculation.....	431
46.4.4 Right-Shift with MSB Feedback.....	431
46.5 Timing Characteristics.....	432
46.5.1 4-bit Fibonacci LFSR Example.....	432
46.5.2 Sequence Generation.....	432
46.6 Usage Examples.....	432
46.7 Design Notes.....	433
46.7.1 1. XOR vs XNOR Feedback.....	433
46.7.2 2. Non-Zero State Protection.....	433
46.7.3 3. Right-Shift Architecture.....	433
46.7.4 4. Polynomial Compatibility.....	433
46.7.5 5. Reset to All Zeros.....	433
46.7.6 Advantages of Fibonacci LFSR.....	433
46.7.7 Disadvantages of Fibonacci LFSR.....	434
46.7.8 When to Use Fibonacci LFSR.....	434

46.7.9 Polynomial Examples for Fibonacci LFSR.....	434
46.7.10 Why the Encoding Differs.....	435
46.7.11 Usage Note.....	435
46.8 Related Modules.....	435
46.9 Testing.....	435
46.10 Navigation.....	436
47 shifter_lfsr_galois.....	437
47.1 Overview.....	437
47.2 Parameters.....	437
47.3 Ports.....	437
47.3.1 Inputs.....	437
47.3.2 Outputs.....	438
47.4 Functional Description.....	438
47.4.1 Key Architectural Differences.....	438
47.4.2 Advantages of Galois Architecture.....	438
47.4.3 Tap Position Processing.....	438
47.4.4 LSB Feedback Generation.....	439
47.4.5 Galois Next State Calculation.....	439
47.4.6 Sequential Update.....	439
47.5 Testing.....	439
47.6 Timing Characteristics.....	440
47.6.1 4-bit Galois LFSR Example.....	440
47.6.2 Sequence Generation.....	440
47.7 Usage Examples.....	440

47.7.1 High-Speed Systems.....	440
47.7.2 Communications.....	440
47.7.3 Cryptography.....	440
47.7.4 Testing.....	440
47.8 Design Notes.....	441
47.8.1 1. Reset to All Ones.....	441
47.8.2 2. Combinational Next State Logic.....	441
47.8.3 3. Conditional XOR Application.....	441
47.8.4 4. Bounds Checking.....	441
47.8.5 5. Right-Shift with Distributed Feedback.....	441
47.8.6 Critical Path Analysis.....	441
47.8.7 Timing Advantages.....	442
47.8.8 Tap Position Mapping.....	442
47.8.9 Example Polynomials.....	442
47.8.10 Galois LFSR Advantages.....	442
47.8.11 Galois LFSR Disadvantages.....	443
47.8.12 When to Choose Galois over Fibonacci.....	443
47.9 Related Modules.....	443
47.10 Navigation.....	443
48 shifter_lfsr.....	444
48.1 Overview.....	444
48.2 Parameters.....	444
48.3 Ports.....	444
48.3.1 Inputs.....	444

48.3.2 Outputs.....	445
48.4 Functional Description.....	445
48.4.1 Feedback Polynomial.....	445
48.4.2 Tap Position Processing.....	445
48.4.3 Feedback Calculation.....	445
48.4.4 Shift Register Operation.....	446
48.5 Timing Characteristics.....	446
48.5.1 Seed Loading and Operation.....	446
48.5.2 Sequence Example (3-bit LFSR, taps=[3,2]).....	446
48.6 Usage Examples.....	446
48.6.1 Pseudo-Random Number Generation.....	446
48.6.2 Built-In Self-Test (BIST).....	447
48.6.3 Scrambling/Descrambling.....	447
48.6.4 Common Use Cases.....	447
48.7 Design Notes.....	447
48.7.1 1. Tap Position Encoding.....	447
48.7.2 2. XNOR Feedback.....	447
48.7.3 3. Cycle Detection.....	447
48.7.4 4. Left-Shift Architecture.....	448
48.7.5 5. Zero State Handling.....	448
48.7.6 LFSR Polynomial Reference.....	448
48.7.7 Example Tap Configuration for 8-bit LFSR.....	448
48.8 Related Modules.....	449
48.9 Testing.....	449

48.10 Navigation.....	449
49 shifter_universal.....	450
49.1 Overview.....	450
49.2 Parameters.....	450
49.3 Ports.....	450
49.3.1 Inputs.....	450
49.3.2 Outputs.....	451
49.4 Functional Description.....	451
49.4.1 Operation Control.....	451
49.4.2 Combinational Logic for Next State.....	451
49.4.3 Sequential Logic for State Update.....	452
49.4.4 Hold Operation (select = 2'b00).....	452
49.4.5 Right Shift Operation (select = 2'b01).....	452
49.4.6 Left Shift Operation (select = 2'b10).....	453
49.4.7 Parallel Load Operation (select = 2'b11).....	453
49.5 Related Modules.....	453
49.6 Timing Characteristics.....	453
49.6.1 Right Shift Operation Example (WIDTH=4).....	453
49.6.2 Parallel Load followed by Left Shift.....	454
49.7 Usage Examples.....	454
49.7.1 Serial Communication Interface.....	454
49.7.2 Multiplication/Division by Powers of 2.....	454
49.7.3 Data Delay Line.....	454
49.7.4 Pattern Generator.....	454

49.7.5 Scan Chain for Testing.....	455
49.7.6 Common Use Cases.....	455
49.8 Design Notes.....	455
49.8.1 1. Bidirectional Serial Interface.....	455
49.8.2 2. Clean Serial Output Management.....	456
49.8.3 3. Asynchronous Reset.....	456
49.8.4 4. Combinational/Sequential Separation.....	456
49.8.5 Chaining Multiple Shifters.....	456
49.8.6 Bidirectional Operation.....	456
49.8.7 Control Logic Integration.....	456
49.9 Testing.....	456
49.10 Navigation.....	457
50 sort.....	458
50.1 Overview.....	458
50.1.1 Module Declaration.....	458
50.2 Parameters.....	458
50.3 Ports.....	459
50.3.1 Inputs.....	459
50.3.2 Outputs.....	459
50.4 Functional Description.....	459
50.4.1 Data Format.....	459
50.4.2 Algorithm: Odd-Even Sort.....	459
50.4.3 Example Execution.....	460
50.4.4 Algorithm Comparison.....	461

50.4.5 Pipeline Structure.....	462
50.4.6 Naming Conventions.....	462
50.4.7 Compare-Swap Logic.....	462
50.4.8 Reset Behavior.....	463
50.4.9 Control Flow.....	463
50.5 Related Modules.....	464
50.6 Timing Characteristics.....	464
50.6.1 Timing Characteristics.....	464
50.6.2 Valid Signal Flow.....	464
50.7 Usage Examples.....	464
50.7.1 Typical Use Cases.....	464
50.7.2 Integration Considerations.....	465
50.7.3 Performance Optimization.....	465
50.8 Design Notes.....	465
50.8.1 1. Parameterizable Design.....	465
50.8.2 2. High Throughput Pipeline.....	465
50.8.3 3. Synthesis-Friendly Design.....	465
50.8.4 4. Packed Data Handling.....	466
50.8.5 Synthesis Results.....	466
50.8.6 Future Enhancements.....	466
50.9 Testing.....	466
50.9.1 Test Levels.....	466
50.9.2 Test Coverage.....	467
50.10 Navigation.....	467

[← Back to Main Index](#) | [rtl-common Index](#)

1 rtl-common Library Overview

The rtl-common library is a collection of fundamental digital design building blocks — parameterizable, synthesis-friendly RTL modules built to a consistent quality bar, ready to drop into a wide range of digital systems. If you’ve hand-rolled a counter or a CRC one too many times (and if you’ve been doing this long enough, you have), this library is where that stops.

1.1 Library Philosophy

1.1.1 Design Principles

Five principles run through everything in here:

- **Modularity:** Each module does one specific function and composes easily with the others
- **Parameterizability:** Configurable width and functionality parameters for maximum reusability
- **Synthesis Friendly:** Optimized for modern synthesis tools with predictable area/timing results
- **Technology Independence:** Clean RTL that works across ASIC and FPGA technologies
- **Broad Coverage:** counters, FIFOs, arbiters, data integrity, and clock/reset control (arithmetic is its own library — [rtl-math](#); CDC is its own too — [rtl-cdc](#))

1.1.2 Quality Standards

Every module in the tree is held to the same bar:

- **Lint Clean:** All modules pass rigorous RTL linting checks
- **Simulation Verified:** Comprehensive testbenches validate functionality
- **Synthesizable:** Written in standard SystemVerilog with no vendor primitives
- **Documentation Complete:** Detailed documentation with usage examples

1.2 Architecture Overview

1.2.1 Module Hierarchy

Per-category counts live in one place — the [count table in index.md](#) — so this tree lists categories only and cannot drift from it.

- rtl-common Library
 - (Arithmetic & Math moved to rtl/math/ – see rtl-math book)
 - Data Integrity
 - Error Detection (CRC, Checksum, Parity)
 - Error Correction (Hamming ECC)
 - Control & Arbitration
 - Round-Robin Arbiters
 - Priority Arbiters
 - Clock & Reset
 - Clock Management (ICG, Dividers)
 - Reset Synchronization
 - Counters & Sequences
 - Binary Counters (Gray/Johnson counters moved to rtl/cdc/)
 - Specialized Counting
 - Data Conversion
 - Code Converters (Binary→BCD; Gray/Johnson moved to rtl/cdc/)
 - Display Decoders
 - Bit Operations
 - Bit Searching & Counting
 - Vector Manipulation
 - Shift & LFSR
 - Barrel Shifters
 - Pseudorandom Generation
 - Memory & Storage
 - FIFO Implementations
 - Content Addressable Memory
 - Utilities
 - PWM Generation
 - Sorting Algorithms

1.3 Functional Categories

1.3.1 1. Arithmetic and Mathematical Operations

Moved. The math_* arithmetic library (adders, subtractors, multipliers, compressors, prefix cells, and the BF16 / FP8 / FP16 / FP32 / IEEE-754 floating-point modules) is now its own subsystem and its own book: [rtl-math](#) — RTL in rtl/math/, tests in val/math/.

1.3.2 2. Data Integrity and Error Management

If data has to survive storage or transmission, these are the blocks that make that happen.

1.3.2.1 Cyclic Redundancy Check (CRC)

- **Generic CRC Engine:** Configurable polynomial, width, and initialization
- **Cascade Implementation:** High-throughput parallel CRC processing
- **XOR-Shift Variants:** Optimized for specific polynomial types

1.3.2.2 Error Correction Coding (ECC)

- **Hamming SEDED:** Single Error Correction, Double Error Detection
- **Configurable Width:** Support for various data widths
- **Encoder/Decoder Pairs:** Complete ECC implementation

1.3.2.3 Simple Integrity Checks

- **Checksum:** Software-compatible checksum algorithms
- **Parity:** Even/odd parity generation and checking

Key Features: - Standards-compliant implementations - Parameterizable for various data widths - Optimized for both latency and throughput scenarios

1.3.3 3. Control Logic and Arbitration

Shared resources need a referee. These blocks are it — the essential components for managing shared resources and coordinating the system.

1.3.3.1 Round-Robin Arbitration

- **Simple Round-Robin:** Basic fair arbitration with minimal overhead
- **Weighted Round-Robin:** Priority-based fair arbitration
- **Advanced Features:** Configurable agent counts and weighting

1.3.3.2 Priority-Based Control

- **Priority Encoders:** Fixed priority arbitration schemes
- **Configurable Priority:** Runtime priority adjustment capability

Applications: - Bus arbitration in multi-master systems - Resource allocation in processors - Quality of Service (QoS) implementations

1.3.4 4. Clock and Reset Management

Power and timing management — the unglamorous foundation of a dependable digital system.

1.3.4.1 Clock Control

- **Integrated Clock Gating (ICG):** Power optimization through controlled clock disabling
- **Clock Dividers:** Frequency synthesis and clock domain generation
- **Pulse Generators:** Controlled clock pulse generation

1.3.4.2 Reset and Synchronization

- **Reset Synchronizers:** Safe reset assertion/deassertion
- **Debouncing:** Input signal stabilization

Power Management Benefits: - Clock gating (ICG) removes switching on idle functional blocks; the actual dynamic-power saving depends on activity factor and is design-specific - Proper reset sequencing for reliable startup - Signal integrity maintenance

1.3.5 5. Counting and Sequence Generation

Fundamental building blocks for timing, addressing, and control.

1.3.5.1 Binary Counters

- **Basic Counters:** Up/down counting with enable/reset
- **Gray and Johnson Counters:** `counter_bingray`, `counter_johnson` — these moved to `rtl/cdc/`; see the [rtl-cdc book](#)

1.3.5.2 Specialized Counting

- **Frequency-Invariant:** Clock-independent counting
- **Load/Clear:** Programmable counter initialization
- **Ring Counters:** One-hot sequence generation

Design Features: - Parameterizable width and functionality - Optional synchronous/asynchronous controls - Overflow/underflow detection

1.3.6 6. Data Conversion and Encoding

Code conversion and display interfacing utilities.

1.3.6.1 Number System Conversion

- **Binary → BCD:** Decimal display conversion

- **Binary ↔ Gray, Johnson decode:** `bin2gray`, `gray2bin`, `johnson2bin` — these moved to `rtl/cdc/`; see the [rtl-cdc book](#)

1.3.6.2 Encoding/Decoding

- **Priority Encoders:** Multiple input to binary conversion
- **Binary Decoders:** Address decoding and control
- **7-Segment Display:** Hexadecimal character display

Applications: - Display interfaces - Address decoding - Safe async signal crossing

1.3.7 7. Bit Manipulation and Searching

Efficient bit-level operations for high-performance computing.

1.3.7.1 Bit Detection

- **Leading Zero Count:** Normalization and floating-point support
- **Find First/Last Set:** Efficient bit searching
- **Priority Detection:** Leading one and trailing one detection

1.3.7.2 Vector Operations

- **Bit Reversal:** Endianness conversion and FFT applications
- **Vector Manipulation:** Bit-level data reorganization

Performance Characteristics: - Logarithmic complexity algorithms where possible - Optimized for synthesis tool recognition - Support for various data widths

1.3.8 8. Shift Operations and LFSRs

Shift-based operations and pseudorandom sequence generation.

1.3.8.1 Shifters

- **Barrel Shifters:** Arbitrary shift/rotate in single cycle
- **Universal Shifters:** Combined left/right, logical/arithmetic shifting

1.3.8.2 Linear Feedback Shift Registers (LFSRs)

- **Fibonacci LFSRs:** External feedback topology
- **Galois LFSRs:** Internal feedback topology
- **Configurable Polynomials:** Various sequence lengths and properties

Applications: - Pseudorandom number generation - Built-in self-test (BIST) patterns - Encryption and scrambling

1.3.9 9. Memory and Storage

Buffering and storage solutions for data flow management.

1.3.9.1 FIFO Implementations

- **Synchronous FIFO:** Single clock domain buffering (`fifo_sync`, `fifo_control`)
- **Asynchronous FIFO:** `fifo_async` moved to `rtl/cdc/`; production FIFOs live in `rtl/amba/gaxi/`

1.3.9.2 Associative Memory

- **Content Addressable Memory (CAM):** Tag matching and lookup

Features: - Configurable depth and width - Full/empty flag generation - Error handling and overflow protection

1.3.10 10. Utility and Specialized Functions

Specialized functions for specific application domains.

1.3.10.1 Signal Generation

- **PWM:** Pulse width modulation for analog control
- **Timing Control:** Precise pulse and timing generation

1.3.10.2 Data Processing

- **Hardware Sorting:** Parallel sorting implementations
- **Algorithmic Processing:** Specialized computational functions

1.4 Performance and Optimization

Picking the right block is half the job — how you configure it decides whether you close timing and meet the power budget.

1.4.1 Synthesis Guidelines

1.4.1.1 Area Optimization

- Use ripple-carry arithmetic for low-area requirements
- Select appropriate FIFO depths based on data flow analysis

- Consider shared resources for infrequently used functions

1.4.1.2 Speed Optimization

- Use parallel prefix adders (Brent-Kung, Han-Carlson) for critical paths
- Implement pipelining in high-throughput applications
- Use barrel shifters for single-cycle shift operations

1.4.1.3 Power Optimization

- Apply clock gating (ICG) to inactive functional blocks
- Use Gray code counters for reduced switching activity
- Implement frequency-invariant designs where applicable

1.4.2 Technology-Specific Considerations

1.4.2.1 ASIC Implementation

- Use dedicated adder and multiplier cells
- Use technology-specific clock gating cells
- Optimize for specific voltage and temperature ranges

1.4.2.2 FPGA Implementation

- Use DSP blocks for arithmetic operations
- Use BRAM for deep FIFOs
- Use the dedicated clock management resources

1.5 Verification and Validation

1.5.1 Testbench Coverage

- **Functional Coverage:** All operational modes and edge cases
- **Directed Tests:** Specific functionality verification
- **Random Testing:** Stress testing with random inputs
- **Corner Cases:** Boundary conditions and error scenarios

1.5.2 Compliance Testing

- **Timing Analysis:** Setup/hold requirements validation
- **Power Analysis:** Dynamic and static power characterization
- **Area Analysis:** Resource utilization across technologies

1.6 Usage Guidelines

1.6.1 Module Selection

1. **Identify Requirements:** Performance, area, and power constraints
2. **Select Architecture:** Choose appropriate implementation (ripple vs. parallel, etc.)
3. **Configure Parameters:** Set widths and options for specific application
4. **Verify Integration:** Ensure proper interfacing and timing

1.6.2 Integration Best Practices

The example below instantiates `math_adder_full_nbit`, which now lives in `rtl/math/` — modules compose across the two libraries.

// Example: Building a complete arithmetic unit (math_ modules are in rtl/math/)*

```
module arithmetic_unit #(
    parameter int DATA_WIDTH = 32
) (
    input logic clk,
    input logic rst_n,
    input logic [DATA_WIDTH-1:0] operand_a,
    input logic [DATA_WIDTH-1:0] operand_b,
    input logic [1:0] operation,
    output logic [DATA_WIDTH-1:0] result,
    output logic overflow
);
```

```
// Use appropriate math modules based on requirements
logic [DATA_WIDTH-1:0] add_result, sub_result, mult_result;
logic add_carry, sub_borrow;
```

```
// High-speed addition using parallel prefix
math_adder_full_nbit #(
    .N(DATA_WIDTH)
```

```

) u_adder (
    .i_a(operand_a), .i_b(operand_b), .i_c(1'b0),
    .ow_sum(add_result), .ow_carry(add_carry)
);

// Subtraction using complement method
math_subtractor_full_nbit #(
    .N(DATA_WIDTH)
) u_subtractor (
    .i_a(operand_a), .i_b(operand_b), .i_b_in(1'b0),
    .ow_d(sub_result), .ow_b(sub_borrow)
);

// Result selection based on operation
always_comb begin
    case (operation)
        2'b00: begin result = add_result; overflow = add_carry;
end
        2'b01: begin result = sub_result; overflow = sub_borrow;
end
        default: begin result = '0; overflow = 1'b0; end
    endcase
end

endmodule

```

1.7 Future Enhancements

1.7.1 Planned Additions

- **DSP Functions:** FFT, filtering, and signal processing primitives
- **Security Primitives:** Hardware security modules and cryptographic functions
- **AI/ML Accelerators:** Matrix operations and neural network primitives

1.7.2 Technology Evolution

- **Emerging Technologies:** Support for new process nodes and device types
- **Tool Integration:** Enhanced synthesis and verification tool compatibility
- **Standards Compliance:** Updates for evolving industry standards

That's the library. The blocks are verified, documented, and built to a consistent bar — so you can spend your design effort on the parts of the system that are actually new.

2 Deficit Round Robin Arbiter

Module: rtl/common/arbiter_deficit_round_robin.sv

2.1 Overview

Deficit round-robin (DRR) arbiter — the sibling wrapper to [arbiter_round_robin_weighted](#) around the same [arbiter_round_robin](#) core. The two disciplines answer different questions:

- **WRR:** shares proportional to **grant count**. Weight 4 means four grants per replenish round, whatever each grant costs.
- **DRR:** shares proportional to **cost served**. Quantum 4 means four cost-units (bytes, beats, bus cycles) per round, however many grants that takes.

With equal-cost requests the two give identical long-run shares — use the cheaper WRR there. Reach for the DRR when a grant's resource usage varies per request: packet lengths, burst sizes, variable-beat DMA descriptors.

```
module arbiter_deficit_round_robin #(
    parameter int CLIENTS      = 4,
    parameter int MAX_QUANTUM  = 16,
    parameter int COST_WIDTH   = 4,
    parameter int WAIT_GNT_ACK = 0
) (
    input logic clk,
    input logic rst_n,
    input logic block_arb,
    input logic [CXQW-1:0] quantum,
    input logic [CXCW-1:0] req_cost,
    input logic [C-1:0] request,
    input logic [C-1:0] grant_ack,
    output logic grant_valid,
    output logic [C-1:0] grant,
    output logic [N-1:0] grant_id
);
```

2.2 Parameters

2.2.1 User-Settable Parameters

Parameter	Type	Default	Description
CLIENTS	int	4	Number of requesting clients (range: 2-32)

Parameter	Type	Default	Description
MAX_QUANTUM	int	16	Exclusive bound on per-client quantum; field width = $\$clog2(\text{MAX_QUANTUM})$ (range: 2-256)
COST_WIDTH	int	4	Width of each client's request-cost input (costs $1..2^{\text{COST_WIDTH}}-1$; range: 1-16)
WAIT_GNT_ACK	int	0	0 = grant completes immediately, 1 = grant held until grant_ack

2.2.2 Derived Parameters

Declared in the parameter list so the port declarations can use them (strict front ends reject body localparams in port ranges); overridable in principle, and must not be — leave them to their derivations. Only DW is a true localparam.

Parameter	Default	Description
QW	$\$clog2(\text{MAX_QUANTUM})$	Quantum field width
DW	$\$clog2(2^{\text{COST_WIDTH}} + \text{MAX_QUANTUM}) + 1$	Deficit counter width — sized so every legal cost is reachable by accumulation, which is the no-livelock guarantee
N	$\$clog2(\text{CLIENTS})$	grant_id width
CXQW / CXCW	$\text{CLIENTS} \times \text{QW} / \text{CLIENTS} \times \text{COST_WIDTH}$	Packed array widths

2.3 Ports

2.3.1 Inputs

Port	Width	Description
clk	1	Clock
rst_n	1	Async active-low reset
block_arb	1	Block all arbitration while high
quantum	CXQW	Packed per-client quanta, client 0 in the low field. Zero disables a client. Changes go through the atomic-update FSM
req_cost	CXCW	Packed cost of each client's HEAD request. Stable while request held; the next frame's cost may be presented as soon as the current frame's grant is observed
request	CLIENTS	Request vector
grant_ack	CLIENTS	Grant acknowledgment (ACK mode only)

2.3.2 Outputs

Port	Width	Description
grant_valid	1	Grant output valid
grant	CLIENTS	One-hot grant vector
grant_id	N	Binary-encoded winner

2.4 Functional Description

2.4.1 The Deficit Discipline

Each client holds a deficit counter (reset 0). A requester is **eligible** when its deficit covers its head cost. When requests exist but nobody can afford service, a **replenish round** adds each requesting client's quantum to its deficit — repeating on consecutive cycles until someone becomes eligible, which is how a cost larger than one quantum is saved up for (multi-round accumulation; the deficit width guarantees termination). On grant completion the winner is debited the served cost and the **remainder carries** to its next request — the carry is what keeps long-run shares on the quantum ratio regardless of how costs divide into it.

Anti-hoarding: a client's deficit clears when its request deasserts (classic DRR empty-queue rule). Service share is earned while competing — an idle client cannot bank deficit and burst later. Hold request through frame gaps if the carry should survive.

2.4.2 The Cost Pipeline

The grant registers one cycle after the arbitration that won it. A consumer naturally pops its frame on grant and presents the next frame's cost immediately — so in the completion cycle `req_cost` may already belong to the *next* frame. The arbiter therefore debits the **arbitration-cycle** cost, captured in an internal one-deep pipeline. Without it, a back-to-back client is debited the wrong frame's cost (this was caught during bring-up by the testbench's deficit mirror, not by inspection — see the test).

2.4.3 Share Example

Quanta [4,2,1,1], every request cost 2:

- Replenish gives deficits [4,2,1,1]: clients 0 and 1 afford immediately, clients 2 and 3 must accumulate a second round to reach 2.
- Long-run cost-units served settle at 4:2:1:1 — the quantum ratio — even though clients 2 and 3 are only served every other round.

2.4.4 Quantum Changes and Disable

Runtime quantum changes pass through the same shadow-register FSM as the WRR's weights (IDLE → BLOCK → DRAIN → UPDATE → STABILIZE; 10 cycles minimum, ~25 with the BLOCK timeout); deficits clear at STABILIZE so old carry cannot distort the new policy. Quantum 0 disables a client entirely. A cost of 0 is defensively served as cost 1.

2.5 Design Notes

Deficit round robin shapes by CREDIT, not by turn. A client with a larger quantum receives proportionally more service over a round rather than more turns, so short-burst and long-burst clients with the same quantum get the same bandwidth, not the same grant count. Size quanta in the unit the downstream consumes.

The compliance model lives in the DV framework. Round-robin order and fairness verdicts come from CocoTBFramework’s ArbiterCompliance; several historical “violations” in this family were defects in that model rather than in any arbiter (common BUG-006 (was COMMON-016) through common BUG-009 (was COMMON-019)).

2.6 Timing Characteristics

Property	Value
Latency	1 cycle steady-state (deficit compare, masking and RR decision combinational; grant registered in the core)
Throughput	1 grant/cycle max
Replenish	1 cycle per round, repeats until affordable
Quantum update	10-25 cycles (FSM; DRAIN occupies 3 cycles, STABILIZE 4)

Critical path: the deficit $\geq$ cost comparator into the request mask into the base arbiter — one comparator wider than the WRR’s credit-nonzero check. At high client counts consider registering eligibility (costs a cycle of latency).

2.7 Usage Examples

Every parameter and port below is taken from the module declaration.

```
arbiter_deficit_round_robin #(
    .CLIENTS          (4),
    .MAX_QUANTUM      (16),
    .COST_WIDTH       (4),
    .WAIT_GNT_ACK     (0),
    .QW                ($clog2(MAX_QUANTUM)),
    .N                 ($clog2(CLIENTS)),
    .C                 (CLIENTS),
    .CXQW              (CLIENTS * QW),
    .CXCW              (CLIENTS * COST_WIDTH),
) u_arbiter_deficit_round_robin (
    .clk                (clk),
    .rst_n              (rst_n),
    .block_arb          (block_arb),
    .quantum             (quantum),
    .req_cost            (req_cost),
    .request             (request),
```

```
.grant_ack          (grant_ack),  
.grant_valid        (grant_valid),  
.grant              (grant),  
.grant_id           (grant_id)  
);
```

2.8 Related Modules

- [arbiter_round_robin](#) — the shared base core
- [arbiter_round_robin_weighted](#) — the grant-count sibling; use it when requests are equal-sized
- [arbiter_round_robin_simple](#), [arbiter_priority_encoder](#)

2.9 Testing

Location: `val/common/test_arbiter_deficit_round_robin.py`

The property under test is cost-proportionality: served cost-units per client must follow the quantum ratio whatever the per-request costs are. Scenarios: equal-cost anchor, mixed random costs (the DRR-defining case), cost > quantum accumulation, anti-hoarding, zero-quantum disable, dynamic quantum change; both ACK modes, 4-16 clients. Every scenario also runs a cycle mirror of the deficit discipline that fails the test on any grant to a client whose deficit did not cover its arbitration-cycle cost.

```
pytest val/common/test_arbiter_deficit_round_robin.py -v
```

2.10 Navigation

- [Index](#) · [Overview](#)

[← Back to Main Index](#) | [rtl-common Index](#)

3 Priority Encoder Arbiter

A high-speed combinational priority encoder built for arbitration. Priority is fixed: lower-indexed clients always have higher priority.

3.1 Overview

The `arbiter_priority_encoder` module gives you single-cycle priority encoding, with optimized implementations for the common client counts (4, 8, 16, 32) built on fully unrolled casez logic. For other client counts, it falls back to a synthesizable loop-based approach. You also get dual request inputs (masked and unmasked) for use in more elaborate arbitration schemes — the round-robin arbiter in this library is built on exactly that trick.

```
module arbiter_priority_encoder #(
    parameter int CLIENTS = 4,           // Number of requesting
clients
    parameter int N = $clog2(CLIENTS)    // Winner ID width (derived)
) (
    input logic [CLIENTS-1:0] requests_masked,    // Masked
request vector
    input logic [CLIENTS-1:0] requests_unmasked,    // Unmasked
request vector
    input logic                                any_masked_requests, // Select masked
vs unmasked
    output logic [N-1:0] winner,                  // Binary-
encoded winner ID
    output logic                                winner_valid      // Winner output
is valid
);
```

3.2 Parameters

3.2.1 User-Settable Parameters

Parameter	Type	Default	Description
CLIENTS	int	4	Number of requesting clients (range: 2-1024)

3.2.2 Derived Parameters

Parameter	Default	Description
N	<code>\$clog2(CLIENTS)</code>	Winner ID width in bits

Note: `N` defaults to `$clog2(CLIENTS)` and should be left alone – but it is an ordinary parameter, not a `localparam`, so it *can* be overridden at instantiation. It is declared that way so it can size the winner port; `arbiter_round_robin` relies on this and passes `.N(N)` explicitly.

3.3 Ports

3.3.1 Inputs

Port	Width	Description
<code>requests_masked</code>	<code>CLIENTS</code>	Masked request vector (higher priority source)
<code>requests_unmasked</code>	<code>CLIENTS</code>	Unmasked request vector (fallback source)
<code>any_masked_requests</code>	<code>1</code>	Select masked (1) or unmasked (0) requests

3.3.2 Outputs

Port	Width	Description
<code>winner</code>	<code>N</code>	Binary-encoded winner client ID
<code>winner_valid</code>	<code>1</code>	Asserted when at least one request is active

3.4 Functional Description

3.4.1 Priority Order

The module implements **fixed priority** arbitration: - **Client 0**: Highest priority (always wins if requesting) - **Client 1**: Second highest priority - ... - **Client CLIENTS-1**: Lowest priority

3.4.2 Request Selection

The module evaluates requests in priority order:

1. If `any_masked_requests = 1`: Use `requests_masked` input
2. If `any_masked_requests = 0`: Use `requests_unmasked` input
3. Find lowest-indexed asserted bit in selected request vector
4. If no requests: `winner = 0` (don't care), `winner_valid = 0`

3.4.3 Combinational Operation

- **Zero cycles latency:** Purely combinational logic
- **No clock or reset:** Outputs update immediately with input changes
- **Single-cycle response:** Winner determined in one combinational path

3.4.4 Fixed Priority vs Round-Robin

Worth knowing what you're signing up for when you pick fixed priority:

Aspect	Fixed Priority	Round-Robin
Fairness	No (low-indexed clients favored)	Yes (all clients equal)
Latency	Deterministic for high-priority	Variable
Starvation	Possible for low-priority	Prevented
Simplicity	Very simple (this module)	More complex
Best Use	Interrupt handling, critical tasks	Data paths, fair scheduling

3.4.5 Priority Inversion

Issue: High-priority client blocked by low-priority client holding resource

Mitigation Strategies: 1. **Priority Inheritance:** Temporarily boost blocked client's priority 2.

Resource Preemption: Force low-priority client to release resource 3. **Priority Ceiling:** Limit maximum priority of resource holders

3.4.6 Optimized Implementations

The module picks its implementation based on client count:

Power-of-2 Counts (4, 8, 16, 32):

```
// Fully unrolled casez logic for maximum performance
always_comb begin
    casez (w_priority_requests)
        4'b???1: begin winner = 2'd0; winner_valid = 1'b1; end //
Client 0
        4'b??10: begin winner = 2'd1; winner_valid = 1'b1; end //
Client 1
        4'b?100: begin winner = 2'd2; winner_valid = 1'b1; end //
Client 2
        4'b1000: begin winner = 2'd3; winner_valid = 1'b1; end //
Client 3
        default: begin winner = 2'd0; winner_valid = 1'b0; end
```

```

    endcase
end

```

Other Client Counts:

```

// Generic loop-based implementation
always_comb begin
    winner = '0;
    winner_valid = 1'b0;

    // Find lowest-indexed requesting client
    for (int i = 0; i < CLIENTS; i++) begin
        if (w_priority_requests[i] && !winner_valid) begin
            winner = i[N-1:0];
            winner_valid = 1'b1;
        end
    end
end

```

3.4.7 Priority Selection Logic

```

// Internal priority request selection
logic [CLIENTS-1:0] w_priority_requests;
assign w_priority_requests = any_masked_requests ?
    requests_masked :
    requests_unmasked;

```

3.5 Timing Characteristics

3.5.1 Latency and Propagation Delay

Metric	Optimized (4,8,16,32)	Generic (<64)	Generic (>64)
Latency	0 cycles (combinational)	0 cycles	0 cycles
Propagation Delay	1-2 LUT levels	2-3 LUT levels	3-4 LUT levels
Max Frequency	>500 MHz (FPGA)	>400 MHz	>300 MHz

3.5.2 Resource Utilization

Client Count	Implementation	LUTs (Est.)	Area Complexity
4	Optimized	~8	O(CLIENTS)
8	Optimized	~16	O(CLIENTS)

Client Count	Implementation	LUTs (Est.)	Area Complexity
16	Optimized	~32	O(CLIENTS)
32	Optimized	~64	O(CLIENTS)
Other	Generic	~CLIENTS×log(CLIENTS)	O(CLIENTS×log(CLIENTS))

3.6 Usage Examples

3.6.1 Standalone Priority Encoder

```
// Simple fixed-priority arbiter (single request input)
arbiter_priority_encoder #(
    .CLIENTS(8)
) u_pri_enc (
    .requests_masked    (8'b0),           // Unused
    .requests_unmasked (client_reqs),     // Primary request vector
    .any_masked_requests(1'b0),          // Always use unmasked
    .winner             (winner_id),
    .winner_valid       (grant_valid)
);

// Convert winner ID to one-hot grant signal
logic [7:0] grant_one_hot;
always_comb begin
    grant_one_hot = '0;
    if (grant_valid) begin
        grant_one_hot[winner_id] = 1'b1;
    end
end
```

3.6.2 Dual-Input Priority Encoder

```
// Masked + unmasked request inputs
arbiter_priority_encoder #(
    .CLIENTS(16)
) u_pri_enc_dual (
    .requests_masked    (masked_reqs),    // Higher priority requests
    .requests_unmasked (all_reqs),        // All requests
    .any_masked_requests(|masked_reqs),   // Select masked if any
    .winner             (winner_id),
    .winner_valid       (grant_valid)
);
```

3.6.3 Integration in Round-Robin Arbiter

```
// Used within arbiter_round_robin.sv
// Masked requests = clients above last winner
// Unmasked requests = all clients
arbiter_priority_encoder #(
    .CLIENTS(CLIENTS)
) u_priority_encoder (
    .requests_masked      (w_requests_masked),
    .requests_unmasked    (w_requests_unmasked),
    .any_masked_requests  (w_any_masked_requests),
    .winner               (w_winner),
    .winner_valid         (w_winner_valid)
);
```

3.6.4 Multi-Level Priority Groups

```
// Implement priority groups: High (C0-C3), Med (C4-C7), Low (C8-C11)
logic [3:0] high_reqs, med_reqs, low_reqs;
logic [1:0] high_winner, med_winner, low_winner;
logic high_valid, med_valid, low_valid;

// Separate encoders for each priority group
arbiter_priority_encoder #(.CLIENTS(4)) u_high_pri (
    .requests_masked(4'b0),
    .requests_unmasked(high_reqs),
    .any_masked_requests(1'b0),
    .winner(high_winner),
    .winner_valid(high_valid)
);

arbiter_priority_encoder #(.CLIENTS(4)) u_med_pri (
    .requests_masked(4'b0),
    .requests_unmasked(med_reqs),
    .any_masked_requests(1'b0),
    .winner(med_winner),
    .winner_valid(med_valid)
);

arbiter_priority_encoder #(.CLIENTS(4)) u_low_pri (
    .requests_masked(4'b0),
    .requests_unmasked(low_reqs),
    .any_masked_requests(1'b0),
    .winner(low_winner),
    .winner_valid(low_valid)
);

// Select winner from highest priority group with requests
```



```

logic [3:0] final_winner;
always_comb begin
    if (high_valid)      final_winner = {2'b00, high_winner};
    else if (med_valid)  final_winner = {2'b01, med_winner};
    else if (low_valid)  final_winner = {2'b10, low_winner};
    else                final_winner = 4'b0;
end

```

3.6.5 Example Priority Scenarios

Four cases worth tracing by hand before you trust the thing in a bigger design.

3.6.5.1 Scenario 1: Single Requester (Client 2)

```

requests_unmasked = 8'b0000_0100
any_masked_requests = 0

```

```

→ winner = 3'd2
→ winner_valid = 1

```

3.6.5.2 Scenario 2: Multiple Requesters (Clients 0, 2, 5)

```

requests_unmasked = 8'b0010_0101
any_masked_requests = 0

```

```

→ winner = 3'd0 (lowest index wins)
→ winner_valid = 1

```

3.6.5.3 Scenario 3: Masked Priority (Clients 3, 6 masked)

```

requests_masked    = 8'b0100_1000
requests_unmasked  = 8'b0110_1101
any_masked_requests = 1

```

```

→ winner = 3'd3 (from masked, index 3 < 6)
→ winner_valid = 1

```

3.6.5.4 Scenario 4: No Requests

```

requests_unmasked = 8'b0000_0000
any_masked_requests = 0

```

```

→ winner = 3'd0 (don't care)
→ winner_valid = 0

```

3.7 Design Notes

3.7.1 When to Use

Appropriate Use Cases: - Fixed-priority arbitration systems - Interrupt controllers - Bus arbitration with priority levels - Resource allocation with service levels - Component within round-robin or weighted arbiters

3.7.2 Performance Optimization

For High-Speed Designs: - Use power-of-2 client counts (4, 8, 16, 32) for optimized implementation - Pipeline outputs if critical path is too long - Consider hierarchical encoding for >32 clients

For Area Optimization: - Non-power-of-2 counts use smaller generic implementation - Share encoder between multiple arbitration paths - Use smallest practical CLIENTS parameter

3.7.3 Synthesis Considerations

Timing Optimization:

```
// Pipeline option for high-speed designs
logic [N-1:0] r_winner;
logic r_winner_valid;
```

```
always_ff @(posedge clk) begin
    r_winner <= winner;
    r_winner_valid <= winner_valid;
end
```

Resource Optimization: - Optimized implementations use casez (efficient mux tree) - Generic implementation uses priority loop (more logic levels) - For CLIENTS > 32, consider hierarchical structure

3.7.4 Common Pitfalls

Here's the part that bites people:

Anti-Pattern 1: Assuming fairness

```
// WRONG: Fixed priority is NOT fair
// Client 0 will always win if requesting
```

Anti-Pattern 2: Ignoring winner_valid

```
// WRONG: Using winner when no requests
if (winner == 2'd0) ... // Ambiguous: valid client 0 or no requests?

// RIGHT: Check winner_valid first
if (winner_valid && winner == 2'd0) ...
```

Anti-Pattern 3: Mixing priority schemes

```
// WRONG: Trying to implement round-robin with priority encoder alone
// Use arbiter_round_robin.sv instead
```

3.8 Related Modules

- **arbiter_round_robin.sv** - Uses this module internally for winner selection
- **arbiter_round_robin_simple.sv** - Alternative simplified arbiter
- **arbiter_round_robin_weighted.sv** - Weighted arbitration with QoS
- **encoder.sv** - General-purpose binary encoder
- **priority_encoder.sv** - Alternative naming (may be same module)

3.9 Testing

From the test suite (val/common/test_arbiter_priority_encoder.py):

- **Coverage:** 95%
- **Key Test Scenarios:**
 - Priority order verification (all bit patterns)
 - Masked vs unmasked selection
 - No requests (winner_valid = 0)
 - All clients requesting simultaneously
 - Single client at a time
 - Power-of-2 sizes (4, 8, 16, 32)
 - Non-power-of-2 sizes (5, 12)

Test Command: `pytest val/common/test_arbiter_priority_encoder.py -v`

3.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

4 Round Robin Arbiter

Fair round-robin arbitration for multiple clients — every requesting client gets equal access to the shared resource, and no client starves.

4.1 Overview

The `arbiter_round_robin` module cycles through requesting clients in order, so access to the shared resource stays fair over time. It is built around a pre-computed mask lookup table (no logic cost at runtime) and delegates winner selection to the `arbiter_priority_encoder` submodule.

4.2 Parameters

Parameter	Type	Default	Description
CLIENTS	int	4	Number of clients
WAIT_GNT_ACK	int	0	When set to 1, waits for grant acknowledgment before moving to next client
N	int	$\$clog2(CLIENTS)$	Internal parameter for address width calculation

4.3 Ports

4.3.1 Inputs

Port	Width	Description
clk	1	System clock
rst_n	1	Active-low asynchronous reset
block_arb	1	When asserted, blocks all arbitration (forces no grants)
request	CLIENTS	Request signals from each client
grant_ack	CLIENTS	Grant acknowledgment signals (used when <code>WAIT_GNT_ACK = 1</code>)

4.3.2 Outputs

Port	Width	Description
grant_valid	1	Indicates when a valid grant is being

Port	Width	Description
		issued
grant	CLIENTS	One-hot encoded grant signals
grant_id	N	Binary encoded ID of the granted client
last_grant	CLIENTS	Previous cycle's grant (for debugging/history)

4.4 Functional Description

4.4.1 Fair Round-Robin Operation

The rotation order is $0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow (\text{CLIENTS}-1) \rightarrow 0$ (see the RTL header). The algorithm ensures fairness by: 1. After serving client i , giving priority to the **higher-indexed** clients ($i+1 \dots \text{CLIENTS}-1$) via `w_win_mask_decode[i]` 2. Checking those masked (higher-index) requests first 3. When no higher-indexed clients are requesting, wrapping around to the **lowest** indices (client 0 upward)

4.4.2 Block Arbitration

Assert `block_arb` and all requests are masked to zero — arbitration is effectively disabled.

A **blocked interval RESETS the rotation**. This is the part that surprises people. `r_last_valid` follows `grant_valid` every cycle, and a block produces no grants, so `r_last_valid` falls to 0 and the mask select drops to its third branch:

```
assign w_curr_mask_decode = grant_valid ? w_win_mask_decode[grant_id]
:
                                r_last_valid ?
w_win_mask_decode[r_last_grant_id] :
                                CLIENTS'(1);
```

`CLIENTS'(1)` masks off everything except client 0, so **the first grant after `block_arb` releases goes to the lowest-numbered requester, not to the client that was next in line**. Traced on a 32-client instance: block released at 25060 ns, and the next two grants went to clients 0 and 3 while the pre-block rotation had reached the high teens.

If you need fairness to survive a blocked interval, `block_arb` is the wrong tool — gate the requests upstream instead, so the arbiter keeps seeing a valid grant history.

4.4.3 Grant Acknowledgment Support

When `WAIT_GNT_ACK = 1`, the arbiter waits for the granted client to acknowledge receipt before updating internal state and moving to the next client.

4.4.4 Key Internal Signals

- `r_last_grant_id`: Tracks the last winner's client ID (smaller than full mask)
- `r_last_valid`: Indicates if last winner should be used for mask generation
- `r_pending_ack`: ACK mode state (indicates ACK pending)
- `r_pending_client`: Which client has pending ACK (only in ACK mode)
- `w_requests_masked`: Requests with priority mask applied
- `w_requests_unmasked`: Raw gated requests without masking

4.4.5 Priority Mechanism

The arbiter is built around a pre-computed mask lookup table:

1. **Mask Lookup Tables**: Pre-computed at elaboration time (no logic cost)
 - `w_win_mask_decode[i] = ~((1 << (i+1)) - 1)`: selects clients **i+1 and above** (all higher indices). After client *i* wins, this is the mask that gives priority to the next-higher clients. This is the only LUT the arbiter actually reads (via `w_curr_mask_decode`).
 - `w_mask_decode[i] = (1 << i) - 1`: covers clients **0 through i-1**. Note: this table is generated but **never read** in the current RTL — it is dead logic left from an earlier scheme, not part of the active datapath.
2. **Fast Request Preprocessing**: Single LUT level for request gating
 - Block arbitration immediately gates all requests
 - Masked and unmasked request vectors computed in parallel
3. **Last Winner Tracking**: Uses client ID instead of full one-hot mask
 - `r_last_grant_id`: More efficient than storing full grant vector
 - `r_last_valid`: Indicates if mask should be applied

4.4.6 Winner Selection

Winner selection is delegated to the `arbiter_priority_encoder` submodule: - Takes both masked and unmasked request vectors - Returns binary-encoded winner ID - Outputs validity signal

```
// Priority encoder selects highest priority requester
arbiter_priority_encoder #(.CLIENTS(CLIENTS), .N(N))
u_priority_encoder (
    .requests_masked      (w_requests_masked),
    .requests_unmasked    (w_requests_unmasked),
    .any_masked_requests  (w_any_masked_requests),
    .winner                (w_winner),
    .winner_valid          (w_winner_valid)
);

// Grant decision with ACK permission check
assign w_should_grant = w_winner_valid && w_any_requests &&
w_can_grant;
```

4.4.7 Mask Update Logic

- **No-ACK Mode:** Mask updates immediately when grant issued (1-cycle round-robin)
- **ACK Mode:** Mask updates only when ACK received (prevents premature rotation)

4.5 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

4.6 Usage Examples

Every parameter and port below is taken from the module declaration.


```

arbiter_round_robin #(
    .CLIENTS          (4),
    .WAIT_GNT_ACK     (0),
    .N                 ($clog2(CLIENTS))
) u_arbiter_round_robin (
    .clk               (clk),
    .rst_n             (rst_n),
    .block_arb         (block_arb),
    .request            (request),
    .grant_ack         (grant_ack),
    .grant_valid       (grant_valid),
    .grant             (grant),
    .grant_id          (grant_id),
    .last_grant        (last_grant)
);

```

4.7 Design Notes

4.7.1 Usage Notes

- On the first grant after reset (no last winner), the mask defaults to client 0, so the lowest-indexed requester wins ties initially; thereafter priority rotates upward from the last winner
- The round-robin nature ensures long-term fairness across all clients
- Grant acknowledgment feature is useful in systems where the granted client needs time to process the grant

4.8 Related Modules

Read out of the RTL, not curated: these are the modules this one instantiates and the modules that instantiate it.

Instantiates: - arbiter_priority_encoder

Instantiated by: - arbiter_deficit_round_robin - arbiter_round_robin_weighted - arbiter_rr_pwm_monbus - axi_read_engine - axi_read_engine_beats - axi_write_engine

4.9 Testing

val/common/test_arbiter_round_robin.py exercises this module. It collects 6 parameter cases at the default REG_LEVEL.

```
source env_python
pytest val/common/test_arbiter_round_robin.py -v
```

4.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

[← Back to Main Index](#) | [rtl-common Index](#)

5 Simple Round-Robin Arbiter

A generic rotating-priority arbiter that gives you fair arbitration among multiple requesting agents using masking/rotation — no if/case ladders in the priority path.

5.1 Overview

The `arbiter_round_robin_simple` module implements round-robin arbitration with fair access to a shared resource among N requesting agents. Fairness comes from remembering the last granted agent and starting the next arbitration from the next agent in sequence.

```
module arbiter_round_robin_simple #(
    parameter int unsigned N = 4,
    parameter int unsigned W = $clog2(N)
) (
    input logic clk,
    input logic rst_n,           // active-low reset
    input logic [N-1:0] request, // request bits [N-1:0]
    output logic grant_valid,    // any grant
    output logic [N-1:0] grant,  // one-hot grant
    output logic [W-1:0] grant_id // encoded grant (undef if
grant_valid==0)
);
```

5.2 Parameters

Parameter	Type	Default	Description
N	int unsigned	4	Number of requesting agents (N >= 2; see note)
W	int unsigned	\$clog2(N)	Width of grant_id. Derived from N – leave it at the default; it is a parameter rather than a localparam only so it can be referenced in the port list.

5.3 Ports

5.3.1 Inputs

Port	Width	Description
clk	1	Clock signal

Port	Width	Description
rst_n	1	Active-low asynchronous reset
request	N	Request bits from N agents [N-1:0]

5.3.2 Outputs

Port	Width	Description
grant_valid	1	Indicates if any grant is active
grant	N	One-hot grant output [N-1:0]
grant_id	$\$clog2(N)$	Binary encoded grant ID (undefined if grant_valid==0)

5.4 Functional Description

5.4.1 Round-Robin Algorithm

The arbiter implements a rotating priority scheme using the following approach:

1. **State Memory:** Maintains r_last_grant register storing the last granted agent ID
2. **Rotation:** Rotates the request vector to prioritize agents after the last granted one
3. **Priority Selection:** Uses lowest-set-bit isolation ($x \& (\sim x + 1)$) for efficient priority selection
4. **Restoration:** Rotates the result back to original bit positions

5.4.2 Key Features

- **Fair Arbitration:** Ensures all requesting agents get equal opportunity over time
- **Efficient Logic:** Uses bit manipulation instead of complex if/case structures
- **Single-Cycle Response:** Combinational grant generation with registered state update
- **Scalable:** Parameterizable for any number of agents ($N \geq 2$). At $N=1$, $\$clog2(1) = 0$ collapses the shift/ID arithmetic into zero-size casts and

zero replications that strict front ends reject (the same degeneracy the [arbiter_single_client](#) page documents for the base arbiter) — use `arbiter_single_client` for the one-agent case

5.4.3 Reset Behavior

- **Power-on Reset:** `r_last_grant` initialized to `N-1`, causing first arbitration to start from agent 0
- **Active Reset:** All state registers cleared, grant outputs become invalid
- **Reset Release:** Next arbitration cycle begins with agent 0 having highest priority

5.4.4 Rotation Logic

```
// Shift amount = last_grant + 1 (mod N)
logic [$clog2(N)-1:0] w_shift_amount;
assign w_shift_amount = (r_last_grant == ($clog2(N))'(N-1)) ? '0 :
(r_last_grant + 1);

// Rotate the request vector RIGHT by w_shift_amount, so that agent
// (last_grant + 1) lands at bit 0 and the lowest-set-bit scan starts
// there.
if (w_shift_amount == '0) begin
    w_rot_req = request;
end else begin
    w_rot_req = (request >> w_shift_amount) | (request << ((W)'(N) -
w_shift_amount));
end
```

The direction is not a free choice. Rotating LEFT by `s` maps rotated bit `j` back to agent $(j - s) \bmod N$, so the scan starts at agent $(N - \text{last} - 1)$ instead of $(\text{last} + 1)$ — a reflection of the priority pointer rather than a rotation. A reflection composed with itself is the identity, so the pointer oscillates between two positions forever: with `N=4` and all four agents requesting, the arbiter granted 0, 3, 0, 3, ... and agents 1 and 2 were never served. Rotating RIGHT maps rotated bit `j` to agent $(j + s) \bmod N$, so the scan starts at $(\text{last} + 1)$ and advances, which is what round-robin means.

5.4.5 Priority Selection

The module uses lowest-set-bit isolation for efficient one-hot selection:

```
// Isolate lowest set bit (one-hot)
w_rot_sel = w_rot_req & ((~w_rot_req) + {{(N-1){1'b0}}, 1'b1});
```

5.4.6 Grant Encoding

A compact one-hot to binary encoder generates the grant ID:

```

always_comb begin
    w_grant_id = r_last_grant; // default to last grant
    for (int i = 0; i < N; i++) begin
        if (w_nxt_grant[i]) w_grant_id = i[$clog2(N)-1:0];
    end
end
end

```

5.5 Timing Characteristics

Characteristic	Value
Setup Time	1 clock cycle for state update
Response Time	Combinational (0 cycles)
Reset Recovery	1 clock cycle

5.6 Usage Examples

5.6.1 Basic 4-Agent Arbiter

```

arbiter_round_robin_simple #(
    .N(4)
) u_arbiter (
    .clk          (clk),
    .rst_n        (rst_n),
    .request       (req_bits),           // [3:0]
    .grant_valid   (grant_valid),
    .grant         (grant_oh),           // [3:0] one-hot
    .grant_id      (grant_bin)           // [1:0] binary
);

```

5.6.2 8-Agent System

```

arbiter_round_robin_simple #(
    .N(8)
) u_8way_arbiter (
    .clk          (clk),
    .rst_n        (rst_n),
    .request       (agent_requests), // [7:0]
    .grant_valid   (resource_granted),
    .grant         (agent_grants),   // [7:0]
    .grant_id      (winning_agent)   // [2:0]
);

```

5.6.3 Arbitration Sequence Example

For N=4 with requests [3:0]:

Cycle	Last Grant	Requests	Rotated	Selected	Grant	Grant ID
0	3 (init)	1111	1111	0001	0001	0
1	0	1110	0111	0001	0010	1
2	1	1101	0111	0001	0100	2
3	2	1011	0111	0001	1000	3
4	3	1111	1111	0001	0001	0

Rotated is the request vector rotated RIGHT by `last_grant + 1`, which places the next agent in priority order at bit 0. Selected isolates that vector's lowest set bit, and Grant rotates it back to the original bit positions.

5.7 Design Notes

5.7.1 Advantages

- **Simple Logic:** Uses efficient bit manipulation instead of complex priority encoders
- **Scalable:** Works for any number of agents with minimal logic overhead
- **Fair:** Guarantees round-robin fairness over time
- **Fast:** Single combinational path from request to grant

5.7.2 Limitations

- **Grant Undefined:** `grant_id` is undefined when `grant_valid` is low
- **Request Requirement:** At least one request must be active for meaningful arbitration
- **Power Consumption:** Rotational logic may consume more power than simpler priority schemes

5.7.3 Synthesis Considerations

- **Area:** Scales approximately as $O(N \log N)$ due to rotation logic
- **Timing:** Critical path through rotation and bit isolation logic
- **Power:** Barrel shifter logic may have higher switching activity
- **Optimization:** Consider using dedicated arbiter primitives for large N

5.8 Related Modules

- `arbiter_round_robin`: Enhanced round-robin arbiter with additional features
- `arbiter_priority_encoder`: Fixed priority arbiter implementation
- `encoder_priority_enable`: Priority encoding utility module

For most round-robin arbitration needs in digital systems, `arbiter_round_robin_simple` is the efficient, fair answer.

5.9 Testing

- **Reset Testing**: Verify proper initialization and first-grant behavior
- **Fairness Testing**: Confirm round-robin sequence over multiple cycles
- **Edge Cases**: Test behavior with single request, all requests, and no requests
- **Timing**: Verify setup/hold requirements for state register updates

5.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

[← Back to Main Index](#) | [rtl-common Index](#)

6 Weighted Round Robin Arbiter

A credit-based weighted round-robin arbiter with Quality of Service (QoS) support. Clients get bandwidth proportional to their weights, and no client can starve.

6.1 Overview

The `arbiter_round_robin_weighted` module layers configurable per-client weights on top of fair round-robin arbitration, giving you proportional bandwidth allocation. Each client receives grants proportional to its assigned weight while starvation is prevented. The module supports an optional acknowledgment protocol and dynamic weight changes with atomic update semantics.

```
module arbiter_round_robin_weighted #(
    parameter int MAX_LEVELS = 16,          // Maximum weight value per
client
    parameter int CLIENTS = 4,              // Number of requesting
clients
    parameter int WAIT_GNT_ACK = 0          // Enable ACK protocol (0=No-
ACK, 1=ACK)
) (
    // Derived parameter - in the list for port-range use; do not
override:
    // localparam int MAX_LEVELS_WIDTH = $clog2(MAX_LEVELS); //
Credit counter width
    // localparam int N = $clog2(CLIENTS); //
Client ID width
    // localparam int C = CLIENTS; //
Convenience alias
    // localparam int MTW = MAX_LEVELS_WIDTH; //
Weight width alias
    // localparam int CXMTW = CLIENTS * MAX_LEVELS_WIDTH; //
Packed weight array width

    input logic clk, // Clock input
    input logic rst_n, // Active-low
asynchronous reset
    input logic block_arb, // Block all
arbitration
    input logic [CXMTW-1:0] max_thresh, // Packed weight
values for all clients
    input logic [C-1:0] request, // Request signals
from clients
    input logic [C-1:0] grant_ack, // Grant
acknowledgment (ACK mode only)
    output logic grant_valid, // Grant output valid
    output logic [C-1:0] grant, // Grant vector (one-
hot)
    output logic [N-1:0] grant_id // Grant client ID

```

(binary encoded)
);

6.2 Parameters

6.2.1 User-Settable Parameters

Parameter	Type	Default	Description
MAX_LEVELS	int	16	Sizes the weight/credit fields at $\$clog2(\text{MAX_LEVELS})$ bits. The largest usable weight is MAX_LEVELS-1 (15 for the default 16), <i>not</i> MAX_LEVELS: a weight of MAX_LEVELS truncates to 0 in the field and silently disables that client ($w_valid_clients[j] = client_weight[j] > 0$). Usable weight range per client: 1 .. MAX_LEVELS-1 (0 = disabled).
CLIENTS	int	4	Number of requesting clients (range: 2-32)
WAIT_GNT_ACK	int	0	Enable ACK protocol (0=No-ACK, 1=ACK required)

6.2.2 Derived Parameters

Parameter	Default	Description
MAX_LEVELS_WIDTH	$\$clog2(\text{MAX_LEVELS})$	Credit counter width in bits
N	$\$clog2(\text{CLIENTS})$	Client ID width in bits
C	CLIENTS	Convenience alias for port widths

Parameter	Default	Description
MTW	MAX_LEVELS_WIDT H	Convenience alias for weight width
CXMTW	CLIENTS * MAX_LEVELS_WIDT H	Total packed weight array width

Note: MAX_LEVELS_WIDTH, N, C, MTW, and CXMTW are derived parameters — declared in the parameter list so the port declarations can use them (strict front ends reject body localparams in port ranges). They *can* be overridden at instantiation, and must not be; leave them to their derivations. (The [arbiter_priority_encoder](#) page describes the same pattern.)

6.3 Ports

6.3.1 Inputs

Port	Width	Description
clk	1	System clock
rst_n	1	Active-low asynchronous reset
block_arb	1	Block all arbitration (external gate)
max_thresh	CXMTW	Packed weight values: {weight[N-1], ..., weight[1], weight[0]}
request	C	Request signals from clients
grant_ack	C	Grant acknowledgment signals (ACK mode only)

6.3.2 Outputs

Port	Width	Description
grant_valid	1	Valid grant indicator
grant	C	Grant vector (one-hot encoded)
grant_id	N	Grant client ID (binary encoded)

6.4 Functional Description

6.4.1 Credit-Based Weighting

Weighted bandwidth allocation here runs on a credit system:

1. **Credit Initialization:** reset loads **1 credit** per client (`r_credit_counter[i] <= MTW'(1)`) — but the shadow weight register resets to all-ones, so for any normal configuration the weight-change FSM runs immediately after reset (with the sub-arbiter blocked) and loads the CONFIGURED weights into the credits at WEIGHT_STABILIZE, before the first grant can issue. The first post-reset round IS weighted; only a config equal to the all-ones reset value skips the init pass
2. **Grant Completion:** When a grant completes, the client's credit counter decrements
3. **Credit Eligibility:** Only clients with non-zero credits are eligible for round-robin arbitration
4. **Global Replenishment:** When no requesting clients have credits, all credits are reloaded

6.4.2 Arbitration Stages

1. **Credit Eligibility:** Filter requests to only clients with credits > 0
2. **Round-Robin Selection:** Base arbiter (`arbiter_round_robin`) selects among eligible clients
3. **Grant Output:** Winning client receives one-hot grant signal
4. **Credit Update:** Winner's credit decrements on completion (immediate or ACK-based)

6.4.3 Weight Ratio Example

For weights [4, 2, 1, 1] with all clients requesting continuously: - **Client 0:** 4 credits per round - **Client 1:** 2 credits per round - **Client 2:** 1 credit per round - **Client 3:** 1 credit per round - **Pattern:** grants are **interleaved, not bursty**. The masking logic excludes the just-granted client whenever more than one client is still eligible (`w_mask_multi_req[j] = w_requesting_eligible[j] && !grant[j]`), so client 0 does **not** receive 4 back-to-back grants while others are eligible. A representative sequence is C0, C1, C2, C3, C0, C1, C0, (bubble), C0, [replenish], repeat — client 0 still gets 4 grants per round, but spread out. - **Bandwidth** (the quantity that actually holds): C0=50%, C1=25%, C2=12.5%, C3=12.5% per round.

Consecutive per-client grants only occur once a client is the **sole** eligible requester (e.g., others have exhausted their credits), at which point the `w_mask_last_client` term lets it keep the resource.

6.4.4 Zero-Weight Client Handling

Clients with `max_thresh = 0` are effectively disabled: - Never accumulate credits - Never participate in arbitration - Credit counters remain at zero

6.4.5 ACK Protocol Support

When `WAIT_GNT_ACK = 1`: - Grant remains asserted until `grant_ack` received - Credit decrement waits for ACK - Weight change FSM waits for pending ACKs - Enables pipelined/posted transaction masters

When `WAIT_GNT_ACK = 0`: - Grant completes immediately (1 cycle) - Credit decrements same cycle as grant - Optimal for simple masters (no pipeline)

6.4.6 Starvation Prevention

The arbiter guarantees starvation-free operation: - All non-zero weight clients eventually served - Maximum starvation time: Sum of all other clients' weights - Global replenishment ensures fairness

6.4.7 Dynamic Weight Changes

6.4.7.1 Weight Management FSM

A finite state machine handles atomic weight updates:

State	Description
WEIGHT_IDLE	Normal operation
WEIGHT_BLOCK	Detect weight change, block new grants
WEIGHT_DRAIN	Wait for pending ACK completion (timer 2 → 3 cycles in state)
WEIGHT_UPDATE	Atomic weight update, reset credits
WEIGHT_STABILIZE	System stabilization (timer 3 → 4 cycles in state)

6.4.7.2 Weight Update Sequence

1. **Detection:** FSM detects `max_thresh` change from shadow register
2. **Block:** New grants blocked, pending grants drain
3. **Drain:** Wait for ACK completion (ACK mode) or fixed delay
4. **Update:** Atomic update of weights, credit counters reset
5. **Stabilize:** System stabilization period
6. **Resume:** Return to normal operation

Total Latency: 10 cycles minimum (DRAIN occupies 3 cycles, STABILIZE 4 — each timed state runs timer+1 cycles); up to ~25 with the 15-cycle BLOCK timeout on a pending ACK

6.4.7.3 Safety Features

- **Shadow Register:** Active weights stored in r_safe_max_thresh for race-free updates
- **Timeout Protection:** 15-cycle timeout prevents FSM lockup
- **Credit Reset:** Credits reset to new weights during WEIGHT_STABILIZE state
- **ACK Awareness:** FSM waits for pending grant_ack before updating weights

6.4.8 Credit Management

```
// Credit decrement on grant completion
if (w_grant_completed[i] && r_credit_counter[i] > 0) begin
    w_credit_counter[i] = r_credit_counter[i] - 1;
end
```

Grant Completion Modes: - **No-ACK Mode:** Completion when grant[i] && grant_valid (same cycle) - **ACK Mode:** Completion when grant[i] && grant_valid && grant_ack[i] (delayed)

6.4.9 Global Replenishment Logic

```
// Replenish when no requesting clients have credits.
// Note w_req_post, not request: w_req_post = block_arb ? '0 :
// request, so while
// block_arb is high there are no requests to replenish for.
assign w_global_replenish = (w_normal_operation && !w_pending_grants
&&
                                (|w_req_post) && !
w_requesting_clients_with_credits);

// On replenish: reload all credit counters to their weights
if (w_global_replenish) begin
    w_credit_counter[i] = client_weight[i];
end
```

6.4.10 Request Masking for Fairness

```
// Only pass eligible clients to round-robin arbiter. Again
w_req_post, so a
// blocked arbiter makes every client ineligible rather than merely
ungranted.
```

```

assign w_requesting_eligible[i] = w_req_post[i] &&
                                   ((w_has_crd[i]) ||
                                   (w_global_replenish &&
w_valid_clients[i]));

// Apply masking for fairness among multiple eligible clients.
// The two terms are OR'd, NOT selected by a ternary: the "eligible &&
!grant"
// term is ALWAYS active, so a lone eligible client can never be
masked to 0.
assign w_mask_multi_req[i]    = w_requesting_eligible[i] && !grant[i];
assign w_mask_last_client[i] = !multiple_eligible &&
                                   w_requesting_eligible[i] &&
                                   (r_credit_counter[i] > 1);
assign w_mask_req[i] = w_mask_multi_req[i] || w_mask_last_client[i];

```

Do not re-implement this as a ternary (`multiple_eligible ? ... : ...`). Dropping the unconditional `eligible && !grant` term would leave a lone eligible client on its last credit (`r_credit_counter == 1`) with `w_mask_req = 0` forever: no grant, so no decrement, so no replenish — a deadlock. The OR form grants it every other cycle.

6.4.11 Sub-Module: `arbiter_round_robin`

The base `arbiter_round_robin` module handles fair selection among eligible clients:

```

arbiter_round_robin #(
    .CLIENTS      (CLIENTS),
    .WAIT_GNT_ACK (WAIT_GNT_ACK)
) u_base_arbiter (
    .clk          (clk),
    .rst_n        (rst_n),
    .block_arb    (w_sub_block_arb),
    .request       (w_mask_req),
    .grant_ack     (grant_ack),
    .grant_valid   (grant_valid),
    .grant         (grant),
    .grant_id      (grant_id),
    .last_grant    (r_last_grant)
);

```

6.5 Timing Characteristics

6.5.1 Latency and Throughput

- **Latency:** 1 cycle in steady state. Credit calculation, eligibility masking and the round-robin decision are all combinational (`request -> w_req_post ->`

w_requesting_eligible -> w_mask_req); the only register stage is the base arbiter's grant <= w_next_grant. Measured on this RTL: a request sampled at edge N produces grant_valid at edge N+1. Two exceptions add cycles – the first grant after reset waits for the weight-init FSM, 10 cycles to return to IDLE traced from the FSM constants (detect 1 + BLOCK 1 + DRAIN 3 + UPDATE 1 + STABILIZE 4) plus one for the grant register, ~11 cycles total – and a global replenish inserts a one-cycle bubble.

- **Throughput:** 1 grant per cycle (maximum)
- **Grant Hold:** No-ACK=1 cycle, ACK=Until grant_ack asserted
- **Weight Update:** 10-25 cycles (FSM: BLOCK→DRAIN→UPDATE→STABILIZE)
- **Reset:** Asynchronous (all credits→1, weights→default)

6.5.2 Critical Paths

1. **Credit Comparison:** Credit counter → eligibility logic
2. **Request Filtering:** Eligible clients → masked requests
3. **Round-Robin:** Base arbiter selection logic

6.5.3 Resource Utilization

CLIENTS	MAX_LEVELS	FFs (Est.)	LUTs (Est.)
4	16	~40	~50
8	16	~70	~90
4	32	~45	~55
8	32	~80	~110

6.5.4 Maximum Frequency

- **Typical:** 300-500 MHz (modern FPGAs)
- **Limiting Factor:** Credit comparison and request filtering logic

6.6 Usage Examples

6.6.1 Basic QoS Arbiter (No-ACK Mode)

```
// 4-client arbiter with QoS weights: 8:4:2:1
localparam int NUM_CLIENTS = 4;
localparam int MAX_WEIGHT = 16;
```



```

localparam int WEIGHT_W = $clog2(MAX_WEIGHT);

logic [NUM_CLIENTS-1:0] client_req, client_grant;
logic grant_vld;
logic [$clog2(NUM_CLIENTS)-1:0] grant_idx;
logic [NUM_CLIENTS*WEIGHT_W-1:0] qos_weights;

// Assign QoS weights (high priority = higher weight)
assign qos_weights = {4'd1, 4'd2, 4'd4, 4'd8}; // {C3, C2, C1, C0}

arbiter_round_robin_weighted #(
    .CLIENTS(NUM_CLIENTS),
    .MAX_LEVELS(MAX_WEIGHT),
    .WAIT_GNT_ACK(0) // No-ACK mode
) u_qos_arbiter (
    .clk          (clk),
    .rst_n        (rst_n),
    .block_arb    (1'b0),
    .max_thresh   (qos_weights),
    .request       (client_req),
    .grant_ack     ('0), // Unused in No-ACK mode
    .grant_valid   (grant_vld),
    .grant         (client_grant),
    .grant_id      (grant_idx)
);

```

6.6.2 Dynamic Weight Adjustment

Slice exactly one client's field. Client j owns bits $[(j+1)*MAX_LEVELS_WIDTH-1 : j*MAX_LEVELS_WIDTH]$, so with $MAX_LEVELS=16$ (4-bit weights) client 0 is $[3:0]$ and client 1 is $[7:4]$. Writing a 4-bit value into a wider slice zero-extends across the neighbour: $r_qos_weights[7:0] <= 4'd15$ sets client 1's weight to 0, and $w_valid_clients[j] = (client_weight[j] > 0)$ means a zero-weight client is never granted again.

```

// Adjust weights based on priority events
logic [NUM_CLIENTS*WEIGHT_W-1:0] r_qos_weights;

always_ff @(posedge clk) begin
    if (high_priority_event) begin
        r_qos_weights[3:0] <= 4'd15; // Boost Client 0 weight
    end else if (normal_priority) begin
        r_qos_weights[3:0] <= 4'd4; // Reset Client 0 weight
    end
end

arbiter_round_robin_weighted #(
    .CLIENTS(NUM_CLIENTS),
    .MAX_LEVELS(MAX_WEIGHT),

```

```

        .WAIT_GNT_ACK(0)
    ) u_dynamic_arbiter (
        .clk          (clk),
        .rst_n        (rst_n),
        .block_arb     (1'b0),
        .max_thresh    (r_qos_weights), // Dynamic weights
        .request       (client_req),
        .grant_ack     ('0'),
        .grant_valid    (grant_vld),
        .grant         (client_grant),
        .grant_id      (grant_idx)
    );

```

6.6.3 ACK Mode for Pipelined Masters

```

// ACK mode arbiter for pipelined masters
arbiter_round_robin_weighted #(
    .CLIENTS(2),
    .MAX_LEVELS(8),
    .WAIT_GNT_ACK(1) // ACK required
) u_ack_arbiter (
    .clk          (clk),
    .rst_n        (rst_n),
    .block_arb    (bus_busy),
    .max_thresh    ({3'd3, 3'd5}), // client1=3, client0=5; fields are
    $clog2(8)=3 bits each
    .request       (m_req),
    .grant_ack     (m_done), // Master completion signal
    .grant_valid    (m_grant_vld),
    .grant         (m_grant),
    .grant_id      (m_id)
);

```

6.7 Design Notes

6.7.1 When to Use Each Mode

No-ACK Mode (WAIT_GNT_ACK=0): - Simple masters without pipelining - Combinational logic masters - Maximum throughput (1 grant/cycle) - Caveat: Grant completes same cycle (no hold)

ACK Mode (WAIT_GNT_ACK=1): - Pipelined masters (e.g., AXI) - Posted transaction buses - Multi-cycle operations - Caveat: Reduced throughput (grant held until ACK)

6.7.2 Weight Selection Guidelines

- **Finer QoS Control:** Increase MAX_LEVELS (but more credit counter bits)
- **Typical Range:** MAX_LEVELS=8 to 32 for most applications

- **Weight Ratio:** Should reflect desired bandwidth ratios
- **Zero Weights:** Effectively disable clients (never granted)

6.7.3 Dynamic Weight Change Best Practices

- **DO NOT** change weights every cycle (causes FSM thrashing)
- **Recommended:** Change weights on policy updates (millisecond timescale)
- **Safe:** FSM ensures atomic updates without race conditions
- **Timeout:** 15-cycle timeout prevents lockup

6.8 Related Modules

- **arbiter_round_robin.sv** - Base round-robin arbiter (used internally)
- **arbiter_round_robin_simple.sv** - Lightweight RR arbiter (no weights)
- **arbiter_priority_encoder.sv** - Fixed priority arbiter

6.9 Testing

From the test suite (`val/common/test_arbiter_round_robin_weighted.py`):

- **Coverage:** 94%
- **Key Test Scenarios:**
 - Weight ratio verification ([4,2,1,1] patterns)
 - Global credit replenishment
 - Dynamic weight changes (atomic updates)
 - ACK mode timing and weight updates
 - Zero weight handling (client disable)
 - Single client optimization
 - FSM timeout protection
 - Starvation prevention

Test Command: `pytest val/common/test_arbiter_round_robin_weighted.py -v`

6.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

[← Back to Main Index](#) | [rtl-common Index](#)

7 Single-Client Arbiter

Module: `arbiter_single_client.sv` **Location:** `rtl/common/` **Status:** Production Ready

7.1 Overview

`arbiter_single_client` is the degenerate, one-requester member of the round-robin arbiter family. It exists to give a single-channel datapath the **exact same registered, ack-held grant lifecycle** as `arbiter_round_robin` running in `WAIT_GNT_ACK` mode, without the parameter degeneracy that breaks the multi-client arbiter when `CLIENTS == 1`.

`arbiter_round_robin` underflows at `CLIENTS == 1` because `$clog2(1) = 0` produces a `grant_id[-1:0]` slice. Historically single-channel builds worked around this by substituting a purely *combinational* passthrough (`grant = request`). That passthrough does **not** reproduce the arbiter's registered grant/ack handshake, so it injects a bubble beat at every burst boundary in the AXI read/write engines that depend on the "request, grant, hold-for-ack" timing. This module is the faithful single-client reduction of those rules.

7.1.1 Key Features

- **Timing-faithful:** Mirrors `arbiter_round_robin` `WAIT_GNT_ACK` `always_ff` rules exactly for one client
- **Registered grant:** Grant is a flip-flop output, held until acknowledged (no combinational passthrough bubble)
- **Ack-held lifecycle:** Grant persists until `grant_ack`, then clears and can re-grant the next cycle
- **Bubble-free bursts:** Preserves the grant timing the AXI read/write engines rely on for back-to-back burst beats
- **Optional ack mode:** `WAIT_GNT_ACK = 0` falls back to a simple registered request-to-grant with no ack hold
- **Trivial decode:** `grant` is a one-hot of width 1 (equal to `grant_valid`); `grant_id` is always 0

7.1.2 Use Cases

The module provides a drop-in single-client arbiter so that a design parameterized down to one channel behaves identically to the same design with two or more channels. Rather than special-casing the `NUM_CHANNELS == 1` build with a passthrough, the datapath instantiates `arbiter_single_client` and gets the identical grant/ack sequencing the general arbiter would have produced for one requester.

- Single-channel builds of a datapath that is otherwise parameterized for N channels
- AXI read/write engines that require the registered, ack-held grant handshake for correct burst sequencing
- Any place a `arbiter_round_robin` instance would be used, but with `CLIENTS == 1`
- Replacing a combinational `grant = request` passthrough that was injecting per-burst bubble beats

Key Benefit: A one-channel build behaves bit-for-bit like the multi-channel arbiter path, eliminating the burst-boundary bubble that a combinational passthrough introduced.

7.2 Parameters

Parameter	Type	Default	Description
WAIT_GNT_ACK	int	1	1 = registered grant is held until <code>grant_ack</code> is received (round-robin ACK lifecycle). 0 = simple registered request-to-grant with no ack hold.

7.3 Ports

7.3.1 Inputs

Port	Width	Description
clk	1	Clock
rst_n	1	Active-low asynchronous reset
block_arb	1	Block arbitration; when high the request is masked and no grant is issued
request	1	The single client's request
grant_ack	1	The single client's grant acknowledgment (used only when <code>WAIT_GNT_ACK = 1</code>)

7.3.2 Outputs

Port	Width	Description
grant_valid	1	Grant valid (registered)
grant	1	One-hot grant of width 1; equal to grant_valid
grant_id	1	Granted client index; always 0 for a single client

7.4 Functional Description

7.4.1 Request Qualification

The incoming request is masked by block_arb first:

```
assign w_req = request && !block_arb;
```

A blocked arbiter behaves as if the client were not requesting at all.

7.4.2 Grant-Eligibility Logic (ACK Mode)

When WAIT_GNT_ACK = 1, three combinational signals gate whether a new grant may be issued:

```
assign w_ack_received = r_pending_ack && grant_ack;           // ack
// seen for the outstanding grant
assign w_can_grant    = !r_pending_ack || w_ack_received;     // free
// to grant, or ack just closed the last grant
assign w_should_grant = w_req && w_can_grant;
```

r_pending_ack records that a grant has been issued and is awaiting its acknowledgment. A new grant can only be launched when there is no pending ack, or when the ack for the current grant arrives this cycle.

7.4.3 Grant Lifecycle (ACK Mode)

The registered state machine is the single-client reduction of the arbiter_round_robin WAIT_GNT_ACK rules. Because there is only one client, w_other_requests is always 0, so the arbiter's Rule 4 (switch to another requester) never applies and Rule 3 simply clears:

State	Condition	Action	grant_valid next	r_pending_ack next
Rule 1	grant_val id == 0	Not granted;	w_should_grant	w_should_grant

State	Condition	Action	grant_valid next	r_pending_ack next
		grant if request ing		
Rule 2	granted, !w_ack_received	Hold the grant	1	1
Rule 3	granted, w_ack_received	Ack received; clear (re-grant next cycle)	0	0

The grant is thus **registered and held** across cycles until the client acknowledges, then it drops for one cycle before it can be re-issued.

7.4.4 No-ACK Fallback Mode

When WAIT_GNT_ACK = 0, the ack machinery is bypassed. grant_valid simply follows a registered version of the qualified request and r_pending_ack stays 0:

```
grant_valid    <= w_should_grant;    // w_can_grant is constant 1 in
this mode
r_pending_ack  <= 1'b0;
```

This gives a one-cycle registered request-to-grant with no hold.

7.4.5 Output Mapping

```
assign grant    = grant_valid;    // one-hot of width 1
assign grant_id = 1'b0;          // single client
```

Because there is exactly one client, grant and grant_valid are identical, and grant_id is a constant 0.

7.5 Timing Characteristics

This module is **sequential**: it contains 1 always_ff block(s), clocked on clk with active-low asynchronous reset rst_n. Outputs derived in those blocks are registered and therefore appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

7.6 Usage Examples

```
// Single-channel build: use arbiter_single_client where a >1-channel
// build would instantiate arbiter_round_robin in WAIT_GNT_ACK mode.
arbiter_single_client #(
    .WAIT_GNT_ACK(1)
) u_arb (
    .clk          (aclk),
    .rst_n        (aresetn),
    .block_arb    (arb_blocked),
    .request       (engine_request),
    .grant_ack     (engine_grant_ack),
    .grant_valid   (engine_grant_valid),
    .grant         (engine_grant),      // width-1 one-hot
    .grant_id      ()                  // always 0, usually unconnected
);
```

7.7 Design Notes

- **Why not a combinational passthrough?** `grant = request` looks correct for one client but loses the registered, ack-held timing. The AXI read/write engines interpret the missing hold as an early grant drop and insert a bubble beat at every burst boundary. This module preserves the exact cycle-by-cycle behavior.
- **grant_id degeneracy.** The general arbiter's `grant_id` is $\$clog2(CLIENTS)$ bits wide, which collapses to an illegal `[-1:0]` slice at `CLIENTS == 1`. Here `grant_id` is simply tied to 0, sidestepping the elaboration problem.
- **Block behavior.** `block_arb` masks the request combinationally, so raising it prevents new grants; an already-pending ack-held grant continues to follow the state machine.
- **Reset.** Active-low asynchronous reset clears both `grant_valid` and `r_pending_ack`.

7.8 Related Modules

Used by single-channel builds of AXI read/write datapath engines that otherwise instantiate `arbiter_round_robin`. The module itself is self-contained — no submodule instantiations.

- [arbiter_round_robin](#) - The general N-client arbiter whose `WAIT_GNT_ACK` timing this module reproduces for one client

- [arbiter_round_robin_simple](#) - Minimal-area round-robin variant
- [arbiter_round_robin_weighted](#) - Weighted / QoS round-robin variant
- [arbiter_priority_encoder](#) - Fixed-priority arbiter

Source: rtl/common/arbiter_single_client.sv, with
rtl/common/arbiter_round_robin.sv as the timing reference, and docs/markdown/rtl-common/index.md.

Last Updated: 2026-07-15

7.9 Testing

No dedicated testbench, by decision. A single-client arbiter grants its one requester unconditionally; the behaviour is verified in situ in STREAM rather than through a standalone bench. Recorded as exempt in the coverage baseline (common TASK-010, was COMMON-024).

Treat any behaviour described on this page as unverified by simulation.

7.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

8 Token-Bucket Request Shaper

Module: rtl/common/arbiter_token_bucket.sv

8.1 Overview

Per-client token-bucket request shaper. It sits in front of any arbiter in the family and gates each client's request by its token balance: tokens accumulate at a configured per-client rate on an external refill tick (up to a burst cap), and one token is spent per completed grant. A client out of tokens simply stops requesting until the next refill.

This is **rate shaping, not fairness** — the arbiter behind it still decides who wins among the requests that pass; the shaper decides how *often* each client may compete. That separation is why it is a free-standing block rather than a mode of any one arbiter: it composes with

[arbiter_round_robin](#) for rate-limited fair sharing, or with

[arbiter_round_robin_weighted](#) / [arbiter_deficit_round_robin](#) for shaped rate *and* weighted share together.

```
module arbiter_token_bucket #(
    parameter int CLIENTS      = 4,
    parameter int MAX_TOKENS   = 64,
    parameter int RATE_WIDTH   = 4,
    parameter int WAIT_GNT_ACK = 0
) (
    input logic clk,
    input logic rst_n,
    input logic refill_tick,
    input logic [CXRW-1:0] rate,
    input logic [CXTW-1:0] bucket_cap,
    input logic [C-1:0] request_in,
    input logic [C-1:0] grant,
    input logic grant_valid,
    input logic [C-1:0] grant_ack,
    output logic [C-1:0] request_out,
    output logic [CXTW-1:0] tokens
);
```

8.2 Parameters

Parameter	Type	Default	Description
CLIENTS	int	4	Number of clients being shaped
MAX_TOKENS	int	64	Exclusive bound on buckets/caps; field width =

Parameter	Type	Default	Description
			$\$clog2(\text{MAX_TOKENS})$
RATE_WIDTH	int	4	Width of each per-client tokens-per-tick rate field
WAIT_GNT_ACK	int	0	Completion contract of the DOWNSTREAM arbiter (must match it)

8.2.1 Derived Parameters (do not override)

Declared as parameter so the elaborator can size the packed ports, not so callers can set them. Overriding one desynchronises it from the parameters above and the design mis-sizes or fails to elaborate.

Derived parameter	Default expression
TW	$\$clog2(\text{MAX_TOKENS})$
CXTW	$\text{CLIENTS} * \text{TW}$

8.3 Ports

Port	Dir	Width	Description
clk	In	1	
rst_n	In	1	
refill_tick	In	1	
rate	In	$[\text{CXRW}-1:0]$	
bucket_cap	In	$[\text{CXTW}-1:0]$	
request_in	In	$[\text{C}-1:0]$	
grant	In	$[\text{C}-1:0]$	
grant_valid	In	1	
grant_ack	In	$[\text{C}-1:0]$	
request_out	Out	$[\text{C}-1:0]$	
tokens	Out	$[\text{CXTW}-1:0]$	

8.4 Functional Description

- **refill_tick is external.** Pair it with `counter_freq_invariant`'s microsecond tick so rates carry real-time meaning across clock frequencies, or with any pulse of your choosing.
- **cap = 0 means UNSHAPED, not blocked** (fail-open). A zeroed config must degrade to “no shaping”, never to “no service”. To block a client, gate its request upstream or zero its weight/quantum in the downstream arbiter. Corollary: real caps live in $1..MAX_TOKENS-1$ — a cap value equal to `MAX_TOKENS` wraps the field to 0 and turns shaping OFF.
- **The cap clamp is an invariant, applied every cycle** — not only at refill. A runtime cap decrease bites immediately; banked burst above the new cap is forfeited (with rate 0 a refill-time-only clamp would never fire at all).
- **Overspend-proof gate.** The downstream grant registers one cycle after arbitration saw `request_out`, so in the completion cycle the bucket register still shows the pre-spend value. The pass gate subtracts the in-flight spend, so a one-token client cannot win twice on one token — the same registered-decision window the DRR's cost pipeline handles.
- **Burst semantics:** a bucket at cap C allows C back-to-back grants, then the client throttles to its refill rate. Reset starts every bucket empty (the conservative choice for a rate limiter).
- **No config-update FSM** — unlike the WRR/DRR there is no cross-client invariant to protect; rate/cap changes take effect per the clamp rules.

8.5 Design Notes

TW and CXTW are derived; do not override them. They are $\lceil \log_2(MAX_TOKENS) \rceil$ and `CLIENTS`
* TW, declared as parameter so the elaborator can size the packed ports. Setting one directly desynchronises it from `MAX_TOKENS` and the packed vectors mis-size.

WAIT_GNT_ACK must match the DOWNSTREAM arbiter. It states the completion contract this shaper is feeding, not a preference; a mismatch means the shaper and the arbiter disagree about when a grant is finished.

8.6 Timing Characteristics

Property	Value
Gate	Combinational: request_in to request_out through one compare
Buckets	Registered; refill-then-clamp-then-spend per cycle
Sustained rate	Exactly rate[i] tokens per tick interval under saturation

8.7 Usage Examples

Every parameter and port below is taken from the module declaration.

```
arbiter_token_bucket #(
    .CLIENTS          (4),
    .MAX_TOKENS        (64),
    .RATE_WIDTH        (4),
    .WAIT_GNT_ACK      (0),
    .TW                ($clog2(MAX_TOKENS)),
    .C                 (CLIENTS),
    .CXTW              (CLIENTS * TW),
    .CXRW              (CLIENTS * RATE_WIDTH),
) u_arbiter_token_bucket (
    .clk               (clk),
    .rst_n             (rst_n),
    .refill_tick       (refill_tick),
    .rate              (rate),
    .bucket_cap        (bucket_cap),
    .request_in        (request_in),
    .grant             (grant),
    .grant_valid       (grant_valid),
    .grant_ack         (grant_ack),
    .request_out       (request_out),
    .tokens            (tokens)
);
```

8.8 Related Modules

- [arbiter_round_robin](#), [arbiter_round_robin_weighted](#), [arbiter_deficit_round_robin](#) — the arbiters this shaper feeds (welded to none of them)

- [counter_freq_invariant](#) — the natural refill_tick source

8.9 Testing

Location: `val/common/test_arbiter_token_bucket.py`

The TB plays the downstream arbiter (registered grants, both ACK modes) and keeps a never-overspend ledger asserted at every completion: cumulative grants can never exceed cumulative banked refill. Scenarios: sustained rate under saturation, burst allowance exactly the cap, cap-0 bypass, and a runtime rate cut that drains then fully blocks.

```
pytest val/common/test_arbiter_token_bucket.py -v
```

8.10 Navigation

- [Index](#) · [Overview](#)

[← Back to Main Index](#) | [rtl-common Index](#)

9 Binary to BCD Converter

Binary in, decimal out — the module you need the moment a number has to end up on a display.

9.1 Overview

The `bin_to_bcd` module converts binary numbers to Binary Coded Decimal (BCD) format using the Double-Dabble algorithm (also known as the Add-3-Then-Shift algorithm). It's a fundamental operation in digital systems for displaying numeric values on seven-segment displays, LCD panels, and other decimal-based output devices.

```
module bin_to_bcd #(
    parameter int WIDTH  = 8,
    parameter int DIGITS = 3
) (
    input logic clk,           // Clock signal
    input logic rst_n,        // Active-low reset signal
    input logic start,
    input logic [WIDTH-1:0] binary,
    output logic [DIGITS*4-1:0] bcd,
    output logic done
);
```

9.2 Parameters

Parameter	Type	Default	Description
WIDTH	int	8	Bit width of the input binary number. Range: 1 to 32 (practical range). Determines conversion complexity and timing. Sizing rule: for n-bit binary input, maximum decimal value is $2^n - 1$.
DIGITS	int	3	Number of BCD digits in the output. Must be sufficient for the maximum input value: $DIGITS \geq \lceil \log_{10}(2^{WIDTH}) \rceil$. Output width is $DIGITS \times 4$ bits (4 bits per BCD digit).

9.2.1 Parameter Relationship Table

WIDTH	Max Binary Value	Max Decimal	Min DIGITS Required	BCD Width
4	15	15	2	8 bits
8	255	255	3	12 bits
10	1023	1023	4	16 bits
12	4095	4095	4	16 bits
16	65535	65535	5	20 bits
20	1048575	1048575	7	28 bits
24	16777215	16777215	8	32 bits

9.3 Ports

9.3.1 Inputs

Port	Width	Description
clk	1	System clock
rst_n	1	Active-low asynchronous reset
start	1	Initiate conversion (pulse)
binary	WIDTH	Binary number to convert

9.3.2 Outputs

Port	Width	Description
bcd	DIGITS×4	BCD result (packed format)
done	1	Conversion complete flag

9.4 Functional Description

9.4.1 The Double-Dabble Algorithm

The Double-Dabble algorithm converts binary to BCD through iterative shifting and adjustment:

1. **Initialize:** BCD accumulator = 0, binary shift register = input value

2. **Check:** For each BCD digit, if value ≥ 5 , add 3
3. **Shift:** Shift entire register (BCD + binary) left by 1 bit
4. **Repeat:** Continue until all binary bits are processed
5. **Result:** BCD accumulator contains the decimal equivalent

9.4.2 Why Add 3?

The “add 3” step exists to prevent BCD overflow during shifting: - **BCD Range:** Each digit must stay within 0-9 (4 bits) - **Problem:** Shifting a BCD digit ≥ 5 creates values > 9 - **Solution:** Adding 3 before shifting ensures proper BCD after shift

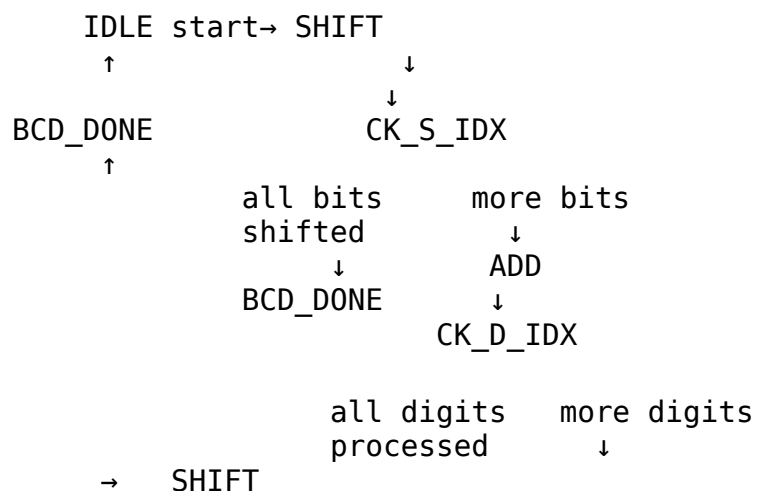
9.4.3 Mathematical Proof

For BCD digit D where $D \geq 5$: - Before shift: $D + 3$ - After shift: $2(D + 3) = 2D + 6$ - For D = 5: $2(5) + 6 = 16 = 0x10$ (BCD: 1,6 - carry generated) - For D = 9: $2(9) + 6 = 24 = 0x18$ (BCD: 2,4 - carry generated)

9.4.4 State Machine

```
typedef enum logic [5:0] {
    IDLE      = 6'b000001, // Wait for start signal
    SHIFT     = 6'b000010, // Shift binary bit into BCD
    CK_S_IDX  = 6'b000100, // Check if all bits shifted
    ADD       = 6'b001000, // Add 3 to digits > 4
    CK_D_IDX  = 6'b010000, // Check if all digits processed
    BCD_DONE  = 6'b100000 // Conversion complete
} fsm_state_t;
```

9.4.5 State Transition Diagram



9.4.6 State Descriptions

9.4.6.1 IDLE State

- **Function:** Wait for start signal
- **Duration:** Variable (until start asserted)
- **Actions:**
 - Monitor start input
 - Initialize counters and registers when start detected
- **Next State:** SHIFT when `start == 1'b1`

9.4.6.2 SHIFT State

- **Function:** Shift one bit from binary to BCD
- **Duration:** 1 clock cycle
- **Actions:**
 - Left-shift BCD accumulator by 1 bit
 - Move MSB of binary into LSB of BCD
 - Left-shift binary register by 1 bit
 - Increment loop counter
- **Next State:** Always CK_S_IDX

9.4.6.3 CK_S_IDX State

- **Function:** Check if all binary bits processed
- **Duration:** 1 clock cycle
- **Actions:**
 - Compare loop counter with WIDTH
 - Reset digit index for ADD phase
- **Next State:**
 - BCD_DONE if `loop_count == WIDTH-1`

- ADD if more bits to process

9.4.6.4 ADD State

- **Function:** Add 3 to BCD digits ≥ 5
- **Duration:** 1 clock cycle
- **Actions:**
 - Extract current BCD digit
 - Add 3 if digit > 4
 - Update BCD accumulator
- **Next State:** Always CK_D_IDX

9.4.6.5 CK_D_IDX State

- **Function:** Check if all digits processed
- **Duration:** 1 clock cycle
- **Actions:**
 - Compare digit index with DIGITS
 - Increment digit index
- **Next State:**
 - SHIFT if digit_index == DIGITS-1
 - ADD if more digits to process

9.4.6.6 BCD_DONE State

- **Function:** Signal completion
- **Duration:** 1 clock cycle
- **Actions:**
 - Assert done flag
 - Prepare for next conversion

- **Next State:** Always IDLE

9.4.7 Register Definitions

```
// FSM state register
fsm_state_t r_fsm_main;

// BCD accumulator - stores the converted result
logic [DIGITS*4-1:0] r_bcd;

// Binary shift register - input value being processed
logic [WIDTH-1:0] r_binary;

// Loop counter - tracks processed bits
localparam int LOOP_COUNT_WIDTH = $clog2(WIDTH + 1);
logic [LOOP_COUNT_WIDTH-1:0] r_loop_count;

// Digit index - tracks current BCD digit being processed
localparam int DIGIT_INDEX_WIDTH = $clog2(DIGITS);
logic [DIGIT_INDEX_WIDTH-1:0] r_digit_index;

// Data valid flag
logic r_dv;
```

9.4.8 Core Processing Logic

```
// Extract current BCD digit being processed
logic [3:0] w_bcd_digit;
assign w_bcd_digit = r_bcd[r_digit_index*4+:4];

// Main FSM
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_fsm_main    <= IDLE;
        r_bcd         <= '0;
        r_binary      <= '0;
        r_digit_index <= '0;
        r_loop_count  <= '0;
        r_dv          <= '0;
    end else begin
        r_dv <= 1'b0; // Default: done is low

        casez (r_fsm_main)
            IDLE: begin
                if (start == 1'b1) begin
                    r_binary    <= binary;
                    r_fsm_main  <= SHIFT;
                end
            endcase
    end
```

```

        r_bcd          <= '0;
        r_loop_count   <= '0;
        r_digit_index  <= '0;
    end else begin
        r_fsm_main <= IDLE;
    end
end

SHIFT: begin
    // Shift BCD left and insert MSB of binary
    r_bcd      <= r_bcd << 1;
    r_bcd[0]    <= r_binary[WIDTH-1];
    r_binary    <= r_binary << 1;
    r_fsm_main  <= CK_S_IDX;
end

CK_S_IDX: begin
    if (r_loop_count == LOOP_COUNT_WIDTH'(WIDTH - 1))
begin
        r_loop_count <= '0;
        r_fsm_main   <= BCD_DONE;
    end else begin
        r_loop_count <= r_loop_count + 1'b1;
        r_digit_index <= '0;
        r_fsm_main   <= ADD;
    end
end

ADD: begin
    if (w_bcd_digit > 4'd4) begin
        r_bcd[(r_digit_index*4)+:4] <= w_bcd_digit + 4'd3;
    end
    r_fsm_main <= CK_D_IDX;
end

CK_D_IDX: begin
    if (r_digit_index == DIGIT_INDEX_WIDTH'(DIGITS - 1))
begin
        r_digit_index <= '0;
        r_fsm_main    <= SHIFT;
    end else begin
        r_digit_index <= r_digit_index + 1'b1;
        r_fsm_main    <= ADD;
    end
end

BCD_DONE: begin

```

```

        r_dv      <= 1'b1;
        r_fsm_main <= IDLE;
    end

    default: begin
        r_fsm_main <= IDLE;
    end
endcase
end
end

```

9.4.9 BCD Format and Encoding

Each BCD digit uses 4 bits to represent decimal values 0-9:

Decimal	BCD (4-bit)	Hex
0	0000	0x0
1	0001	0x1
2	0010	0x2
3	0011	0x3
4	0100	0x4
5	0101	0x5
6	0110	0x6
7	0111	0x7
8	1000	0x8
9	1001	0x9
Invalid	1010-1111	0xA-0xF

The output bcd uses packed BCD format where multiple digits are concatenated:

```

// For DIGITS = 3, bcd[11:0] contains:
// bcd[11:8]  = hundreds digit
// bcd[7:4]   = tens digit
// bcd[3:0]   = ones digit

```

```

// Example: decimal 123 → bcd = 12'h123

```

Extracting individual digits from the packed vector:

```

// Extract individual digits from packed BCD
logic [3:0] ones, tens, hundreds;

assign ones      = bcd[3:0];

```

```

assign tens      = bcd[7:4];
assign hundreds = bcd[11:8];

// Convert to ASCII for display
logic [7:0] ascii_ones, ascii_tens, ascii_hundreds;

assign ascii_ones    = 8'h30 + ones;      // Add ASCII '0'
assign ascii_tens    = 8'h30 + tens;
assign ascii_hundreds = 8'h30 + hundreds;

```

9.5 Related Modules

Nothing in the tree instantiates this module and it instantiates nothing: it is a leaf, used directly by whatever design needs it. Its nearest neighbours in rtl/common/ are:

- (none in this category)

9.6 Timing Characteristics

9.6.1 Conversion Latency

The total conversion time depends on the input width and number of digits:

Formula: Latency = (WIDTH-1) × (2×DIGITS + 2) + 4 clock cycles (measured from the cycle start is accepted to the cycle done asserts). The +4 tail is SHIFT, CK_S_IDX, BCD_DONE, and one more cycle for the registered done: r_dv is set in the BCD_DONE state and done = r_dv only reads 1 the following cycle, after the FSM is back in IDLE.

The per-bit cost is dominated by the add-3 phase: each shift is followed by a full ADD → CK_D_IDX pass over **every** digit, so each bit position costs 2×DIGITS cycles there in addition to its SHIFT and CK_S_IDX cycles. The CK_S_IDX and CK_D_IDX states are distinct clock cycles, not free.

9.6.2 Latency Breakdown

- **SHIFT + CK_S_IDX:** 2 cycles per input bit → 2 × WIDTH cycles.
- **ADD + CK_D_IDX:** 2 × DIGITS cycles after each shift **except the last** (the final bit skips the add phase and goes straight to BCD_DONE) → (WIDTH-1) × 2 × DIGITS cycles.
- **Completion:** 1 cycle for BCD_DONE state.
- **Total:** (WIDTH-1) × (2×DIGITS + 2) + 3 cycles.

9.6.3 Latency Examples

WIDTH	DIGITS	Total Cycles	@ 100MHz	@ 50MHz
4	2	$3 \times 6 + 4 = 22$	220ns	440ns
8	3	$7 \times 8 + 4 = 60$	600ns	1.20μs
12	4	$11 \times 10 + 4 = 114$	1.14μs	2.28μs
16	5	$15 \times 12 + 4 = 184$	1.84μs	3.68μs
20	7	$19 \times 16 + 4 = 308$	3.08μs	6.16μs

9.6.4 Pipeline Considerations

The conversion is inherently sequential, making pipelining challenging: - **Data Dependency:** Each iteration depends on previous results - **State-Based:** FSM prevents overlapping conversions - **Optimization:** Consider parallel converters for high throughput

9.6.5 Synthesis Results (Typical FPGA)

Configuration	LUTs	FFs	BRAM	Max Freq	Notes
8-bit, 3-digit	85	45	0	250 MHz	Basic config
12-bit, 4-digit	145	65	0	220 MHz	Medium size
16-bit, 5-digit	220	85	0	200 MHz	Larger design
20-bit, 7-digit	350	125	0	180 MHz	Complex design

9.6.6 Power Consumption

- **Dynamic Power:** Proportional to conversion frequency
- **Static Power:** Minimal (standard CMOS)
- **Optimization:** Clock gating during IDLE state
- **Trade-off:** Performance vs. power efficiency

9.7 Usage Examples

Every parameter and port below is read from the module declaration.

```
bin_to_bcd #(
    .WIDTH          (8),
    .DIGITS          (3)
) u_bin_to_bcd (
    .clk             (clk),
    .rst_n           (rst_n),
    .start           (start),
    .binary          (binary),
    .bcd             (bcd),
    .done            (done)
);
```

9.8 Design Notes

9.8.1 Optimization Techniques

9.8.1.1 Early Termination Optimization

```
// Optimize for small numbers by detecting leading zeros
logic [WIDTH-1:0] leading_zero_mask;
logic [$clog2(WIDTH):0] significant_bits;

// Count significant bits
count_leading_zeros #(.WIDTH(WIDTH)) clz_inst (
    .data(binary),
    .clz(leading_zeros)
);

assign significant_bits = WIDTH - leading_zeros;

// Modify loop termination condition
always_ff @(posedge clk) begin
    // ... other logic ...

    CK_S_IDX: begin
        if (r_loop_count == significant_bits - 1) begin
            r_fsm_main <= BCD_DONE; // Early termination
        end else begin
            // Continue normal processing
        end
    end
end
```

9.8.1.2 Parallel Digit Processing

```
// Process all digits in parallel during ADD state
logic [3:0] digit_array [DIGITS];
logic [3:0] adjusted_digits [DIGITS];

// Extract all digits
genvar i;
generate
    for (i = 0; i < DIGITS; i++) begin : extract_digits
        assign digit_array[i] = r_bcd[i*4+:4];
        assign adjusted_digits[i] = (digit_array[i] > 4) ?
                                   (digit_array[i] + 3) :
digit_array[i];
    end
endgenerate

// Update all digits simultaneously
always_ff @(posedge clk) begin
    // ... other states ...

    ADD: begin
        for (int j = 0; j < DIGITS; j++) begin
            r_bcd[j*4+:4] <= adjusted_digits[j];
        end
        r_fsm_main <= CK_S_IDX; // Skip CK_D_IDX state
    end
end
```

9.8.1.3 Pipelined Implementation

```
// Multi-stage pipeline for high throughput
module bin_to_bcd_pipelined #(
    parameter int WIDTH = 8,
    parameter int DIGITS = 3,
    parameter int STAGES = 4 // Pipeline depth
) (
    input logic clk,
    input logic rst_n,
    input logic valid_in,
    input logic [WIDTH-1:0] binary_in,
    output logic valid_out,
    output logic [DIGITS*4-1:0] bcd_out
);

// Pipeline stages
logic [STAGES-1:0] valid_pipe;
logic [WIDTH-1:0] binary_pipe [STAGES];
logic [DIGITS*4-1:0] bcd_pipe [STAGES];
```

```

// Stage processing
generate
    for (genvar s = 0; s < STAGES; s++) begin : pipeline_stages
        logic [7:0] bits_to_process;
        assign bits_to_process = WIDTH * s / STAGES;

        // Process subset of bits in each stage
        // Implementation depends on specific algorithm
partitioning
        end
    endgenerate

endmodule

```

9.8.2 Common Design Patterns

9.8.2.1 Pattern 1: Auto-Converting Display Interface

```

module auto_display_interface #(
    parameter int WIDTH = 10
) (
    input logic [WIDTH-1:0] sensor_data,
    input logic data_valid,
    output logic [6:0] display_segments[4],
    output logic display_valid
);

// Auto-trigger conversion on new data
logic prev_data_valid;
logic start_conversion;

always_ff @(posedge clk) begin
    prev_data_valid <= data_valid;
end

assign start_conversion = data_valid && !prev_data_valid;

// BCD converter and display driver
// ... (implementation similar to previous examples)

endmodule

```

9.8.2.2 Pattern 2: Buffered Converter with FIFO

```

module buffered_bcd_converter #(
    parameter int WIDTH = 12,
    parameter int DIGITS = 4,

```

```

    parameter int BUFFER_DEPTH = 8
) (
    input  logic          clk,
    input  logic          rst_n,
    input  logic [WIDTH-1:0] binary_in,
    input  logic          write_enable,
    output logic [DIGITS*4-1:0] bcd_out,
    output logic          bcd_valid,
    output logic          buffer_full,
    output logic          buffer_empty
);

// Input FIFO
logic [WIDTH-1:0] fifo_data_out;
logic fifo_read_enable, fifo_empty_int;

fifo_sync #(
    .DATA_WIDTH(WIDTH),
    .DEPTH(BUFFER_DEPTH)
) input_buffer (
    .clk(clk),
    .rst_n(rst_n),
    .write(write_enable),
    .wr_data(binary_in),
    .wr_full(buffer_full),
    .wr_almost_full(),
    .read(fifo_read_enable),
    .rd_data(fifo_data_out),
    .rd_empty(fifo_empty_int),
    .rd_almost_empty()
);

// BCD converter control
logic conversion_idle;

assign fifo_read_enable = !fifo_empty_int && conversion_idle;
assign buffer_empty = fifo_empty_int;

// BCD converter instance
bin_to_bcd #(
    .WIDTH(WIDTH),
    .DIGITS(DIGITS)
) converter (
    .clk(clk),
    .rst_n(rst_n),
    .start(fifo_read_enable),
    .binary(fifo_data_out),

```

```

        .bcd(bcd_out),
        .done(bcd_valid)
    );

    // Track converter state
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            conversion_idle <= 1;
        end else begin
            if (fifo_read_enable) begin
                conversion_idle <= 0;
            end else if (bcd_valid) begin
                conversion_idle <= 1;
            end
        end
    end
end
endmodule

```

9.8.3 Troubleshooting Guide

9.8.3.1 Issue 1: Incorrect BCD Output

Symptoms: Output doesn't match expected decimal value **Causes:** - Insufficient DIGITS parameter
 - Width mismatch between binary input and internal registers - BCD digit corruption during processing

Debug Steps:

```

// Add internal signal monitoring
always_ff @(posedge clk) begin
    if (dut.r_fsm_main == ADD && dut.w_bcd_digit > 4) begin
        $display("ADD operation: digit=%d, new_value=%d",
            dut.w_bcd_digit, dut.w_bcd_digit + 3);
    end
end
end

```

9.8.3.2 Issue 2: Conversion Never Completes

Symptoms: done signal never asserts **Causes:** - FSM stuck in loop - Incorrect loop termination condition - Clock or reset issues

Debug Steps:

```

// Monitor FSM state transitions
always_ff @(posedge clk) begin
    $display("State: %s, Loop: %d, Digit: %d",
        dut.r_fsm_main.name(), dut.r_loop_count,
        dut.r_digit_index);
end
end

```

9.8.3.3 Issue 3: Timing Violations

Symptoms: Synthesis reports timing failures **Causes:** - Long combinational paths in ADD logic - High fan-out on BCD register - Insufficient pipeline stages

Solutions:

```
// Add pipeline registers
logic [DIGITS*4-1:0] bcd_pipe;
always_ff @(posedge clk) begin
    bcd_pipe <= r_bcd;
end

// Use parallel digit processing
// (Shown in optimization section)
```

9.9 Testing

9.9.1 Test Strategy

```
module tb_bin_to_bcd;

    // Test parameters
    parameter int WIDTH = 8;
    parameter int DIGITS = 3;
    parameter int MAX_VALUE = (1 << WIDTH) - 1;

    // DUT signals
    logic clk, rst_n, start, done;
    logic [WIDTH-1:0] binary;
    logic [DIGITS*4-1:0] bcd;

    // Test control
    logic [WIDTH-1:0] test_vector;
    logic [31:0] expected_decimal;
    logic [DIGITS*4-1:0] expected_bcd;

    // DUT instantiation
    bin_to_bcd #(
        .WIDTH(WIDTH),
        .DIGITS(DIGITS)
    ) dut (.*);

    // Clock generation
    initial clk = 0;
    always #5 clk = ~clk;
```

```

// Test sequence
initial begin
    rst_n = 0;
    start = 0;
    binary = 0;

    #100 rst_n = 1;

    // Test all possible values
    for (int i = 0; i <= MAX_VALUE; i++) begin
        test_conversion(i);
    end

    // Boundary tests
    test_conversion(0);
    test_conversion(1);
    test_conversion(MAX_VALUE);

    // Random tests
    repeat (1000) begin
        test_conversion($random % (MAX_VALUE + 1));
    end

    $display("All tests completed successfully!");
    $finish;
end

// Test task
task test_conversion(input [WIDTH-1:0] value);
    begin
        binary = value;
        expected_decimal = value;
        expected_bcd = decimal_to_bcd(expected_decimal);

        @(posedge clk);
        start = 1;
        @(posedge clk);
        start = 0;

        wait(done);
        @(posedge clk);

        if (bcd != expected_bcd) begin
            $error("Mismatch: binary=%d, expected_bcd=%h,
actual_bcd=%h",
                value, expected_bcd, bcd);
        end
    end
endtask

```



```

        end else begin
            $display("PASS: binary=%d → bcd=%h", value, bcd);
        end
    end
endtask

```

```

// Reference function
function [DIGITS*4-1:0] decimal_to_bcd(input [31:0] decimal);
    logic [31:0] temp;
    logic [DIGITS*4-1:0] result;
    begin
        temp = decimal;
        for (int i = 0; i < DIGITS; i++) begin
            result[i*4+:4] = temp % 10;
            temp = temp / 10;
        end
        decimal_to_bcd = result;
    end
endfunction

```

```
endmodule
```

9.9.2 Coverage Model

```
covergroup bcd_conversion_cg;
```

```

    // Input value coverage
    cp_binary_value: coverpoint binary {
        bins zero = {0};
        bins small[] = {[1:9]};
        bins medium[] = {[10:99]};
        bins large[] = {[100:255]}; // For 8-bit
        bins max_val = {2**WIDTH - 1};
    }

    // State coverage
    cp_fsm_states: coverpoint dut.r_fsm_main {
        bins all_states[] = {IDLE, SHIFT, CK_S_IDX, ADD, CK_D_IDX,
BCD_DONE};
        bins state_transitions[] = (IDLE => SHIFT),
                                   (SHIFT => CK_S_IDX),
                                   (CK_S_IDX => ADD),
                                   (CK_S_IDX => BCD_DONE),
                                   (ADD => CK_D_IDX),
                                   (CK_D_IDX => ADD),
                                   (CK_D_IDX => SHIFT),
                                   (BCD_DONE => IDLE);
    }

```

```

// BCD digit coverage
cp_bcd_digits: coverpoint bcd {
    bins valid_bcd[] = {[12'h000:12'h999]}; // Valid BCD range
}

// Loop iteration coverage
cp_loop_count: coverpoint dut.r_loop_count {
    bins all_counts[] = {[0:WIDTH-1]};
}

// Cross coverage
cross_value_states: cross cp_binary_value, cp_fsm_states;

```

endcovergroup

9.9.3 Formal Verification Properties

```

// Properties for formal verification
module bin_to_bcd_properties #(
    parameter int WIDTH = 8,
    parameter int DIGITS = 3
);

// Bind to DUT
bind bin_to_bcd bin_to_bcd_properties #(WIDTH, DIGITS) props_inst
(.*);

// Basic properties
property conversion_eventually_completes;
    @(posedge clk) disable iff (!rst_n)
    start |-> ##[1:200] done; // Adjust bound based on max latency
endproperty

property bcd_output_valid;
    @(posedge clk) disable iff (!rst_n)
    done |-> valid_bcd(bcd);
endproperty

property conversion_correctness;
    logic [31:0] expected_decimal;
    @(posedge clk) disable iff (!rst_n)
    (start, expected_decimal = binary) |->
    ##[1:200] (done && (bcd_to_decimal(bcd) == expected_decimal));
endproperty

// Helper functions

```

```

function bit valid_bcd(logic [DIGITS*4-1:0] bcd_val);
    for (int i = 0; i < DIGITS; i++) begin
        if (bcd_val[i*4+:4] > 9) return 0;
    end
    return 1;
endfunction

function [31:0] bcd_to_decimal(logic [DIGITS*4-1:0] bcd_val);
    logic [31:0] result = 0;
    for (int i = 0; i < DIGITS; i++) begin
        result += bcd_val[i*4+:4] * (10**i);
    end
    return result;
endfunction

// Assertions
assert property (conversion_eventually_completes);
assert property (bcd_output_valid);
assert property (conversion_correctness);

```

endmodule

That covers the Binary to BCD converter end to end — algorithm, implementation, verification, and optimization. Double-Dabble is sequential by nature, but it gives you reliable, predictable conversion for a wide range of digital display applications.

9.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

[← Back to Main Index](#) | [rtl-common Index](#)

10 CAM Tag

Associative tag storage: you find entries by content, not by address.

10.1 Overview

The `cam_tag` module implements a Content Addressable Memory (CAM) for tag storage and lookup. Lookups are associative — you find data by content, not by address — which is exactly what cache implementations, translation lookaside buffers (TLBs), and routing tables need.

10.2 Parameters

Parameter	Type	Default	Description
ENABLE	int	1	Enable/disable CAM functionality
N	int	8	Width of each tag in bits
DEPTH	int	16	Number of tag entries in the CAM

10.3 Ports

10.3.1 Inputs

Port	Width	Description
clk	1	System clock
rst_n	1	Active-low asynchronous reset
tag_in_status	N	Tag to search for (lookup operation)
mark_valid	1	Signal to add a new valid tag
tag_in_valid	N	Tag value to mark as valid
mark_invalid	1	Signal to invalidate an existing tag
tag_in_invalid	N	Tag value to mark as

Port	Width	Description
		invalid

10.3.2 Outputs

Port	Width	Description
tags_empty	1	Indicates when CAM contains no valid entries
tags_full	1	Indicates when CAM is completely full
tag_status	1	Result of lookup operation (1 if tag found, 0 if not found)

10.4 Functional Description

10.4.1 Tag Lookup

Continuously compares tag_in_status against all valid entries: - Returns 1 if tag found among valid entries - Returns 0 if tag not found or entry is invalid

10.4.2 Tag Insertion

When mark_valid is asserted and CAM is not full: 1. Store tag_in_valid at w_next_valid_loc
2. Set corresponding valid bit 3. Only occurs when ENABLE != 0

10.4.3 Tag Invalidation

When mark_invalid is asserted and matching tag found: 1. Clear the tag value at matching location 2. Clear corresponding valid bit

10.4.4 Enable Control

When ENABLE = 0: - Tag insertion is disabled - Tag lookup and invalidation still function - Useful for disabling CAM without losing existing data

10.4.5 Allocation Strategy

- **First Available:** Uses the **lowest-indexed** free location (see the search below). Functional behavior is correct either way; only the index order matters when matching allocations to waveforms.
- **Replacement:** No automatic replacement policy (must manually invalidate)
- **Overflow Protection:** Prevents insertion when full

10.4.6 Storage Arrays

```
logic [N-1:0]      r_tag_array [0:DEPTH-1]; // Tag storage array
logic [DEPTH-1:0] r_valid;                // Valid bit for each
entry
```

10.4.7 Search Logic

Three parallel search operations run side by side:

10.4.7.1 1. Next Available Location Search

```
always_comb begin
    w_next_valid_loc = -1;
    for (int i=DEPTH-1; i >= 0; i--)
        if (r_valid[i] == 1'b0)
            w_next_valid_loc = i;
end
```

- Iterates from the highest index down to 0, overwriting `w_next_valid_loc` on every free slot — so the **last** assignment (index 0, if free) wins
- Returns the **lowest-indexed** free location (e.g., all slots free → slot 0)
- Returns -1 if no free locations available

10.4.7.2 2. Valid Tag Match Search

```
always_comb begin
    w_match_loc = -1;
    for (int i = 0; i < DEPTH; i++)
        if (r_valid[i] == 1'b1 && tag_in_status == r_tag_array[i])
            w_match_loc = i;
end
```

- Searches for exact tag match among valid entries
- Returns index of matching entry or -1 if no match

10.4.7.3 3. Invalid Tag Match Search

```
always_comb begin
    w_match_invalid_loc = -1;
    for (int i = 0; i < DEPTH; i++)
        if (r_valid[i] == 1'b1 && tag_in_invalid == r_tag_array[i])
            w_match_invalid_loc = i;
end
```

- Used for invalidation operations
- Finds valid entries that match the invalidation tag

10.4.8 State Management

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_valid <= 'b0;
        for (int i = 0; i < DEPTH; i++) begin
            r_tag_array[i] <= 'b0;
        end
    end else begin
        if (mark_valid && !tags_full && (ENABLE != 0)) begin
            r_tag_array[w_next_valid_loc] <= tag_in_valid;
            r_valid[w_next_valid_loc] <= 1'b1;
        end else if (mark_invalid && w_match_invalid_loc >= 0) begin
            r_tag_array[w_match_invalid_loc] <= 'b0;
            r_valid[w_match_invalid_loc] <= 1'b0;
        end
    end
end
```

10.4.9 Status Signals

- tags_empty = ~|r_valid: NOR of all valid bits
- tags_full = &r_valid: AND of all valid bits
- tag_status: Result of tag lookup operation

10.5 Related Modules

Nothing in the tree instantiates this module and it instantiates nothing: it is a leaf, used directly by whatever design needs it. Its nearest neighbours in rtl/common/ are:

- (none in this category)
-

10.6 Timing Characteristics

This module is **sequential**: it contains clocked logic (via always_ff or the repository's ALWAYS_FF_RST macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

10.7 Usage Examples

Every parameter and port below is taken from the module declaration.

```
cam_tag #(
    .ENABLE          (1),
    .N               (8),
    .DEPTH           (16)
) u_cam_tag (
    .clk             (clk),
    .rst_n           (rst_n),
    .tag_in_status   (tag_in_status),
    .mark_valid      (mark_valid),
    .tag_in_invalid  (tag_in_invalid),
    .mark_invalid    (mark_invalid),
    .tag_in_invalid  (tag_in_invalid),
    .tags_empty      (tags_empty),
    .tags_full       (tags_full),
    .tag_status      (tag_status)
);
```

10.8 Design Notes

10.8.1 Usage Considerations

- **Timing:** All lookups are combinational (single cycle)
- **Capacity:** Monitor `tags_full` before insertions
- **Conflicts:** No protection against duplicate tag insertion
- **Performance:** Search time is constant regardless of occupancy
- **Power:** All entries are searched simultaneously (high power for large depths)

10.9 Testing

`val/common/test_cam_tag.py` exercises this module. It collects 2 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/common/test_cam_tag.py -v
```

10.10 **Navigation**

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

[← Back to Main Index](#) | [rtl-common Index](#)

11 Clock Divider

Multiple divided clocks from one input clock, all tapped off a single shared counter so they come out phase-aligned.

WARNING: NOT recommended for functional clocks! Use PLL/MMCM/clock manager primitives instead. This module is intended for testbenches, debug outputs, and non-critical timing applications.

11.1 Overview

The `clock_divider` module generates multiple divided clock signals from a single input clock using configurable pick-off points from a shared master counter. Because every output taps the same counter, the divided clocks come out synchronized and phase-aligned — the behavior you need for multi-rate digital signal processing, communication systems, and power management applications.

```
module clock_divider #(
    parameter int N                = 4, // Number of output clocks
    parameter int PO_WIDTH        = 8, // Width of the Pick off
    registers
    parameter int COUNTER_WIDTH = 64 // Width of the counter
) (
    input logic clk, // Input clock
    signal
    input logic rst_n, // Active low
    reset signal
    input logic [(N*PO_WIDTH-1):0] pickoff_points, // the pick off
    point config registers
    output logic [N-1:0] divided_clk // Divided clock
    signals
);
```

11.2 Parameters

Parameter	Type	Default	Description
N	int	4	Number of independently configurable output clocks. Range: 1 to 16 (practical limit). Determines number of output clock domains and resource usage (N registers + N muxes).

Parameter	Type	Default	Description
PO_WIDTH	int	8	Bit width of each pick-off point configuration register. Range: 4 to 8 bits. Must be $> \lceil \log_2(\text{COUNTER_WIDTH}) \rceil$ to avoid truncation — equivalently $\text{PO_WIDTH} \geq \lceil \log_2(\text{COUNTER_WIDTH} + 1) \rceil$, which ensures PO_WIDTH can hold the value COUNTER_WIDTH without truncation. Examples: CW=16 needs $\text{PO} \geq 5$, CW=32 needs $\text{PO} \geq 6$, CW=64 needs $\text{PO} \geq 7$. Larger width allows finer counter bit selection but uses more configuration storage. Typical: PO_WIDTH=8 for COUNTER_WIDTH up to 128.
COUNTER_WIDTH	int	64	Bit width of the master binary counter. Range: 2 to 64 bits (practical range). Determines maximum division ratio ($2^{\text{COUNTER_WIDTH}}$). Example: WIDTH=16 $\rightarrow$ max division ratio = 65536. Larger counters enable slower frequencies but consume more resources.

11.2.1 Parameter Relationships

- **Addressing Constraint:** $PO_WIDTH > \lceil \log_2(COUNTER_WIDTH) \rceil$ (strictly greater — the RTL \$fatal's at elaboration if $PO_WIDTH \leq \lceil \log_2(COUNTER_WIDTH) \rceil$). E.g. $COUNTER_WIDTH=16$ needs $PO_WIDTH \geq 5$.
- **Division Range:** Each output can divide by 2^1 to $2^{COUNTER_WIDTH}$
- **Configuration Width:** Total configuration bits = $N \times PO_WIDTH$

11.3 Ports

11.3.1 Inputs

Port	Width	Description
clk	1	Master input clock (source to be divided)
rst_n	1	Active-low asynchronous reset
pickoff_points	$N \times PO_WIDTH$	Packed pickoff point configuration array

pickoff_points Format:

Bit Packing: {po[N-1], ..., po[1], po[0]}

Each po[i] (PO_WIDTH bits): Counter bit index to sample for output i

11.3.2 Outputs

Port	Width	Description
divided_clk	N	Divided clock outputs (registered)

11.4 Functional Description

11.4.1 Counter Operation

- Free-running binary counter increments every input clock cycle
- Wraps at $2^{COUNTER_WIDTH}$ (no overflow flag)
- Each bit toggles at half the frequency of the previous bit
 - Bit 0: $clk/2$ (toggles every cycle)
 - Bit 1: $clk/4$ (toggles every 2 cycles)

- Bit N: $\text{clk}/2^{(N+1)}$

11.4.2 Pickoff Point Selection

- Each output i samples `counter[pickoff_points[i]]`
- Division ratio = $2^{(\text{pickoff_point}+1)}$
- Example: `pickoff=3` → samples `counter[3]` → divides by 16

11.4.3 Out-of-Range Handling

- If `pickoff_points[i] ≥ COUNTER_WIDTH`, clamps to `COUNTER_WIDTH-1` (MSB)
- Prevents illegal bit indexing
- MSB provides slowest possible division

11.4.4 Output Registration

- All `divided_clk` outputs are registered for glitch-free operation
- Adds 1 cycle latency but prevents combinational glitches
- Reset clears all outputs to 0 (low phase)

11.4.5 Division Ratios (100MHz Input Clock)

<code>pickoff_point</code>	Counter Bit	Division Ratio	Output Freq	Period
0	[0]	2	50MHz	20ns
1	[1]	4	25MHz	40ns
2	[2]	8	12.5MHz	80ns
3	[3]	16	6.25MHz	160ns
4	[4]	32	3.125MHz	320ns
5	[5]	64	1.5625MHz	640ns
10	[10]	2048	48.8kHz	20.5μs
15	[15]	65536	1.5kHz	655μs
19	[19]	1048576	95Hz	10.5ms
23	[23]	16777216	6Hz	168ms

11.4.6 Duty Cycle

- All divided clocks have 50% duty cycle

- Result of sampling counter bit (toggle behavior)
- High for $2^{(\text{pickoff_point})}$ input cycles
- Low for $2^{(\text{pickoff_point})}$ input cycles

11.4.7 Internal Signals

```
logic [COUNTER_WIDTH-1:0] r_divider_counters; // Free-running binary counter
localparam int ADDR_WIDTH = $clog2(COUNTER_WIDTH); // Address width
```

11.5 Timing Characteristics

11.5.1 Latency

- **Output Latency:** 1 cycle (registered output)
- **Clock-to-Q:** Standard flip-flop delay
- **Propagation:** Counter increment → mux → output register

11.5.2 Phase Relationships

- All divided clocks share same counter → phase-locked to each other
- Divided clocks have phase offset relative to input clock
- Phase offset depends on counter reset value (0)

11.5.3 Jitter

- Input clock jitter propagates to outputs
- Additional quantization jitter: ± 1 input clock cycle
- For low-jitter clocks: Use dedicated PLL/clock synthesis

11.5.4 Maximum Frequency

- **Limiting Factor:** Counter width and mux depth
- **Typical Performance:**
 - COUNTER_WIDTH=16: ~500 MHz (modern FPGAs)
 - COUNTER_WIDTH=32: ~400 MHz
 - COUNTER_WIDTH=64: ~300 MHz

11.5.5 Resource Scaling

COUNTER_WIDTH	N Outputs	FFs	LUTs (mux, est.)	fmax (Est.)
16	4	20	~12	500 MHz
32	4	36	~28	400 MHz
64	4	68	~52	300 MHz
16	8	24	~24	500 MHz

FF counts are exact (COUNTER_WIDTH counter bits + N output registers). The LUT column is $N * \text{ceil}((\text{COUNTER_WIDTH}-1)/5)$ for the pickoff muxes on a 6-LUT architecture, excluding the clamp comparators; earlier revisions of this table listed 8/10/12, which did not scale with the mux width and understated the wide-counter cases by several times. fmax figures remain unsourced estimates.

11.6 Usage Examples

11.6.1 Four Debug Clocks

```
// Generate clk/4, clk/16, clk/256, clk/65536
logic [31:0] pickoff_cfg;
assign pickoff_cfg = {8'd15, 8'd7, 8'd3, 8'd1}; // {po3, po2, po1, po0}

clock_divider #(
    .N(4), // 4 outputs
    .PO_WIDTH(8), // 8-bit pickoff point
    .COUNTER_WIDTH(16) // 16-bit counter (max div = 65536)
) u_clk_div (
    .clk (clk_100mhz),
    .rst_n (rst_n),
    .pickoff_points(pickoff_cfg),
    .divided_clk ({clk_1p5khz, clk_390khz, clk_6p25mhz, clk_25mhz})
);
```

11.6.2 Baud Rate Generator (Approximate)

```
// For 9600 baud from 100MHz:
// Ideal: 100MHz / 9600 ≈ 10417 (not power-of-2)
// Closest power-of-2: 2^13 = 8192 → 100MHz/8192 = 12.2kHz
// WARNING: 12.2kHz vs 9.6kHz is a ~27% error -- FAR beyond the ~2-3%
// a UART
// can tolerate, and it is NOT a 16x-oversampling clock for 9600 baud
// either.
// A power-of-2 divider CANNOT hit standard baud rates. For a real
// UART use
// counter_load_clear (arbitrary modulus), not this divider.
```

```

logic [7:0] baud_pickoff = 8'd12; // 2^13 = 8192 division
(illustrative only)

clock_divider #(
    .N(1),
    .PO_WIDTH(8),
    .COUNTER_WIDTH(16)
) u_baud_clk (
    .clk          (sys_clk),
    .rst_n        (rst_n),
    .pickoff_points(baud_pickoff),
    .divided_clk   (uart_baud_x16) // 16× oversampling clock
);

```

Note: For precise baud rates (not power-of-2), use counter_load_clear instead.

11.6.3 Runtime-Programmable Divider

```

// APB-configurable clock divider
logic [7:0] cfg_pickoff_0, cfg_pickoff_1;

always_ff @(posedge pclk) begin
    if (pwrite && paddr == ADDR_PICKOFF_0) cfg_pickoff_0 <=
pdata[7:0];
    if (pwrite && paddr == ADDR_PICKOFF_1) cfg_pickoff_1 <=
pdata[7:0];
end

clock_divider #(
    .N(2),
    .PO_WIDTH(8),
    .COUNTER_WIDTH(24)
) u_cfg_clk_div (
    .clk          (core_clk),
    .rst_n        (core_rst_n),
    .pickoff_points({cfg_pickoff_1, cfg_pickoff_0}),
    .divided_clk   ({clk_out_1, clk_out_0})
);

```

11.6.4 Test Clock Generation

```

// Generate 8 test clocks for debug
clock_divider #(
    .N(8), // 8 debug clocks
    .PO_WIDTH(8),
    .COUNTER_WIDTH(32) // Support ultra-slow clocks
) u_test_clks (
    .clk          (ref_clk),

```



```

        .rst_n          (test_rst_n),
        .pickoff_points({8'd20, 8'd18, 8'd16, 8'd14, 8'd12, 8'd10, 8'd8,
8'd6}),
        .divided_clk    (test_clk_bus)
    );

```

11.7 Design Notes

11.7.1 Changing Pickoff Points at Runtime

```

// To avoid glitches when changing pickoff configuration:
// 1. Assert rst_n (reset)
// 2. Change pickoff_points
// 3. Wait for configuration to settle
// 4. Deassert rst_n
// 5. Outputs start with new division ratios

// The sequence spans three clocks, so it is a small FSM in the logic
// that owns
// the configuration registers. It cannot be written as one always_ff
// with
// embedded `@(posedge cfg_clk)` statements: IEEE 1800 allows exactly
// one event
// control on an always_ff and none inside its body, so that form does
// not
// compile.

```

```

typedef enum logic [1:0] {CFG_IDLE, CFG_LOAD, CFG_RELEASE}
cfg_state_t;

```

```

cfg_state_t          r_cfg_state;
logic [(N*PO_WIDTH)-1:0] r_pickoff;
logic                r_div_rst_n;

```

```

always_ff @(posedge cfg_clk or negedge cfg_rst_n) begin
    if (!cfg_rst_n) begin
        r_cfg_state <= CFG_IDLE;
        r_div_rst_n <= 1'b1;
        r_pickoff    <= DEFAULT_PICKOFF_POINTS;
    end else begin
        case (r_cfg_state)
            CFG_IDLE: if (cfg_write) begin
                r_div_rst_n <= 1'b0;           // 1. hold the divider in
reset
                r_cfg_state <= CFG_LOAD;
            end
            CFG_LOAD: begin

```

```

        r_pickoff    <= cfg_data;      // 2. safe now: outputs
are held low
        r_cfg_state <= CFG_RELEASE;
    end
    CFG_RELEASE: begin
        r_div_rst_n <= 1'b1;          // 3. restart on the new
ratios
        r_cfg_state <= CFG_IDLE;
    end
    default: r_cfg_state <= CFG_IDLE;
endcase
end
end

```

```

clock_divider #(
    .N              (N),
    .PO_WIDTH       (PO_WIDTH),
    .COUNTER_WIDTH  (COUNTER_WIDTH)
) u_div (
    .clk             (cfg_clk),
    .rst_n           (r_div_rst_n),    // divider reset is driven by the
FSM above
    .pickoff_points (r_pickoff),
    .divided_clk     (divided_clk)
);

```

Note that `divided_clk` is held low for the two cycles the FSM spends in reset; downstream logic must tolerate the gap rather than assume a continuous clock. Treat these outputs as enables into a synchronous fabric, not as clocks driving a separate domain – see the warning under Synthesis Considerations.

11.7.2 Synthesis Considerations

Resource Utilization: - **Flip-Flops:** COUNTER_WIDTH (counter) + N (output registers) - **LUTs:** one COUNTER_WIDTH-to-1 mux per output (pickoff selection), plus a clamp comparator per output. On a 6-input LUT architecture a CW-to-1 bit mux costs about $\text{ceil}((CW-1)/5)$ LUTs, so the mux cost scales with COUNTER_WIDTH – it is not a fixed overhead. clamp logic. These are analytic estimates, not synthesis results.

Timing Optimization: - **Critical Path:** Counter increment → mux → output register - **For high-frequency operation:** - Reduce COUNTER_WIDTH (if possible) - Pipeline counter output (add register stage) - Use dedicated clock resources (PLL/MMCM)

Power Optimization: - **Clock Gating:** Gate counter when no divided clocks needed - **Selective Outputs:** Only generate divided clocks actually used - **Example:**

```

logic counter_enable;
assign counter_enable = |divided_clk_enable;    // Only run when needed

```

```

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) r_divider_counters <= 0;
    else if (counter_enable) r_divider_counters <= r_divider_counters
+ 1;
end

```

11.7.3 Important Warnings

This is the part that bites people, so read it twice.

11.7.3.1 *NOT for Functional Clocks*

Do NOT use for: - Primary system clocks - Clocking synchronous logic - Production designs requiring precise timing

Use PLL/MMCM/Clock Manager instead: - Dedicated clock synthesis primitives - Clean clock tree - Proper static timing analysis (STA) - Phase-locked loops for low jitter - Precise frequency generation

11.7.3.2 *Derived Clock Hazards*

Using divided_clk as a clock creates a **derived clock**: - Complicates timing analysis - STA tools may not properly constrain - Potential setup/hold violations - Clock domain crossing issues

11.7.3.3 *Only Power-of-2 Divisions*

- This module ONLY supports power-of-2 division ratios (2, 4, 8, 16, ...)
- For arbitrary ratios (e.g., divide by 7, 100, 1000): Use counter_load_clear

11.7.3.4 *Glitches During Configuration Changes*

- Changing pickoff_points at runtime can cause glitches
- Always reset module when changing configuration
- Do NOT change configuration while outputs are actively used

11.8 Related Modules

- counter_bin.sv - Simple binary counter (related, not instantiated here — clock_divider keeps its own r_divider_counters array)
- counter_freq_invariant.sv - Time-based counter with 1MHz tick
- counter_load_clear.sv - Arbitrary division ratios (not power-of-2)
- clock_pulse.sv - Configurable pulse generator

- `clock_gate_ctrl.sv` - Clock gating for power management

11.9 Testing

11.9.1 Test Scenarios

1. **Single Output:** N=1 with various pickoff points
2. **Multiple Outputs:** N=4 with different divisions
3. **Out-of-Range Clamping:** $\text{pickoff} \geq \text{COUNTER_WIDTH}$
4. **Runtime Changes:** Modify `pickoff_points` dynamically
5. **Reset Behavior:** All outputs low after reset
6. **Division Verification:** Measure output frequencies
7. **Edge Cases:**
 - `pickoff=0` (divide by 2)
 - `pickoff=COUNTER_WIDTH-1` (slowest division)
 - `pickoff > COUNTER_WIDTH` (clamping)

11.9.2 Coverage Points

```
covergroup clock_divider_cg @(posedge clk);
  cp_pickoff_range: coverpoint pickoff_points[7:0] {
    bins low = {[0:3]};
    bins mid = {[4:11]};
    bins high = {[12:15]};
    bins oob = {[16:255]}; // Out-of-bounds (will clamp)
  }

  cp_outputs: coverpoint divided_clk {
    bins all_outputs[] = {[1:$]};
  }
endcovergroup
```

11.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

[← Back to Main Index](#) | [rtl-common Index](#)

12 Clock Gate Controller

Stops the clock automatically when a block goes idle — one of the cheapest dynamic-power wins available to you.

12.1 Overview

The `clock_gate_ctrl` module is a clock gating controller that stops the clock automatically when a circuit block goes idle, significantly reducing dynamic power consumption. You get a configurable idle timeout, immediate wake-up capability, and safe clock gating through integrated clock gate (ICG) cells.

12.2 Module Interface

```
module clock_gate_ctrl #(
    parameter int IDLE_CNTR_WIDTH = 4    // the ONLY overridable
parameter
) (
    // Inputs
    input logic          clk_in,
    input logic          aresetn,
    input logic          cfg_cg_enable,    // Global
clock gate enable
    input logic [IDLE_CNTR_WIDTH-1:0] cfg_cg_idle_count, // Idle
countdown value
    input logic          wakeup,          // Signal to
wake up the block

    // Outputs
    output logic          clk_out,
    output logic          gating          // clock gating indicator
);
    localparam int N = IDLE_CNTR_WIDTH; // derived alias, NOT a
parameter
```

12.3 Parameters

Parameter	Type	Default	Description
IDLE_CNTR_WIDTH	int	4	Bit width of the idle timeout counter. Range: 2 to 16 bits (practical range). Determines maximum idle timeout ($2^{\text{IDLE_CNTR_WIDTH}} - 1$ cycles). Power trade-

Parameter	Type	Default	Description
			off: larger counters allow longer timeouts but consume more power. This is the ONLY overridable parameter.

12.3.1 Derived Parameters

Parameter	Default	Description
N	IDLE_CNTR_WIDTH	Internal alias for the counter width, used in port/signal widths. Declared as a <code>localparam int</code> in the module body, not a parameter — set only <code>IDLE_CNTR_WIDTH</code> when instantiating. N cannot be overridden: <code>#(.N(8))</code> is an elaboration error.

12.4 Ports

12.4.1 Inputs

Port	Width	Type	Description
clk_in	1	logic	Input clock to be gated
aresetn	1	logic	Active-low asynchronous reset
cfg_cg_enable	1	logic	Global clock gating enable control
cfg_cg_idle_count	N	logic	Idle timeout value (configuration)
wakeup	1	logic	Immediate wake-up signal

12.4.2 Outputs

Port	Width	Type	Description
clk_out	1	logic	Gated output clock

Port	Width	Type	Description
gating	1	logic	Clock gating status indicator

12.5 Functional Description

12.5.1 Why Clock Gating?

Clock gating is one of the most effective power reduction techniques we have: - **Dynamic Power:** $P = C \times V^2 \times f \times \alpha$ (α = switching activity) - **Clock Networks:** Typically consume 20-40% of total chip power - **Gating Benefits:** Can reduce power by 50-90% in idle blocks - **Area Overhead:** Minimal (< 5% for typical designs)

12.5.2 Clock Gating Safety

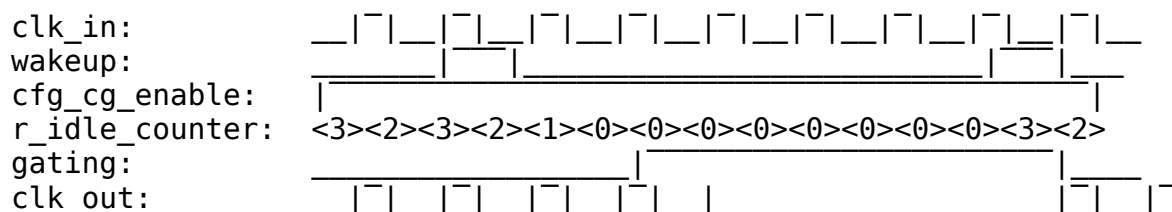
Proper clock gating requires: 1. **Glitch-Free:** No spurious clock edges during gating transitions 2. **Setup/Hold:** Adequate timing margins around gating decisions 3. **Reset Handling:** Proper behavior during reset conditions 4. **Tool Support:** Use of qualified ICG (Integrated Clock Gate) cells

12.5.3 State Machine Behavior

The controller operates as a countdown timer with three states:

1. **ACTIVE State:** Counter loaded with `cfg_cg_idle_count`
2. **COUNTDOWN State:** Counter decrements each cycle when no activity
3. **GATED State:** Counter at zero, clock is gated off

12.5.4 Timing Diagram



12.5.5 Idle Counter Logic

```

logic [N-1:0] r_idle_counter;

always_ff @(posedge clk_in or negedge aresetn) begin
    if (!aresetn) begin
        r_idle_counter <= cfg_cg_idle_count;
    end else begin

```



```

        if (wakeup || !cfg_cg_enable) begin
            // On wakeup or global disable, reset counter
            r_idle_counter <= cfg_cg_idle_count;
        end else if (r_idle_counter != 'h0) begin
            // Normal counting operation - decrement when not zero
            r_idle_counter <= r_idle_counter - 1'b1;
        end
        // When counter reaches zero, it stays at zero
    end
end

```

12.5.6 Gating Control Logic

```

wire w_gate_enable = cfg_cg_enable && !wakeup && (r_idle_counter ==
'h0);

```

```

// Instantiate the ICG cell
icg u_icg (
    .clk(clk_in),
    .en(~w_gate_enable), // Enable is active high in ICG
    .gclk(gclk_out)
);

```

```

assign gating = w_gate_enable;

```

12.5.7 The ICG Cell

```

module icg (
    input logic clk, // Input clock
    input logic en, // Enable (active high)
    output logic gclk // Gated clock output
);

    // Implementation varies by technology library
    // Typically includes:
    // - Latch to avoid glitches
    // - AND gate for gating
    // - Proper setup/hold timing

endmodule

```

A simplified conceptual implementation (the real thing is technology-specific):

```

// Simplified ICG implementation (technology-specific)
module icg_generic (
    input logic clk,
    input logic en,
    output logic gclk
);

```

```

logic en_latched;

// Latch enable on falling edge to avoid glitches
always_latch begin
    if (~clk)
        en_latched = en;
end

// Gate clock with latched enable
assign gclk = clk & en_latched;

```

endmodule

12.5.8 Technology-Specific ICG Implementations

```

// Example for TSMC technology
module icg_tsmc (
    input logic clk,
    input logic en,
    output logic gclk
);

    // Technology-specific implementation
    CKGTPBUF_X1M_A9TH u_icg (
        .CK(clk),
        .E(en),
        .GCK(gclk)
    );

```

endmodule

```

// Example for Quartus technology
module icg_quartus (
    input logic clk,
    input logic en,
    output logic gclk
);

    // Quartus FPGA implementation
    logic en_reg;

    always_ff @(negedge clk) begin
        en_reg <= en;
    end

```

```

    assign gclk = clk & en_reg;

endmodule

// Generic behavioral model
module icg_behavioral (
    input logic clk,
    input logic en,
    output logic gclk
);

    logic en_latch;

    // Latch enable on falling edge to prevent glitches
    always_latch begin
        if (!clk)
            en_latch = en;
    end

    // Gate the clock
    assign gclk = clk & en_latch;

endmodule

```

12.5.9 Timing and Performance

12.5.10 Resource Utilization

Configuration	LUTs	FFs	ICG Cells	Area Overhead
4-bit counter	8	4	1	< 2%
8-bit counter	15	8	1	< 3%
16-bit counter	30	16	1	< 5%

12.6 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

12.7 Usage Examples

12.7.1 CPU Core Clock Gating

```
module cpu_clock_gating (
    input logic clk_cpu,
    input logic rst_n,

    // CPU control signals
    input logic cpu_active,
    input logic interrupt_pending,
    input logic debug_mode,

    // Power management
    input logic [3:0] idle_threshold,
    input logic power_save_enable,

    // Output clocks
    output logic clk_cpu_gated,
    output logic clk_debug,
    output logic cpu_sleeping
);

    logic cpu_wakeup;
    logic debug_wakeup;

    // CPU wake-up conditions
    assign cpu_wakeup = cpu_active || interrupt_pending;

    // Debug interface always active in debug mode
    assign debug_wakeup = debug_mode || cpu_wakeup;

    // Main CPU clock gating
    clock_gate_ctrl #(
        .IDLE_CNTR_WIDTH(4)
    ) cpu_gate (
        .clk_in(clk_cpu),
```

```

        .aresetn(rst_n),
        .cfg_cg_enable(power_save_enable),
        .cfg_cg_idle_count(idle_threshold),
        .wakeup(cpu_wakeup),
        .clk_out(clk_cpu_gated),
        .gating(cpu_sleeping)
    );

    // Debug clock gating (shorter timeout)
    clock_gate_ctrl #(
        .IDLE_CNTR_WIDTH(3)
    ) debug_gate (
        .clk_in(clk_cpu),
        .aresetn(rst_n),
        .cfg_cg_enable(power_save_enable && !debug_mode),
        .cfg_cg_idle_count(3'd2), // Short timeout for debug
        .wakeup(debug_wakeup),
        .clk_out(clk_debug),
        .gating() // Unused
    );

```

endmodule

12.7.2 Peripheral Interface Clock Management

```

module peripheral_clock_manager (
    input logic      clk_system,
    input logic      rst_n,

    // Peripheral activity signals
    input logic      uart_active,
    input logic      spi_active,
    input logic      i2c_active,
    input logic      timer_active,

    // Configuration
    input logic [7:0] power_config, // [7:4] = enable bits, [3:0]
    = timeout

    // Gated clocks
    output logic      clk_uart,
    output logic      clk_spi,
    output logic      clk_i2c,
    output logic      clk_timer,

    // Status
    output logic [3:0] peripheral_sleeping

```

```

);

logic [3:0] idle_timeout;
logic [3:0] gate_enables;

assign idle_timeout = power_config[3:0];
assign gate_enables = power_config[7:4];

// UART clock gating
clock_gate_ctrl #(4) uart_cg (
    .clk_in(clk_system),
    .aresetn(rst_n),
    .cfg_cg_enable(gate_enables[0]),
    .cfg_cg_idle_count(idle_timeout),
    .wakeup(uart_active),
    .clk_out(clk_uart),
    .gating(peripheral_sleeping[0])
);

// SPI clock gating
clock_gate_ctrl #(4) spi_cg (
    .clk_in(clk_system),
    .aresetn(rst_n),
    .cfg_cg_enable(gate_enables[1]),
    .cfg_cg_idle_count(idle_timeout),
    .wakeup(spi_active),
    .clk_out(clk_spi),
    .gating(peripheral_sleeping[1])
);

// I2C clock gating
clock_gate_ctrl #(4) i2c_cg (
    .clk_in(clk_system),
    .aresetn(rst_n),
    .cfg_cg_enable(gate_enables[2]),
    .cfg_cg_idle_count(idle_timeout),
    .wakeup(i2c_active),
    .clk_out(clk_i2c),
    .gating(peripheral_sleeping[2])
);

// Timer clock gating (different timeout)
clock_gate_ctrl #(4) timer_cg (
    .clk_in(clk_system),
    .aresetn(rst_n),
    .cfg_cg_enable(gate_enables[3]),
    .cfg_cg_idle_count(4'd1), // Quick timeout for timers

```

```

        .wakeup(timer_active),
        .clk_out(clk_timer),
        .gating(peripheral_sleeping[3])
    );

```

endmodule

12.7.3 Memory Subsystem Clock Gating

```

module memory_clock_gating (
    input logic      clk_memory,
    input logic      rst_n,

    // Memory access signals
    input logic      mem_read,
    input logic      mem_write,
    input logic      mem_refresh_req,
    input logic      cache_miss,

    // Configuration
    input logic [2:0] mem_power_mode, // 0=always on, 7=aggressive
    gating

    // Gated clocks
    output logic      clk_mem_ctrl,    // Memory controller
    output logic      clk_mem_data,    // Data path
    output logic      clk_mem_refresh, // Refresh logic

    // Status
    output logic [2:0] mem_gating_status
);

logic [3:0] timeout_ctrl, timeout_data, timeout_refresh;
logic mem_active, data_active, refresh_active;

// Activity detection
assign mem_active = mem_read || mem_write || cache_miss;
assign data_active = mem_read || mem_write;
assign refresh_active = mem_refresh_req;

// Timeout configuration based on power mode
always_comb begin
    case (mem_power_mode)
        3'd0: {timeout_ctrl, timeout_data, timeout_refresh} =
{4'd15, 4'd15, 4'd15}; // Always on
        3'd1: {timeout_ctrl, timeout_data, timeout_refresh} =
{4'd10, 4'd8, 4'd12}; // Conservative
    
```

```

        3'd2: {timeout_ctrl, timeout_data, timeout_refresh} =
{4'd8, 4'd6, 4'd10};    // Moderate
        3'd3: {timeout_ctrl, timeout_data, timeout_refresh} =
{4'd6, 4'd4, 4'd8};    // Balanced
        3'd4: {timeout_ctrl, timeout_data, timeout_refresh} =
{4'd4, 4'd2, 4'd6};    // Aggressive
        3'd5: {timeout_ctrl, timeout_data, timeout_refresh} =
{4'd2, 4'd1, 4'd4};    // Very aggressive
        3'd6: {timeout_ctrl, timeout_data, timeout_refresh} =
{4'd1, 4'd0, 4'd2};    // Maximum power save
        3'd7: {timeout_ctrl, timeout_data, timeout_refresh} =
{4'd0, 4'd0, 4'd1};    // Ultra power save
    endcase
end

```

```

// Memory controller clock gating

```

```

clock_gate_ctrl #(4) ctrl_cg (
    .clk_in(clk_memory),
    .aresetn(rst_n),
    .cfg_cg_enable(mem_power_mode != 0),
    .cfg_cg_idle_count(timeout_ctrl),
    .wakeuper(mem_active),
    .clk_out(clk_mem_ctrl),
    .gating(mem_gating_status[0])
);

```

```

// Data path clock gating

```

```

clock_gate_ctrl #(4) data_cg (
    .clk_in(clk_memory),
    .aresetn(rst_n),
    .cfg_cg_enable(mem_power_mode != 0),
    .cfg_cg_idle_count(timeout_data),
    .wakeuper(data_active),
    .clk_out(clk_mem_data),
    .gating(mem_gating_status[1])
);

```

```

// Refresh logic clock gating

```

```

clock_gate_ctrl #(4) refresh_cg (
    .clk_in(clk_memory),
    .aresetn(rst_n),
    .cfg_cg_enable(mem_power_mode > 2), // Only gate in aggressive
modes
    .cfg_cg_idle_count(timeout_refresh),
    .wakeuper(refresh_active),
    .clk_out(clk_mem_refresh),
    .gating(mem_gating_status[2])
);

```


endmodule

12.7.4 Adaptive Clock Gating with Activity Monitoring

```
module adaptive_clock_gating #(
    parameter int ACTIVITY_WINDOW = 256 // Activity monitoring window
) (
    input logic      clk_in,
    input logic      rst_n,

    // Activity signals
    input logic [7:0] activity_vector,

    // Configuration
    input logic      adaptive_enable,
    input logic [3:0] base_timeout,

    // Outputs
    output logic      clk_out,
    output logic      gating_active,
    output logic [7:0] activity_level // Activity percentage
);

    logic [15:0] activity_counter;
    logic [7:0] activity_window_counter;
    logic [3:0] adaptive_timeout;
    logic block_active;

    // Activity monitoring
    always_ff @(posedge clk_in or negedge rst_n) begin
        if (!rst_n) begin
            activity_counter <= 0;
            activity_window_counter <= 0;
            activity_level <= 0;
        end else begin
            // Count activity in current window
            if (|activity_vector)
                activity_counter <= activity_counter + 1;

            activity_window_counter <= activity_window_counter + 1;

            // Update activity level at end of window
            if (activity_window_counter == ACTIVITY_WINDOW - 1) begin
                activity_level <= activity_counter[15:8]; // Scale to
0-255
                activity_counter <= 0;
            end
        end
    end
```

```

        activity_window_counter <= 0;
    end
end
end

// Adaptive timeout based on activity level
always_comb begin
    if (!adaptive_enable) begin
        adaptive_timeout = base_timeout;
    end else begin
        case (activity_level)
            8'h00: adaptive_timeout = base_timeout + 4'd8; // Very
low activity
            8'h01: adaptive_timeout = base_timeout + 4'd6; // Low
activity
            8'h02: adaptive_timeout = base_timeout + 4'd4; //
Medium-low activity
            8'h04: adaptive_timeout = base_timeout + 4'd2; //
Medium activity
            8'h08: adaptive_timeout = base_timeout; //
Normal activity
            default: adaptive_timeout = (base_timeout > 2) ?
base_timeout - 2 : 0; // High activity
        endcase
    end
end

// Activity detection
assign block_active = |activity_vector;

// Clock gating controller
clock_gate_ctrl #(4) adaptive_cg (
    .clk_in(clk_in),
    .aresetn(rst_n),
    .cfg_cg_enable(1'b1),
    .cfg_cg_idle_count(adaptive_timeout),
    .wakeup(block_active),
    .clk_out(clk_out),
    .gating(gating_active)
);

endmodule

```

12.7.5 Hierarchical Clock Gating System

```

module hierarchical_clock_gating (
    input logic clk_root,
    input logic rst_n,

```

```

// Global control
input logic      global_cg_enable,
input logic [3:0] global_timeout,

// Subsystem activity
input logic      cpu_active,
input logic      dsp_active,
input logic      peripheral_active,
input logic      memory_active,

// Hierarchical clocks
output logic      clk_cpu_domain,
output logic      clk_dsp_domain,
output logic      clk_peripheral_domain,
output logic      clk_memory_domain,

// Power monitoring
output logic [3:0] domain_gating_status,
output logic [15:0] power_savings_estimate
);

logic system_active;
logic clk_system_gated;
logic system_gated;

// System-level activity detection
assign system_active = cpu_active || dsp_active ||
peripheral_active || memory_active;

// Top-level system clock gating
clock_gate_ctrl #(4) system_cg (
    .clk_in(clk_root),
    .aresetn(rst_n),
    .cfg_cg_enable(global_cg_enable),
    .cfg_cg_idle_count(global_timeout + 4'd4), // Longer timeout
for system level
    .wakeup(system_active),
    .clk_out(clk_system_gated),
    .gating(system_gated)
);

// CPU domain gating
clock_gate_ctrl #(4) cpu_cg (
    .clk_in(clk_system_gated),
    .aresetn(rst_n),

```

```

        .cfg_cg_enable(global_cg_enable && !system_gated),
        .cfg_cg_idle_count(global_timeout),
        .wakeup(cpu_active),
        .clk_out(clk_cpu_domain),
        .gating(domain_gating_status[0])
    );

    // DSP domain gating
    clock_gate_ctrl #(4) dsp_cg (
        .clk_in(clk_system_gated),
        .aresetn(rst_n),
        .cfg_cg_enable(global_cg_enable && !system_gated),
        .cfg_cg_idle_count(global_timeout - 1), // Aggressive for DSP
        .wakeup(dsp_active),
        .clk_out(clk_dsp_domain),
        .gating(domain_gating_status[1])
    );

    // Peripheral domain gating
    clock_gate_ctrl #(4) peripheral_cg (
        .clk_in(clk_system_gated),
        .aresetn(rst_n),
        .cfg_cg_enable(global_cg_enable && !system_gated),
        .cfg_cg_idle_count(global_timeout + 1), // Conservative for
peripherals
        .wakeup(peripheral_active),
        .clk_out(clk_peripheral_domain),
        .gating(domain_gating_status[2])
    );

    // Memory domain gating
    clock_gate_ctrl #(4) memory_cg (
        .clk_in(clk_system_gated),
        .aresetn(rst_n),
        .cfg_cg_enable(global_cg_enable && !system_gated),
        .cfg_cg_idle_count(global_timeout + 2), // Conservative for
memory
        .wakeup(memory_active),
        .clk_out(clk_memory_domain),
        .gating(domain_gating_status[3])
    );

    // Power savings estimation (simplified)
    logic [3:0] gated_domains;
    assign gated_domains = domain_gating_status[3:0];

    always_ff @(posedge clk_root or negedge rst_n) begin

```

```

        if (!rst_n) begin
            power_savings_estimate <= 0;
        end else begin
            // Rough power savings estimate (percentage)
            case ({system_gated, gated_domains})
                5'b1_0000: power_savings_estimate <= 16'd9000; // 90%
savings
                5'b0_1111: power_savings_estimate <= 16'd7000; // 70%
savings
                5'b0_1110: power_savings_estimate <= 16'd6000; // 60%
savings
                5'b0_1100: power_savings_estimate <= 16'd5000; // 50%
savings
                5'b0_1000: power_savings_estimate <= 16'd3000; // 30%
savings
                5'b0_0100: power_savings_estimate <= 16'd2000; // 20%
savings
                5'b0_0010: power_savings_estimate <= 16'd1500; // 15%
savings
                5'b0_0001: power_savings_estimate <= 16'd1000; // 10%
savings
                default: power_savings_estimate <= 16'd0000; // 0%
savings
            endcase
        end
    end
endmodule

```

12.8 Design Notes

12.8.1 Glitch-Free Clock Switching

```

module glitch_free_clock_gate (
    input logic clk_in,
    input logic rst_n,
    input logic gate_enable,
    input logic wakeup,
    output logic clk_out
);

    logic gate_enable_sync;
    logic gate_request;

    // Synchronize gate enable to avoid glitches
    always_ff @(posedge clk_in or negedge rst_n) begin
        if (!rst_n) begin

```

```

        gate_enable_sync <= 0;
        gate_request <= 0;
    end else begin
        gate_enable_sync <= gate_enable;
        gate_request <= gate_enable_sync && !wakeup;
    end
end

```

```

// Use ICG with synchronized control
icg u_icg (
    .clk(clk_in),
    .en(~gate_request),
    .gclk(clk_out)
);

```

endmodule

12.8.2 Multi-Level Clock Gating

```

module multi_level_clock_gate #(
    parameter int LEVELS = 3
) (
    input    logic          clk_in,
    input    logic          rst_n,
    input    logic [LEVELS-1:0] level_enables,
    input    logic [LEVELS-1:0] level_wakeups,
    input    logic [4*LEVELS-1:0] level_timeouts,
    output    logic          clk_out,
    output    logic [LEVELS-1:0] level_gating_status
);

logic [LEVELS:0] clk_levels; // Include input clock
assign clk_levels[0] = clk_in;

genvar i;
generate
    for (i = 0; i < LEVELS; i++) begin : level_gates
        logic [3:0] timeout;
        assign timeout = level_timeouts[i*4+:4];

        clock_gate_ctrl #(4) level_cg (
            .clk_in(clk_levels[i]),
            .aresetn(rst_n),
            .cfg_cg_enable(level_enables[i]),
            .cfg_cg_idle_count(timeout),
            .wakeup(|level_wakeups[LEVELS-1:i]), // Wake if any
            .clk_out(clk_levels[i+1]),

```

```

        .gating(level_gating_status[i])
    );
end
endgenerate

```

```

assign clk_out = clk_levels[LEVELS];

```

```

endmodule

```

12.8.3 Measuring Power Savings

```

module power_analysis_wrapper (
    input logic clk_in,
    input logic rst_n,
    input logic cfg_cg_enable,
    input logic [3:0] cfg_cg_idle_count,
    input logic wakeup,
    output logic clk_out,
    output logic gating,
    output logic [15:0] power_savings_percent
);

    // Clock gate controller
    clock_gate_ctrl #(4) cg_ctrl (
        .clk_in(clk_in),
        .aresetn(rst_n),
        .cfg_cg_enable(cfg_cg_enable),
        .cfg_cg_idle_count(cfg_cg_idle_count),
        .wakeup(wakeup),
        .clk_out(clk_out),
        .gating(gating)
    );

    // Power monitoring
    logic [31:0] total_cycles;
    logic [31:0] gated_cycles;
    logic [31:0] ungated_cycles;

    always_ff @(posedge clk_in or negedge rst_n) begin
        if (!rst_n) begin
            total_cycles <= 0;
            gated_cycles <= 0;
            ungated_cycles <= 0;
        end else begin
            total_cycles <= total_cycles + 1;
            if (gating)
                gated_cycles <= gated_cycles + 1;
            else

```

```

        ungated_cycles <= ungated_cycles + 1;
    end
end

// Calculate power savings percentage
always_ff @(posedge clk_in) begin
    if (total_cycles > 1000) begin // Avoid division by small
numbers
        power_savings_percent <= (gated_cycles * 100) /
total_cycles;
    end else begin
        power_savings_percent <= 0;
    end
end

endmodule

```

12.8.4 Synthesis Constraints

```

# SDC constraints for clock gating
create_clock -name clk_in -period 10.0 [get_ports clk_in]

# Clock gating timing
set_case_analysis 0 [get_pins u_icg/en]
set_case_analysis 1 [get_pins u_icg/en]

# Setup/hold for gating control
set_input_delay -clock clk_in -max 2.0 [get_ports wakeup]
set_input_delay -clock clk_in -min 0.5 [get_ports wakeup]

# Clock gating cell timing
set_clock_gating_check -setup 0.2 -hold 0.1 [get_pins u_icg/en]

# UPF power intent for clock gating
create_power_domain PD_MAIN
create_power_domain PD_GATED

# Clock gating specification
set_domain_supply_net PD_MAIN -primary_power_net VDD -
primary_ground_net VSS
set_domain_supply_net PD_GATED -primary_power_net VDD -
primary_ground_net VSS

# Clock gating control
create_power_switch PS_GATED \
    -domain PD_GATED \
    -input_supply_port {vin VDD} \

```



```

-output_supply_port {vout VDD_GATED} \
-control_port {ctrl gating} \
-on_state {on vin {!ctrl}}

```

12.8.5 Design Guidelines

Timeout selection:

```

// Guidelines for timeout values
parameter TIMEOUT_GUIDELINES = {
    "CPU cores: 4-16 cycles",
    "Memory controllers: 8-32 cycles",
    "Peripherals: 16-64 cycles",
    "Debug interfaces: 2-8 cycles",
    "Always-on domains: Disable gating"
};

```

Activity signal design:

```

// Good activity signal examples
logic cpu_active;
assign cpu_active = instruction_fetch ||
                    data_access ||
                    interrupt_processing ||
                    debug_access;

// Avoid overly broad activity signals
logic bad_activity;
assign bad_activity = |all_system_signals; // Too broad

// Avoid overly narrow activity signals
logic narrow_activity;
assign narrow_activity = specific_register_write; // Too narrow

```

Hierarchical gating strategy:

```

System Level (Longest timeout)
CPU Domain (Medium timeout)
    ALU (Short timeout)
    Cache (Medium timeout)
    Debug (Very short timeout)
DSP Domain (Short timeout)
Peripheral Domain (Long timeout)
Memory Domain (Medium timeout)

```

Verification checklist: - [] Verify gating occurs after correct timeout - [] Verify immediate wakeup functionality - [] Test global enable/disable - [] Check reset behavior - [] Validate power savings measurements - [] Test all timeout configurations - [] Verify clock integrity (no glitches) - [] Check setup/hold timing margins

12.8.6 Common Applications

1. **CPU Power Management:** Core, cache, and pipeline clock gating
2. **Peripheral Controllers:** UART, SPI, I2C, timer clock gating
3. **Memory Subsystems:** Controller, data path, refresh logic gating
4. **Graphics/Video:** Pixel processing, display controller gating
5. **Communication Blocks:** Baseband, RF, protocol stack gating
6. **Test and Debug:** Scan chains, JTAG, debug interface gating
7. **AI/ML Accelerators:** Compute engine, memory interface gating

12.8.7 Troubleshooting Guide

Common Issues: 1. **Clock Glitches:** Use qualified ICG cells, proper setup/hold margins 2. **Timing Violations:** Ensure adequate timing margins for gating signals 3. **Unexpected Gating:** Check activity signal connectivity and logic 4. **Power Not Saved:** Verify clock tree reaches gated logic 5. **Reset Issues:** Ensure proper reset sequencing and distribution

Debug Techniques: 1. **Simulation:** Monitor gating signals and clock activity 2. **Power Analysis:** Use tools to measure actual power savings 3. **Timing Analysis:** Check setup/hold violations on gating paths 4. **Logic Analyzer:** Capture real-time gating behavior in hardware 5. **Power Profiling:** Measure current consumption with/without gating

12.9 Related Modules

Read out of the RTL, not curated: these are the modules this one instantiates and the modules that instantiate it.

Instantiates: - icg

Instantiated by: - amba_clock_gate_ctrl

12.10 Testing

12.10.1 Test Bench

```
module tb_clock_gate_ctrl;

    parameter IDLE_CNTR_WIDTH = 4;

    logic clk_in, aresetn;
```

```

logic cfg_cg_enable;
logic [IDLE_CNTR_WIDTH-1:0] cfg_cg_idle_count;
logic wakeup;
logic clk_out, gating;

// Clock generation
initial clk_in = 0;
always #5ns clk_in = ~clk_in; // 100MHz

// DUT instantiation
clock_gate_ctrl #(
    .IDLE_CNTR_WIDTH(IDLE_CNTR_WIDTH)
) dut (.*);

// Test sequence
initial begin
    // Initialize
    aresetn = 0;
    cfg_cg_enable = 0;
    cfg_cg_idle_count = 4'd5;
    wakeup = 0;

    #100ns aresetn = 1;

    // Test basic gating
    test_basic_gating();

    // Test immediate wakeup
    test_immediate_wakeup();

    // Test global disable
    test_global_disable();

    // Test different timeouts
    test_timeout_values();

    $display("All tests completed!");
    $finish;
end

// Test basic clock gating functionality
task test_basic_gating();
    begin
        $display("Testing basic clock gating...");
    end

```

```

    cfg_cg_enable = 1;
    cfg_cg_idle_count = 4'd3;

    // Wait for gating to occur
    wait_for_gating();

    // Verify clock is gated
    check_clock_gated();

    // Wake up and verify clock restoration
    wakeup = 1;
    #20ns wakeup = 0;
    check_clock_active();
end
endtask

// Wait for clock gating to occur
task wait_for_gating();
    int timeout_cycles;
    begin
        timeout_cycles = 0;
        while (!gating && timeout_cycles < 100) begin
            @(posedge clk_in);
            timeout_cycles++;
        end

        if (!gating) begin
            $error("Clock gating did not occur within timeout");
        end else begin
            $display("Clock gating occurred after %d cycles",
timeout_cycles);
        end
    end
endtask

// Check that clock is properly gated
task check_clock_gated();
    logic prev_clk_out;
    int stable_cycles;
    begin
        prev_clk_out = clk_out;
        stable_cycles = 0;

        repeat (10) begin
            @(posedge clk_in);
            if (clk_out == prev_clk_out) begin

```

```

        stable_cycles++;
    end else begin
        stable_cycles = 0;
        prev_clk_out = clk_out;
    end
end

    if (stable_cycles >= 5) begin
        $display("PASS: Clock is properly gated");
    end else begin
        $error("FAIL: Clock is not properly gated");
    end
end
endtask

// Check that clock is active
task check_clock_active();
    int edge_count;
    begin
        edge_count = 0;

        repeat (20) begin
            @(posedge clk_in);
            if ($rose(clk_out)) edge_count++;
        end

        if (edge_count >= 8) begin
            $display("PASS: Clock is active after wakeup");
        end else begin
            $error("FAIL: Clock not properly restored after
wakeup");
        end
    end
endtask

endmodule

```

12.10.2 Coverage Model

```

covergroup clock_gate_cg @(posedge clk_in);

    cp_idle_count: coverpoint cfg_cg_idle_count {
        bins short_timeout[] = {[0:3]};
        bins medium_timeout[] = {[4:10]};
        bins long_timeout[] = {[11:15]};
    }

```

```

cp_counter_values: coverpoint dut.r_idle_counter {
    bins countdown[] = {[0:15]};
    bins zero = {0};
}

cp_gating_enable: coverpoint cfg_cg_enable {
    bins enabled = {1};
    bins disabled = {0};
}

cp_wakeup: coverpoint wakeup {
    bins active = {1};
    bins inactive = {0};
}

cp_gating_status: coverpoint gating {
    bins gated = {1};
    bins ungated = {0};
}

// Cross coverage
cross_timeout_gating: cross cp_idle_count, cp_gating_status;
cross_wakeup_gating: cross cp_wakeup, cp_gating_status;

```

endcovergroup

12.10.3 Formal Verification Properties

module clock_gate_properties;

```

// Bind to DUT
bind clock_gate_ctrl clock_gate_properties props_inst (.*);

// Property: Immediate wakeup
property immediate_wakeup;
    @(posedge clk_in) disable iff (!aresetn)
    wakeup |-> ##1 !gating;
endproperty

// Property: Gating after timeout
property gating_after_timeout;
    @(posedge clk_in) disable iff (!aresetn)
    cfg_cg_enable && !wakeup && (dut.r_idle_counter == 0) |->
gating;
endproperty

// Property: Counter behavior

```

```

property counter_countdown;
    @(posedge clk_in) disable iff (!aresetn)
    cfg_cg_enable && !wakeup && (dut.r_idle_counter > 0) | =>
    (dut.r_idle_counter == $past(dut.r_idle_counter) - 1);
endproperty

// Property: Global disable
property global_disable;
    @(posedge clk_in) disable iff (!aresetn)
    !cfg_cg_enable | -> !gating;
endproperty

// Assertions
assert property (immediate_wakeup);
assert property (gating_after_timeout);
assert property (counter_countdown);
assert property (global_disable);

```

endmodule

Done right, the clock gate controller is essential power management: significant dynamic power reduction for minimal area overhead and design complexity. Getting it right means careful attention to timing, safety, and verification — that's what buys you reliable operation and the full power savings.

12.11 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

13 clock_pulse

13.1 Overview

The `clock_pulse` module is a periodic pulse generator with a configurable period: it emits a single-cycle pulse every `WIDTH` clock cycles. It's the part you reach for whenever something in the system needs to happen on a schedule — timing generation, heartbeat signals, sampling triggers, periodic events of any kind.

13.2 Parameters

Parameter	Type	Default	Description
<code>WIDTH</code>	<code>int</code>	<code>10</code>	Period of the pulse generation in clock cycles

`WIDTH` can range from 2 to $2^{31}-1$. The parameter is declared `int`, which is 32-bit SIGNED, so $2^{32}-1$ is not representable. It sets the pulse frequency ($f_{\text{clk}} / \text{WIDTH}$). The pulse itself is always exactly 1 clock cycle wide, which makes the duty cycle $1/\text{WIDTH}$ — 10% for `WIDTH=10`.

13.3 Ports

```
module clock_pulse #(
    parameter int WIDTH = 10 // PERIOD in clock cycles (the pulse
                             // always exactly 1 cycle wide)
    itself is
) (
    input logic clk, // Input clock signal
    input logic rst_n, // Input reset signal
    output logic pulse // Output pulse signal
);
```

Port	Direction	Width	Description
<code>clk</code>	Input	1	System clock input
<code>rst_n</code>	Input	1	Active-low asynchronous reset
<code>pulse</code>	Output	1	Periodic pulse output

13.4 Functional Description

13.4.1 Internal Counter

```
// WIDTH is the PERIOD; the counter only needs to hold 0..WIDTH-1,
// i.e.
// $clog2(WIDTH) bits (NOT WIDTH bits).
localparam int CW = (WIDTH < 2) ? 1 : $clog2(WIDTH);

logic [CW-1:0] r_counter;
logic [CW-1:0] w_width_minus_one;

// Properly sized period-1 constant
assign w_width_minus_one = CW'(WIDTH - 1);
```

13.4.2 Core Logic

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_counter <= 'b0;
        pulse      <= 'b0;
    end else begin
        if (r_counter < w_width_minus_one)
            r_counter <= r_counter + 1'b1;
        else
            r_counter <= 'b0;

        pulse <= (r_counter == w_width_minus_one);
    end
end
```

13.4.3 Operation Principles

1. **Counter:** Free-running counter from 0 to WIDTH-1
2. **Pulse Generation:** pulse is **registered** — pulse <= (r_counter == WIDTH-1) — so it asserts on the cycle **after** the counter reaches WIDTH-1, i.e. during the cycle when the counter has wrapped back to 0.
3. **Auto-Reset:** Counter wraps to 0 after reaching maximum
4. **Synchronous:** All operations synchronized to input clock
5. **Single Cycle:** Pulse duration is exactly one clock cycle

That second point is the one people get wrong, so let's be blunt about it: the registered comparison means pulse never coincides with the terminal count. It lands one cycle later, on the wrapped-to-zero count. Every timing diagram below follows from that.

13.5 Related Modules

Nothing in the tree instantiates this module and it instantiates nothing: it is a leaf, used directly by whatever design needs it. Its nearest neighbours in `rtl/common/` are:

- `clock_divider`
- `clock_gate_ctrl`

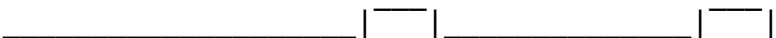
13.6 Timing Characteristics

13.6.1 Timing Characteristics

- **Period:** WIDTH clock cycles
- **Frequency:** $f_{\text{clk}} / \text{WIDTH}$
- **Pulse Width:** 1 clock cycle
- **Duty Cycle:** $1/\text{WIDTH}$
- **Phase:** Because the comparison is registered, pulse is high one cycle **after** `r_counter == WIDTH-1` — that is, during the `r_counter == 0` cycle of each period, not on the WIDTH-1 count itself.

13.6.2 Basic Operation (WIDTH=4)

Watch where pulse actually lands — high during the `< 0 >` cell that immediately follows `< 3 >`, one cycle after the counter hit WIDTH-1. That's the registered comparison at work:

Clock: 
Counter: `< 0 > < 1 > < 2 > < 3 > < 0 > < 1 > < 2 > < 3 > < 0 >`
Pulse: 

(The first `< 0 >` is the post-reset state, where pulse is still low.)

13.6.3 Reset Behavior

Clock: 
Reset_n: 
Counter: `< 0 > < 1 > < 2 > < 3 > < 0 > < 0 > < 1 > < 2 > < 3 > < 0 >`
Pulse: 

Two things worth seeing here, both consequences of the pulse being registered (pulse `<= (r_counter == w_width_minus_one)`):

- **The pulse that reset swallows.** The counter reaches 3 in the fourth cell, so a pulse would normally appear in the fifth — but reset is asserted exactly there, and reset forces pulse ≤ 0 . That pulse never happens.
- **Recovery costs a full period.** After reset releases, the counter restarts from 0 and the first pulse appears in the cell *after* it next reaches 3 — the final cell above, not the one where the counter reads 3.

13.6.4 Different WIDTH Values

WIDTH	Period	Duty Cycle	Use Case
2	2 cycles	50%	Clock divider
4	4 cycles	25%	Sampling
10	10 cycles	10%	Timing reference
100	100 cycles	1%	Heartbeat
1000	1000 cycles	0.1%	Slow events

13.7 Usage Examples

Every parameter and port below is read from the module declaration.

```
clock_pulse #(
    .WIDTH          (10)
) u_clock_pulse (
    .clk            (clk),
    .rst_n          (rst_n),
    .pulse          (pulse)
);
```

13.8 Design Notes

13.8.1 Resource Utilization

WIDTH	Counter Bits	LUTs	FFs	Max Freq
8	3	8	4	500MHz
16	4	12	5	450MHz
32	5	18	6	400MHz
1024	10	28	11	350MHz

13.8.2 Synthesis Considerations

For high-frequency applications, pipeline the comparison:

```
// For high-frequency applications, pipeline the comparison (WIDTH >=  
2).  
// IMPORTANT: because the registered compare_result ALSO gates the  
counter  
// wrap, it must be produced from the WIDTH-2 threshold, not WIDTH-1.  
Comparing  
// against WIDTH-1 here makes the counter sequence 0..WIDTH,0 and  
yields a  
// period of WIDTH+1, not WIDTH. With WIDTH-2 the period is exactly  
WIDTH and  
// the pulse still lands on the r_counter==0 cycle, matching the base  
module.  
module clock_pulse_pipelined #(  
    parameter int WIDTH = 1000  
) (  
    input    logic clk,  
    input    logic rst_n,  
    output logic pulse  
) ;  
  
    localparam int CW = (WIDTH < 2) ? 1 : $clog2(WIDTH);  
    logic [CW-1:0] r_counter;  
    logic          compare_result;  
  
    // Register the comparison one count early so the pipelined wrap  
keeps a  
// WIDTH-cycle period.  
    always_ff @(posedge clk or negedge rst_n) begin  
        if (!rst_n) compare_result <= 1'b0;  
        else          compare_result <= (r_counter == CW'(WIDTH - 2));  
    end  
  
    always_ff @(posedge clk or negedge rst_n) begin  
        if (!rst_n) begin  
            r_counter <= 'b0;  
            pulse      <= 1'b0;  
        end else begin  
            r_counter <= compare_result ? '0 : r_counter + 1'b1;  
            pulse      <= compare_result;  
        end  
    end  
  
endmodule
```

13.8.3 Design Considerations

1. **Choose Appropriate WIDTH:** Balance between resolution and resource usage
2. **Consider Reset Timing:** Ensure clean startup behavior
3. **Validate Timing:** Verify pulse frequency matches requirements
4. **Plan for Variation:** Consider process, voltage, temperature effects
5. **Monitor Resource Usage:** Large counters can impact timing and area
6. **Use Enables:** Add enable signals for conditional operation
7. **Document Timing:** Clearly specify pulse rates and relationships

13.8.4 Common Applications

1. **Timing References:** System heartbeats, watchdog timers
2. **Sampling Systems:** ADC triggers, data acquisition timing
3. **Communication:** Baud rate generation, protocol timing
4. **Memory Systems:** Refresh timing, access scheduling
5. **Test Equipment:** Pattern generation, stimulus timing
6. **Power Management:** Activity monitoring, timeout generation
7. **Display Systems:** Refresh rates, sync generation

That's the whole module. It's a small thing, but precise periodic timing shows up in nearly every design you'll ever ship — get this one right once, parameterize it well, and reuse it forever.

13.9 Testing

13.9.1 Comprehensive Test Bench

```
module tb_clock_pulse;

    parameter WIDTH = 10;

    logic clk, rst_n, pulse;

    // Clock generation
```

```

initial clk = 0;
always #5ns clk = ~clk; // 100MHz

// DUT instantiation
clock_pulse #(.WIDTH(WIDTH)) dut (.*);

// Test sequence
initial begin
    rst_n = 0;
    #100ns rst_n = 1;

    // Test basic pulse generation
    test_basic_pulse_generation();

    // Test reset behavior
    test_reset_behavior();

    // Test timing accuracy
    test_timing_accuracy();

    $display("All tests completed!");
    $finish;
end

// Test basic pulse generation
task test_basic_pulse_generation();
    int pulse_count = 0;
    int cycle_count = 0;
    begin
        $display("Testing basic pulse generation (WIDTH=%0d)...",
WIDTH);

        // Monitor for several pulse periods. WIDTH*3 + 1 edges:
        `@(posedge clk)`
        // resumes BEFORE the NBA update, so pulses are first
        visible at edges
        // WIDTH+1, 2*WIDTH+1, 3*WIDTH+1 (see test_reset_behavior
        for the trace).
        repeat (WIDTH * 3 + 1) begin
            @(posedge clk);
            cycle_count++;
            if (pulse) pulse_count++;
        end

        // Verify pulse count
        if (pulse_count == 3) begin

```

```

        $display("PASS: Correct number of pulses generated");
    end else begin
        $error("FAIL: Expected 3 pulses, got %0d",
pulse_count);
    end
end
endtask

// Test reset behavior -- the two consequences of the registered
pulse
task test_reset_behavior();
    int pulses_during_reset = 0;
    int cycles_to_first_pulse = 0;
    begin
        $display("Testing reset behavior...");

        // Line the counter up so a pulse is due, then assert
reset in
        // exactly that cycle. Reset forces pulse <= 0, so that
pulse is
        // swallowed and never appears.
        repeat (WIDTH - 1) @(posedge clk);
        rst_n = 0;
        repeat (WIDTH * 2) begin
            @(posedge clk);
            if (pulse) pulses_during_reset++;
        end
        if (pulses_during_reset == 0) begin
            $display("PASS: no pulse while reset is asserted");
        end else begin
            $error("FAIL: %0d pulse(s) during reset",
pulses_during_reset);
        end

        // Recovery costs a full period: the counter restarts from
0, so the
        // first pulse lands one cycle AFTER it next reaches
WIDTH-1.
        @(negedge clk);
        rst_n = 1;
        // Expect the first pulse on the (WIDTH+1)th sampled edge,
not the
        // WIDTHth. Three separate cycles are involved and it is
easy to
        // lose one: the counter reaches WIDTH-1 at edge WIDTH-1,
`pulse`
        // is NBA-assigned at edge WIDTH, and `@(posedge clk)`
resumes in

```

```

so the          // the Active region -- BEFORE that edge's NBA update --
                // high value is first visible at edge WIDTH+1.
                forever begin
                    @(posedge clk);
                    cycles_to_first_pulse++;
                    if (pulse) break;
                    if (cycles_to_first_pulse > WIDTH * 2) begin
                        $error("FAIL: no pulse within two periods of reset
release");
                    end
                    break;
                end
            end
            if (cycles_to_first_pulse == WIDTH + 1) begin
                $display("PASS: first pulse %0d sampled edges after
release",
                        WIDTH + 1);
            end else begin
                $error("FAIL: expected first pulse at cycle %0d, got
%0d",
                        WIDTH + 1, cycles_to_first_pulse);
            end
        end
    endtask

    // Test timing accuracy
    task test_timing_accuracy();
        time pulse_times[10];
        time pulse_periods[9];
        int i;
        real avg_period, expected_period;
        begin
            $display("Testing timing accuracy...");

            expected_period = WIDTH * 10.0; // 10ns clock period

            // Capture pulse timing
            for (i = 0; i < 10; i++) begin
                @(posedge pulse);
                pulse_times[i] = $time;
            end

            // Calculate periods
            for (i = 0; i < 9; i++) begin
                pulse_periods[i] = pulse_times[i+1] - pulse_times[i];
            end
        end
    endtask

```



```

    // Calculate average period
    avg_period = 0;
    for (i = 0; i < 9; i++) begin
        avg_period += pulse_periods[i];
    end
    avg_period = avg_period / 9.0;

    // Check accuracy (within 1% tolerance)
    if (abs(avg_period - expected_period) < expected_period *
0.01) begin
        $display("PASS: Average period = %.1f ns (expected
%.1f ns)",
                avg_period, expected_period);
    end else begin
        $error("FAIL: Average period = %.1f ns (expected %.1f
ns)",
                avg_period, expected_period);
    end
end
endtask

function real abs(real value);
    abs = (value >= 0) ? value : -value;
endfunction

endmodule

```

13.9.2 Coverage Model

```

covergroup clock_pulse_cg @(posedge clk);

    cp_counter_values: coverpoint dut.r_counter {
        bins zero = {0};
        bins low[] = {[1:WIDTH/4]};
        bins mid[] = {[WIDTH/4+1:3*WIDTH/4]};
        bins high[] = {[3*WIDTH/4+1:WIDTH-2]};
        bins max = {WIDTH-1};
    }

    cp_pulse: coverpoint pulse {
        bins asserted = {1};
        bins deasserted = {0};
    }

    cp_reset: coverpoint rst_n {
        bins reset = {0};
        bins normal = {1};
    }

```

```

}

// Transition coverage
cp_pulse_edges: coverpoint pulse {
    bins rising = (0 ==> 1);
    bins falling = (1 ==> 0);
    bins stable_low = (0 ==> 0);
    bins stable_high = (1 ==> 1);
}

// Cross coverage
cross_counter_pulse: cross cp_counter_values, cp_pulse;

```

endcovergroup

13.9.3 Formal Properties

module clock_pulse_properties;

```

// Bind to DUT
bind clock_pulse clock_pulse_properties props_inst (.*)

// Property: reaching max count produces a pulse on the NEXT
cycle.
// The comparison is registered (pulse <= r_counter == WIDTH-1),
so the
// implication is |==> (next cycle), NOT |-> (same cycle).
property pulse_after_max_count;
    @(posedge clk) disable iff (!rst_n)
    (dut.r_counter == WIDTH-1) |==> pulse;
endproperty

// Property: a pulse implies the PREVIOUS count was max
property pulse_implies_prev_max;
    @(posedge clk) disable iff (!rst_n)
    pulse |-> $past(dut.r_counter == WIDTH-1);
endproperty

// Property: Counter wraps correctly
property counter_wrap;
    @(posedge clk) disable iff (!rst_n)
    (dut.r_counter == WIDTH-1) |==> (dut.r_counter == 0);
endproperty

// Property: Counter increments
property counter_increment;
    @(posedge clk) disable iff (!rst_n)

```

```

        (dut.r_counter < WIDTH-1) | =>
        (dut.r_counter == $past(dut.r_counter) + 1);
    endproperty

    // Property: Reset behavior
    property reset_behavior;
        @(posedge clk)
        !rst_n | => (dut.r_counter == 0) && !pulse;
    endproperty

    // Assertions
    assert property (pulse_after_max_count);
    assert property (pulse_implies_prev_max);
    assert property (counter_wrap);
    assert property (counter_increment);
    assert property (reset_behavior);

endmodule

```

13.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

14 Binary Counter with Load and Variable Increment

14.1 Overview

The `counter_bin_load` module takes basic binary counting and adds three distinct operating modes: standard +1 increment, variable-amount increment, and direct load. It's built specifically for FIFO pointer management, where drop/flush operations need to jump the read pointer by arbitrary amounts. The wraparound is FIFO-style — at $2 \times \text{MAX}$, with an MSB toggle for full/empty detection. If you've ever had to discard a burst of FIFO entries mid-stream, this is the counter that makes it painless.

14.2 Module Interface

```
module counter_bin_load #(
    parameter int WIDTH = 5,      // Counter bit width
    parameter int MAX = 10       // Maximum count value (wraps at MAX-
1)
) (
    // Derived localparam (computed internally - not user-settable):
    // localparam int WRAP_BOUNDARY = 2 * MAX; // Wraparound boundary

    input logic clk,              // Clock input
    input logic rst_n,            // Active-low
asynchronous reset
    input logic enable,           // Count +1 enable
    input logic add_enable,       // Variable increment
enable
    input logic [WIDTH-1:0] add_value, // Value to add
    input logic load,             // Load enable
(highest priority)
    input logic [WIDTH-1:0] load_value, // Value to load
    output logic [WIDTH-1:0] counter_bin_curr, // Current count
(register)
    output logic [WIDTH-1:0] counter_bin_next // Next count
(combination)
);
```

14.3 Parameters

14.3.1 User-Settable Parameters

Parameter	Type	Default	Description
WIDTH	int	5	Counter bit width (range: 2-64)
MAX	int	10	Maximum count value

Parameter	Type	Default	Description
			before wraparound (range: 2 to $2^{(WIDTH-1)}$)

WIDTH Guidelines: - **Minimum:** 2 bits (required for MSB inversion behavior) - **Typical:** FIFO address width + 1 (e.g., 4 for 8-entry FIFO: 3 addr + 1 MSB) - **Maximum:** 64 bits (practical limit for synthesis)

MAX Constraints: - Must fit within WIDTH-1 bits (excludes MSB) - Example: WIDTH=4 → MAX ≤ 8 (3 bits for addresses 0-7, 1 MSB for full/empty) - FIFO depth is typically MAX (e.g., MAX=8 → 8-entry FIFO)

14.3.2 Derived Localparam (Computed Internally)

Localparam	Computation	Description
WRAP_BOUNDARY	$2 \times \text{MAX}$	Wraparound boundary for add operations

Note: WRAP_BOUNDARY is computed internally and cannot be overridden by users.

14.4 Ports

14.4.1 Inputs

Port	Width	Description
clk	1	System clock (rising edge active)
rst_n	1	Active-low asynchronous reset
enable	1	Standard +1 increment enable
add_enable	1	Variable increment enable
add_value	WIDTH	Value to add when add_enable=1
load	1	Load enable (highest priority operation)
load_value	WIDTH	Value to load when load=1

14.4.2 Outputs

Port	Width	Description
counter_bin_curr	WIDTH	Current counter value (registered output)
counter_bin_next	WIDTH	Next counter value (combinational preview)

14.5 Functional Description

14.5.1 Operation Priority

The module implements a three-level priority hierarchy:

1. **Load Operation** (Highest Priority): `load = 1`
 - Directly sets `counter_bin_next = load_value`
 - Used for: Flush all, jump to specific position
2. **Variable Increment**: `add_enable = 1` (when `load = 0`)
 - Adds `add_value` to current count: `counter_bin_next = counter_bin_curr + add_value`
 - Wraps at $2 \times \text{MAX}$: Subtracts $2 \times \text{MAX}$ if $\text{sum} \geq 2 \times \text{MAX}$
 - Used for: Drop multiple entries, bulk operations
3. **Standard Increment**: `enable = 1` (when `load = 0` and `add_enable = 0`)
 - Adds 1 to current count: `counter_bin_next = counter_bin_curr + 1`
 - Special FIFO wraparound: MSB toggles at $\text{MAX}-1$, lower bits clear
 - Used for: Normal FIFO read/write pointer increment
4. **Hold**: All enables = 0
 - Maintains current value: `counter_bin_next = counter_bin_curr`

14.5.2 FIFO-Optimized Wraparound

Standard Increment Mode (enable=1):

For MAX=8, WIDTH=4:

Count sequence: 0, 1, 2, 3, 4, 5, 6, 7, 8 (toggle MSB, clear lower)

Binary: 0000 → 0001 → ... → 0111 → 1000 (MSB inverts, lower bits = 0)

0000 → 0001 → ... → 0111 → 1000 → 1001 → ... → 1111 → 0000

Key Feature: MSB toggles when lower bits reach MAX-1 - This creates a $2 \times \text{MAX}$ count range: 0 to $(2 \times \text{MAX} - 1)$ - MSB different between write/read pointers → FIFO full or empty

Variable Increment Mode (add_enable=1):

Wraparound at $2 \times \text{MAX}$:

Example: counter_bin_curr = 14, add_value = 5, MAX = 8, WRAP_BOUNDARY = 16

Sum = 14 + 5 = 19 $\geq$ 16 → counter_bin_next = 19 - 16 = 3

Key Feature: Full 0 to $(2 \times \text{MAX} - 1)$ range with modulo arithmetic

14.5.3 Full/Empty Detection Pattern

// In FIFO controller:

logic full, empty;

assign full = (wr_ptr[WIDTH-2:0] == rd_ptr[WIDTH-2:0]) &&
(wr_ptr[WIDTH-1] != rd_ptr[WIDTH-1]); *// MSB different*

assign empty = (wr_ptr[WIDTH-2:0] == rd_ptr[WIDTH-2:0]) &&
(wr_ptr[WIDTH-1] == rd_ptr[WIDTH-1]); *// MSB same*

Why This Works: - Normal operation: wr_ptr advances, MSBs eventually differ → full - After wraparound: rd_ptr catches up, MSBs match again → empty - The MSB does lap-counter duty for the circular buffer — it's how you tell “wrapped, so full” apart from “caught up, so empty.”

14.5.4 Priority Logic

always_comb begin

counter_bin_next = counter_bin_curr; *// Default: hold*

if (load) **begin**

// Highest priority: Direct load

counter_bin_next = load_value;

end else if (add_enable) **begin**

*// Variable increment with wraparound at 2*MAX*

w_sum_ext = {1'b0, counter_bin_curr} + {1'b0, add_value};

if (w_sum_ext >= WRAP_BOUNDARY) **begin**

counter_bin_next = w_sum_ext[WIDTH-1:0] -
WIDTH'(WRAP_BOUNDARY);

```

    end else begin
        counter_bin_next = w_sum_ext[WIDTH-1:0];
    end

    end else if (enable) begin
        // Standard +1 with MSB toggle at MAX-1
        if (counter_bin_curr[WIDTH-2:0] == (MAX - 1)) begin
            counter_bin_next = {~counter_bin_curr[WIDTH-1], {(WIDTH-1)
{1'b0}}}};
        end else begin
            counter_bin_next = counter_bin_curr + 1;
        end
    end
    // else: hold (default assignment)
end

```

14.5.5 Registered Output

```

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        counter_bin_curr <= 'b0; // Reset to 0
    else
        counter_bin_curr <= counter_bin_next;
end

```

Reset Behavior: Counter initializes to 0 on `rst_n = 0`

14.5.6 Combinational Next-Value Preview

The `counter_bin_next` output gives you a one-cycle lookahead of the counter value. Handy for: -
FIFO full/empty prediction - Pre-computing next address - Pipelined operations

14.6 Usage Examples

14.6.1 FIFO Read Pointer with Drop Support

```

// 8-entry FIFO with drop capability
localparam int FIFO_DEPTH = 8;
localparam int PTR_WIDTH = $clog2(FIFO_DEPTH) + 1; // +1 for MSB

logic [PTR_WIDTH-1:0] rd_ptr, rd_ptr_next;
logic rd_en, drop_valid;
logic [PTR_WIDTH-1:0] drop_count;

counter_bin_load #(
    .WIDTH(PTR_WIDTH), // 4 bits: 3 for addr (0-7) + 1 MSB
    .MAX(FIFO_DEPTH) // Wrap at 8
) u_rd_ptr (
    .clk (clk),

```



```

        .rst_n          (rst_n),
        .enable         (rd_en & !fifo_empty),           // Normal read
        .add_enable     (drop_valid),                     // Drop multiple
        .add_value      (drop_count),                     // How many to
drop
        .load           (flush_all),                       // Flush operation
        .load_value     (wr_ptr),                         // Jump to write
ptr
        .counter_bin_curr (rd_ptr),
        .counter_bin_next (rd_ptr_next)
    );

```

```

// Use lower bits for memory address
assign rd_addr = rd_ptr[PTR_WIDTH-2:0];

```

14.6.2 FIFO Write Pointer

```

logic [PTR_WIDTH-1:0] wr_ptr, wr_ptr_next;
logic wr_en;

```

```

counter_bin_load #(
    .WIDTH(PTR_WIDTH),
    .MAX(FIFO_DEPTH)
) u_wr_ptr (
    .clk          (clk),
    .rst_n       (rst_n),
    .enable      (wr_en & !fifo_full),
    .add_enable   (1'b0),           // No variable increment
for write
    .add_value    ('0),
    .load        (flush_all),       // Reset to read pointer
    .load_value  (rd_ptr),
    .counter_bin_curr (wr_ptr),
    .counter_bin_next (wr_ptr_next)
);

```

```

assign wr_addr = wr_ptr[PTR_WIDTH-2:0];

```

14.6.3 Full/Empty Status Logic

```

// Full: Lower bits match, MSB different
assign fifo_full = (wr_ptr[PTR_WIDTH-2:0] == rd_ptr[PTR_WIDTH-2:0]) &&
    (wr_ptr[PTR_WIDTH-1] != rd_ptr[PTR_WIDTH-1]);

// Empty: All bits match
assign fifo_empty = (wr_ptr == rd_ptr);

// Count calculation (handles wraparound)

```

```

logic [PTR_WIDTH-1:0] fifo_count;
assign fifo_count = (wr_ptr >= rd_ptr) ? (wr_ptr - rd_ptr) :
                                     (2*FIFO_DEPTH - rd_ptr +
wr_ptr);

```

14.6.4 Packet Drop Controller

```

// Drop N packets from FIFO
logic [7:0] packets_to_drop;
logic drop_start;

// State machine for multi-cycle drop
typedef enum logic [1:0] {
    IDLE,
    DROPPING
} drop_state_t;
drop_state_t drop_state;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        drop_state <= IDLE;
        drop_valid <= 1'b0;
        drop_count <= '0;
    end else begin
        case (drop_state)
            IDLE: begin
                if (drop_start && packets_to_drop > 0) begin
                    drop_valid <= 1'b1;
                    drop_count <= packets_to_drop;
                    drop_state <= DROPPING;
                end
            end

            DROPPING: begin
                drop_valid <= 1'b0; // Single-cycle pulse
                drop_state <= IDLE;
            end
        endcase
    end
end

// Connect to counter
counter_bin_load #(
    .WIDTH(PTR_WIDTH),
    .MAX(FIFO_DEPTH)
) u_rd_ptr (
    .clk      (clk),
    .rst_n    (rst_n),

```

```

        .enable      (rd_en & !fifo_empty),
        .add_enable  (drop_valid),
        .add_value   (drop_count),
        .load        (1'b0),
        .load_value  ('0),
        .counter_bin_curr(rd_ptr),
        .counter_bin_next(rd_ptr_next)
    );

```

14.7 Timing Characteristics

14.7.1 Latency and Throughput

Property	Value
Increment latency	1 cycle (registered output)
Next-value availability	0 cycles (combinational counter_bin_next)
Throughput	1 operation per cycle
Reset recovery	1 cycle (asynchronous reset)

14.7.2 Operation Timing Diagrams

Standard Increment (enable=1):

Cycle:	0	1	2	3	4	5	6	7	8	9
clk:										
enable:										
curr:	0000	0000	0001	0010	0011	0100	0101	0110	0111	
next:	0000	0001	0010	0011	0100	0101	0110	0111		

Variable Increment (add_enable=1, add_value=3):

Cycle:	0	1	2	3
clk:				
add_en:				
add_val:				
curr:	0010	0010	0101	
next:	0010	0101	0101	

↑ +3 ↑ add_en low -> hold (next = curr)

Load Operation (load=1, load_value=12):

Cycle:	0	1	2
clk:			
load:			
load_val:			
curr:	0011	0011	1100

```
next:      0011  1100  1100
           ↑ load  ↑ load low -> hold (next = curr)
```

14.7.3 Resource Utilization

Configuration	FFs	LUTs	Description
WIDTH=4, MAX=8	4	~12	Minimal (8-entry FIFO)
WIDTH=5, MAX=16	5	~16	Small (16-entry FIFO)
WIDTH=9, MAX=256	9	~30	Medium (256-entry FIFO)
WIDTH=13, MAX=4096	13	~45	Large (4K-entry FIFO)

Area Scaling: Approximately linear with WIDTH due to adder and comparator

14.7.4 Maximum Frequency

- **Typical:** 300-500 MHz (modern FPGAs)
- **Limiting Factor:** Add operation with wraparound detection
- **Optimization:** Pipeline add path for >500 MHz

14.8 Testing

Test suite location: `val/common/test_counter_bin_load.py`

Key Test Scenarios: - Standard increment (enable=1) - Variable increment with various `add_value` - Load operation priority verification - Wraparound behavior at $2 \times \text{MAX}$ - MSB toggle at $\text{MAX}-1$ for standard increment - Full/empty condition verification - Priority hierarchy (load > add > enable) - Reset behavior

Test Command:

```
pytest val/common/test_counter_bin_load.py -v
```

14.9 Design Notes

14.9.1 When to Use `counter_bin_load`

Appropriate Use Cases: - FIFO read/write pointers with drop/flush - Circular buffer management with skip operations - Packet queue pointers with bulk discard - Ring buffer pointers with reset capability - Any counter needing jump-to-position capability

14.9.2 When to Use Alternatives

Use counter_bin.sv if: - Only need standard +1 increment - No load or variable increment required - Simpler interface preferred

Use counter_load_clear.sv if: - Need load/clear but not FIFO wraparound - Don't need MSB toggle behavior - Variable increment not required

14.9.3 Parameter Selection Guidelines

WIDTH Calculation:

```
// For FIFO with N entries:
localparam int FIFO_DEPTH = 16;
localparam int PTR_WIDTH = $clog2(FIFO_DEPTH) + 1; // +1 for MSB

// Example: 16-entry FIFO
// FIFO_DEPTH = 16
// PTR_WIDTH = 4 + 1 = 5 bits
// Lower 4 bits = address (0-15)
// MSB = full/empty indicator
```

MAX Selection: - Set MAX equal to FIFO depth - MAX must fit in WIDTH-1 bits

14.9.4 Critical Paths

1. **Add Operation:** Extended addition + wraparound comparison
 - Longest path: counter_bin_curr + add_value → overflow detection → modulo
2. **Standard Increment:** Comparison + MSB toggle
 - Path: Lower bits comparison → MSB invert + clear
3. **Load Operation:** Direct assignment
 - Shortest path: Mux select

Optimization Tip: If the add operation is your critical path, pipeline it:

```
// Pipeline add operation
logic [WIDTH-1:0] r_add_result;
always_ff @(posedge clk) begin
    r_add_result <= counter_bin_curr + add_value;
end
```

14.9.5 Common Pitfalls

Anti-Pattern 1: Incorrect WIDTH for FIFO depth

```
// WRONG: WIDTH = $clog2(FIFO_DEPTH)
localparam int WIDTH = $clog2(16); // = 4 bits
// Problem: No MSB for full/empty detection!

// RIGHT: WIDTH = $clog2(FIFO_DEPTH) + 1
localparam int WIDTH = $clog2(16) + 1; // = 5 bits
// Lower 4 bits = address, MSB = full/empty indicator
```

Anti-Pattern 2: Ignoring operation priority

```
// WRONG: Trying to increment while load is active
load <= 1'b1;
enable <= 1'b1; // This will be IGNORED!
// Result: Counter loads load_value, enable has no effect

// RIGHT: Ensure only one operation active
load <= drop_all;
enable <= rd_en & !drop_all; // Mutually exclusive
```

Anti-Pattern 3: add_value exceeds 2×MAX

```
// WRONG: add_value larger than valid range
// MAX = 8, add_value = 20
// Result: Multiple wraparounds, unpredictable behavior

// RIGHT: Constrain add_value
assert property (@(posedge clk) add_enable |-> add_value < 2*MAX);
```

Anti-Pattern 4: Using wrong bits for memory address

```
// WRONG: Using full counter including MSB
assign mem_addr = wr_ptr; // Includes MSB!
// Result: Addresses 0-15 appear as 0-15 and 16-31

// RIGHT: Use only lower bits
assign mem_addr = wr_ptr[WIDTH-2:0]; // Exclude MSB
```

Every one of these has bitten someone on a real project. Anti-Pattern 1 is the most common — the missing MSB doesn't show up until you try to distinguish full from empty, which is to say, at the worst possible moment.

14.10 Related Modules

- **counter_bin.sv** - Simple binary counter (no load or variable increment)
- **counter_load_clear.sv** - Counter with load/clear (no FIFO wraparound)

- **counter_freq_invariant.sv** - Time-based counter (millisecond/microsecond timing)
- **gaxi_drop_fifo_sync.sv** - Uses counter_bin_load for drop-capable FIFO
- **gaxi_fifo_sync.sv** - Standard synchronous FIFO with this counter

14.11 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

15 Binary Counter for FIFOs

15.1 Overview

The `counter_bin` module is a binary counter with one job: FIFO (First-In-First-Out) buffer addressing. Instead of a plain wrap, it inverts the MSB and clears the lower bits when it reaches the maximum value. That folded counting pattern is exactly what circular buffer management wants — the MSB becomes your lap marker.

15.2 Module Interface

```
module counter_bin #(
    parameter int WIDTH = 5,
    parameter int MAX    = 10
) (
    input  logic          clk,
    input  logic          rst_n,
    input  logic          enable,
    output logic [WIDTH-1:0] counter_bin_curr,
    output logic [WIDTH-1:0] counter_bin_next
);
```

15.3 Parameters

Parameter	Type	Default	Description
WIDTH	int	5	Total width of the counter in bits
MAX	int	10	Maximum count value before special wrap behavior

WIDTH must be ≥ 2 for proper MSB inversion operation, and it determines the counter's bit width and maximum representable value.

MAX ranges from 2 to $2^{(WIDTH-1)}$ **inclusive**. The RTL computes `w_max_val = (MAX-1)` in `WIDTH-1` bits, and `MAX-1 = 2^{(WIDTH-1)} - 1` fits exactly, so `MAX = 2^{(WIDTH-1)}` is legal (the doc's own validation asserts `MAX <= 2** (WIDTH-1)`, and both examples use it: `#(.WIDTH(4), .MAX(8))` and `#(.WIDTH(11), .MAX(1024))`). This matters because power-of-2 FIFO depths sit exactly at `MAX = 2^{(WIDTH-1)}`. The one hard constraint: `MAX-1` must fit within the lower `WIDTH-1` bits.

15.4 Ports

15.4.1 Inputs

Port	Width	Description
clk	1	System clock input
rst_n	1	Active-low asynchronous reset
enable	1	Counter enable control

15.4.2 Outputs

Port	Width	Description
counter_bin_curr	WIDTH	Current counter value (registered)
counter_bin_next	WIDTH	Next counter value (combinational)

15.5 Functional Description

15.5.1 Internal Signals

```
logic [WIDTH-2:0] w_max_val;  
assign w_max_val = (WIDTH-1)'(MAX - 1);
```

15.5.2 Special Wrap Logic

Here's the wrap behavior that makes this a FIFO counter rather than a generic one:

```
always_comb begin  
    if (enable) begin  
        if (counter_bin_curr[WIDTH-2:0] == w_max_val)  
            counter_bin_next = {~counter_bin_curr[WIDTH-1], {(WIDTH -  
1)'1'b0}}};  
        else  
            counter_bin_next = counter_bin_curr + 1;  
        end else begin  
            counter_bin_next = counter_bin_curr;  
        end  
    end  
end
```

15.5.3 Normal Counting

When `counter_bin_curr[WIDTH-2:0] < MAX-1`, nothing exotic happens: - Counter increments normally by 1 - MSB remains unchanged - Standard binary counting behavior

15.5.4 Wrap Condition

When `counter_bin_curr[WIDTH-2:0] == MAX-1`: - MSB is inverted:
`~counter_bin_curr[WIDTH-1]` - Lower bits are cleared: `{(WIDTH-1){1'b0}}` - Creates a “folded” counting pattern

15.5.5 Enable Control

- When `enable = 1`: Counter operates normally
- When `enable = 0`: Counter holds current value

15.5.6 FIFO Application Context

15.5.7 Address Generation

This pattern earns its keep in FIFO addressing because: 1. **Circular Addressing**: Lower bits provide circular buffer addresses 2. **Overflow Detection**: MSB toggles indicate buffer wrap-around 3. **Full/Empty Logic**: Comparing MSBs between read/write pointers

15.5.8 FIFO Pointer Comparison

```
// FIFO empty when pointers exactly equal
```

```
assign fifo_empty = (rd_ptr == wr_ptr);
```

```
// FIFO full when addresses equal but MSBs differ
```

```
assign fifo_full = (rd_ptr[WIDTH-2:0] == wr_ptr[WIDTH-2:0]) &&  
                  (rd_ptr[WIDTH-1] != wr_ptr[WIDTH-1]);
```

15.5.9 Advanced Variants

15.5.10 Address Extraction

```
// Extract memory address (lower bits only)
```

```
assign mem_addr = counter_bin_curr[WIDTH-2:0];
```

```
// Extract wrap flag (MSB only)
```

```
assign wrap_flag = counter_bin_curr[WIDTH-1];
```

15.5.11 Gray Code Conversion

For asynchronous FIFOs, convert to Gray code:

```
logic [WIDTH-1:0] gray_ptr;  
bin2gray #(.WIDTH(WIDTH)) b2g (  
    .binary(counter_bin_curr),  
    .gray(gray_ptr)  
);
```

15.6 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

15.6.1 Timing

- **Enable to Next:** Single logic level
- **Current to Next:** Increment/wrap logic depth
- **Critical Path:** Wrap detection and MSB inversion
- **Setup Time:** Standard flip-flop requirements
- **Clock-to-Q:** Standard flip-flop propagation
- **Reset Recovery:** One cycle after reset deassertion

15.6.2 Maximum Frequency

- **Limiting Factor:** Wrap detection and MSB inversion logic
- **Typical Performance:** 300-500 MHz in modern FPGAs
- **Optimization:** Consider registering intermediate signals for high speed

15.6.3 Power Consumption

- **Dynamic Power:** Proportional to toggle rate
- **Optimization:** Use enable signal to reduce unnecessary toggles
- **Clock Gating:** Gate clock when FIFO operations are idle

15.7 Usage Examples

15.7.1 1. Small FIFO Buffer (8 entries)

```
counter_bin #(
    .WIDTH(4),      // 4-bit counter
    .MAX(8)         // 8 entries (0-7)
) wr_ptr_counter (
    .clk(clk),
    .rst_n(rst_n),
    .enable(wr_enable),
    .counter_bin_curr(wr_ptr),
    .counter_bin_next(wr_ptr_next)
);
```

15.7.2 2. Large FIFO Buffer (1024 entries)

```
counter_bin #(
    .WIDTH(11),     // 11-bit counter (10 bits address + 1 MSB)
    .MAX(1024)      // 1024 entries (0-1023)
) rd_ptr_counter (
    .clk(clk),
    .rst_n(rst_n),
    .enable(rd_enable),
    .counter_bin_curr(rd_ptr),
    .counter_bin_next(rd_ptr_next)
);
```

15.7.3 3. Asymmetric FIFO

```
// Write pointer - 16 entries
counter_bin #(.WIDTH(5), .MAX(16)) wr_cnt (...);
```

```
// Read pointer - different size for width conversion
counter_bin #(.WIDTH(6), .MAX(32)) rd_cnt (...);
```

15.7.4 Counting Sequence Example

15.7.5 Configuration: WIDTH=4, MAX=6

```
Initial: 0000 (MSB=0, lower=000)
         0001
         0010
         0011
         0100
         0101 <- MAX-1 reached
Wrap:    1000 <- MSB inverted (0→1), lower bits cleared
         1001
         1010
         1011
```

```

1100
1101 <- MAX-1 reached again
Wrap: 0000 <- MSB inverted (1→0), lower bits cleared

```

15.7.6 Pattern Analysis

- **First Half:** MSB=0, counts 0 to MAX-1
- **Second Half:** MSB=1, counts 0 to MAX-1
- **Full Cycle:** 2×MAX states total ## Testing

15.7.7 Test Scenarios

1. **Basic Increment:** Verify normal counting behavior
2. **Wrap Boundary:** Test MSB inversion at MAX-1
3. **Enable Control:** Verify hold behavior when disabled
4. **Reset Operation:** Confirm proper reset to zero
5. **Full Sequence:** Complete cycle through all states

15.7.8 Coverage Points

```

covergroup counter_bin_cg @(posedge clk);
  cp_lower_bits: coverpoint counter_bin_curr[WIDTH-2:0] {
    bins all_values[] = {[0:MAX-1]};
  }
  cp_msb: coverpoint counter_bin_curr[WIDTH-1] {
    bins low = {0};
    bins high = {1};
  }
  cp_wrap: coverpoint (counter_bin_curr[WIDTH-2:0] == w_max_val);
  cp_enable: coverpoint enable;
endcovergroup

```

15.8 Design Notes

15.8.1 Resource Utilization

- **Flip-Flops:** WIDTH registers for current counter
- **LUTs:** Increment, wrap detection, and MSB inversion logic
- **Optimization:** Synthesis tools typically optimize the wrap logic efficiently

15.8.2 Timing Optimization

```
// Optional: Pipeline for high-frequency operation
logic [WIDTH-1:0] counter_bin_pipe;
always_ff @(posedge clk) begin
    counter_bin_pipe <= counter_bin_next;
end
```

15.8.3 Parameter Validation

```
initial begin
    assert (WIDTH >= 2)
        else $error("WIDTH must be at least 2 for MSB operation");
    assert (MAX <= (2**(WIDTH-1)))
        else $error("MAX too large for given WIDTH");
    assert (MAX > 0)
        else $error("MAX must be positive");
end
```

15.8.4 Common Issues and Solutions

Issue 1: Incorrect FIFO Size - Problem: MAX parameter doesn't match actual FIFO depth - **Solution:** Ensure MAX equals the number of FIFO entries

Issue 2: Width Mismatch - Problem: WIDTH too small for MAX value - **Solution:** Set $WIDTH \geq \lceil \log_2(2*MAX) \rceil$ for full range

Issue 3: Timing Violations - Problem: Wrap logic creates long combinational paths - **Solution:** Pipeline the next value calculation or reduce MAX

15.9 Related Modules

- **counter_bingray** - Provides both binary and Gray code outputs
- **counter** - Basic counter without wrap behavior
- **counter_load_clear** - Loadable counter with different wrap behavior
- Standard FIFO controllers that use this addressing scheme

15.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

16 Frequency Invariant Counter

16.1 Overview

The `counter_freq_invariant` module divides an arbitrary input clock down to a 1 MHz tick, so your design can count real time without knowing its clock frequency at compile time. You pick the division factor at run time through `freq_sel`, which indexes a lookup table built at elaboration time from the `MIN_FREQ_MHZ`, `MAX_FREQ_MHZ`, `NUM_FREQ_ENTRIES` and `FREQ_STRATEGY` parameters.

The whole module rests on one convenient identity: **the division factor and the clock frequency in MHz are the same number**. A 100 MHz clock has 100 cycles per microsecond, so dividing by 100 yields a 1 MHz tick. The LUT just stores frequencies in MHz and uses them directly as divisors. Elegant, once you see it.

One bit of history: an earlier revision of this module had a hardcoded 68-entry frequency table and a fixed 4-bit `freq_sel`. That table is gone; the LUT is now generated from parameters and `freq_sel` is sized to match `NUM_FREQ_ENTRIES`.

16.2 Module Interface

```
module counter_freq_invariant #(
    // User parameters
    parameter int COUNTER_WIDTH      = 16,      // Width of output
    // microsecond counter
    parameter int MIN_FREQ_MHZ       = 5,        // Lowest supported clock
    (MHz)
    parameter int MAX_FREQ_MHZ       = 220,      // Highest supported
    clock (MHz)
    parameter int NUM_FREQ_ENTRIES   = 16,        // Number of LUT entries
    parameter int FREQ_STRATEGY      = 0,        // 0 = LINEAR, 1 = POW2
    parameter bit  DEBUG_LUT         = 1'b0,     // Print LUT at time 0

    // Derived parameters (do not override)
    parameter int SEL_WIDTH          = (NUM_FREQ_ENTRIES > 1) ?
    $clog2(NUM_FREQ_ENTRIES) : 1,
    parameter int DIV_WIDTH          = $clog2(MAX_FREQ_MHZ + 1),
    parameter int PRESCALER_MAX      = 2 ** DIV_WIDTH
) (
    input  logic                clk,
    input  logic                rst_n,
    input  logic                sync_reset_n,
    input  logic [SEL_WIDTH-1:0] freq_sel,
    output logic [COUNTER_WIDTH-1:0] o_counter,
    output logic                tick
);
```

16.3 Parameters

counter_freq_invariant user parameters

Parameter	Type	Default	Description
COUNTER_WIDTH	int	16	Width of the microsecond counter. Wraps at $2^{\text{COUNTER_WIDTH}}$.
MIN_FREQ_MHZ	int	5	Lowest clock frequency in the LUT, in MHz. Must be ≥ 1 .
MAX_FREQ_MHZ	int	220	Highest clock frequency in the LUT, in MHz. Must be $\geq \text{MIN_FREQ_MHZ}$.
NUM_FREQ_ENTRIES	int	16	Number of LUT entries. Must be ≥ 1 . Sets SEL_WIDTH.
FREQ_STRATEGY	int	0	LUT distribution: 0 = LINEAR, 1 = POW2.
DEBUG_LUT	bit	1'b0	Print the generated LUT at simulation time 0. Simulation only.

Derived parameters – do not override. SEL_WIDTH, DIV_WIDTH and PRESCALER_MAX appear in the parameter list only so they can size ports and internal counters. They're computed from the user parameters:

Derived parameters at the default configuration

Derived	Expression	Value at defaults
SEL_WIDTH	$\$clog2(\text{NUM_FREQ_ENTRIES}), \text{min } 1$	4
DIV_WIDTH	$\$clog2(\text{MAX_FREQ_MHZ} + 1)$	8
PRESCALER_MAX	2^{**}DIV_WIDTH	256

Bad combinations get caught early: the three \$error checks in the param_check initial block reject $\text{MIN_FREQ_MHZ} < 1$, $\text{MAX_FREQ_MHZ} < \text{MIN_FREQ_MHZ}$, and $\text{NUM_FREQ_ENTRIES} < 1$ at elaboration.

16.4 Ports

16.4.1 Inputs

Port	Width	Description
clk	1	Input clock, any frequency covered by the LUT
rst_n	1	Active-low asynchronous reset
sync_reset_n	1	Synchronous run/reset. 0 holds the counter cleared; 1 runs.
freq_sel	SEL_WIDTH	LUT index selecting the division factor

16.4.2 Outputs

Port	Width	Description
o_counter	COUNTER_WIDTH	Microsecond counter, wraps at $2^{\text{COUNTER_WIDTH}}$
tick	1	Single-cycle pulse, once per microsecond

One naming gotcha: the output is o_counter, not counter.

16.5 Functional Description

```
freq_sel ---> [ LUT mux ] ---> division_factor
      |
clk, rst_n ---> [ change detect ] ---> clear_pulse
      |
      v
      [ counter_load_clear ] ---> prescaler_done
      |
      v
      [ counter + tick reg ] ---> o_counter, tick
```

16.5.1 LUT Generation Strategies

The LUT is built at elaboration time by a generate loop that evaluates a pure integer function per index, so synthesis infers a mux or ROM rather than any runtime arithmetic. That's what you want — no multiplier hiding in your timing report.

LINEAR (FREQ_STRATEGY = 0, default)

```
freq[i] = MIN_FREQ_MHZ + (MAX_FREQ_MHZ - MIN_FREQ_MHZ) * i /
(NUM_FREQ_ENTRIES - 1)
```

Uniform steps across the range. Integer division truncates. Good for FPGA bring-up where predictable coverage of the operating range matters.

POW2 (FREQ_STRATEGY = 1)

```
freq[i] = min(MIN_FREQ_MHZ * 2^i, MAX_FREQ_MHZ)
```

Doubling from the minimum, clamped at the maximum. Gives finer resolution at the low end and saturates once MAX_FREQ_MHZ is reached.

Default LUT (LINEAR, 5-220 MHz, 16 entries)

Default LINEAR LUT. Division factor equals the clock frequency in MHz.

freq_sel	Clock (MHz)	Cycles per us	freq_sel	Clock (MHz)	Cycles per us
0	5	5	8	119	119
1	19	19	9	134	134
2	33	33	10	148	148
3	48	48	11	162	162
4	62	62	12	177	177
5	76	76	13	191	191
6	91	91	14	205	205
7	105	105	15	220	220

Set DEBUG_LUT and the table your parameters actually produce prints at time 0 — much better than working it out by hand and being wrong.

Same table under POW2

Same range, same entry count, FREQ_STRATEGY = 1: you get 5, 10, 20, 40, 80, 160 MHz and then saturation at 220 MHz for indices 6 through 15. POW2 only earns its keep when NUM_FREQ_ENTRIES is close to $\log_2(\text{MAX}/\text{MIN}) + 1$; beyond that the remaining entries are duplicates.

16.5.2 Division Factor Lookup

```
logic [DIV_WIDTH-1:0] w_div_table [NUM_FREQ_ENTRIES];
```

```
generate
  for (genvar gi = 0; gi < NUM_FREQ_ENTRIES; gi++) begin :
    gen_div_entry
      assign w_div_table[gi] = DIV_WIDTH'(freq_mhz_at_idx(gi));
    end
  endgenerate
```

```

logic [DIV_WIDTH-1:0] w_division_factor;
assign w_division_factor = w_div_table[freq_sel];

```

16.5.3 Change Detection

A clear pulse fires whenever freq_sel changes or sync_reset_n is low. One subtlety worth noticing: r_clear_pulse resets to 1, not 0, so the counter starts life in the cleared state.

```

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_prev_freq_sel <= '0;
        r_clear_pulse   <= 1'b1;    // start in reset state
    end else begin
        r_prev_freq_sel <= freq_sel;
        r_clear_pulse   <= (freq_sel != r_prev_freq_sel) || !
sync_reset_n;
    end
end

```

16.5.4 Prescaler

```

counter_load_clear #(
    .MAX(PRESCALER_MAX)
) prescaler_counter (
    .clk      (clk),
    .rst_n    (rst_n),
    .clear    (r_clear_pulse),
    .increment (1'b1),
    .load     (1'b1),
    .loadaval (w_division_factor - DIV_WIDTH'(1)),
    .done     (w_prescaler_done),
    .count    ()
);

```

16.5.5 Counter and Tick

tick pulses on **every** prescaler completion, once per microsecond. It is not gated on the counter reaching its maximum — people assume otherwise, and it leads to off-by-2^N bugs in timeout logic. Don't.

```

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        o_counter <= '0;
        tick      <= 1'b0;
    end else begin
        if (r_clear_pulse) begin
            o_counter <= '0;
            tick      <= 1'b0;
        end else if (w_prescaler_done && sync_reset_n) begin
            o_counter <= o_counter + 1'b1;
        end
    end

```

```

        tick      <= 1'b1;
    end else begin
        tick <= 1'b0;
    end
end
end
end

```

16.6 Timing Characteristics

16.6.1 Timing

counter_freq_invariant timing properties

Property	Value
Tick period	division_factor input clock cycles = 1 us
Counter wrap period	2 ^{COUNTER_WIDTH} microseconds
Latency	2 cycles (prescaler stage plus output register)
Reconfiguration	Immediate: a freq_sel change clears and reloads

At the default COUNTER_WIDTH of 16, the counter wraps every 65536 us, or about 65.5 ms. Size your timeout logic accordingly.

16.6.2 Resource Use

Resource use by element

Element	Size
Prescaler counter	$\$c\log_2(\text{PRESCALER_MAX})$ bits
Microsecond counter	COUNTER_WIDTH bits
Change detection	SEL_WIDTH + 1 flops
LUT	NUM_FREQ_ENTRIES × DIV_WIDTH bits of constant mux or ROM

16.6.3 Critical Path

The critical path runs through the prescaler increment and its terminal-count comparison. Raising MAX_FREQ_MHZ widens DIV_WIDTH and lengthens that path — something to watch if you're pushing frequency.

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

16.7 Usage Examples

16.7.1 1. Microsecond timebase on an unknown FPGA clock

```
logic [15:0] usec_count;
logic      usec_tick;

counter_freq_invariant #(
    .COUNTER_WIDTH    (16),
    .MIN_FREQ_MHZ     (5),
    .MAX_FREQ_MHZ     (220),
    .NUM_FREQ_ENTRIES (16),
    .FREQ_STRATEGY    (0)
) u_timebase (
    .clk      (sys_clk),
    .rst_n    (sys_rst_n),
    .sync_reset_n(1'b1),
    .freq_sel  (cfg_freq_sel), // 4 bits at these parameters
    .o_counter (usec_count),
    .tick      (usec_tick)
);
```

16.7.22. ASIC range, 100 MHz to 1 GHz

```
counter_freq_invariant #(
    .COUNTER_WIDTH    (16),
    .MIN_FREQ_MHZ     (100),
    .MAX_FREQ_MHZ     (1000),
    .NUM_FREQ_ENTRIES (32),
    .FREQ_STRATEGY    (0)
) u_timer (
    .clk      (core_clk),
    .rst_n    (core_rst_n),
    .sync_reset_n(1'b1),
    .freq_sel  (cfg_freq_sel), // 5 bits at NUM_FREQ_ENTRIES = 32
    .o_counter (usec_count),
    .tick      (usec_tick)
);
```

DIV_WIDTH becomes $\lceil \log_2(1001) \rceil = 10$ bits here, and PRESCALER_MAX 1024.

16.7.33. Millisecond timeout built on the tick

```
logic [9:0] r_msec;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) r_msec <= '0;
    else if (usec_tick && usec_count[9:0] == 10'd0) r_msec <= r_msec +
1'b1;
end
```

A cheaper alternative is to watch a bit of o_counter directly: bit 9 toggles approximately every 512 us.

16.7.44. Gating the timer with sync_reset_n

```
// Hold the timer cleared while calibration is running
counter_freq_invariant #(
    .COUNTER_WIDTH(16)
) u_cal_timer (
    .clk          (clk),
    .rst_n        (rst_n),
    .sync_reset_n(!enter_calibration_mode),
    .freq_sel     (cfg_freq_sel),
    .o_counter    (cal_usec),
    .tick         (cal_tick)
);
```

16.8 Testing

Test: val/common/test_counter_freq_invariant.py

Scenarios worth covering:

1. Every freq_sel index produces the expected tick period for its LUT entry
2. Changing freq_sel mid-operation clears and restarts cleanly
3. Asynchronous reset and sync_reset_n both clear o_counter and tick
4. o_counter wraps correctly at $2^{\text{COUNTER_WIDTH}}$
5. Both FREQ_STRATEGY values generate the expected LUT
6. NUM_FREQ_ENTRIES = 1 degenerate case elaborates and runs

16.9 Design Notes

16.9.1 Choosing the LUT Range

MIN_FREQ_MHZ and MAX_FREQ_MHZ should bracket the clock frequencies the design will actually see. Widening the range without raising NUM_FREQ_ENTRIES coarsens the LINEAR steps, and a clock that falls between two LUT entries produces a tick that is off by the rounding error. At the defaults the step is about 14 MHz, so a 70 MHz clock selecting index 5 (76 MHz) ticks roughly 8% slow. That's not an error term you can wave away in a watchdog.

If you need an exact tick at one known frequency, there's a cleaner path: set NUM_FREQ_ENTRIES to 1 and MIN_FREQ_MHZ equal to MAX_FREQ_MHZ equal to that frequency. SEL_WIDTH clamps to 1 in that case and freq_sel should be tied to 0.

16.10 Related Modules

- **counter_bin** - basic binary counter
- **counter_load_clear** - programmable counter, used here as the prescaler
- **clock_divider** - integer clock division producing a divided clock, not a tick
- **clock_pulse** - configurable pulse generator

16.11 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

17 Counter with Load and Clear

17.1 Overview

The `counter_load_clear` module is a counter with a loadable terminal count and a clear — the workhorse you reach for when a plain counter isn't quite enough. You get precise control over counting behavior through configurable match values, which makes it a natural fit for timing applications, protocol implementations, and configurable delay generation.

17.2 Module Interface

```
module counter_load_clear #(
    parameter int MAX = 32'd32
) (
    input logic          clk,
    input logic          rst_n,
    input logic          clear,
    input logic          increment,
    input logic          load,
    input logic [$clog2(MAX)-1:0] loadval,
    output logic [$clog2(MAX)-1:0] count,
    output logic         done
);
```

17.3 Parameters

Parameter	Type	Default	Description
MAX	int	32'd32	Maximum possible count value

MAX ranges from 2 to $2^{32}-1$ (the RTL header states the same). MAX=1 gives $\$clog2(1) = 0$, so `loadval` and `count` would be zero-width. The parameter determines the counter width and the maximum loadable value: counter width = $\$clog2(MAX)$.

17.4 Ports

17.4.1 Inputs

Port	Width	Description
clk	1	System clock
rst_n	1	Active-low asynchronous reset
clear	1	Synchronous clear

Port	Width	Description
increment	1	(highest priority) Enable counter increment
load	1	Load new match value
loadval	$\$clog2(MAX)$	New terminal count value

17.4.2 Outputs

Port	Width	Description
count	$\$clog2(MAX)$	Current counter value
done	1	Terminal count reached flag

17.5 Functional Description

17.5.1 Internal Registers

```

logic [ $\$clog2(MAX) - 1:0$ ] count;           // Current count value
logic [ $\$clog2(MAX) - 1:0$ ] r_match_val;      // Terminal count register

```

17.5.2 Priority Logic

Control is priority-based, and the order matters:

1. **Asynchronous Reset** (Highest Priority)
 - Resets both count and r_match_val to 0
 - Independent of clock edge
2. **Load Operation** (High Priority)
 - Updates r_match_val with loadval
 - Can occur simultaneously with other operations
3. **Clear Operation** (Medium Priority)
 - Immediately sets count to 0

- Overrides increment operation

4. Increment Operation (Lowest Priority)

- Advances counter or wraps to 0 at terminal count
- Only when increment is asserted

17.5.3 Main Control Logic

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        count <= 'b0;
        r_match_val <= 'b0;
    end else begin
        // Load has independent priority
        if (load) r_match_val <= loadval;

        // Clear has highest operational priority
        if (clear) begin
            count <= 'b0;
        end else if (increment) begin
            count <= (count == r_match_val) ? 'b0 : count + 'b1;
        end
    end
end
```

17.5.4 Done Signal Generation

```
assign done = (count == r_match_val);
```

17.5.5 Advanced Variants

17.5.6 1. Dual-Rate Timer

```
// Timer with fast/slow modes
logic fast_mode;
logic [15:0] timer_period;

always_comb begin
    timer_period = fast_mode ? 16'd99 : 16'd999; // 100 vs 1000
cycles
end
```

```
counter_load_clear #(.MAX(65536)) dual_timer (
    .clk(clk),
    .rst_n(rst_n),
    .clear(timer_done), // Auto-restart
    .increment(1'b1),
```

```

        .load(timer_done),           // Reload on completion
        .loadval(timer_period),
        .count(timer_value),
        .done(timer_done)
    );

```

17.5.72. PWM Generator

// Simple PWM with configurable duty cycle

```
logic [7:0] duty_cycle; // 0-255
```

```
logic pwm_out;
```

```

counter_load_clear #(.MAX(256)) pwm_counter (
    .clk(clk),
    .rst_n(rst_n),
    .clear(1'b0),
    .increment(1'b1),
    .load(1'b1),
    .loadval(8'd255),           // Always count 0-255
    .count(pwm_count),
    .done()
);

```

```
assign pwm_out = (pwm_count < duty_cycle);
```

17.5.83. Burst Generator

// Generate bursts of N pulses

```
logic [7:0] burst_length;
```

```
logic start_burst, burst_active, burst_complete;
```

```

counter_load_clear #(.MAX(256)) burst_counter (
    .clk(clk),
    .rst_n(rst_n),
    .clear(start_burst),
    .increment(burst_active),
    .load(start_burst),
    .loadval(burst_length - 1),
    .count(burst_count),
    .done(burst_complete)
);

```

// State machine for burst control

```

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        burst_active <= 1'b0;
    end else begin
        if (start_burst)
            burst_active <= 1'b1;
    end

```

```

        else if (burst_complete)
            burst_active <= 1'b0;
    end
end

```

17.5.94. Communication Timeout

```

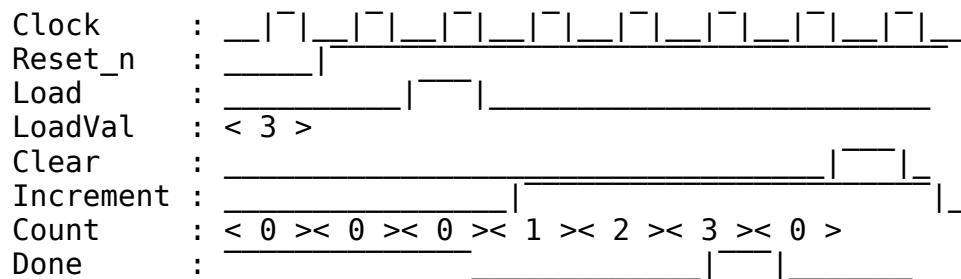
// UART receive timeout
logic rx_active, rx_timeout, char_received;
parameter RX_TIMEOUT_CYCLES = 1000;

counter_load_clear #(.MAX(2048)) rx_timer (
    .clk(clk),
    .rst_n(rst_n),
    .clear(char_received),           // Reset on new character
    .increment(rx_active),          // Only count when receiving
    .load(rx_active && !char_received),
    .loadval(RX_TIMEOUT_CYCLES-1),
    .count(),
    .done(rx_timeout)
);

```

17.6 Timing Characteristics

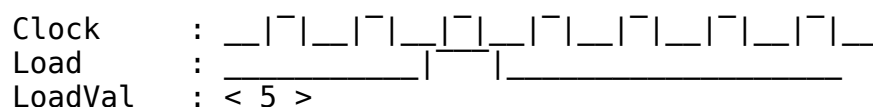
17.6.1 Basic Operation



done is combinational (count == r_match_val) and both reset to 0, so it is **high out of reset** and stays high until the load writes a non-zero match value. It is not a pulse that only appears at terminal count.

17.6.2 Load During Count

Load only writes r_match_val; it does **not** stall the count. With increment held high, count advances every cycle (including the load cycle). Here load asserts while count == 2, so r_match_val becomes 5 starting the next cycle — before count reaches the old match value 3 — so the counter never wraps at 3 and instead runs to the new match 5:



Increment	:																	
Count	:	<	0	><	1	><	2	><	3	><	4	><	5	><	0	>		
r_match	:	<	3	><	3	><	3	><	5	><	5	><	5	><	5	>		
Done	:														—			

(If load had instead arrived after count already reached 3, the counter would have wrapped at 3 first — the outcome depends on load timing relative to the old match value.)

17.6.3 Maximum Frequency

- **Typical:** 300-500 MHz in modern FPGAs
- **Critical Path:** Increment + comparison logic
- **Optimization:** Consider pipelining for extreme speeds

17.6.4 Latency

- **Load Operation:** 1 cycle to update match value
- **Clear Operation:** 1 cycle to reset count
- **Done Signal:** Combinational (0 cycles)

17.7 Usage Examples

17.7.11. Basic Counting Mode

```
// Simple timer: count to 100
counter_load_clear #(.MAX(1000)) timer (
    .clk(clk),
    .rst_n(rst_n),
    .clear(1'b0),
    .increment(1'b1),
    .load(1'b1),
    .loadval(8'd99),    // Count 0-99 (100 cycles)
    .count(timer_count),
    .done(timer_done)
);
```

17.7.22. Configurable Delay Mode

```
// Variable delay generator
logic [7:0] delay_setting;
logic start_delay, delay_complete;

counter_load_clear #(.MAX(256)) delay_gen (
    .clk(clk),
    .rst_n(rst_n),
```

```

        .clear(start_delay),           // Start new delay
        .increment(1'b1),
        .load(start_delay),           // Load new value when starting
        .loadval(delay_setting),
        .count(),
        .done(delay_complete)
    );

```

17.7.33. Retriggerable Timer Mode

// Watchdog timer that resets on activity

logic activity_detected, timeout;

```

counter_load_clear #(.MAX(10000)) watchdog (
    .clk(clk),
    .rst_n(rst_n),
    .clear(activity_detected), // Reset on activity
    .increment(1'b1),
    .load(1'b1),
    .loadval(16'd9999),       // 10000 cycle timeout
    .count(),
    .done(timeout)
);

```

1. **Timer Modules:** Configurable timing generation
2. **Protocol Engines:** Timeout and retry mechanisms
3. **PWM Controllers:** Variable duty cycle generation
4. **Burst Generators:** Programmable pulse trains
5. **Watchdog Timers:** System monitoring and reset
6. **Clock Dividers:** Variable frequency division
7. **Test Pattern Generators:** Configurable test sequences

17.7.4 Design Patterns

17.7.5 Pattern 1: Auto-Reloading Timer

```

wire auto_reload = timer_done;
counter_load_clear timer (
    .clear(auto_reload),
    .load(auto_reload),
    .loadval(PERIOD-1),
    .done(timer_done),

```

```
);    ...
```

17.7.6 Pattern 2: Conditional Counting

```
wire count_enable = condition_met && !paused;
counter_load_clear conditional_timer (
    .increment(count_enable),
    ...
);
```

17.7.7 Pattern 3: Dynamic Period Adjustment

```
logic period_changed;
wire reload_needed = timer_done || period_changed;

counter_load_clear adaptive_timer (
    .clear(reload_needed),
    .load(reload_needed),
    .loadval(current_period),
    ...
);
```

17.8 Testing

17.8.1 Test Scenarios

1. **Basic Counting:** Verify normal increment operation
2. **Terminal Count:** Test done signal generation
3. **Load Operation:** Verify match value updates
4. **Clear Priority:** Test clear overrides increment
5. **Simultaneous Operations:** Load + clear + increment
6. **Boundary Conditions:** Count = 0, count = match_val
7. **Reset Behavior:** Async reset during various operations

17.8.2 Coverage Model

```
covergroup load_clear_cg @(posedge clk);
    cp_count: coverpoint count {
        bins zero = {0};
        bins low = {[1:MAX/4]};
        bins mid = {[MAX/4+1:3*MAX/4]};
        bins high = {[3*MAX/4+1:MAX-1]};
    }
endcovergroup
```

```

}

cp_match_val: coverpoint r_match_val {
    bins zero = {0};
    bins low = {[1:MAX/4]};
    bins mid = {[MAX/4+1:3*MAX/4]};
    bins high = {[3*MAX/4+1:MAX-1]};
}

cp_operations: coverpoint {clear, increment, load} {
    bins clear_only = {3'b100};
    bins inc_only = {3'b010};
    bins load_only = {3'b001};
    bins clear_inc = {3'b110};
    bins clear_load = {3'b101};
    bins inc_load = {3'b011};
    bins all_ops = {3'b111};
    bins no_ops = {3'b000};
}

cp_done: coverpoint done;

// Cross coverage
cross_done_ops: cross cp_operations, done;
endcovergroup

```

17.8.3 Assertions

```

// Count stays within the match value ONLY for the load-then-count
pattern.
// This property does NOT hold in general: if a smaller match is
loaded while
// count is already above it (load 5, count up to 5, then load 2),
then
// count (5) > r_match_val (2) and the counter free-runs modulo
2^WIDTH until it
// happens to equal the new match again -- there is no terminal-count
clamp,
// only the equality wrap. Do not copy this assertion into a testbench
that
// dynamically REDUCES the terminal count; it will fail on correct
RTL.
property count_bounds;    // holds only when match is never reduced
below count
    @(posedge clk) disable iff (!rst_n)
    count <= r_match_val;
endproperty

```



```

// Done should be true when count equals match
property done_when_match;
    @(posedge clk) disable iff (!rst_n)
        (count == r_match_val) |-> done;
endproperty

// Clear should reset count to 0
property clear_resets_count;
    @(posedge clk) disable iff (!rst_n)
        clear |=> (count == 0);
endproperty

// Load should update match value
property load_updates_match;
    @(posedge clk) disable iff (!rst_n)
        load |=> (r_match_val == $past(loadval));
endproperty

assert property (count_bounds);
assert property (done_when_match);
assert property (clear_resets_count);
assert property (load_updates_match);

```

17.9 Design Notes

17.9.1 Resource Usage

- **Flip-Flops:** $\$clog2(\text{MAX})$ for count + $\$clog2(\text{MAX})$ for match = $2 \times \$clog2(\text{MAX})$
- **LUTs:** Increment logic, comparator, and control logic

17.9.2 Timing Optimization

```

// For high-speed operation, pipeline the comparison
logic done_pipe;
always_ff @(posedge clk) begin
    done_pipe <= (count == r_match_val);
end

```

17.9.3 Parameter Validation

```

initial begin
    assert (MAX > 0) else $error("MAX must be positive");
    assert (MAX <= 2**30) else $warning("Large MAX may impact
synthesis");
end

```

17.10 Related Modules

- **counter** - Basic counter without load/clear
- **counter_freq_invariant** - Uses this module for prescaling
- Standard timer and PWM controller implementations

17.11 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

18 Basic Counter

18.1 Overview

The counter module is as simple as it gets: a configurable up-counter that raises a tick pulse when it reaches its maximum value. Use it for basic timing jobs where all you need is a periodic signal.

18.2 Module Interface

```
module counter #(
    parameter int MAX = 32767
) (
    input logic clk,
    input logic rst_n,
    output logic tick
);
```

18.3 Parameters

Parameter	Type	Default	Description
MAX	int	32767	Maximum count value before wrapping to zero

MAX can be any positive integer. The counter width is automatically calculated as $\lceil \log_2(\text{MAX}+1) \rceil$.

18.4 Ports

18.4.1 Inputs

Port	Width	Description
clk	1	System clock input
rst_n	1	Active-low asynchronous reset

18.4.2 Outputs

Port	Width	Description
tick	1	Pulse output when counter reaches MAX

18.5 Functional Description

18.5.1 Counter Register

logic [**\$clog2**(MAX+1)-1:0] r_count;

- **Width:** Automatically sized to accommodate MAX value
- **Reset Value:** '0 (zero)
- **Operation:** Increments every clock cycle until reaching MAX

18.5.2 State Machine Behavior

Behaviorally, it's a tiny state machine:

1. **Reset State:** r_count = 0, tick = 0
2. **Counting State:** r_count increments each cycle
3. **Maximum State:** r_count = MAX, tick = 1 for one cycle
4. **Wrap State:** r_count returns to 0, cycle repeats

18.5.3 Main Counter Logic

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_count <= '0;
    end else begin
        if (r_count == MAX[$clog2(MAX+1)-1:0]) begin
            r_count <= '0;
        end else begin
            r_count <= r_count + 1'b1;
        end
    end
end
```

18.5.4 Tick Generation

```
assign tick = (r_count == MAX[$clog2(MAX+1)-1:0]);
```

18.5.5 Key Features

- **Automatic Width Sizing:** Counter width adapts to MAX parameter
- **Wrap-around Operation:** Automatically returns to zero after MAX
- **Single Cycle Tick:** Tick pulse lasts exactly one clock cycle

- **Asynchronous Reset:** Immediate reset capability

18.6 Timing Characteristics

18.6.1 Period Calculation

- **Total Period:** $\text{MAX} + 1$ clock cycles
- **Tick Frequency:** $f_{\text{clk}} / (\text{MAX} + 1)$
- **Duty Cycle:** $1 / (\text{MAX} + 1)$

18.6.2 Examples

MAX Value	Counter Width	Period (cycles)	Tick Frequency
7	3 bits	8	$f_{\text{clk}}/8$
15	4 bits	16	$f_{\text{clk}}/16$
99	7 bits	100	$f_{\text{clk}}/100$
999	10 bits	1000	$f_{\text{clk}}/1000$

18.6.3 Resource Usage

- **LUTs:** Approximately $\lceil \log_2(\text{MAX}+1) \rceil$ LUTs for the counter
- **FFs:** $\lceil \log_2(\text{MAX}+1) \rceil$ flip-flops for the counter register
- **Logic Levels:** Single level for tick generation

18.6.4 Maximum Frequency

- **Maximum Frequency:** Limited by counter increment logic
- **Reset Recovery:** One clock cycle after reset deassertion
- **Propagation Delay:** Tick output has minimal combinational delay

18.6.5 Power Optimization

- **Clock Gating:** Consider gating when tick output not needed
- **Reset Strategy:** Asynchronous reset reduces reset tree loading
- **Parameter Selection:** Smaller MAX values reduce power consumption

18.7 Usage Examples

18.7.11. Basic Timer (1ms tick at 100MHz)

```
counter #(
    .MAX(99999) // 100,000 cycles = 1ms at 100MHz
) timer_1ms (
    .clk(clk_100mhz),
    .rst_n(rst_n),
    .tick(tick_1ms)
);
```

18.7.22. Heartbeat Generator (1Hz at 50MHz)

```
counter #(
    .MAX(49999999) // 50M cycles = 1 second at 50MHz
) heartbeat (
    .clk(clk_50mhz),
    .rst_n(rst_n),
    .tick(heartbeat_1hz)
);
```

18.7.33. Simple Divider (Divide by 8)

```
counter #(
    .MAX(7) // 8 cycles total
) div8 (
    .clk(clk_in),
    .rst_n(rst_n),
    .tick(clk_div8)
);
```

1. **Periodic Timers:** Generating regular time intervals
2. **Clock Division:** Creating slower clock enables
3. **Timeout Generation:** Watchdog timers and timeouts
4. **Sampling Control:** Periodic data sampling triggers
5. **LED Blinking:** Visual indicators and status displays
6. **Protocol Timing:** Communication protocol timeouts

18.8 Testing

18.8.1 Test Scenarios

1. **Basic Counting:** Verify counting from 0 to MAX

2. **Wrap Behavior:** Confirm return to zero after MAX
3. **Tick Generation:** Verify single-cycle tick pulse
4. **Reset Functionality:** Test asynchronous reset operation
5. **Parameter Sweep:** Test various MAX values

18.8.2 Coverage Points

- Counter reaches MAX value
- Counter wraps from MAX to 0
- Tick pulse generation timing
- Reset during counting operation

18.9 Design Notes

18.9.1 Parameter Constraints

```
initial begin
    assert (MAX > 0) else $error("MAX must be positive");
    assert (MAX < 2**31) else $warning("Large MAX may impact
synthesis");
end
```

18.9.2 Optimization Hints

- **Binary MAX Values:** Powers of 2 minus 1 optimize better (7, 15, 31, etc.)
- **Tool Directives:** Consider synthesis attributes for timing-critical paths
- **Reset Inference:** Ensure reset is properly inferred as asynchronous

18.9.3 Limitations

1. **No Enable Control:** Always counting when not in reset
2. **Up-Counter Only:** Cannot count down or bidirectionally
3. **Fixed Increment:** Always increments by 1
4. **No Load Capability:** Cannot load arbitrary values
5. **Single Output:** Only provides tick signal, not count value

Know these going in. If you need any of them, you want `counter_load_clear` — don't bolt an enable onto this one and call it done.

18.10 Related Modules

- **`counter_load_clear`** - Adds load and clear functionality
- **`counter_bin`** - Specialized for FIFO applications
- **`counter_freq_invariant`** - Frequency-independent timing
- **`clock_pulse`** - Similar functionality with different interface

18.11 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

19 Ring Counter

19.1 Overview

The `counter_ring` module is the classic ring counter: a single '1' bit circulating through a shift register. The result is a one-hot encoded sequence — exactly what you want for sequential enable signals, walking LED patterns, or multi-phase operations where precisely one stage may be active at a time.

19.2 Module Interface

```
module counter_ring #(
    parameter int WIDTH = 4
) (
    input wire      clk,
    input wire      rst_n,
    input wire      enable,
    output reg [WIDTH-1:0] ring_out
);
```

19.3 Parameters

Parameter	Type	Default	Description
WIDTH	int	4	Number of stages in the ring counter

WIDTH must be ≥ 2 . At WIDTH=1 the rotate expression `{ring_out[0], ring_out[WIDTH-1:1]}` becomes the illegal reversed part-select `ring_out[0:1]` and the module does not elaborate. The counter generates WIDTH unique states, with a single '1' bit rotating through WIDTH positions.

19.4 Ports

19.4.1 Inputs

Port	Width	Description
clk	1	System clock input
rst_n	1	Active-low asynchronous reset
enable	1	Counter enable control

19.4.2 Outputs

Port	Width	Description
ring_out	WIDTH	Ring counter output (one-hot encoded)

19.5 Functional Description

19.5.1 Ring Counter Principle

A ring counter is just a circular shift register: - **Initialization:** Single '1' bit in LSB position - **Operation:** The '1' bit rotates right each clock cycle - **Feedback:** LSB connects back to MSB (no inversion) - **States:** WIDTH unique one-hot states

19.5.2 Mathematical Representation

For each clock cycle when enabled (`ring_out <= {ring_out[0], ring_out[WIDTH-1:1]}`):

```
ring_out[i] = ring_out[i+1] for i = 0 to WIDTH-2
ring_out[WIDTH-1] = ring_out[0]
```

From reset 0001 this gives 0001 -> 1000 -> 0100 -> 0010 -> 0001, matching the state tables below.

19.5.3 Core Logic

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        ring_out <= 'b1; // Initialize with LSB set
    end else if (enable) begin
        // Right rotate operation
        ring_out <= {ring_out[0], ring_out[WIDTH-1:1]};
    end
end
```

19.5.4 Operation Breakdown

1. **Reset State:** `ring_out = 'b1` (only LSB set)
2. **Rotation:** `{ring_out[0], ring_out[WIDTH-1:1]}`
 - LSB becomes new MSB
 - All other bits shift right (toward the LSB)
3. **Enable Control:** Only advances when enable is high
4. **Circular:** After WIDTH cycles, returns to initial state

19.5.5 State Sequences

19.5.6 4-bit Ring Counter (WIDTH=4)

Step	ring_out	Binary	One-Hot Position
0	0001	Reset state	Position 0 active
1	1000	After 1 clock	Position 3 active
2	0100	After 2 clocks	Position 2 active
3	0010	After 3 clocks	Position 1 active
4	0001	After 4 clocks	Back to start

19.5.7 3-bit Ring Counter (WIDTH=3)

Step	ring_out	Active Stage
0	001	Stage 0
1	100	Stage 2
2	010	Stage 1
3	001	Stage 0 (repeat)

19.5.8 8-bit Ring Counter Example

Step 0: 00000001 (Stage 0)
Step 1: 10000000 (Stage 7)
Step 2: 01000000 (Stage 6)
Step 3: 00100000 (Stage 5)
Step 4: 00010000 (Stage 4)
Step 5: 00001000 (Stage 3)
Step 6: 00000100 (Stage 2)
Step 7: 00000010 (Stage 1)
Step 8: 00000001 (Stage 0) - Cycle repeats

19.5.9 Properties and Characteristics

19.5.10 One-Hot Encoding

Ring counters hand you one-hot encoding for free: - **Exactly One Bit Set**: Always exactly one bit is '1' - **Unique States**: Each state is distinctly different - **Easy Decode**: No additional decoding logic needed - **Glitch-Free**: Clean transitions between states

19.5.11 Self-Correcting Behavior

An honest word about self-correction — ring counters mostly don't: - **All Zeros**: Will remain stuck (not self-correcting) - **Multiple Ones**: Will maintain multiple bits indefinitely - **Single One**: Will operate correctly

If an upset can land you in an illegal state, see the self-correcting variant under Advanced Variants.

19.5.12 Comparison with Other Counters

Counter Type	States	Decode Logic	One-Hot	Power
Binary	2^N	Complex	No	Low
Gray	2^N	Moderate	No	Low
Johnson	$2 \times N$	Moderate	No	Low
Ring	N	None	Yes	Medium

19.5.13 Advanced Variants

19.5.14 1. Self-Correcting Ring Counter

```
// Add error correction for stuck states
logic error_detected;
logic [WIDTH-1:0] corrected_ring;

assign error_detected = (ring_out == 'b0) || ($countones(ring_out) != 1);

always_comb begin
    if (error_detected)
        corrected_ring = 'b1; // Force to valid state
    else
        corrected_ring = ring_out;
end
```

19.5.15 2. Bidirectional Ring Counter

```
logic direction; // 0 = right, 1 = left

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        ring_out <= 'b1;
    end else if (enable) begin
        if (direction)
            // Left rotate
            ring_out <= {ring_out[WIDTH-2:0], ring_out[WIDTH-1]};
        else
```

```

        // Right rotate
        ring_out <= {ring_out[0], ring_out[WIDTH-1:1]};
    end
end

```

19.5.16 3. Programmable Ring Counter

```

logic [WIDTH-1:0] init_pattern;
logic load_pattern;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        ring_out <= 'b1;
    end else if (load_pattern) begin
        ring_out <= init_pattern;
    end else if (enable) begin
        ring_out <= {ring_out[0], ring_out[WIDTH-1:1]};
    end
end

```

19.6 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

19.6.1 Maximum Frequency

- **Typical:** 400-600 MHz in modern FPGAs
- **Limitation:** Shift register timing
- **Advantage:** Very simple logic path

19.6.2 Power Consumption

- **Dynamic:** Proportional to switching frequency
- **Optimization:** Higher switching than binary counters

- **Trade-off:** Simplicity vs. power efficiency

19.7 Usage Examples

19.7.1 1. Sequential Enable Generator

```
counter_ring #(
    .WIDTH(4)
) seq_enable (
    .clk(clk),
    .rst_n(rst_n),
    .enable(sequence_active),
    .ring_out(stage_enables)
);

// Use each bit as an enable for different stages
assign stage0_enable = stage_enables[0];
assign stage1_enable = stage_enables[1];
assign stage2_enable = stage_enables[2];
assign stage3_enable = stage_enables[3];

// Example: Pipeline stage control
always_ff @(posedge clk) begin
    if (stage0_enable) stage0_data <= input_data;
    if (stage1_enable) stage1_data <= process_stage0(stage0_data);
    if (stage2_enable) stage2_data <= process_stage1(stage1_data);
    if (stage3_enable) output_data <= process_stage2(stage2_data);
end
```

19.7.2.2. LED Chaser Display

```
// Create walking LED pattern
counter_ring #(
    .WIDTH(8)
) led_chaser (
    .clk(slow_clk),           // ~2Hz for visible effect
    .rst_n(rst_n),
    .enable(chase_enabled),
    .ring_out(led_pattern)
);

// Connect directly to LEDs
assign leds[7:0] = led_pattern;

// Optional: Bidirectional chaser
logic direction;
logic [7:0] led_display;
```

```

always_comb begin
    if (direction)
        led_display = led_pattern;
    else
        led_display = {led_pattern[0], led_pattern[7:1]}; // Reverse
end

```

19.7.33. Memory Bank Sequencer

```

counter_ring #(
    .WIDTH(4)
) bank_sequencer (
    .clk(clk),
    .rst_n(rst_n),
    .enable(memory_active),
    .ring_out(bank_select)
);

// Memory bank control
logic [3:0] bank_cs_n;
assign bank_cs_n = ~bank_select; // Active low chip selects

// Example: Round-robin memory access
always_comb begin
    mem_addr = base_addr;
    mem_data_out = 32'h0;

    case (bank_select)
        4'b0001: begin
            mem_addr = bank0_addr;
            mem_data_out = bank0_data;
        end
        4'b0010: begin
            mem_addr = bank1_addr;
            mem_data_out = bank1_data;
        end
        4'b0100: begin
            mem_addr = bank2_addr;
            mem_data_out = bank2_data;
        end
        4'b1000: begin
            mem_addr = bank3_addr;
            mem_data_out = bank3_data;
        end
    endcase
end

```

19.7.44. State Machine Controller

```
// Simple state machine using ring counter
counter_ring #(
    .WIDTH(5)
) state_machine (
    .clk(clk),
    .rst_n(rst_n),
    .enable(state_advance),
    .ring_out(current_state)
);

// Decode states
assign state_idle    = current_state[0];
assign state_request = current_state[1];
assign state_wait    = current_state[2];
assign state_active  = current_state[3];
assign state_done    = current_state[4];

// State machine logic
always_ff @(posedge clk) begin
    case (1'b1) // One-hot case statement
        state_idle:    state_advance <= start_request;
        state_request: state_advance <= request_granted;
        state_wait:    state_advance <= ready_signal;
        state_active:  state_advance <= operation_complete;
        state_done:    state_advance <= 1'b1; // Always advance from
done
        default:      state_advance <= 1'b0;
    endcase
end
```

19.7.55. Clock Domain Multiplexer

```
// Sequentially select different clock domains
counter_ring #(
    .WIDTH(3)
) clk_mux_ctrl (
    .clk(control_clk),
    .rst_n(rst_n),
    .enable(mux_advance),
    .ring_out(clk_select)
);

// Clock multiplexer (use proper clock muxing in real design)
logic selected_clk;
always_comb begin
    case (clk_select)
        3'b001: selected_clk = clk_domain_0;
```



```

        3'b010: selected_clk = clk_domain_1;
        3'b100: selected_clk = clk_domain_2;
        default: selected_clk = 1'b0;
    endcase
end

```

19.8 Testing

19.8.1 Test Scenarios

1. **Reset Behavior:** Verify initialization to single '1' bit
2. **Rotation Sequence:** Check complete WIDTH-step cycle
3. **Enable Control:** Test hold behavior when disabled
4. **One-Hot Property:** Ensure exactly one bit set always
5. **Cycle Completion:** Verify return to initial state

19.8.2 Coverage Model

```

covergroup ring_cg @(posedge clk);
    cp_ring_state: coverpoint ring_out {
        bins valid_states[] = {1, 2, 4, 8, 16, 32, 64, 128}; // For
WIDTH=8
        bins invalid_states[] = default;
    }

    cp_enable: coverpoint enable {
        bins enabled = {1};
        bins disabled = {0};
    }

    cp_one_hot: coverpoint $countones(ring_out) {
        bins exactly_one = {1};
        bins zero = {0};
        bins multiple = {[2:WIDTH]};
    }

    // Verify complete cycle
    cp_transitions: coverpoint ring_out {
        bins cycle[] = (1 => 8), (8 => 4), (4 => 2), (2 => 1); // For
WIDTH=4
    }
endcovergroup

```

19.8.3 Assertions

```
// Exactly one bit should be set (except during reset)
property one_hot_property;
    @(posedge clk) disable iff (!rst_n)
        $countones(ring_out) == 1;
endproperty

// After reset, should start with LSB set
property reset_state;
    @(posedge clk)
        $rose(rst_n) |-> ring_out == 1;
endproperty

// When enabled, should rotate right
property rotation_property;
    @(posedge clk) disable iff (!rst_n)
        enable | => ring_out == {$past(ring_out[0]), $past(ring_out[WIDTH-1:1])};
endproperty

assert property (one_hot_property);
assert property (reset_state);
assert property (rotation_property);
```

19.9 Design Notes

19.9.1 Resource Utilization

- **Flip-Flops:** WIDTH registers
- **LUTs:** Minimal - just shift logic
- **Routing:** Simple interconnections

19.9.2 Optimization Tips

```
// Use SRL (Shift Register LUT) inference when appropriate
(* SRL_STYLE = "register" *) reg [WIDTH-1:0] ring_out;

// For very wide counters, consider using dedicated shift register
primitives
```

19.9.3 Timing Considerations

- **Clock Skew:** Ensure clean clock distribution
- **Setup/Hold:** Standard flip-flop requirements

- **Routing Delay:** Minimal due to simple structure

19.9.4 Common Pitfalls

1. Invalid Initial States

```
// Problem: Multiple bits set
ring_out <= 4'b1001; // Invalid - two bits set
```

```
// Solution: Use proper initialization
ring_out <= 4'b0001; // Valid - exactly one bit
```

2. Stuck at Zero

```
// Problem: All zeros state
if (ring_out == 'b0) begin
    ring_out <= 'b1; // Force recovery
end
```

3. Width Mismatch

```
// Problem: Initialization doesn't match width
parameter WIDTH = 8;
ring_out <= 4'b0001; // Wrong - should be 8'b00000001
```

```
// Solution: Use proper width
ring_out <= 'b1; // Correct - sized automatically
```

19.10 Related Modules

- **counter_johnson** - Twisted-ring counter with $2 \times \text{WIDTH}$ states
- **counter_bingray** - Binary/Gray counter for different applications
- Standard shift register implementations
- One-hot state machines

19.11 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

20 Count Leading Zeros

20.1 Overview

The `count_leading_zeros` module counts how many consecutive zero bits sit at the beginning (MSB side) of a data word. This is fundamental computer arithmetic — you'll find it inside floating-point normalization, priority encoders, and half the bit-manipulation tricks in the book.

The scan starts at `data[WIDTH-1]` and proceeds downward, stopping at the first set bit:

```
data = 32'h8000_0000 -> clz = 0    (MSB is already set)
data = 32'h0000_0001 -> clz = 31   (31 zeros above the LSB)
data = 32'h0000_0000 -> clz = WIDTH
```

For the complementary count from the LSB upward, use [count_trailing_zeros](#).

History - do not reintroduce input reversal. Earlier revisions of this module scanned from bit 0 upward, which computes *trailing* zeros, not leading zeros, despite the module name. Callers compensated by bit-reversing their input before connecting it. Both the module and every caller have been corrected: the module now performs a true MSB-down scan, and callers pass their operand straight in. Any reversal generate block or `w*_reversed` wire feeding this module is stale and must not be re-added. If you actually want the LSB-up count, instantiate `count_trailing_zeros` instead of reversing the input.

20.2 Module Interface

```
module count_leading_zeros #(
    parameter int WIDTH = 32
) (
    input logic [WIDTH-1:0] data,
    output logic [$clog2(WIDTH):0] clz
);
```

20.3 Parameters

Parameter	Type	Default	Description
WIDTH	int	32	Bit width of the input data

WIDTH can be any positive integer ≥ 1 ; 8, 16, 32, and 64 are the common values for standard data widths. It determines the output width and algorithm complexity, and it's the only parameter.

20.4 Ports

20.4.1 Inputs

Port	Width	Description
data	WIDTH	Input data word to analyze

20.4.2 Outputs

Port	Width	Description
clz	$\$clog2(WIDTH)+1$	Count of leading zeros

The output width $\$clog2(WIDTH)+1$ ensures the port can represent all possible counts: - **Range:** 0 to WIDTH (inclusive) - **Example:** For WIDTH=32, output is 6 bits (0-32 requires 6 bits) - **Special Case:** All zeros input produces CLZ = WIDTH

20.5 Functional Description

20.5.1 Function-Based Approach

```
function automatic [$clog2(WIDTH):0] clz_func;
    input [WIDTH-1:0] input_data;
    logic found;
    begin
        clz_func = 0;
        found = 1'b0;
        // Scan from the MSB downward.
        for (int i = WIDTH - 1; i >= 0; i--) begin
            if (!input_data[i] && !found) begin
                clz_func += 1;
            end else begin
                found = 1'b1; // Stop counting when first '1' found
            end
        end
    end
endfunction
```

20.5.2 Bit Scanning Process

The algorithm scans from MSB to LSB: 1. **Initialize:** $clz_func = 0$, $found = 0$ 2. **Scan Loop:** For each bit position i from $WIDTH-1$ down to 0: - If $data[i] == 0$ AND no '1' found yet: increment count - If $data[i] == 1$: set found flag, stop counting 3. **Result:** Final count is the number of zeros above the highest set bit

20.5.3 Why MSB-to-LSB Scanning?

“Leading” refers to the bits that lead the word when it is written out, which is the MSB side. The loop therefore starts at data[WIDTH-1] and walks down: - **Bit Order**: data[0] is LSB, data[WIDTH-1] is MSB - **Termination**: The found flag latches at the highest set bit, so all zeros below it are ignored - only the contiguous run above it is counted - **Count Logic**: clz = (WIDTH-1) - position_of_highest_set_one, and clz = WIDTH when no bit is set

Because the scan direction already matches the definition, the input must be fed in directly. Reversing data before connecting it turns this module back into a trailing zero counter - see the history note above.

20.5.4 Advanced Variants

20.5.5 1. Pipeline Version for High Speed

```
module count_leading_zeros_pipelined #(
    parameter int WIDTH = 32,
    parameter int STAGES = 2
) (
    input logic clk,
    input logic rst_n,
    input logic [WIDTH-1:0] data,
    output logic [$clog2(WIDTH):0] clz
);

// Stage 1: Partial counting
logic [WIDTH-1:0] data_s1;
logic [$clog2(WIDTH):0] partial_clz_s1;

always_ff @(posedge clk) begin
    data_s1 <= data;
    // Count leading zeros in upper half
    partial_clz_s1 <= clz_upper_half(data);
end

// Stage 2: Complete counting
always_ff @(posedge clk) begin
    if (partial_clz_s1 == WIDTH/2) begin
        // All upper bits are zero, count in lower half
        clz <= WIDTH/2 + clz_lower_half(data_s1);
    end else begin
        // Found '1' in upper half
        clz <= partial_clz_s1;
    end
end

endmodule
```

20.5.62. Hierarchical Implementation

```
// Divide and conquer approach for large widths
module count_leading_zeros_hierarchical #(
    parameter int WIDTH = 64
) (
    input logic [WIDTH-1:0] data,
    output logic [$clog2(WIDTH):0] clz
);

localparam int HALF_WIDTH = WIDTH/2;

logic [$clog2(HALF_WIDTH):0] clz_upper, clz_lower;
logic upper_all_zeros;

// Count in upper half
count_leading_zeros #(.WIDTH(HALF_WIDTH)) upper_clz (
    .data(data[WIDTH-1:HALF_WIDTH]),
    .clz(clz_upper)
);

// Count in lower half
count_leading_zeros #(.WIDTH(HALF_WIDTH)) lower_clz (
    .data(data[HALF_WIDTH-1:0]),
    .clz(clz_lower)
);

assign upper_all_zeros = (clz_upper == HALF_WIDTH);

// Combine results
assign clz = upper_all_zeros ? (HALF_WIDTH + clz_lower) : clz_upper;

endmodule
```

20.5.73. LUT-Based Implementation (Small Widths)

```
// Optimized for small widths using a wildcard case statement.
// casez is required so that '?' is treated as a don't-care.
module count_leading_zeros_lut #(
    parameter int WIDTH = 8
) (
    input logic [WIDTH-1:0] data,
    output logic [$clog2(WIDTH):0] clz
);

always_comb begin
    casez (data)
```

```

        8'b00000000: clz = 8;
        8'b00000001: clz = 7;
        8'b0000001?: clz = 6;
        8'b000001?: clz = 5;
        8'b00001???: clz = 4;
        8'b0001????: clz = 3;
        8'b001?????: clz = 2;
        8'b01?????: clz = 1;
        8'b1?????: clz = 0;
        default:      clz = 0;
    endcase
end

endmodule

```

20.6 Usage Examples

20.6.1 1. Floating-Point Normalization

```

// IEEE 754 floating-point normalization
logic [31:0] mantissa_raw;
logic [5:0] shift_amount;
logic [31:0] normalized_mantissa;

count_leading_zeros #(.WIDTH(32)) norm_clz (
    .data(mantissa_raw),
    .clz(shift_amount)
);

// Normalize mantissa by shifting left
assign normalized_mantissa = mantissa_raw << shift_amount;

// Adjust exponent accordingly
logic [7:0] adjusted_exponent;
assign adjusted_exponent = raw_exponent - shift_amount;

```

20.6.22. Priority Encoder

```

// Find highest priority request
logic [15:0] request_vector;
logic [4:0] leading_zeros;
logic [3:0] highest_priority;
logic any_request;

count_leading_zeros #(.WIDTH(16)) priority_enc (
    .data(request_vector),
    .clz(leading_zeros)
);

```



```

assign any_request = (leading_zeros != 16);
assign highest_priority = any_request ? (15 - leading_zeros) : 4'b0;

```

```

// Example: request_vector = 16'b0000_0100_1000_0000
// Highest set bit is bit 10
// CLZ = 5 (five leading zeros above bit 10)
// highest_priority = 15 - 5 = 10

```

20.6.33. Bit Width Calculation

```

// Determine minimum bits needed to represent a number

```

```

logic [31:0] number;
logic [5:0] min_bits_needed;

```

```

count_leading_zeros #(.WIDTH(32)) bit_width_calc (
    .data(number),
    .clz(leading_zeros)
);

```

```

assign min_bits_needed = (number == 0) ? 1 : (32 - leading_zeros);

```

```

// Examples:
// number = 0      → min_bits = 1
// number = 1      → min_bits = 1
// number = 255    → min_bits = 8
// number = 256    → min_bits = 9
// number = 65535  → min_bits = 16

```

20.6.44. Log2 Calculation

```

// Calculate floor(log2(x)) for x > 0

```

```

logic [31:0] input_value;
logic [4:0] log2_result;
logic valid;

```

```

count_leading_zeros #(.WIDTH(32)) log2_calc (
    .data(input_value),
    .clz(leading_zeros)
);

```

```

assign valid = (input_value != 0);
assign log2_result = valid ? (31 - leading_zeros) : 5'b0;

```

```

// Examples:
// input = 1      → log2 = 0
// input = 2      → log2 = 1

```

```
// input = 4    → log2 = 2
// input = 7    → log2 = 2 (floor)
// input = 8    → log2 = 3
```

20.6.55. Data Compression - Run Length Encoding

```
// Count leading zeros for compression
logic [63:0] data_block;
logic [6:0] zero_run_length;

count_leading_zeros #(.WIDTH(64)) rle_counter (
    .data(data_block),
    .clz(zero_run_length)
);

// Use in compression algorithm
if (zero_run_length > THRESHOLD) begin
    // Encode as zero run
    compressed_data = {RLE_CODE, zero_run_length};
end else begin
    // Encode as literal data
    compressed_data = {LITERAL_CODE, data_block};
end
```

1. **CPU/DSP Cores:** Instruction implementation (CLZ instruction)
2. **Floating-Point Units:** Mantissa normalization
3. **Network Processors:** Longest prefix matching
4. **Compression Engines:** Run-length encoding optimization
5. **Memory Controllers:** Priority arbitration
6. **Graphics Processors:** Texture coordinate processing
7. **Cryptographic Units:** Bit manipulation operations

20.6.6 Examples and Truth Tables

20.6.7 8-bit Examples (WIDTH=8)

The count is set by the **highest** set bit; bits below it never affect the result.

Input (data)	Binary	Highest set bit	Leading Zeros	CLZ Output
8'b00000000	00000000	None	8	8
0				
8'b00000001	00000001	Bit 0	7	7
1				
8'b00000010	00000010	Bit 1	6	6
0				
8'b00000100	00000100	Bit 2	5	5
0				
8'b00001000	00001000	Bit 3	4	4
0				
8'b00010000	00010000	Bit 4	3	3
0				
8'b00100000	00100000	Bit 5	2	2
0				
8'b01000000	01000000	Bit 6	1	1
0				
8'b10000000	10000000	Bit 7	0	0
0				
8'b11111111	11111111	Bit 7	0	0
1				
8'b10101010	10101010	Bit 7	0	0
0				
8'b00010101	00010101	Bit 4	3	3
1				
8'b00110000	00110000	Bit 5	2	2
0				

20.6.8 32-bit Examples

Input	Hex	Leading Zeros	CLZ
32'h00000000	0x00000000	32	32
32'h00000001	0x00000001	31	31
32'h00000080	0x00000080	24	24

Input	Hex	Leading Zeros	CLZ
32'h00008000	0x00008000	16	16
32'h00800000	0x00800000	8	8
32'h80000000	0x80000000	0	0
32'hFFFFFFFF	0xFFFFFFFF	0	0

20.7 Timing Characteristics

20.7.1 Resource Utilization

WIDTH	LUTs (Typical)	Delay Levels	Max Frequency
8	15-20	3-4	500+ MHz
16	30-40	4-5	400+ MHz
32	60-80	5-6	300+ MHz
64	120-150	6-7	250+ MHz

20.7.2 Timing Characteristics

- **Combinational Delay:** $O(\log(\text{WIDTH}))$ for tree implementations
- **Critical Path:** Through the priority encoding logic
- **Scalability:** Linear increase in logic for function implementation

20.8 Testing

20.8.1 Test Scenarios

1. **Boundary Cases:** All zeros, all ones, single bit patterns
2. **Random Patterns:** Comprehensive random testing
3. **Systematic Sweep:** Test all possible leading zero counts
4. **Corner Cases:** Maximum width values, alternating patterns

20.8.2 Coverage Model

```
covergroup clz_cg;
  cp_leading_zeros: coverpoint clz {
    bins zero_clz = {0};
    bins low_clz = {[1:WIDTH/4]};
    bins mid_clz = {[WIDTH/4+1:3*WIDTH/4]};
  }
endgroup
```

```

        bins high_clz = {[3*WIDTH/4+1:WIDTH-1]};
        bins all_zeros = {WIDTH};
    }

    cp_data_patterns: coverpoint data {
        bins all_zeros = {'0'};
        bins all_ones = {'1'};
        bins single_bit[] = {1, 2, 4, 8, 16, 32}; // For appropriate
WIDTH
        bins msb_set = {1 << (WIDTH-1)};
        bins mixed[] = {[1:2** (WIDTH-1)-1]};
    }
endcovergroup

```

20.8.3 Assertions

```

// CLZ should never exceed WIDTH
property clz_bounds;
    clz <= WIDTH;
endproperty

// All zeros should give CLZ = WIDTH
property all_zeros_case;
    (data == '0') |-> (clz == WIDTH);
endproperty

// MSB set should give CLZ = 0
property msb_set_case;
    data[WIDTH-1] |-> (clz == 0);
endproperty

// Relationship between CLZ and first '1' position
property clz_correctness;
    logic [WIDTH-1:0] shifted;
    assign shifted = data << clz;
    (data != '0') |-> shifted[WIDTH-1];
endproperty

assert property (clz_bounds);
assert property (all_zeros_case);
assert property (msb_set_case);
assert property (clz_correctness);

```

20.9 Design Notes

20.9.1 Tool-Specific Attributes

```
// Xilinx: Prevent SRL inference for better timing  
(* SRL_STYLE = "register" *) logic [WIDTH-1:0] data_reg;
```

```
// Quartus: Use specific LUT implementation  
// synthesis attribute LUT_SIZE of clz_func is 6;
```

20.9.2 Area vs. Speed Trade-offs

```
// For area optimization: Use iterative approach  
// For speed optimization: Use tree-based or LUT approach  
// For power optimization: Add enable signals and clock gating
```

20.10 Related Modules

- [count_trailing_zeros](#) - the LSB-up counterpart. Use CLZ when you care about the magnitude of a value (normalization, log2, bit width); use CTZ when you care about alignment or the lowest pending request.
- Population count (number of '1' bits)
- Find first set (FFS) / Find last set (FLS)
- Priority encoders
- Barrel shifters

20.11 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

21 count_trailing_zeros

21.1 Overview

The `count_trailing_zeros` module counts how many consecutive zero bits sit at the end (LSB side) of a data word. It's the mirror image of `count_leading_zeros`, and it's the natural primitive for alignment checks, lowest-set-bit extraction, and picking the least significant pending request out of a vector.

The scan starts at `data[0]` and walks upward, stopping at the first set bit:

```
data = 32'h0000_0001 -> ctz = 0    (bit 0 is already set)
data = 32'h8000_0000 -> ctz = 31   (31 zeros below the MSB)
data = 32'h0000_0000 -> ctz = WIDTH
```

For the complementary count from the MSB downward, use [count_leading_zeros](#).

21.1.1 Choosing Between CLZ and CTZ

Question you are asking	Module
How big is this value? (normalization, log2, minimum bit width)	<code>count_leading_zeros</code>
How is this value aligned? (address alignment, burst size, power-of-two stride)	<code>count_trailing_zeros</code>
Which is the highest priority request set?	<code>count_leading_zeros</code>
Which is the lowest index request set?	<code>count_trailing_zeros</code>

Don't emulate one by bit-reversing the input to the other. Both modules exist; picking the right one costs no extra logic and keeps your intent visible in the netlist.

21.1.2 Module Declaration

```
module count_trailing_zeros #(
    parameter int WIDTH = 32
) (
    input logic [WIDTH-1:0] data,
    output logic [$clog2(WIDTH):0] ctz
);
```

21.2 Parameters

Parameter	Default	Description
WIDTH	32	Bit width of the input data

WIDTH can be any positive integer ≥ 1 ; 8, 16, 32, and 64 are the common values for standard data widths. It sets the output width and the algorithm complexity, and it's the only parameter.

21.3 Ports

Port	Direction	Width	Description
data	Input	WIDTH	Input data word to analyze
ctz	Output	$\$clog2(WIDTH) + 1$	Count of trailing zeros

The output width $\$clog2(WIDTH) + 1$ lets the port represent every possible count: - **Range:** 0 to WIDTH (inclusive) - **Example:** For WIDTH=32, output is 6 bits (0-32 requires 6 bits) - **Special Case:** All zeros input produces CTZ = WIDTH

21.4 Functional Description

21.4.1 Function-Based Approach

```
function automatic [ $clog2(WIDTH):0 ] ctz_func;
    input [WIDTH-1:0] input_data;
    logic found;
    begin
        ctz_func = 0;
        found = 1'b0;
        // Scan from the LSB upward.
        for (int i = 0; i < WIDTH; i++) begin
            if (!input_data[i] && !found) begin
                ctz_func += 1;
            end else begin
                found = 1'b1; // Stop counting when first '1' found
            end
        end
    end
endfunction
```

21.4.2 Bit Scanning Process

The algorithm scans from LSB to MSB: 1. **Initialize:** $ctz_func = 0$, $found = 0$ 2. **Scan Loop:** For each bit position i from 0 to $WIDTH-1$: - If $data[i] == 0$ AND no '1' found yet: increment count - If $data[i] == 1$: set found flag, stop counting 3. **Result:** Final count is the number of zeros below the lowest set bit

21.4.3 Why LSB-to-MSB Scanning?

“Trailing” means the bits that trail the word when it’s written out — the LSB side. So the loop starts at data[0] and walks up: - **Bit Order**: data[0] is LSB, data[WIDTH-1] is MSB -

Termination: The found flag latches at the lowest set bit, so every bit above it is ignored - only the contiguous run below it is counted - **Count Logic**: ctz = position_of_lowest_set_one, and ctz = WIDTH when no bit is set

21.4.4 8-bit Examples (WIDTH=8)

The count is set by the **lowest** set bit; bits above it never affect the result.

Input (data)	Binary	Lowest set bit	Trailing Zeros	CTZ Output
8'b00000000	0000000 0	None	8	8
8'b00000001	0000000 1	Bit 0	0	0
8'b00000010	0000001 0	Bit 1	1	1
8'b00000100	0000010 0	Bit 2	2	2
8'b00001000	0000100 0	Bit 3	3	3
8'b00010000	0001000 0	Bit 4	4	4
8'b00100000	0010000 0	Bit 5	5	5
8'b01000000	0100000 0	Bit 6	6	6
8'b10000000	1000000 0	Bit 7	7	7
8'b11111111	1111111 1	Bit 0	0	0
8'b10101010	1010101 0	Bit 1	1	1
8'b00110000	0011000 0	Bit 4	4	4

21.4.5 32-bit Examples

Input	Hex	Trailing Zeros	CTZ
32'h00000000	0x00000000	32	32
32'h00000001	0x00000001	0	0
32'h00000080	0x00000080	7	7
32'h00008000	0x00008000	15	15
32'h00800000	0x00800000	23	23
32'h80000000	0x80000000	31	31
32'hFFFFFFFF	0xFFFFFFFF	0	0

21.4.6 Relationship to CLZ

For a single-bit input data == (1 < k), the two modules are complementary:

```
ctz = k
clz = WIDTH - 1 - k
ctz + clz = WIDTH - 1
```

That identity holds **only** for one-hot inputs. For general data the two counts are independent, because CLZ is decided by the highest set bit and CTZ by the lowest. For example 8'b00110000 gives clz = 2 and ctz = 4, which sum to 6, not 7. Worth remembering before you write a check that assumes the identity holds for arbitrary data.

21.5 Timing Characteristics

21.5.1 Resource Utilization

WIDTH	LUTs (Typical)	Delay Levels	Max Frequency
8	15-20	3-4	500+ MHz
16	30-40	4-5	400+ MHz
32	60-80	5-6	300+ MHz
64	120-150	6-7	250+ MHz

21.5.2 Timing Characteristics

- **Combinational Delay:** $O(\log(\text{WIDTH}))$ for tree implementations
- **Critical Path:** Through the priority encoding logic
- **Scalability:** Linear increase in logic for the function implementation

21.6 Usage Examples

21.6.1 Address Alignment Detection

```
// Determine the largest power-of-two boundary an address is aligned
to
logic [31:0] address;
logic [5:0] alignment_log2;

count_trailing_zeros #(.WIDTH(32)) align_ctz (
    .data(address),
    .ctz(alignment_log2)
);

// alignment_log2 = 0  -> byte aligned only
// alignment_log2 = 2  -> 4-byte aligned
// alignment_log2 = 6  -> 64-byte aligned (cache line)
// alignment_log2 = 32 -> address is zero, aligned to everything
```

21.6.2 Maximum Burst Size for a DMA Transfer

```
// A burst may not cross its natural alignment boundary, so the
alignment of the
// start address caps the burst size.
logic [31:0] start_addr;
logic [5:0] addr_align;
logic [5:0] max_burst_log2;

count_trailing_zeros #(.WIDTH(32)) burst_ctz (
    .data(start_addr),
    .ctz(addr_align)
);

// Clamp against the protocol maximum (e.g. 4KB = 12)
assign max_burst_log2 = (addr_align > 6'd12) ? 6'd12 : addr_align;
```

21.6.3 Lowest-Index Request Arbitration

```
// Fixed priority arbiter favouring the lowest requester index
logic [15:0] request_vector;
logic [4:0] trailing_zeros;
logic [3:0] granted_index;
logic any_request;

count_trailing_zeros #(.WIDTH(16)) req_ctz (
    .data(request_vector),
    .ctz(trailing_zeros)
);
```

```

assign any_request  = (trailing_zeros != 16);
assign granted_index = any_request ? trailing_zeros[3:0] : 4'b0;

// Example: request_vector = 16'b0000_0100_1000_0000
// Lowest set bit is bit 7
// CTZ = 7, so requester 7 is granted

```

21.6.4 Isolating and Clearing the Lowest Set Bit

```

// Iterate over set bits one at a time (software-style "x & -x" in
// hardware)
logic [31:0] pending;
logic [5:0]  next_index;
logic [31:0] lowest_bit_mask;
logic [31:0] pending_next;

count_trailing_zeros #(.WIDTH(32)) pend_ctz (
    .data(pending),
    .ctz(next_index)
);

assign lowest_bit_mask = pending & (~pending + 32'd1); // isolates
// bit next_index
assign pending_next    = pending & ~lowest_bit_mask;  // clears it

```

21.6.5 Power-of-Two Detection

```

// A non-zero value is a power of two exactly when its only set bit is
// the lowest one
logic [31:0] value;
logic [5:0]  value_ctz;
logic        is_power_of_two;

count_trailing_zeros #(.WIDTH(32)) pow2_ctz (
    .data(value),
    .ctz(value_ctz)
);

assign is_power_of_two = (value != 0) && (value == (32'd1 <<
value_ctz));

// value = 64    -> ctz = 6,  1<<6 = 64    -> true
// value = 96    -> ctz = 5,  1<<5 = 32    -> false
// value = 0      -> false by the explicit guard

```

21.7 Design Notes

21.7.1 Hierarchical Implementation

```
// Divide and conquer approach for large widths
module count_trailing_zeros_hierarchical #(
    parameter int WIDTH = 64
) (
    input logic [WIDTH-1:0] data,
    output logic [$clog2(WIDTH):0] ctz
);

localparam int HALF_WIDTH = WIDTH/2;

logic [$clog2(HALF_WIDTH):0] ctz_upper, ctz_lower;
logic lower_all_zeros;

// Count in lower half
count_trailing_zeros #(.WIDTH(HALF_WIDTH)) lower_ctz (
    .data(data[HALF_WIDTH-1:0]),
    .ctz(ctz_lower)
);

// Count in upper half
count_trailing_zeros #(.WIDTH(HALF_WIDTH)) upper_ctz (
    .data(data[WIDTH-1:HALF_WIDTH]),
    .ctz(ctz_upper)
);

assign lower_all_zeros = (ctz_lower == HALF_WIDTH);

// Combine results
assign ctz = lower_all_zeros ? (HALF_WIDTH + ctz_upper) : ctz_lower;

endmodule
```

21.7.2 LUT-Based Implementation (Small Widths)

```
// Optimized for small widths using a wildcard case statement.
// casez is required so that '?' is treated as a don't-care.
module count_trailing_zeros_lut #(
    parameter int WIDTH = 8
) (
    input logic [WIDTH-1:0] data,
    output logic [$clog2(WIDTH):0] ctz
);
```

```

always_comb begin
    casez (data)
        8'b???????1: ctz = 0;
        8'b??????10: ctz = 1;
        8'b?????100: ctz = 2;
        8'b????1000: ctz = 3;
        8'b???10000: ctz = 4;
        8'b??100000: ctz = 5;
        8'b?1000000: ctz = 6;
        8'b10000000: ctz = 7;
        default:      ctz = 8;    // all zeros
    endcase
end

endmodule

```

21.7.3 Mask-Based Implementation

```

// Isolate the lowest set bit, then one-hot encode its position.
// Often maps well to carry-chain logic on FPGAs.
logic [WIDTH-1:0] w_lowest_one;

assign w_lowest_one = data & (~data + 1'b1);
// w_lowest_one is one-hot (or all zero), so a plain one-hot encoder
yields ctz

```

21.7.4 Synthesis Trade-offs

```

// For area optimization: Use the iterative function approach
// For speed optimization: Use the hierarchical or LUT approach
// For power optimization: Add enable signals and clock gating

```

21.7.5 Common Applications

1. **CPU/DSP Cores:** Instruction implementation (CTZ / FFS instruction)
2. **Memory Controllers:** Address alignment and burst-size derivation
3. **DMA Engines:** Legal transfer size given a start address
4. **Schedulers and Arbiters:** Lowest-index pending request selection
5. **Allocators:** Free-list and bitmap scanning
6. **Interrupt Controllers:** Lowest pending interrupt vector

21.8 Related Modules

- [count_leading_zeros](#) - the MSB-down counterpart. See the selection table at the top of this page before choosing between them.
- Population count (number of '1' bits)
- Find first set (FFS) / Find last set (FLS)
- Priority encoders
- Barrel shifters

21.9 Testing

21.9.1 Test Scenarios

1. **Boundary Cases:** All zeros, all ones, single bit patterns
2. **Random Patterns:** Comprehensive random testing
3. **Systematic Sweep:** Test all possible trailing zero counts
4. **Corner Cases:** Maximum width values, alternating patterns
5. **Cross-Check:** For one-hot inputs, confirm `ctz + clz == WIDTH-1` against `count_leading_zeros`

21.9.2 Coverage Model

```
covergroup ctz_cg;
  cp_trailing_zeros: coverpoint ctz {
    bins zero_ctz    = {0};
    bins low_ctz     = {[1:WIDTH/4]};
    bins mid_ctz     = {[WIDTH/4+1:3*WIDTH/4]};
    bins high_ctz    = {[3*WIDTH/4+1:WIDTH-1]};
    bins all_zeros   = {WIDTH};
  }

  cp_data_patterns: coverpoint data {
    bins all_zeros   = {'0'};
    bins all_ones    = {'1'};
    bins single_bit[] = {1, 2, 4, 8, 16, 32}; // For appropriate
WIDTH
    bins lsb_set     = {1};
    bins msb_set     = {1 << (WIDTH-1)};
    bins mixed[]     = {[1:2** (WIDTH-1) - 1]};
  }
endgroup
```

```

    }
endcovergroup

```

21.9.3 Assertions

```

// CTZ should never exceed WIDTH
property ctz_bounds;
    ctz <= WIDTH;
endproperty

// All zeros should give CTZ = WIDTH
property all_zeros_case;
    (data == '0') |-> (ctz == WIDTH);
endproperty

// LSB set should give CTZ = 0
property lsb_set_case;
    data[0] |-> (ctz == 0);
endproperty

// Relationship between CTZ and the lowest '1' position
// Legal SVA: no continuous assigns inside a property body -- fold the
// expression in directly (or use a `let`)
property ctz_correctness;
    (data != '0') |-> ((data >> ctz) & 1'b1);
endproperty

// CTZ must select exactly the isolated lowest set bit
property ctz_isolates_lowest;
    (data != '0') |-> ((data & (~data + 1)) == (1 << ctz));
endproperty

// Concurrent asserts need a clocking event (or a default clocking
block);
// this module is combinational, so sample on the consumer's clock:
always @(posedge clk) begin
    assert property (ctz_bounds);
    assert property (all_zeros_case);
    assert property (lsb_set_case);
    assert property (ctz_correctness);
    assert property (ctz_isolates_lowest);
end

```

21.10 Navigation

- [← Back to rtl-common Index](#)

- [← Back to Main Documentation Index](#)

22 dataint_checksum

22.1 Overview

The `dataint_checksum` module is a simple accumulating checksum calculator. It keeps adding incoming data values into a running checksum you can use for basic data integrity verification. Nothing fancy — that's the point.

22.1.1 Module Declaration

```
module dataint_checksum #(
    parameter int WIDTH = 8
) (
    input logic          clk,
    rst_n,
    input logic          reset,
    input logic          valid,
    input logic [WIDTH-1:0] data,
    output logic [WIDTH-1:0] chksum
);
```

22.2 Parameters

Parameter	Default	Description
WIDTH	8	Width of the data and checksum in bits

22.3 Ports

Port	Direction	Width	Description
clk	Input	1	Clock signal
rst_n	Input	1	Active-low asynchronous reset
reset	Input	1	Synchronous reset signal
valid	Input	1	Data valid signal - when high, data is accumulated
data	Input	WIDTH	Input data to be added to checksum
chksum	Output	WIDTH	Current checksum value

22.4 Functional Description

22.4.1 Operation

The module keeps an internal counter (`r_count`) that accumulates incoming data: - On reset (either asynchronous `rst_n` or synchronous `reset`), the counter clears to zero - When `valid` is asserted, the input data is added to the current counter value - The checksum output is simply the current counter value

22.4.2 Reset Behavior

Two reset mechanisms, and you should know which one wins: 1. **Asynchronous Reset:** `rst_n` (active-low) immediately clears the counter 2. **Synchronous Reset:** `reset` (active-high) clears the counter on the next clock edge

22.4.3 Key Features

- **Simple Accumulation:** Uses basic addition for checksum calculation
- **Parameterizable Width:** Supports any data width through the `WIDTH` parameter
- **Dual Reset Support:** Both asynchronous and synchronous reset capabilities
- **Enable Control:** Data accumulation only occurs when `valid` is asserted

22.4.4 Special Considerations

- **Overflow Handling:** The checksum wraps around when the accumulated value exceeds the maximum representable in `WIDTH` bits — expected behavior for a checksum, not a bug
- **Continuous Output:** The checksum output is always valid and reflects the current accumulated value
- **Reset Priority:** Asynchronous reset (`rst_n`) takes precedence over synchronous reset (`reset`)

22.4.5 State Machine

No formal FSM here — it operates as a simple accumulator with two states: - **RESET:** Counter is zero - **ACCUMULATING:** Counter adds valid data inputs

22.5 Related Modules

Nothing in the tree instantiates this module and it instantiates nothing: it is a leaf, used directly by whatever design needs it. Its nearest neighbours in `rtl/common/` are:

- dataint_crc
 - dataint_crc_xor_shift
 - dataint_crc_xor_shift_cascade
 - dataint_ecc_hamming_decode_secdec
 - dataint_ecc_hamming_encode_secdec
-

22.6 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

22.7 Usage Examples

```
// 16-bit checksum calculator
dataint_checksum #(
    .WIDTH(16)
) checksum_calc (
    .clk(clk),
    .rst_n(rst_n),
    .reset(soft_reset),
    .valid(data_valid),
    .data(input_data),
    .chksum(calculated_checksum)
);
```

22.8 Design Notes

22.8.1 Common Applications

- Simple data integrity checking
- Packet checksum calculation

- Basic error detection in data streams
- Accumulation of data values for verification purposes

22.9 Testing

val/common/test_dataint_checksum.py exercises this module. It collects 4 parameter cases at the default REG_LEVEL.

```
source env_python
pytest val/common/test_dataint_checksum.py -v
```

22.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

23 dataint_crc

23.1 Overview

The `dataint_crc` module is a generic Cyclic Redundancy Check (CRC) computation engine — configurable polynomials, data widths, and processing options, built for flexible, high-performance CRC calculation. It's a full CRC calculation engine for data integrity applications, communication protocols, and storage systems, with support for configurable polynomial widths, data widths, input/output reflection, and cascade processing for high-throughput applications.

23.1.1 Module Declaration

```
module dataint_crc #(
    parameter int DATA_WIDTH = 64,           // Input data width
    parameter int CRC_WIDTH = 64,           // CRC polynomial width
    parameter int REF_IN = 1,               // Reflect input data
    parameter int REF_OUT = 1               // Reflect output data
) (
    // Derived localparam (computed internally - not user-settable):
    parameter int CHUNKS = DATA_WIDTH / 8, // Derived -- do NOT
    // override (port width dependency)
    // localparam int CW = CRC_WIDTH;        // CRC width alias
    // localparam int DW = DATA_WIDTH;      // Data width alias
    // localparam int CH = CHUNKS;           // Chunks alias
    input logic [CRC_WIDTH-1:0] POLY,        // CRC polynomial
    input logic [CRC_WIDTH-1:0] POLY_INIT,   // Initial CRC
    value
    input logic [CRC_WIDTH-1:0] XOROUT,      // Output XOR mask
    input logic clk,                        // Clock
    input logic rst_n,                      // Active-low
    reset
    input logic load_crc_start,              // Load initial
    CRC
    input logic load_from_cascade,           // Load from
    cascade
    input logic [CHUNKS-1:0] cascade_sel,    // Cascade
    selection (one-hot)
    input logic [DATA_WIDTH-1:0] data,       // Input data
    output logic [CRC_WIDTH-1:0] crc        // CRC output
);
```

23.2 Parameters

23.2.1 User-Settable Parameters

Parameter	Default	Description
DATA_WIDTH	64	Input data width in

Parameter	Default	Description
CRC_WIDTH	64	bits (must be multiple of 8) CRC polynomial width in bits (8, 16, 32, or 64)
REFIN	1	Reflect input data bytes (1=reflect, 0=direct)
REFOUT	1	Reflect output CRC (1=reflect, 0=direct)

23.2.2 Calculated Parameters (Do Not Override)

Parameter	Formula	Description
CHUNKS (CH)	DATA_WIDTH/8	Number of 8-bit processing chunks per cycle
CW	CRC_WIDTH	Convenience alias for CRC width
DW	DATA_WIDTH	Convenience alias for data width

Note: CW, DW, and CH are body localparams and cannot be overridden. CHUNKS is different: it is a derived parameter (the cascade_sel port needs it, and a body localparam elaborates too late for a port width), so it *can* be overridden at instantiation — and must not be. The RTL's own parameter comment says the same: derived, not configurable, leave the default alone.

23.3 Ports

Port	Direction	Width	Description
POLY	Input	CRC_WIDTH	CRC polynomial coefficient
POLY_INIT	Input	CRC_WIDTH	Initial CRC value (seed)
XOROUT	Input	CRC_WIDTH	Final XOR mask for output
clk	Input	1	System clock

Port	Direction	Width	Description
rst_n	Input	1	Active-low asynchronous reset
load_crc_start	Input	1	Load initial CRC value
load_from_cascade	Input	1	Load CRC from cascade output
cascade_sel	Input	CHUNKS	One-hot cascade stage selection
data	Input	DATA_WIDTH	Input data for CRC calculation
crc	Output	CRC_WIDTH	Computed CRC value (registered)

23.4 Functional Description

23.4.1 CRC Algorithm Implementation

The CRC calculation is parallel and runs through four stages:

1. **Input Data Conditioning:** Optional bit reflection per REFIN parameter
2. **Cascade Processing:** Parallel processing through multiple XOR-shift stages
3. **Output Conditioning:** Optional bit reflection per REFOUT parameter
4. **Final XOR:** Application of XOROUT mask to final result

23.4.2 Key Features

- **Parallel Processing:** Processes multiple bytes simultaneously for high throughput
- **Configurable Reflection:** Support for MSB-first and LSB-first bit ordering
- **Cascade Architecture:** Enables partial CRC computation and continuation

- **Standards Compliance:** Configurable for standard CRC algorithms (CRC-32, CRC-64, etc.)

23.4.3 Input Data Reflection

```
// Input reflection based on REFIN parameter
if (REFIN != 0) begin : gen_reflect_inputs
    for (genvar i = 0; i < CH; i++) begin : gen_ch_reflect
        for (genvar j = 0; j < 8; j++) begin : gen_bit_reflect
            assign w_block_data[i][j] = data[i*8+7-j]; // Bit
        reversal
        end
    end
end else begin : gen_direct_assign_inputs
    for (genvar i = 0; i < CH; i++) begin : gen_ch_direct
        assign w_block_data[i] = data[i*8+:8]; // Direct assignment
    end
end
```

23.4.4 Cascade Processing Architecture

```
// Chain of XOR-shift cascade stages
for (i = 0; i < CH; i++) begin : gen_xor_shift_cascade
    if (i == 0) begin : gen_xor_cascade_0
        dataint_crc_xor_shift_cascade #(.CRC_WIDTH(CRC_WIDTH))
        cascade_0 (
            .block_input(r_crc_value), // Start with current CRC
            .poly(w_poly),
            .data_input(w_block_data[i]),
            .block_output(w_cascade[i])
        );
    end else begin : gen_xor_cascade_N
        dataint_crc_xor_shift_cascade #(.CRC_WIDTH(CRC_WIDTH))
        cascade_N (
            .block_input(w_cascade[i-1]), // Chain from previous
            stage
            .poly(w_poly),
            .data_input(w_block_data[i]),
            .block_output(w_cascade[i])
        );
    end
end
```

23.4.5 Cascade Selection Logic

```
// Select appropriate cascade output based on data length
always_comb begin
    w_selected_cascade_output = POLY_INIT; // Default
    for (int i = 0; i < CH; i++) begin
        if (cascade_sel[i]) begin
```

```

        w_selected_cascade_output = w_cascade[i];
    end
end
end

```

23.5 Timing Characteristics

23.5.1 Throughput

Data Width	CRC Width	Max Frequency	Throughput
32-bit	32-bit	~400 MHz	12.8 Gbps
64-bit	32-bit	~350 MHz	22.4 Gbps
128-bit	64-bit	~300 MHz	38.4 Gbps

23.5.2 Latency

Property	Value
Pipeline stages	2 cycles (input conditioning + CRC computation)
Reset recovery	1 cycle
Cascade selection	0 cycles (combinational)

23.5.3 Critical Paths

1. **Cascade Chain:** Data flows through all cascade stages
2. **Output Reflection:** Bit reversal for REFOUT
3. **Register Setup:** Loading CRC state register

23.6 Usage Examples

23.6.1 Basic CRC Calculation

```

logic [63:0] input_data;
logic [31:0] crc_result;
logic data_valid, crc_ready;

```

```

dataint_crc #(
    .DATA_WIDTH(64),
    .CRC_WIDTH(32),
    .REFIN(1),
    .REFOUT(1)
) u_crc_engine (

```

```

        .POLY(32'h04C11DB7),
        .POLY_INIT(32'hFFFFFFFF),
        .XOROUT(32'hFFFFFFFF),
        .clk(clk),
        .rst_n(rst_n),
        .load_crc_start(data_valid),      // pulse: seed accumulator with
POLY_INIT
        .load_from_cascade(capture_crc),  // pulse: capture the cascade
result
        .cascade_sel(8'h80),              // ONE-HOT: select the 8-byte
(full) stage
        .data(input_data),
        .crc(crc_result)
    );

```

The accumulator is only updated through the cascade path. Data reaches crc *only* via load_from_cascade; tying load_from_cascade to 0 (and using cascade_sel = 8'hFF) makes the engine reload POLY_INIT on every load_crc_start and emit a constant (reflect(POLY_INIT) ^ XOROUT), never a CRC of input_data. This is the mistake everyone makes with this module exactly once. A single-shot calculation must: (1) pulse load_crc_start to seed, (2) present data, then (3) pulse load_from_cascade with a **one-hot** cascade_sel selecting the stage for the number of valid bytes (8'h80 for all 8). cascade_sel is one-hot, not a mask — 8'hFF only works by highest-set-bit priority. See the streaming example below for the general pattern.

23.6.2 Streaming CRC with Cascade

```

// Process variable-length packets using cascade selection
logic [63:0] packet_data;
logic [7:0] byte_count; // Number of valid bytes (1-8)
logic [7:0] cascade_select;

// Convert byte count to one-hot cascade selection
always_comb begin
    cascade_select = '0;
    if (byte_count > 0 && byte_count <= 8) begin
        cascade_select[byte_count-1] = 1'b1;
    end
end

dataint_crc #(
    .DATA_WIDTH(64),
    .CRC_WIDTH(32)
) u_stream_crc (
    .clk(clk),

```

```

        .rst_n(rst_n),
        .load_crc_start(packet_start),
        .load_from_cascade(packet_continue),
        .cascade_sel(cascade_select),
        .data(packet_data),
        .crc(streaming_crc),
        // ... other connections
    );

```

23.6.3 Multi-Stage CRC Pipeline

```

// Pipeline multiple CRC engines for high throughput
logic [63:0] stage_data [0:3];
logic [31:0] stage_crc [0:3];

genvar stage;
generate
    for (stage = 0; stage < 4; stage++) begin : gen_crc_pipeline
        dataint_crc #(
            .DATA_WIDTH(64),
            .CRC_WIDTH(32)
        ) u_pipeline_crc (
            .clk(clk),
            .rst_n(rst_n),
            .data(stage_data[stage]),
            .crc(stage_crc[stage]),
            // ... configure for pipeline operation
        );
    end
endgenerate

```

23.6.4 CRC-32 (IEEE 802.3)

```

dataint_crc #(
    .DATA_WIDTH(32),
    .CRC_WIDTH(32),
    .REFIN(1),
    .REFOUT(1)
) u_crc32 (
    .POLY(32'h04C11DB7),
    .POLY_INIT(32'hFFFFFFFF),
    .XOROUT(32'hFFFFFFFF),
    // ... other connections
);

```

23.6.5 CRC-16 (CCITT)

```

dataint_crc #(
    .DATA_WIDTH(16),
    .CRC_WIDTH(16),

```

```

        .REFIN(0),
        .REFOUT(0)
    ) u_crc16 (
        .POLY(16'h1021),
        .POLY_INIT(16'hFFFF),
        .XOROUT(16'h0000),
        // ... other connections
    );

```

23.6.6 CRC-64 (ECMA-182)

```

dataint_crc #(
    .DATA_WIDTH(64),
    .CRC_WIDTH(64),
    .REFIN(0),
    .REFOUT(0)
) u_crc64 (
    .POLY(64'h42F0E1EBA9EA3693),
    .POLY_INIT(64'h0000000000000000), // init 0
    .XOROUT(64'h0000000000000000),   // xorout 0
    // ... other connections
);

```

23.7 Design Notes

23.7.1 Area Optimization

- **Polynomial Optimization:** Use polynomials with fewer terms
- **Width Reduction:** Match CRC_WIDTH to actual requirements
- **Cascade Stages:** Reduce CHUNKS for lower throughput applications

23.7.2 Power Optimization

- **Clock Gating:** Gate unused cascade stages
- **Data Gating:** Prevent unnecessary switching when not processing

Between the configurable polynomial, the reflection options, and the cascade architecture, `dataint_crc` covers most of the CRC standards you'll meet in practice — set it up once per protocol and let it run.

23.8 Related Modules

- **`dataint_crc_xor_shift_cascade`** - Building block for cascade stages
- **`dataint_crc_xor_shift`** - Simple XOR-shift CRC implementation

- **dataint_checksum** - Alternative integrity checking method
- **dataint_parity** - Simpler parity-based error detection

23.9 Testing

23.9.1 Setup Requirements

```
// Data must be stable before clock edge
timing_spec: assert property (
    @(posedge clk) $stable(data) throughout (load_crc_start)
);

// Cascade selection must be stable
cascade_timing: assert property (
    @(posedge clk) $stable(cascade_sel) throughout (load_from_cascade)
);
```

23.9.2 Verification Notes

- **Polynomial Validation:** Verify against known CRC test vectors
- **Cascade Testing:** Test all cascade selection combinations
- **Reflection Testing:** Verify bit ordering for REFIN/REFOUT modes
- **Standards Compliance:** Validate against published CRC standards

23.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

24 dataint_crc_xor_shift_cascade

24.1 Overview

Eight instances of `dataint_crc_xor_shift`, chained back to back so a full byte of data gets chewed through in a single combinational operation. Byte in, updated CRC out — no clock involved.

24.1.1 Module Declaration

```
module dataint_crc_xor_shift_cascade #(
    parameter int CRC_WIDTH = 32
) (
    input  [CRC_WIDTH-1:0] block_input,
    input  [CRC_WIDTH-1:0] poly,
    input   [7:0] data_input,
    output [CRC_WIDTH-1:0] block_output
);
```

24.2 Parameters

Parameter	Default	Description
CRC_WIDTH	32	Width of the CRC register and polynomial

24.3 Ports

Port	Direction	Width	Description
block_input	Input	CRC_WIDTH	Initial CRC state for the byte
poly	Input	CRC_WIDTH	CRC polynomial
data_input	Input	8	8-bit data byte to process
block_output	Output	CRC_WIDTH	Final CRC state after processing 8 bits

24.4 Functional Description

24.4.1 Cascade Operation

Eight CRC calculation stages, chained end to end: 1. **Stage 0**: Processes the MSB of the data byte against the initial CRC state 2. **Stages 1-6**: Each processes the next data bit against the previous stage's output 3. **Stage 7**: Processes the LSB and produces the final result

24.4.2 Data Bit Ordering

Data bits run MSB to LSB: - `data_input[7]` is processed first - `data_input[0]` is processed last - This matches typical serial transmission order

24.4.3 Architecture Overview

`block_input` → [Stage 0] → [Stage 1] → ... → [Stage 7] → `block_output`
 `data[7]` `data[6]` `data[0]`

24.4.4 Generate Block Structure

The cascade is built from a parameterized generate block:

```
generate
  for (i = 0; i < 8; i++) begin : gen_xor_shift_cascade
    if (i == 0) begin : gen_xor_shift_0
      // First stage uses block_input
      dataint_crc_xor_shift #(.CRC_WIDTH(CRC_WIDTH)) stage (
        .stage_input(block_input),
        .poly(poly),
        .new_bit(data_input[7-i]),
        .stage_output(w_cascade[i])
      );
    end else begin : gen_xor_shift_N
      // Subsequent stages chain previous output
      dataint_crc_xor_shift #(.CRC_WIDTH(CRC_WIDTH)) stage (
        .stage_input(w_cascade[i-1]),
        .poly(poly),
        .new_bit(data_input[7-i]),
        .stage_output(w_cascade[i])
      );
    end
  end
endgenerate
```

24.4.5 Intermediate Signals

- **w_cascade[0:7]**: Array holding intermediate CRC states
- **Fixed Width**: Each element is `CRC_WIDTH` bits wide

- **Sequential Chain:** Each stage feeds the next

24.4.6 Key Features

- **Parallel Byte Processing:** 8 bits in one combinational operation, no clock required — suitable for high-speed applications
- **Bit Order Handling:** MSB-first processing (network byte order), with the bit index calculated as `data_input[7-i]`, matching standard CRC bit processing order
- **Scalable Architecture:** Easy to modify for different byte sizes; generate blocks enable clean scaling, parameterized for different CRC widths

24.5 Timing Characteristics

24.5.1 Combinational Delay

Total delay is the sum of 8 CRC stage delays: - Each stage: ~2-3 gate delays - Total cascade: ~16-24 gate delays - May require pipelining for very high frequencies

Do that math against your clock period before you promise timing closure — 24 gate levels is not free at 400 MHz.

24.5.2 Critical Path

The critical path runs through all 8 stages:

`block_input` → Stage0 → Stage1 → ... → Stage7 → `block_output`

24.6 Usage Examples

24.6.1 Basic Usage

```
dataint_crc_xor_shift_cascade #(
    .CRC_WIDTH(16)
) crc_byte_processor (
    .block_input(current_crc_state),
    .poly(16'h1021),           // CRC-16-CCITT
    .data_input(data_byte),
    .block_output(updated_crc_state)
);
```

24.6.2 Multi-Byte Processing

```
// Process 4 bytes (32 bits) of data
wire [CRC_WIDTH-1:0] stage_output[0:3];
```

```

genvar i;
generate
  for (i = 0; i < 4; i++) begin : gen_multi_byte
    dataint_crc_xor_shift_cascade #(.CRC_WIDTH(CRC_WIDTH)) cascade
    (
      .block_input(i == 0 ? initial_crc : stage_output[i-1]),
      .poly(polyynomial),
      .data_input(data_bytes[i]),
      .block_output(stage_output[i])
    );
  end
endgenerate

assign final_crc = stage_output[3];

```

24.7 Design Notes

24.7.1 Performance Optimization

For high-frequency operation: - Consider adding pipeline registers between cascades - Balance latency vs. throughput requirements - You may need to pipeline within the 8-stage cascade

For area: - Each stage requires $\sim \text{CRC_WIDTH}$ XOR gates - Total area scales with $\text{CRC_WIDTH} \times 8$ - Consider time-multiplexed alternatives for area-critical applications

24.7.2 Integration with the CRC System

This module is used by `dataint_crc` for: - Processing data in byte-sized chunks - Building wider data processing capability - Maintaining processing efficiency

On the interface with the parent module: - Multiple cascade instances handle wide data - Cascade selection allows intermediate results - Enables flexible data width support

24.7.3 Applications

- High-speed packet processing
- Real-time data integrity checking
- Communication protocol implementations
- Storage system error detection
- Building block for wider CRC processors

24.8 Related Modules

- **dataint_crc_xor_shift**: Single-bit processing building block

- **dataint_crc**: Parent CRC calculation module
- Used together to create flexible CRC calculation systems

24.9 Testing

No dedicated testbench for this module. It has no `val/**/test_dataint_crc_xor_shift_cascade.py`; it is exercised indirectly, through the tests of the modules that instantiate it:

- `dataint_crc` – `val/common/test_dataint_crc.py`

Indirect coverage exercises this module only in the configurations its parents elaborate. A parameter or mode no parent uses is untested.

24.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

25 dataint_crc_xor_shift

25.1 Overview

The fundamental CRC step, applied to exactly one input bit: shift the current CRC state, and XOR in the polynomial when the feedback bit says to.

- **Single Bit Processing:** One bit per operation, no more
- **Combinational Logic:** Pure combinational implementation (no clock)
- **Parameterizable Width:** Any CRC width you need
- **Optimized Design:** Breaks the circular dependency with an intermediate signal

25.1.1 Module Declaration

```
module dataint_crc_xor_shift #(
    parameter int CRC_WIDTH = 32
) (
    input [CRC_WIDTH-1:0] stage_input,
    input [CRC_WIDTH-1:0] poly,
    input new_bit,
    output [CRC_WIDTH-1:0] stage_output
);
```

25.2 Parameters

Parameter	Default	Description
CRC_WIDTH	32	Width of the CRC register and polynomial

25.3 Ports

Port	Direction	Width	Description
stage_input	Input	CRC_WIDTH	Current CRC state
poly	Input	CRC_WIDTH	CRC polynomial
new_bit	Input	1	New data bit to process

Port	Direction	Width	Description
stage_output	Output	CRC_WIDTH	Updated CRC state

25.4 Functional Description

25.4.1 Core CRC Operation

This is the classic CRC shift-and-XOR, broken into three moves: 1. **Feedback Calculation**: XOR the new input bit with the MSB of the current CRC 2. **Shift Operation**: Shift the CRC register left by one position 3. **Polynomial XOR**: If the feedback bit is 1, XOR with the polynomial

25.4.2 Mathematical Foundation

The CRC operation follows this algorithm:

```
feedback = new_bit ⊕ CRC[MSB]
CRC_new[0] = feedback
CRC_new[MSB:1] = CRC[MSB-1:0] ⊕ (polynomial[MSB:1] & {WIDTH-1{feedback}})
```

25.4.3 Circular Dependency Solution

A naive write-up of this has a circular dependency — you'd be computing the output from itself. The fix is to compute the feedback bit first, on its own, then reuse it:

```
logic feedback_bit;

// Calculate feedback first
assign feedback_bit = new_bit ^ stage_input[CRC_WIDTH-1];

// Then use it for output calculation
assign stage_output[0] = feedback_bit;
assign stage_output[CRC_WIDTH-1:1] = stage_input[CRC_WIDTH-2:0] ^
    ({CRC_WIDTH-1{feedback_bit}} & poly[CRC_WIDTH-1:1]);
```

25.4.4 Bit-wise Operations

- **LSB Assignment**: Always gets the feedback bit
- **MSBs Assignment**: Shifted previous bits XORed with the polynomial when feedback is active
- **Conditional XOR**: Bit replication builds the conditional XOR mask

25.4.5 Feedback Bit Logic

The feedback bit runs the whole show: - It decides whether the polynomial gets applied - It's the XOR of the input bit and the current MSB - It gates the polynomial application

25.4.6 Polynomial Application

The polynomial goes in conditionally: - Only bits [MSB:1] of the polynomial are used (bit 0 is implicit) - Applied through an AND mask built by replicating the feedback bit - XORed with the shifted CRC bits

25.4.7 No State Elements

This module is purely combinational: - No clock, no reset - Instantaneous response to input changes - Can be cascaded for multi-bit processing

25.4.8 Mathematical Verification

For a polynomial $P(x)$ and data bit d : - Feedback = 0: it's a plain left shift - Feedback = 1: left shift, then XOR the polynomial

That's CRC polynomial division, done in gates.

25.5 Timing Characteristics

- **Latency:** Combinational (zero clock cycles)
- **Throughput:** One bit per clock when wrapped in a sequential system
- **Area:** Minimal — a few XOR gates and multiplexers
- **Power:** Low — just simple combinational logic

25.6 Usage Examples

You rarely instantiate this block alone — it's a building block.

25.6.1 Single Bit Processing

```
dataint_crc_xor_shift #(.CRC_WIDTH(16)) crc_stage (  
    .stage_input(current_crc),  
    .poly(crc_polynomial),  
    .new_bit(data_bit),  
    .stage_output(next_crc)  
);
```

25.6.2 Cascaded for Multiple Bits

Chain as many instances as you have bits to process:

```

// Process 8 bits. Do NOT also `assign stage[0] = initial_crc;` --
// stage[0] is
// driven by the i==0 instance below, and a second continuous driver
// on the same
// wire is an elaboration error (X contention in simulation).
wire [CRC_WIDTH-1:0] stage[0:7];

genvar i;
generate
    for (i = 0; i < 8; i++) begin
        dataint_crc_xor_shift #(.CRC_WIDTH(CRC_WIDTH)) stage_inst (
            .stage_input(i == 0 ? initial_crc : stage[i-1]),
            .poly(polyynomial),
            .new_bit(data_byte[7-i]),
            .stage_output(stage[i])
        );
    end
endgenerate

assign final_crc = stage[7];

```

25.7 Design Notes

25.7.1 Applications

- Building block for CRC calculation engines
- Serial CRC processors
- Component in parallel CRC architectures
- Educational CRC implementations
- Custom protocol CRC calculators

25.8 Related Modules

Read out of the RTL, not curated: these are the modules this one instantiates and the modules that instantiate it.

Instantiated by: - dataint_crc_xor_shift_cascade

25.9 Testing

No dedicated testbench for this module. It has no `val/**/test_dataint_crc_xor_shift.py`. It is exercised indirectly, through the tests of modules that instantiate it (directly or further up):

- `dataint_crc – val/**/test_dataint_crc.py`

Indirect coverage exercises this module only in the configurations those parents elaborate. A parameter or mode no parent uses is untested.

Treat any behaviour described on this page as unverified by simulation.

25.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

26 dataint_ecc_hamming_decode_secded

26.1 Overview

A Hamming decoder with SECDED — Single Error Correction, Double Error Detection. It decodes Hamming-encoded data, detects and fixes single-bit errors, and raises a flag when two bits have gone bad (which it can see but not repair).

26.1.1 Module Declaration

```
module dataint_ecc_hamming_decode_secded #(
    parameter int WIDTH = 4,
    parameter int DEBUG = 0
) (
    input logic clk,
    input logic rst_n,
    input logic enable,
    input logic [WIDTH+ParityBits:0] hamming_data,
    output logic [WIDTH-1:0] data,
    output logic error_detected,
    output logic double_error_detected
);
```

26.2 Parameters

Parameter	Default	Description
WIDTH	4	Width of original data in bits
DEBUG	0	Enable debug output (1=enabled, 0=disabled)

26.2.1 Calculated Parameters

Parameter	Formula	Description
ParityBits	$\$clog2(WIDTH + \$clog2(WIDTH) + 1)$	Number of Hamming parity bits
TotalWidth	$WIDTH + ParityBits + 1$	Total encoded data width

26.3 Ports

Port	Direction	Width	Description
clk	Input	1	Clock signal

Port	Direction	Width	Description
rst_n	Input	1	Active-low asynchronous reset
enable	Input	1	Enable decoding operation
hamming_data	Input	TotalWidth	Encoded data with ECC bits
data	Output	WIDTH	Decoded original data
error_detected	Output	1	Single or double error detected
double_error_detected	Output	1	Double error specifically detected

26.4 Functional Description

26.4.1 Decoding Process

1. **Syndrome Generation:** Calculate the parity check syndrome
2. **Overall Parity Check:** Verify the SECDED bit
3. **Error Classification:** Work out the error type and location
4. **Error Correction:** Fix single-bit errors
5. **Data Extraction:** Pull the corrected original data back out

26.4.2 Error Detection Logic

Overall Parity	Syndrome	Error Type	Action
Match	Zero	No Error	Pass data through
Mismatch	Non-zero	Single Error (Hamming)	Correct using syndrome
Mismatch	Zero	Single Error (SECDED)	No correction needed
Match	Non-zero	Double Error	Flag error, no correction

26.4.3 Key Functions

26.4.3.1 *bit_position(k)* Function

Tells you where data bit *k* ended up in the encoded stream:

```
function automatic integer bit_position(input integer k);
    integer j, pos;
    begin
        pos = k + 1; // Account for parity bit at position 0
        for (j = 0; j < ParityBits; j++) begin
            if (pos >= (2 ** j)) pos = pos + 1; // Skip parity
positions
        end
        bit_position = pos - 1; // Convert to 0-based index
    end
endfunction
```

26.4.3.2 *get_covered_bits(parity_bit)* Function

Hands back a bitmask of every position a given parity bit covers:

```
function automatic [TotalWidth-1:0] get_covered_bits(input integer
parity_bit);
    integer j;
    begin
        get_covered_bits = 0;
        // FIXED: Exclude SECDED bit from Hamming parity calculations
        for (j = 0; j < TotalWidth - 1; j++) begin
            if (((j + 1) >> parity_bit) & 1)
                get_covered_bits[j] = 1'b1;
        end
    end
endfunction
```

26.4.4 Syndrome Generation

```
always_comb begin : create_syndrome_covered_bits
    if (enable) begin
        for (i = 0; i < ParityBits; i++) begin
            parity_pos = (2 ** i) - 1; // Parity bit positions
            w_syndrome_in[i] = hamming_data[parity_pos]; // Stored
parity
            w_syndrome[i] = 1'b0;

            w_covered_bits = get_covered_bits(i);
            for (bit_index = 0; bit_index < TotalWidth; bit_index++)
                // Calculate parity over covered bits (excluding
parity bit itself)
        end
    end
end
```

```

        if (w_covered_bits[bit_index] && (bit_index !=
parity_pos))
            w_syndrome[i] = w_syndrome[i] ^
hamming_data[bit_index];
        end

        // Compare with stored parity to get syndrome
        w_syndrome[i] = w_syndrome[i] ^ w_syndrome_in[i];
    end
end
end

```

26.4.5 Overall Parity Check

```

always_comb begin : check_overall_parity
    w_overall_parity_in = hamming_data[TotalWidth-1]; // SECEDED bit
    if (enable) begin
        w_overall_parity = ^hamming_data[TotalWidth-2:0]; //
Calculated parity
    end else begin
        w_overall_parity = 'b0;
    end
end
end

```

26.4.6 Control States

There's no fancy FSM here — just enable gating two logical states:

1. **IDLE**: enable = 0 - No processing
2. **DECODE**: enable = 1 - Active decoding and error correction

26.4.7 Error Correction Logic

```

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_data_with_parity <= 'b0;
        error_detected <= 'b0;
        double_error_detected <= 'b0;
    end else if (enable) begin
        r_data_with_parity <= hamming_data; // Default assignment
        error_detected <= 'b0;
        double_error_detected <= 'b0;

        // SECEDED error detection logic
        if (w_overall_parity != w_overall_parity_in) begin
            error_detected <= 1'b1;

            if (w_syndrome != {ParityBits{1'b0}}) begin

```

```

        // Single-bit error in Hamming data - correct it
        r_data_with_parity[w_syndrome_0_based] <=
            ~hamming_data[w_syndrome_0_based];
    end
    // Single-bit error in SECDED bit - no correction needed

end else if (w_syndrome != {ParityBits{1'b0}}) begin
    // Double-bit error detected
    error_detected <= 1'b1;
    double_error_detected <= 1'b1;
end
end
end

```

26.4.8 Data Extraction

```

genvar j;
generate
    for (j = 0; j < WIDTH; j++) begin : gen_block
        assign data[j] = r_data_with_parity[bit_position(j)];
    end
endgenerate

```

That peels just the original data bits out of the corrected encoded word.

26.4.9 Error Handling Details

Syndrome interpretation: - **Syndrome = 0**: No error in the Hamming bits - **Syndrome ≠ 0**: Points to the error bit position (1-based) - **Zero-based Syndrome**: `w_syndrome_0_based = w_syndrome - 1`

The SECDED bit is what lets you tell apart: - Single-bit errors (correctable) - Double-bit errors (detectable, but you're out of luck on correction) - No errors

The correction sequence: 1. Calculate the syndrome from the received parity bits 2. Check overall parity (the SECDED bit) 3. Single error detected? Flip the bit at the syndrome position 4. Double error detected? Flag it, but leave the data alone

26.4.10 Error Correction Capabilities

- **Single Error Correction**: Automatically fixes single-bit errors
- **Double Error Detection**: Flags double-bit errors but can't correct them
- **Error Location**: The syndrome points right at the bad bit
- **SECDED Protection**: The extra parity bit buys you double-error detection

26.4.11 Debug Support

Fair warning: the `DEBUG` parameter is a **no-op** today. The RTL's only nod to it is an empty initial `begin if (DEBUG != 0) begin ... end end` placeholder with no `$display` statements. Turn it on and you'll get exactly no output in the current implementation.

26.5 Timing Characteristics

- **Latency:** One clock cycle for error correction
- **Throughput:** One word per clock cycle
- **Detection Rate:** 100% for single and double errors
- **Critical Path:** Syndrome calculation and error correction
- **Pipeline Stages:** Consider pipelining if you're pushing frequency
- **Area:** Scales with data width and parity requirements
- **Power:** Moderate — the combinational parity math costs you

26.6 Usage Examples

26.6.1 Basic Decoder

```
dataint_ecc_hamming_decode_secded #(
    .WIDTH(8),
    .DEBUG(0)
) hamming_decoder (
    .clk(clk),
    .rst_n(rst_n),
    .enable(decode_enable),
    .hamming_data(encoded_input),
    .data(decoded_data),
    .error_detected(single_or_double_error),
    .double_error_detected(uncorrectable_error)
);
```

26.6.2 Error Handling System

```
always_ff @(posedge clk) begin
    if (error_detected) begin
        if (double_error_detected) begin
            // Log uncorrectable error
            error_log <= error_log + 1;
            // Request retransmission
            request_retransmit <= 1'b1;
        end else begin
```

```

        // Log corrected error
        corrected_errors <= corrected_errors + 1;
        // Continue with corrected data
    end
end
end

```

26.7 Design Notes

26.7.1 Applications

- ECC memory systems
- Communication error correction
- Storage system integrity
- Real-time data processing
- Safety-critical systems
- Network protocol implementation

26.8 Related Modules

- **dataint_ecc_hamming_encode_secdec**: Corresponding encoder
- Used together for complete ECC system implementation

26.9 Testing

val/common/test_dataint_ecc_hamming_secdec.py covers this module. It builds an ecc_secdec_wrapper instantiating the encoder and decoder together and drives injected single- and double-bit errors through the pair.

The test is named for the PAIR rather than the module, which is why an earlier audit looking only for val/**/test_dataint_ecc_hamming_decode_secdec.py reported it as having no coverage. It does.

Treat any behaviour described on this page as unverified by simulation.

26.10 Navigation

- [← Back to rtl-common Index](#)

- [← Back to Main Documentation Index](#)

27 dataint_ecc_hamming_encode_secded

27.1 Overview

The encoder half of a SECDED Hamming scheme (Single Error Correction, Double Error Detection). It generates the check bits that let the far end detect up to 2-bit errors and correct single-bit errors in the data.

27.1.1 Module Declaration

```
module dataint_ecc_hamming_encode_secded #(
    parameter int WIDTH = 4,
    parameter int DEBUG = 0
) (
    input logic [WIDTH-1:0] data,
    output logic [TotalWidth-1:0] encoded_data
);
```

27.2 Parameters

Parameter	Default	Description
WIDTH	4	Width of input data in bits
DEBUG	0	Enable debug output (1=enabled, 0=disabled)

27.2.1 Calculated Parameters

Parameter	Formula	Description
ParityBits	$\$clog2(WIDTH + \$clog2(WIDTH) + 1)$	Number of Hamming parity bits
TotalWidth	$WIDTH + ParityBits + 1$	Total encoded data width (includes SECDED bit)

27.2.2 Example Configurations

4-bit data (the common example): - Data bits: 4 - Parity bits: 3 (positions 0, 1, 3) - SECDED bit: 1 (position 7) - Total width: 8 bits

8-bit data: - Data bits: 8 - Parity bits: 4 (positions 0, 1, 3, 7) - SECDED bit: 1 (position 12) - Total width: 13 bits

16-bit data: - Data bits: 16 - Parity bits: 5 (positions 0, 1, 3, 7, 15) - SECDED bit: 1 (position 21) - Total width: 22 bits

27.3 Ports

Port	Direction	Width	Description
data	Input	WIDTH	Original data to be encoded
encoded_data	Output	TotalWidth	Encoded data with parity and SECDED bits

27.4 Functional Description

27.4.1 Hamming Code Structure

The encoded word carries three things: 1. **Data Bits**: Your original data, parked at specific positions 2. **Parity Bits**: Hamming parity bits at the power-of-2 positions (1, 2, 4, 8, ...) 3. **SECDED Bit**: One extra overall parity bit for double error detection

27.4.2 Bit Position Layout

Position:	1	2	3	4	5	6	7	8	9	10	11	12	...
Content:	P1	P2	D1	P4	D2	D3	D4	P8	D5	D6	D7	D8	...

Where: P = Parity bit, D = Data bit

27.4.3 Key Functions

27.4.3.1 *bit_position(k) Function*

Works out where data bit k belongs in the encoded stream:

```
function automatic integer bit_position(input integer k);
    integer j, pos;
    begin
        pos = k + 1; // Start at k+1 to account for parity at
        position 0
        for (j = 0; j < ParityBits; j++) begin
            if (pos >= (2 ** j)) pos = pos + 1; // Skip parity
            positions
        end
        bit_position = pos - 1; // Convert to 0-based index
    end
endfunction
```

27.4.3.2 *get_covered_bits(parity_bit) Function*

Returns a bitmask of every position a specific parity bit covers:

```
function automatic [TotalWidth-1:0] get_covered_bits(input integer
parity_bit);
```

```

integer j;
begin
    get_covered_bits = 'b0;
    for (j = 0; j < TotalWidth; j++) begin
        // Check if position j+1 has parity_bit set in its binary
representation
        if (|(((j + 1) >> parity_bit) & 1))
            get_covered_bits[j] = 1'b1;
    end
end
endfunction

```

27.4.4 Encoding Algorithm

27.4.4.1 Step 1: Data Placement

```

// Initialize with zeros
w_data_with_parity = {TotalWidth{1'b0}};

// Insert data bits into correct positions
for (i = 0; i < WIDTH; i++) begin
    w_data_with_parity[bit_position(i)] = data[i];
end

```

27.4.4.2 Step 2: Parity Calculation

```

// Calculate each Hamming parity bit
for (i = 0; i < ParityBits; i++) begin
    parity_pos = (2 ** i) - 1; // Positions 0, 1, 3, 7, 15, ...
    w_data_with_parity[parity_pos] = 1'b0; // Initialize

    // XOR all covered positions
    w_covered_bits = get_covered_bits(i);
    for (bit_index = 0; bit_index < TotalWidth; bit_index++) begin
        if (w_covered_bits[bit_index]) begin
            w_data_with_parity[parity_pos] =
                w_data_with_parity[parity_pos] ^
w_data_with_parity[bit_index];
        end
    end
end
end

```

27.4.4.3 Step 3: SECDED Bit Calculation

```

// Calculate overall parity for SECDED
w_data_with_parity[TotalWidth-1] = ^w_data_with_parity[TotalWidth-
2:0];

```

27.4.5 Error Correction Capabilities

Single error correction: - Can correct any single-bit error in the encoded data - Uses the Hamming parity bits to locate the error - The decoder flips the bad bit

Double error detection: - The SECDED bit is what spots double-bit errors - Can't fix them, but it flags them - No silent data corruption

Error Type	Detection	Correction
No Error	Yes	N/A
Single Bit Error	Yes	Yes
Double Bit Error	Yes	No
Triple+ Bit Error	Partial	No

27.4.6 Combinational Design

- **No Clock Required:** Purely combinational logic
- **Immediate Output:** Encoded data shows up as soon as the input settles
- **Stateless Operation:** No internal state, nothing to reset

27.4.7 Scalable Architecture

- **Parameterizable Width:** Any data width you like
- **Automatic Sizing:** The parity bit count is worked out for you
- **Efficient Layout:** Optimal bit positioning for the Hamming code

27.4.8 Debug Support

Note that DEBUG is a **no-op** in this module: the RTL never references it outside its parameter declaration, so there's no debug output to be had. The parameter is reserved for future use.

27.5 Timing Characteristics

This module is **purely combinational** – it contains no `always_ff` and no latch, so it holds no state and adds no clock cycles. Its outputs settle a propagation delay after its inputs, and it introduces no latency into a pipeline that instantiates it.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

27.6 Usage Examples

27.6.1 Basic Encoding

```
// 8-bit data encoder
dataint_ecc_hamming_encode_secDED #(
    .WIDTH(8),
    .DEBUG(0)
) hamming_encoder (
    .data(input_data),
    .encoded_data(ecc_encoded_data)
);
```

27.6.2 With Debug

```
// Debug-enabled encoder for verification
dataint_ecc_hamming_encode_secDED #(
    .WIDTH(4),
    .DEBUG(1)
) debug_encoder (
    .data(test_data),
    .encoded_data(encoded_output)
);
```

27.7 Design Notes

27.7.1 Applications

- Memory systems (ECC RAM)
- Communication protocols
- Storage systems
- Critical data processing
- Aerospace and automotive systems
- Any application requiring data integrity

27.8 Related Modules

- **dataint_ecc_hamming_decode_secDED**: Corresponding decoder module
- Used together for complete SECDED error correction system

27.9 Testing

`val/common/test_dataint_ecc_hamming_secded.py` covers this module. It builds an `ecc_secded_wrapper` instantiating the encoder and decoder together and drives injected single- and double-bit errors through the pair.

The test is named for the PAIR rather than the module, which is why an earlier audit looking only for `val/**/test_dataint_ecc_hamming_encode_secded.py` reported it as having no coverage. It does.

Treat any behaviour described on this page as unverified by simulation.

27.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

28 dataint_parity

28.1 Overview

A generic parity generator and checker in one. It computes parity bits over data chunks and verifies incoming parity against them, supporting both even and odd schemes across as many data segments as you carve your bus into.

28.1.1 Module Declaration

```
module dataint_parity #(
    parameter int CHUNKS = 4, // Number of chunks to check parity
    parameter int WIDTH  = 32 // Total width of the data
) (
    input logic [WIDTH-1:0] data_in, // Data input
    input logic [CHUNKS-1:0] parity_in, // Parity input for
checking
    input logic parity_type, // 1=even, 0=odd
    output logic [CHUNKS-1:0] parity, // Generated parity bits
    output logic [CHUNKS-1:0] parity_err // Error indicators
);
```

28.2 Parameters

Parameter	Default	Description
CHUNKS	4	Number of data chunks to process
WIDTH	32	Total width of input data in bits

28.2.1 Calculated Parameters

Parameter	Formula	Description
ChunkSize	WIDTH / CHUNKS	Size of each chunk in bits
ExtraBits	WIDTH % CHUNKS	Remaining bits if WIDTH not evenly divisible

28.3 Ports

Port	Direction	Width	Description
data_in	Input	WIDTH	Input data to calculate/check parity

Port	Direction	Width	Description
parity_in	Input	CHUNKS	Expected parity bits for verification
parity_type	Input	1	Parity type (1=even, 0=odd)
parity	Output	CHUNKS	Calculated parity bits
parity_err	Output	CHUNKS	Error flags (1=parity mismatch)

28.4 Functional Description

28.4.1 Dual Operation Modes

The module does both jobs at once: 1. **Generates Parity**: Calculates parity for each data chunk 2. **Checks Parity**: Compares that calculated parity with the input parity

28.4.2 Parity Types

- **Even Parity** (parity_type = 1): The parity bit makes the total 1s even
- **Odd Parity** (parity_type = 0): The parity bit makes the total 1s odd

28.4.3 Data Chunking

Data gets divided into chunks, with a small wrinkle for widths that don't divide evenly: - Most chunks: ChunkSize bits each - The last chunk picks up any extra bits when $WIDTH \% CHUNKS \neq 0$

28.4.4 Generate Block Architecture

```

genvar i;
generate
  for (i = 0; i < CHUNKS; i++) begin : gen_parity
    // Calculate bounds for each chunk
    localparam int LowerBound = i * ChunkSize;
    localparam int UpperBound = (i < CHUNKS - 1) ?
      ((i + 1) * ChunkSize) - 1 : WIDTH - 1;

    // Calculate parity for this chunk
    wire calculated_parity = parity_type ?
      ^data_in[UpperBound:LowerBound] :           // Even parity
      ~^data_in[UpperBound:LowerBound];           // Odd parity
  end
endgenerate

```



```

        assign parity[i] = calculated_parity;
        assign parity_err[i] = (calculated_parity != parity_in[i]);
    end
endgenerate

```

28.4.5 Chunk Boundary Calculation

Each chunk's bounds are worked out statically: - **Regular Chunks** ($i < \text{CHUNKS}-1$): Fixed-size chunks - **Last Chunk** ($i = \text{CHUNKS}-1$): Mops up any remainder bits

28.4.6 Parity Calculation Logic

For each chunk: 1. **XOR Reduction**: $\text{\^data_in}[\text{UpperBound}:\text{LowerBound}]$ 2. **Parity Type Application**: - Even parity: take the XOR result as-is - Odd parity: invert it

28.4.7 Error Detection

Error checking is just a compare — calculated against expected:

```

assign parity_err[i] = (calculated_parity != parity_in[i]);

```

28.4.8 Example Configurations

32-bit data, 4 chunks (8 bits each):

```

Chunk 0: data_in[7:0]      → parity[0], parity_err[0]
Chunk 1: data_in[15:8]     → parity[1], parity_err[1]
Chunk 2: data_in[23:16]    → parity[2], parity_err[2]
Chunk 3: data_in[31:24]    → parity[3], parity_err[3]

```

30-bit data, 4 chunks (uneven division):

```

Chunk 0: data_in[6:0]      (7 bits) → parity[0], parity_err[0]
Chunk 1: data_in[13:7]     (7 bits) → parity[1], parity_err[1]
Chunk 2: data_in[20:14]    (7 bits) → parity[2], parity_err[2]
Chunk 3: data_in[29:21]    (9 bits) → parity[3], parity_err[3]

```

28.4.9 Key Features

- **Scalable Architecture**: Parameterizable chunk count, flexible data width, uneven divisions handled for you
- **Simultaneous Operation**: Generation and checking run concurrently, each chunk minds its own business, and all chunks evaluate at the same time
- **Parity Flexibility**: You can flip the parity type during operation; all chunks share the same parity type; both common parity schemes covered

28.5 Related Modules

Nothing in the tree instantiates this module and it instantiates nothing: it is a leaf, used directly by whatever design needs it. Its nearest neighbours in rtl/common/ are:

- dataint_checksum
 - dataint_crc
 - dataint_crc_xor_shift
 - dataint_crc_xor_shift_cascade
 - dataint_ecc_hamming_decode_secdec
-

28.6 Timing Characteristics

- **Combinational Logic:** Zero clock delay
- **Critical Path:** XOR tree depth $\approx \log_2(\text{ChunkSize})$
- **Propagation Delay:** Minimal for typical chunk sizes
- **Area:** Grows linearly with CHUNKS and WIDTH; just XOR trees, so very little logic per chunk
- **Power:** Low — simple combinational logic, and power tracks the data switching

28.7 Usage Examples

28.7.1 Basic Parity Generation

```
dataint_parity #(
    .CHUNKS(8),
    .WIDTH(64)
) parity_gen (
    .data_in(input_data),
    .parity_in(8'b0),           // Not used for generation
    .parity_type(1'b1),        // Even parity
    .parity(generated_parity),
    .parity_err()              // Not used for generation
);
```

28.7.2 Parity Checking

```
dataint_parity #(
    .CHUNKS(4),
    .WIDTH(32)
) parity_check (
    .data_in(received_data),
    .parity_in(received_parity),
    .parity_type(1'b0),           // Odd parity
    .parity(recalc_parity),
    .parity_err(error_flags)
);

// Check for any errors
wire any_parity_error = |error_flags;
```

28.7.3 Error Correction System

```
logic [CHUNKS-1:0] error_location;
logic [WIDTH-1:0] corrected_data;

dataint_parity #(.CHUNKS(CHUNKS), .WIDTH(WIDTH)) parity_checker (
    .data_in(received_data),
    .parity_in(received_parity),
    .parity_type(even_parity),
    .parity(recalc_parity),
    .parity_err(error_location)
);

// Simple error correction (single-bit errors only)
always_comb begin
    corrected_data = received_data;
    for (int i = 0; i < CHUNKS; i++) begin
        if (error_location[i]) begin
            // Flip bits in the erroneous chunk
            // (Simplified - real correction needs more sophisticated
            logic)
        end
    end
end
```

28.8 Design Notes

28.8.1 Applications

- Memory parity checking
- Communication protocol verification

- Data bus integrity monitoring
- Storage system error detection
- Real-time parity generation and concurrent error checking
- Multi-segment data validation and protocol compliance verification
- Building block for ECC systems and larger error correction schemes
- Interface protection, debug and validation tools

Related application areas: - **ECC Systems:** Building block for Hamming codes - **Network Protocols:** Ethernet, UART parity - **Memory Systems:** DRAM/SRAM parity protection - **Communication:** RS-232, SPI, I2C error detection

28.8.2 Chunk Size Selection

- **Power of 2:** Usually the friendliest for alignment
- **Protocol Requirements:** Match whatever your system demands
- **Error Granularity:** Smaller chunks pin errors down tighter

28.8.3 Parity Type Selection

- **Even Parity:** The more common choice in digital systems
- **Odd Parity:** Gives you all-zeros detection
- **System Compatibility:** Match the existing standard

28.8.4 Width Considerations

- **Alignment:** Think about data bus alignment
- **Extra Bits:** Handle remainder bits sensibly
- **Performance:** Balance chunk count against granularity

28.9 Testing

val/common/test_dataint_parity.py exercises this module. It collects 16 parameter cases at the default REG_LEVEL.

```
source env_python
pytest val/common/test_dataint_parity.py -v
```

28.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

29 debounce

29.1 Overview

Kills mechanical switch bounce by sampling button inputs on a regular tick and refusing to believe a button until the same state shows up for several consecutive samples.

29.2 Parameters

Parameter	Default	Description
N	4	Number of buttons/input signals
DEBOUNCE_DELAY	4	Number of consecutive stable samples required
PRESSED_STATE	1	Logic level when button is pressed (1 for normally open, 0 for normally closed)

29.3 Ports

Port	Direction	Width	Description
clk	Input	1	System clock signal
rst_n	Input	1	Active-low reset signal
long_tick	Input	1	~10ms sampling tick signal (controls sampling rate)
button_in	Input	N	Raw button input signals to be debounced
button_out	Output	N	Debounced button output signals

29.4 Functional Description

29.4.1 Core Algorithm

The module tracks each button's recent history in a shift register:

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        for (int i = 0; i < N; i++) begin
```

```

        r_shift_regs[i] <= {DEBOUNCE_DELAY{1'b0}};
    end
end else if (long_tick) begin
    for (int i = 0; i < N; i++) begin
        r_shift_regs[i] <= {
            r_shift_regs[i][DEBOUNCE_DELAY-2:0],
            PRESSED_STATE ? button_in[i] : ~button_in[i]
        };
    end
end
end
end

```

29.4.2 Sampling Control

- Only samples inputs when long_tick fires
- Over-sampling invites false triggering — this avoids it
- Typical long_tick period: ~10ms

29.4.3 Button Type Support

- **Normally Open (NO):** PRESSED_STATE = 1
 - Button reads '0' when not pressed, '1' when pressed
- **Normally Closed (NC):** PRESSED_STATE = 0
 - Button reads '1' when not pressed, '0' when pressed
- The input inversion handles NC buttons transparently

29.4.4 Debounce Logic

- Keeps DEBOUNCE_DELAY samples per button
- The output only goes high when every sample in the shift register is '1'
- Which means the button has to sit stable in the pressed state for the whole delay

```

always_comb begin
    for (int i = 0; i < N; i++) begin
        // Output 1 when shift register shows stable pressed state
        (all 1s)
        w_debounced_signals[i] = &r_shift_regs[i];
    end
end
end

```

29.4.5 Output Registration

- The final output is registered, so transitions come out clean and glitch-free
- Resets to all zeros on system reset

29.5 Timing Characteristics

29.5.1 Debounce Delay

- **Total delay** = DEBOUNCE_DELAY × long_tick period
- **Default:** 4 × 10ms = 40ms debounce time
- **Minimum stable time:** The button has to hold steady for the full delay period

29.5.2 Response Time

- **Press detection:** DEBOUNCE_DELAY ticks after the button stabilizes high (the output is &r_shift_regs[i], so it needs DEBOUNCE_DELAY consecutive high samples to assert)
- **Release detection:** ~1 tick (+1 clk). The output is an AND-reduce, so the first 0 shifted in on release immediately clears it — release is **not** debounced beyond a single sample.
- **Asymmetric:** press is fully debounced; release propagates ~DEBOUNCE_DELAY× faster. Do NOT rely on symmetric debounce timing.

29.6 Usage Examples

Every parameter and port below is taken from the module declaration.

```
debounce #(
    .N                (4),
    .DEBOUNCE_DELAY  (4),
    .PRESSED_STATE    (1)
) u_debounce (
    .clk              (clk),
    .rst_n            (rst_n),
    .long_tick        (long_tick),
    .button_in        (button_in),
    .button_out       (button_out)
);
```

29.7 Design Notes

29.7.1 Applications

- Mechanical pushbuttons and switches
- Rotary encoder inputs (with the delay set appropriately)
- Any digital input prone to contact bounce
- Both normally open and normally closed switch types

29.7.2 Design Considerations

- **Delay tuning:** Set `DEBOUNCE_DELAY` to match your switch's characteristics
- **Tick frequency:** `long_tick` should run much slower than the bounce duration
- **Multiple buttons:** All buttons share the same debounce parameters
- **Reset behavior:** All outputs go low on reset, no matter what the buttons are doing

29.8 Related Modules

Read out of the RTL, not curated: these are the modules this one instantiates and the modules that instantiate it.

Instantiated by: - `cdc_counter_display_top`

29.9 Testing

`val/common/test_debounce.py` exercises this module. It collects 4 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/common/test_debounce.py -v
```

29.10 Navigation

- [← Back to rtl-common Index](#)

- [← Back to Main Documentation Index](#)

30 decoder

30.1 Overview

Converts an M-bit binary encoded input to a one-hot N-bit output where $N = 2^M$. Standard binary-to-one-hot decoding — nothing exotic.

30.2 Parameters

Parameter	Default	Description
M	4	Width of binary input
N	$2^M = 16$	Width of one-hot output

30.3 Ports

Port	Direction	Width	Description
encoded	Input	M	Binary encoded input value
data	Output	N	One-hot decoded output vector

30.4 Functional Description

30.4.1 Operation Principle

- Each output bit maps to one possible input value
- Only one output bit is high at any time (that's the one-hot part)
- Output bit i is high when the encoded input equals i

30.4.2 Truth Table Example (M=2, N=4)

encoded[1:0]	data[3]	data[2]	data[1]	data[0]
00	0	0	0	1
01	0	0	1	0
10	0	1	0	0

encoded[1:0]	data[3]	data[2]	data[1]	data[0]
11	1	0	0	0

30.4.3 Key Characteristics

- **Combinational logic:** No clock dependency, immediate response
- **One-hot output:** Exactly one bit high for valid inputs
- **Complete decoding:** All possible input combinations decoded
- **No default initialization:** because $N = 2^M$ covers every input value, exactly one output is high at all times — the outputs are never all-zero, and the RTL contains no default assignment

30.4.4 Core Logic

A generate loop spins up one comparator per output bit:

```
genvar i;
generate
    for (i = 0; i < N; i++) begin : gen_DECODER_LOOP
        assign data[i] = (encoded == i) ? 1'b1 : 1'b0;
    end
endgenerate
```

30.4.5 Parameterization

- **Scalable width:** M sets the input width
- **Automatic sizing:** N falls out as 2^M
- **Generate loops:** Synthesizes cleanly for any size

30.5 Timing Characteristics

- **Setup/hold:** None (purely combinational)
- **Output changes:** Follow the inputs immediately

30.6 Usage Examples

30.6.1 Memory Address Decoding

```
decoder #(.M(2), .N(4)) addr_decoder (
    .encoded(address[1:0]),
    .data(chip_select) // 4 chip selects
);
```

30.6.2 State Machine Output Decoding

```
decoder #(.M(3), .N(8)) state_decoder (  
    .encoded(current_state),  
    .data(state_outputs) // 8 control signals  
);
```

30.7 Design Notes

30.7.1 Applications

- **Address decoding:** Memory or register select signals
- **State machine outputs:** Decode binary state to control signals
- **Multiplexer control:** Generate select signals for data routing
- **Interrupt controllers:** Decode interrupt vectors
- **Bus decoding:** Generate chip select signals

30.7.2 Synthesis Considerations

- **Resource usage:** Typically lands in LUT-based logic
- **Propagation delay:** Single LUT delay in most FPGA architectures
- **Fan-out:** Each input bit feeds multiple output comparisons

30.7.3 Other Notes

- No error checking for invalid inputs (though all combinations are valid)
- Purely combinational: the generate loop drives every data[i] exactly once, so all outputs are defined at all times (no separate initialization needed — and none exists in the RTL)
- The generate loop structure scales efficiently to any required size

30.8 Related Modules

- **Encoder:** Performs inverse operation (one-hot to binary)
- **Priority Encoder:** Handles multiple simultaneous inputs
- **Multiplexer:** Often used together for data routing

30.9 Testing

`val/common/test_decoder.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/common/test_decoder.py -v
```

30.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

31 encoder

31.1 Overview

Converts a one-hot N-bit input back into its binary index — the inverse of a decoder. Feed it a one-hot vector, get out the position of the asserted bit in binary.

31.2 Parameters

Parameter	Default	Description
N	8	Width of one-hot input vector

31.3 Ports

Port	Direction	Width	Description
decoded	Input	N	One-hot input vector
data	Output	$\lceil \log_2(N) \rceil$	Binary encoded output representing the position of the set bit

31.4 Functional Description

31.4.1 Operation Principle

- Scans the input vector from LSB to MSB
- Outputs the binary representation of the highest indexed set bit
- Which makes it a **priority encoder** when multiple bits are set (higher index wins)

31.4.2 Truth Table Example (N=8, output width=3)

decoded[7:0]	data[2:0]	Notes
00000001	000	Bit 0 set
00000010	001	Bit 1 set
00000100	010	Bit 2 set
00001000	011	Bit 3 set
00010000	100	Bit 4 set

decoded[7:0]	data[2:0]	Notes
00100000	101	Bit 5 set
01000000	110	Bit 6 set
10000000	111	Bit 7 set
00000000	000	No bits set
10000001	111	Multiple bits - highest wins

31.4.3 Key Characteristics

- **Priority encoding:** Higher-indexed bits take precedence
- **Zero default:** Outputs 0 when no input bits are set
- **Combinational:** Immediate response to input changes
- **Self-sizing:** Output width comes from $\$clog2(N)$

31.4.4 Core Algorithm

A combinational always block runs the priority search:

```
always_comb begin
    data = 0; // Default output when no bits set
    for (int i = 0; i < N; i++) begin
        if (decoded[i]) data = $clog2(N)'(i);
    end
end
```

31.4.5 Automatic Width Calculation

- **Output width:** $\$clog2(N)$ bits
- **Example:** $N=8 \rightarrow 3\text{-bit output}$, $N=16 \rightarrow 4\text{-bit output}$
- **Optimal sizing:** No wasted bits in the output

31.4.6 Priority Behavior

The for-loop gives you priority for free: - Later iterations overwrite earlier results - So the highest-indexed set bit decides the final output - For invalid one-hot inputs, it quietly behaves as a priority encoder

31.4.7 Behavioral Notes

- **For-loop order:** Creates LSB-to-MSB priority (higher index wins)

- **Default assignment:** Ensures output has known value for all inputs
- **Type casting:** `$clog2(N)'(i)` ensures proper bit width matching

31.5 Timing Characteristics

- **Propagation delay:** Depends on input width and synthesis
- **Critical path:** Runs through the for-loop comparisons
- **Setup/hold:** None (purely combinational)

31.6 Usage Examples

31.6.1 Interrupt Controller

```
encoder #(.N(8)) int_encoder (
    .decoded(interrupt_pending),
    .data(interrupt_id)
);
```

31.6.2 Arbiter Output Encoding

```
encoder #(.N(16)) grant_encoder (
    .decoded(grant_vector),
    .data(granted_id)
);
```

31.7 Design Notes

31.7.1 Applications

For valid one-hot inputs: - **Interrupt acknowledgment:** Convert interrupt vector to binary ID - **Resource arbitration:** Encode granted request to requester ID - **State machine encoding:** Convert one-hot state to binary - **Position encoding:** Find position of active element

For priority encoding: - **Multiple interrupt handling:** Encode highest priority interrupt - **Error reporting:** Report highest severity error condition - **Resource allocation:** Grant to highest priority requester

31.7.2 Input Validation

- **Assumes one-hot input** for standard encoder behavior
- **Handles multiple bits** gracefully (acts as priority encoder)
- **Zero input** produces zero output (whether that's what you want is up to you)

31.7.3 Synthesis Implications

- **Resource usage:** Typically synthesizes to multiplexer logic
- **Optimization:** Modern synthesizers chew through the for-loop just fine
- **Scalability:** Performance degrades gradually as N grows

31.8 Related Modules

- **Decoder:** Performs inverse operation (binary to one-hot)
- **Priority Encoder with Enable:** Enhanced version with enable control
- **Find First Set:** Similar functionality with different search order

31.9 Testing

`val/common/test_encoder.py` exercises this module. It collects 2 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/common/test_encoder.py -v
```

31.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

32 encoder_priority_enable

32.1 Overview

Encodes the highest priority (highest index) asserted bit in the input vector, with an enable control signal. Where the basic encoder gets its priority behavior as a side effect, this one searches MSB to LSB on purpose.

32.2 Parameters

Parameter	Default	Description
WIDTH	8	Input vector width

32.3 Ports

Port	Direction	Width	Description
priority_in	Input	WIDTH	Priority input vector (higher index = higher priority)
enable	Input	1	Enable signal (active high)
encode	Output	$\$clog2(WIDTH)$	Binary encoded output of highest priority bit position

32.4 Functional Description

32.4.1 Operation Principle

- **True priority encoding:** Searches from MSB (highest priority) to LSB
- **Enable control:** Only runs when enable is asserted
- **Found flag:** Blocks multiple assignments, so the highest priority wins
- **Clean disable:** Outputs zero when disabled

32.4.2 Priority Resolution Example (WIDTH=8)

priority_in[7:0]	enable	encode[2:0]	Notes
00000001	1	000	Only bit 0 set
10000001	1	111	Bit 7 has highest

priority_in[7:0]	enable	encode[2:0]	Notes
01010101	1	110	priority Bit 6 is highest set
11111111	1	111	Bit 7 wins among all
10000001	0	000	Disabled - output zero
00000000	1	000	No bits set

32.4.3 Key Differences from the Basic Encoder

Feature	Basic Encoder	Priority Encoder with Enable
Search Direction	LSB to MSB	MSB to LSB
Priority	Higher index overwrites	True highest priority
Enable Control	None	Enable signal required
Found Flag	Implicit	Explicit prevention of overwrites

32.4.4 Core Algorithm

```

logic found;

always_comb begin
    // Default assignments to prevent latches
    encode = '0;
    found = 1'b0;

    if (enable == 1'b1) begin
        // Find the highest priority bit using found flag to prevent
        overwrites
        for (int i = WIDTH-1; i >= 0; i--) begin
            if (priority_in[i] == 1'b1 && !found) begin
                encode = $clog2(WIDTH)'(i);
                found = 1'b1;
            end
        end
    end
end

```

32.4.5 True Priority Encoding

- **MSB-first search:** for (int i = WIDTH-1; i >= 0; i--)
- **Single assignment:** The found flag stops any overwrites
- **Highest wins:** The first match (highest index) sets the output

32.4.6 Enable Control

- **Conditional operation:** Only active when enable = 1
- **Clean disable:** Output forced to zero when disabled
- **Power savings:** Can gate operation when not needed

32.4.7 Latch Prevention

```
// Default assignments to prevent latches
encode = '0;
found = 1'b0;
```

- Gives every signal path a defined value
- Keeps the synthesizer from warning about incomplete assignments

32.5 Timing Characteristics

- **Propagation delay:** Depends on WIDTH and synthesis optimization
- **Critical path:** Through the priority resolution loop
- **Enable response:** Immediate when enable changes
- **Setup/hold:** None (purely combinational)

32.6 Usage Examples

32.6.1 Interrupt Controller

```
encoder_priority_enable #(.WIDTH(8)) irq_encoder (
    .priority_in(interrupt_requests),
    .enable(interrupt_enable),
    .encode(highest_priority_irq)
);
```

32.6.2 Resource Arbitration

```
encoder_priority_enable #(.WIDTH(16)) arbiter (  
    .priority_in(request_vector),  
    .enable(arbitration_enable),  
    .encode(granted_requestor)  
);
```

32.6.3 Error Reporting

```
encoder_priority_enable #(.WIDTH(4)) error_encoder (  
    .priority_in({fatal_error, severe_error, warning, info}),  
    .enable(error_reporting_on),  
    .encode(error_level)  
);
```

32.7 Design Notes

32.7.1 Applications

Multi-level interrupt controllers: - CPU interrupt prioritization - Nested interrupt handling - Real-time system scheduling

Bus arbitration: - Multi-master bus systems - DMA channel arbitration - Memory access prioritization

Error and exception handling: - Exception priority resolution - Multi-source error reporting - Diagnostic priority encoding

32.7.2 Priority Assignment

- **Bit assignment:** Higher index = higher priority
- **Planning:** Assign critical functions to higher-indexed bits
- **Documentation:** Write the priority hierarchy down — you'll thank yourself

32.7.3 Performance Scaling

- **Linear complexity:** Search time increases with WIDTH
- **Synthesis optimization:** Modern tools optimize priority encoding well
- **Resource usage:** Typically maps to priority encoder primitives

32.7.4 Enable Usage Patterns

- **Power management:** Disable when priority encoding not needed

- **Conditional operation:** Enable based on system state
- **Test/debug:** Disable during certain test modes

32.7.5 Synthesis Considerations

- **Resource efficient:** Maps well to FPGA priority encoder resources
- **Timing closure:** Generally meets timing with proper constraints
- **Area scaling:** Grows approximately linearly with WIDTH

32.8 Related Modules

- **Basic Encoder:** Simpler version without enable or true priority
- **Find Last Set:** Similar MSB-first search functionality
- **Arbiter:** Often uses priority encoders for grant generation

32.9 Testing

`val/common/test_encoder_priority_enable.py` exercises this module. It collects 2 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/common/test_encoder_priority_enable.py -v
```

32.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

33 fifo_control

33.1 Overview

This is the shared brain that generates full/empty status flags for both FIFO variants (sync and async, the latter with either pointer encoding). All the tricky pointer arithmetic and mode-aware timing lives here.

33.2 Parameters

Parameter	Default	Description
ADDR_WIDTH	3	Address width
DEPTH	8	FIFO depth (default was 16 until common BUG-004 (was COMMON-014): 16 against ADDR_WIDTH=3 broke the module's own $DEPTH \leq 2^{ADDR_WIDTH}$ rule and wr_almost_full could never assert)
ALMOST_WR_MARGIN	1	Almost full threshold
ALMOST_RD_MARGIN	1	Almost empty threshold
REGISTERED	0	Output mode: 0=mux, 1=flop

33.3 Ports

Port	Direction	Width	Description
wr_clk	Input	1	Write domain clock
wr_rst_n	Input	1	Write domain active-low reset
rd_clk	Input	1	Read domain clock
rd_rst_n	Input	1	Read domain active-low reset
wr_ptr_bin	Input	ADDR_WIDTH+1	Write pointer (binary, next value)
wdom_rd_ptr_bin	Input	ADDR_WIDTH+1	Read pointer synchronized to write

Port	Direction	Width	Description
			domain
rd_ptr_bin	Input	ADDR_WIDT H+1	Read pointer (binary, next value)
rdom_wr_ptr_bin	Input	ADDR_WIDT H+1	Write pointer synchronized to read domain
count	Output	ADDR_WIDT H+1	Current FIFO occupancy count
wr_full	Output	1	Write domain full flag
wr_almost_full	Output	1	Write domain almost full flag
rd_empty	Output	1	Read domain empty flag
rd_almost_empty	Output	1	Read domain almost empty flag

33.4 Functional Description

33.4.1 Dual-Domain Design

The module straddles two clock domains: - **Write domain**: Generates full and almost_full flags - **Read domain**: Generates empty and almost_empty flags

33.4.2 Pointer Arithmetic Foundation

All the status generation boils down to **pointer comparison with wraparound detection**:

```
// XOR the MSBs to detect wraparound condition
assign w_wdom_ptr_xor = wr_ptr_bin[AW] ^ wdom_rd_ptr_bin[AW];
assign w_rdom_ptr_xor = rd_ptr_bin[AW] ^ rdom_wr_ptr_bin[AW];
```

33.4.3 Full Detection Logic

```
assign w_wr_full_d = (w_wdom_ptr_xor &&
                     (wr_ptr_bin[AW-1:0] == wdom_rd_ptr_bin[AW-1:0]));
```

The algorithm: - **Condition 1**: The MSBs must differ ($w_wdom_ptr_xor = 1$) - **Condition 2**: The LSBs must be equal - **Meaning**: The write pointer has “lapped” the read pointer

A visual example (DEPTH=8, ADDR_WIDTH=3):

Pointers: [wrap_bit][address_bits]

Case 1: Not Full
 wr_ptr = 0_101 (address 5, no wrap)
 rd_ptr = 0_010 (address 2, no wrap)
 → MSBs same, not full

Case 2: Full
 wr_ptr = 1_010 (address 2, wrapped once)
 rd_ptr = 0_010 (address 2, not wrapped)
 → MSBs differ, addresses same = FULL

33.4.4 Almost Full Logic

Occupancy calculation:

```
assign w_almost_full_count = (w_wdom_ptr_xor) ?
  ((AW+1)'(D) - wdom_rd_ptr_bin[AW-1:0] + wr_ptr_bin[AW-1:0]) :
  (wr_ptr_bin[AW-1:0] - wdom_rd_ptr_bin[AW-1:0]);
```

Two cases handled: 1. **No wraparound** ($w_wdom_ptr_xor = 0$): - Count = $wr_ptr - rd_ptr$ -
 Just a simple subtraction

2. **Wraparound** ($w_wdom_ptr_xor = 1$):

- Count = $DEPTH - rd_ptr + wr_ptr$
- Accounts for the circular buffer wrapping around

The threshold:

```
assign w_wr_almost_full_d = w_almost_full_count >= (AW+1)'(AFT);
// Where AFT = DEPTH - ALMOST_WR_MARGIN
```

33.4.5 Empty Detection Logic - Mode Aware

The critical innovation: mode-aware write pointer selection.

```
generate
  if (REGISTERED == 1) begin : gen_flop_mode
    // FLOP mode: Use previous cycle's write pointer
    logic [ADDR_WIDTH:0] r_rdom_wr_ptr_bin_delayed;

    always_ff @(posedge rd_clk or negedge rd_rst_n) begin
      if (!rd_rst_n) begin
        r_rdom_wr_ptr_bin_delayed <= '0;
      end else begin
        r_rdom_wr_ptr_bin_delayed <= rdom_wr_ptr_bin;
      end
    end
  end
end
```

```

        assign w_wr_ptr_for_empty = r_rdom_wr_ptr_bin_delayed;
    end else begin : gen_mux_mode
        // MUX mode: Use current write pointer
        assign w_wr_ptr_for_empty = rdom_wr_ptr_bin;
    end
endgenerate

```

Why mode-aware detection?

In MUX mode (REGISTERED = 0): - **Data availability:** Immediate (combinational read) - **Write pointer:** Use the current value - **Reasoning:** Data is available the moment it's written

In FLOP mode (REGISTERED = 1): - **Data availability:** Delayed by 1 cycle (registered read) - **Write pointer:** Use the delayed value - **Reasoning:** Data isn't available until the next clock cycle

The empty detection algorithm:

```

assign w_rd_empty_d = (!w_rdom_ptr_xor_for_empty &&
    (rd_ptr_bin[AW:0] == w_wr_ptr_for_empty[AW:0]));

```

- **Condition 1:** The MSBs must be the same (no wrap difference)
- **Condition 2:** All bits must be equal
- **Meaning:** The read pointer has caught up to the write pointer

33.4.6 Almost Empty Logic

Almost empty uses standard timing regardless of FIFO mode:

```

assign w_almost_empty_count = (w_rdom_ptr_xor) ?
    ((AW+1)'(D) - rd_ptr_bin[AW-1:0] + rdom_wr_ptr_bin[AW-1:0]) :
    (rdom_wr_ptr_bin[AW-1:0] - rd_ptr_bin[AW-1:0]);

assign w_rd_almost_empty_d = w_almost_empty_count <= (AW+1)'(AET);
// Where AET = ALMOST_RD_MARGIN

```

33.4.7 Count Generation

```

// Combinational occupancy...
logic [AW:0] w_count, r_count;
assign w_count = (w_rdom_ptr_xor) ?
    (rdom_wr_ptr_bin[AW-1:0] - rd_ptr_bin[AW-1:0] + (AW+1)'(D)) :
    (rdom_wr_ptr_bin[AW-1:0] - rd_ptr_bin[AW-1:0]);

// ...but the exported count is REGISTERED in flop mode (r_count is
w_count
// delayed one rd_clk); mux mode exports the combinational value
directly.
assign count = (REGISTERED == 1) ? r_count : w_count;

```

How to read the count: - **Range:** 0 to DEPTH - **Zero:** FIFO empty - **DEPTH:** FIFO full - **Uses:** Flow control, occupancy monitoring

33.4.8 Flag Registration

Write domain flags:

```
always_ff @(posedge wr_clk, negedge wr_rst_n) begin
    if (!wr_rst_n) begin
        wr_full <= 'b0;
        wr_almost_full <= 'b0;
    end else begin
        wr_full <= w_wr_full_d;
        wr_almost_full <= w_wr_almost_full_d;
    end
end
```

Read domain flags:

```
always_ff @(posedge rd_clk, negedge rd_rst_n) begin
    if (!rd_rst_n) begin
        rd_empty <= 'b1;           // Reset to empty
        rd_almost_empty <= 'b0;
    end else begin
        rd_empty <= w_rd_empty_d;
        rd_almost_empty <= w_rd_almost_empty_d;
    end
end
```

Reset behavior: - **Full flags:** Reset to 0 (not full) - **Empty flags:** Reset to 1 (empty) - **Almost flags:** Reset to 0

33.4.9 Width Casting Fix

```
// The occupancy count needs AW+1 bits, and D (= DEPTH) must be cast
// to AW+1
// bits -- NOT AW bits. AW'(D) TRUNCATES: for depth=16, AW=4, AW'(16)
// = 4'b0000
// (wrong!), whereas (AW+1)'(16) = 5'b10000 (correct). The count
// register is
// widened to [AW:0] to hold the full range 0..DEPTH.
logic [AW:0] w_almost_full_count;
```

```
assign w_almost_full_count = (w_wdom_ptr_xor) ?
    ((AW+1)'(D) - wdom_rd_ptr_bin[AW-1:0] + wr_ptr_bin[AW-1:0]) :
    (wr_ptr_bin[AW-1:0] - wdom_rd_ptr_bin[AW-1:0]);
```

That (AW+1)'(D) cast (with the widened [AW:0] count) is the only thing standing between you and a truncated wraparound occupancy. Cast to AW'(D) instead and you reintroduce the

truncation bug: e.g. DEPTH=16 with wr=2, rd=14 should give occupancy $16-14+2 = 4$, but AW' (16) = 0 yields $0-14+2$ (garbage).

33.5 Timing Characteristics

- **Minimal overhead:** Efficient pointer arithmetic
- **Low latency:** Single cycle flag updates
- **Resource efficient:** Shared across multiple FIFO types

33.6 Usage Examples

Every parameter and port below is taken from the module declaration.

```
fifo_control #(
    .ADDR_WIDTH      (3),
    .DEPTH           (8),
    .ALMOST_WR_MARGIN (1),
    .ALMOST_RD_MARGIN (1),
    .REGISTERED      (0)
) u_fifo_control (
    .wr_rst_n        (wr_rst_n),
    .rd_clk           (rd_clk),
    .rd_rst_n        (rd_rst_n),
    .wr_clk           (wr_clk),
    .wr_ptr_bin       (wr_ptr_bin),
    .wdom_rd_ptr_bin  (wdom_rd_ptr_bin),
    .rd_ptr_bin       (rd_ptr_bin),
    .rdom_wr_ptr_bin  (rdom_wr_ptr_bin),
    .count            (count),
    .wr_full          (wr_full),
    .wr_almost_full   (wr_almost_full),
    .rd_empty         (rd_empty),
    .rd_almost_empty  (rd_almost_empty)
);
```

33.7 Design Notes

33.7.1 Key Design Insights

Wraparound handling: - **MSB significance:** The extra bit is your wraparound detector - **Circular arithmetic:** Proper modulo DEPTH calculations - **Comparison logic:** XOR-based wraparound detection

Timing considerations: - **Synchronizer delay**: Accounted for in the conservative design - **Mode awareness**: Different timing for different output modes - **Flag updates**: Registered for clean transitions

Conservative design: - **Over-reporting**: Flags may assert slightly early - **Safety margin**: Prevents overflow/underflow - **Synchronizer latency**: Built into the safety margins

33.7.2 Applications

- **Status monitoring**: Provides comprehensive FIFO state
- **Flow control**: Enables back-pressure and rate matching
- **Debug/verification**: The count output earns its keep during debug

33.8 Related Modules

- **fifo_sync** and **fifo_async**: both FIFO variants share this status logic (binary or Johnson pointers). The former **fifo_async_div2** is retired — **fifo_async** with **USE_JOHNSON=1** replaces it.

33.9 Testing

No dedicated testbench for this module. It has no `val/**/test_fifo_control.py`; it is exercised indirectly, through the tests of the modules that instantiate it:

- **fifo_sync** – `val/common/test_fifo_sync.py`
- **gaxi_drop_fifo_sync** – `val/amba/test_gaxi_drop_fifo_sync.py`
- **gaxi_fifo_sync** – `val/amba/test_gaxi_fifo_sync.py`

Indirect coverage exercises this module only in the configurations its parents elaborate. A parameter or mode no parent uses is untested.

33.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

34 fifo_sync

34.1 Overview

A synchronous First-In-First-Out buffer for when everything lives in a single clock domain. Depth, data width, and output modes are all configurable, and you get the full set of status signals.

34.2 Parameters

Parameter	Default	Description
MEM_STYLE	FIFO_AUTO	Memory implementation (FIFO_AUTO/SRL/BRAM) — see note below
REGISTERED	0	Output mode: 0=mux mode (combinational), 1=flop mode (registered)
DATA_WIDTH	4	Width of data bus
DEPTH	4	FIFO depth in words
ALMOST_WR_MARGIN	1	Almost full threshold
ALMOST_RD_MARGIN	1	Almost empty threshold

MEM_STYLE details: In this module every branch honours REGISTERED: with REGISTERED=0 the BRAM branch gives a *combinational* read (assign rd_data = mem[r_rd_addr]), not a registered one. Note that fifo_async differs here — its BRAM branch is unconditionally registered — so do not carry that assumption across. **Caveat:** MEM_STYLE=FIFO_BRAM with REGISTERED=0 asks for an asynchronous read from a block RAM, which real BRAM cannot do; synthesis will ignore the ram_style/ramstyle attribute and map to LUTRAM instead. Use REGISTERED=1 when you actually want block RAM.

34.3 Ports

Port	Direction	Width	Description
clk	Input	1	System clock
rst_n	Input	1	Active-low reset
write	Input	1	Write enable signal
wr_data	Input	DATA_WIDTH	Data to write into FIFO

Port	Direction	Width	Description
read	Input	1	Read enable signal
wr_full	Output	1	Write domain full flag
wr_almost_full	Output	1	Write domain almost full flag
rd_data	Output	DATA_WIDTH	Data read from FIFO
rd_empty	Output	1	Read domain empty flag
rd_almost_empty	Output	1	Read domain almost empty flag

34.4 Functional Description

34.4.1 Core Components

1. **Binary counters** for the read/write pointers
2. **Memory array** for data storage
3. **FIFO control logic** for status generation
4. **Output multiplexing** based on the REGISTERED parameter

34.4.2 Memory Organization

```

logic [DATA_WIDTH-1:0] mem [DEPTH];           // Memory array (declared
inside the                                     // MEM_STYLE generate branch
so all                                         // accesses share one scope)
assign r_wr_addr = r_wr_ptr_bin[AW-1:0];      // Write address
assign r_rd_addr = r_rd_ptr_bin[AW-1:0];      // Read address

```

34.4.3 Pointer Management

Binary counters drive both the read and write pointers:

```

counter_bin #(.WIDTH(AW + 1), .MAX(D)) write_pointer_inst (
    .clk(clk),
    .rst_n(rst_n),

```



```

        .enable(write && !wr_full),
        .counter_bin_curr(r_wr_ptr_bin),
        .counter_bin_next(w_wr_ptr_bin_next)
    );

```

Pointer characteristics: - **Width:** $\$clog2(DEPTH) + 1$ bits (the extra bit catches wraps) - **Increment:** Only when the operation is valid (write && !full, read && !empty) - **Wraparound:** Automatic modulo DEPTH counting

34.4.4 Memory Operations

Write operation:

```

always_ff @(posedge clk) begin
    if (write && !wr_full) begin // the !wr_full guard is REQUIRED:
        it stops a
        mem[r_wr_addr] <= wr_data; // write-while-full from
        clobbering unread data
    end
end

```

Read operation — dual mode:

```

if (REGISTERED != 0) begin : g_flop
    logic [DATA_WIDTH-1:0] r_rd_data;
    `ALWAYS_FF_RST(clk, rst_n,
        if (!rst_n) r_rd_data <= '0;
        else r_rd_data <= mem[r_rd_addr];
    )
    assign rd_data = r_rd_data;

end else begin : g_mux
    // Mux mode - non-registered output
    assign rd_data = mem[r_rd_addr];
end

```

34.4.5 Status Flag Generation

Status comes out of the shared fifo_control module: - **Full detection:** Based on pointer comparison with wraparound handling - **Almost full:** When remaining space $\leq$ ALMOST_WR_MARGIN - **Empty detection:** When read pointer equals write pointer - **Almost empty:** When available data $\leq$ ALMOST_RD_MARGIN

34.4.6 Operating Modes

Mux mode (REGISTERED = 0): - **Read latency:** 0 cycles (combinational output) - **Data availability:** Immediate after write - **Use case:** Low-latency applications - **Timing:** Read data changes combinationally with the address

Flop mode (REGISTERED = 1): - **Read latency:** 1 cycle (registered output) - **Data availability:** 1 cycle after read enable - **Use case:** High-speed designs, timing closure - **Timing:** Read data stable for the full clock cycle

34.4.7 Functional Behavior

Write operations: - **Condition:** write && !wr_full - **Action:** Store data at the write pointer, bump the pointer - **Blocking:** Writes ignored when the FIFO is full - **Status:** Full flags track the new occupancy

Read operations: - **Condition:** read && !rd_empty - **Action:** Advance the read pointer (data handling depends on mode) - **Blocking:** Reads ignored when the FIFO is empty - **Status:** Empty flags track the new occupancy

Reset behavior: - **Pointers:** Reset to 0 - **Flags:** Full flags $\rightarrow$ 0, Empty flags $\rightarrow$ 1 - **Data:** Read data cleared in flop mode - **Memory:** Contents undefined, but that doesn't matter

34.5 Timing Characteristics

34.5.1 Write Sequence

clk _|_|_||_|_||_|_||_|_
write _____|_____
wr_data =====[D0]=====
wr_full _____|_____ (when FIFO becomes full)

34.5.2 Read Sequence - Mux Mode

```

clk      _|_|_||_|_||_|_||_|_
read     _____|_____
rd_data  =====[ D0 ]===== (immediate)

```

34.5.3 Read Sequence - Flop Mode

clk _|_|_||_|_||_|_
read _____|_____
rd data ==[D0]== (1 cycle delay)

34.5.4 Performance Characteristics

- **Throughput:** 1 operation per clock cycle (when not full/empty)
- **Latency:** 0 cycles (mux) or 1 cycle (flop) for read
- **Occupancy:** 0 to DEPTH words
- **Efficiency:** 100% bandwidth utilization possible

34.6 Usage Examples

Every parameter and port below is taken from the module declaration.

```
fifo_sync #(
    .MEM_STYLE          (FIFO_AUTO),
    .REGISTERED         (0),
    .DATA_WIDTH         (4),
    .DEPTH              (4),
    .ALMOST_WR_MARGIN   (1),
    .ALMOST_RD_MARGIN   (1)
) u_fifo_sync (
    .rst_n              (rst_n),
    .clk                (clk),
    .write              (write),
    .wr_data            (wr_data),
    .wr_full            (wr_full),
    .wr_almost_full     (wr_almost_full),
    .read              (read),
    .rd_data            (rd_data),
    .rd_empty           (rd_empty),
    .rd_almost_empty    (rd_almost_empty)
);
```

34.7 Design Notes

34.7.1 Mode Selection Guidelines

- **Choose Mux Mode when:** Latency is critical, moderate clock speeds
- **Choose Flop Mode when:** High clock speeds, timing closure issues
- **Performance impact:** Flop mode costs you 1 cycle of latency but buys fmax

Typical use cases: - **Low latency applications (mux mode):** Data streaming with minimal delay, real-time processing pipelines, clock domain buffers - **High speed applications (flop mode):** High-frequency designs, timing-critical paths, pipelined architectures

34.7.2 Sizing Considerations

- **Depth:** Must accommodate worst-case burst sizes
- **Almost flags:** Set margins based on producer/consumer response times
- **Data width:** Should match datapath requirements

34.7.3 Error Detection

Note: the current RTL does **not** contain any runtime `$display` overflow/underflow checks. The `!wr_full` write guard is the only overflow protection. If you need write-while- full / read-while-empty telemetry, add assertions in your own testbench.

34.8 Related Modules

- **fifo_async:** For clock domain crossing applications
- **fifo_control:** Shared status flag generation logic
- **counter_bin:** Binary counter implementation

34.9 Testing

34.9.1 WaveDrom Visualization

The WaveDrom test generates detailed timing diagrams for synchronous FIFO operation:

```
# Generate synchronous FIFO waveforms (single clock domain)
pytest val/common/test_fifo_sync_wavedrom.py -v
```

Waveform scenarios generated:

1. Write-Fill-Read-Empty Cycle

- Basic synchronous FIFO operation
- Simple binary pointer management
- No CDC complexity (single clock domain)

2. Back-to-Back Operations

- Maximum throughput demonstration
- Minimal inter-transaction delays
- Sequential write and read bursts

3. Simultaneous Write-Read (Ping-Pong)

- Read and write in same cycle
- Steady-state FIFO operation
- Demonstrating simultaneous access capability

4. Flag Transitions

- Full/empty flag behavior at boundaries
- Almost-full/almost-empty thresholds
- Flow control signaling

Key characteristics vs. async FIFOs:

- **Single Clock Domain:** No CDC complexity, simpler design
- **Binary Pointers:** Direct addressing, no Gray code conversion
- **Zero CDC Latency:** Flags update immediately (no synchronization delay)
- **Simultaneous Access:** Can read and write in same cycle

Comparison tests:

- `test_fifo_async_wavedrom.py` - Gray code CDC (power-of-2 depths)

34.9.2 Functional Test Suite

- `val/common/test_fifo_buffer.py` - Full functional verification
- `val/common/test_fifo_sync_wavedrom.py` - WaveDrom timing diagrams

Run the tests:

```
# Full functional test (basic/medium/full levels)
pytest val/common/test_fifo_buffer.py -v
```

```
# WaveDrom waveform generation
pytest val/common/test_fifo_sync_wavedrom.py -v
```

34.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

35 find_first_set

35.1 Overview

Finds the index of the least significant bit (LSB) that's set to '1' in the input vector — a priority encoder that plays favorites with the low bits.

35.2 Parameters

Parameter	Default	Description
WIDTH	32	Width of input data vector — the module's only parameter

35.3 Ports

Port	Direction	Width	Description
data	Input	WIDTH	Input data vector to search
index	Output	$\$clog2(WIDTH)$	Index of the first (lowest) set bit

35.4 Functional Description

35.4.1 Search Strategy

- **Direction:** LSB to MSB (left to right, ascending index)
- **Priority:** Lower indices have higher priority
- **First match:** Stops at the first '1' bit encountered
- **Found flag:** Prevents multiple assignments

35.4.2 Truth Table Examples (WIDTH=8)

data[7:0]	index[2:0]	Description
00000001	000	Bit 0 is first set
00000010	001	Bit 1 is first set
00000100	010	Bit 2 is first set
00001000	011	Bit 3 is first set
00010000	100	Bit 4 is first set

data[7:0]	index[2:0]	Description
00100000	101	Bit 5 is first set
01000000	110	Bit 6 is first set
10000000	111	Bit 7 is first set
00000000	000	No bits set - default
10000001	000	Multiple bits - LSB wins
11111111	000	All bits set - bit 0 wins

35.4.3 Key Characteristics

- **LSB priority:** Multiple bits set? You get the lowest index
- **Zero default:** Returns 0 when no bits are set
- **Deterministic:** Same input always produces same output
- **Combinational:** Immediate response to input changes

35.4.4 Core Algorithm

```

localparam int N = $clog2(WIDTH);
logic w_found;

always_comb begin
    index = {N{1'b0}}; // Default value if no bit is set
    w_found = 1'b0;

    for (int i = 0; i < WIDTH; i++) begin
        if (data[i] && !w_found) begin
            index = i[N-1:0]; // Ensure correct bit width
            w_found = 1'b1;
        end
    end
end
end

```

35.4.5 Width Parameterization

- **Automatic sizing:** Output width calculated as $\$clog2(WIDTH)$
- **Examples:**
 - WIDTH=8 → index[2:0] (3 bits)

- WIDTH=16 → index[3:0] (4 bits)
- WIDTH=32 → index[4:0] (5 bits)

35.4.6 Found Flag Logic

```
if (data[i] && !w_found) begin
    index = i[N-1:0];
    w_found = 1'b1;
end
```

- **Purpose:** Keeps the first found result from being overwritten
- **Mechanism:** Once w_found is set, no further assignments occur
- **Result:** Only the lowest index is captured

35.4.7 Bit Width Handling

```
index = i[N-1:0]; // Ensure correct bit width
```

- **Type safety:** Explicit width matching keeps synthesis warnings away
- **Truncation:** Safely handles loop variable width vs. output width

35.5 Timing Characteristics

35.5.1 Propagation Delay

- **Worst case:** Multiple LUT delays (higher bits set)
- **Average case:** Depends on typical input patterns

35.5.2 Critical Path

- **Path length:** Increases with WIDTH
- **Optimization:** Modern synthesizers create efficient priority encoders
- **Scaling:** Generally logarithmic complexity in hardware

35.6 Usage Examples

35.6.1 Interrupt Processing

```
find_first_set #(.WIDTH(8)) irq_ffs (
    .data(interrupt_pending),
    .index(lowest_priority_irq)
);
```


Process the lowest-numbered interrupt first — fair servicing of interrupt sources.

35.6.2 Resource Allocation

```
find_first_set #(.WIDTH(16)) resource_ffs (  
    .data(available_resources),  
    .index(allocated_resource_id)  
);
```

Allocate the lowest-numbered available resource; round-robin allocation when combined with masking.

35.6.3 Error Detection

```
find_first_set #(.WIDTH(4)) error_ffs (  
    .data(error_status),  
    .index(first_error_type)  
);
```

Report the first error condition detected, prioritizing error handling by position.

35.6.4 Bit Manipulation

```
find_first_set #(.WIDTH(32)) bit_scan (  
    .data(search_pattern),  
    .index(first_set_position)  
);
```

Bit scanning operations, pattern analysis and parsing.

35.6.5 Round-Robin Arbitration

```
// Mask off already-serviced requests  
assign masked_requests = pending_requests & ~service_mask;  
  
find_first_set #(.WIDTH(N)) arbiter_ffs (  
    .data(masked_requests),  
    .index(next_grant)  
);
```

```
// Update mask to exclude granted request  
assign next_mask = service_mask | (1 << next_grant);
```

35.6.6 Free List Management

```
find_first_set #(.WIDTH(P00L_SIZE)) free_finder (  
    .data(free_pool_bitmap),  
    .index(allocated_entry)  
);
```

```
// Mark entry as allocated
assign updated_bitmap = free_pool_bitmap & ~(1 << allocated_entry);
```

35.6.7 Packet Classification

```
find_first_set #(.WIDTH(NUM_RULES)) rule_matcher (
    .data(rule_match_vector),
    .index(matched_rule_id)
);
```

35.7 Design Notes

35.7.1 Algorithm Comparison

Versus find_last_set:

Aspect	Find First Set	Find Last Set
Search direction	LSB → MSB	MSB → LSB
Priority	Lower index wins	Higher index wins
Use case	Fair allocation	Priority allocation

Versus a priority encoder:

Aspect	Find First Set	Priority Encoder
Priority direction	LSB has priority	MSB has priority
Application	Round-robin systems	Hierarchical systems

35.7.2 Design Considerations

Input validation: - **All zeros**: Returns 0 (may need validation in application) - **Valid range**: All input patterns handled gracefully - **No error conditions**: Always produces a result

Performance optimization: - **Synthesis**: Let tools optimize the priority encoding - **Pipelining**: Consider registering for high-speed applications - **Parallel**: Can process multiple vectors simultaneously

Width selection: - **Power of 2**: Often most efficient for synthesis - **Arbitrary width**: Supported but may have overhead - **Large widths**: Consider hierarchical implementation

35.7.3 Synthesis Considerations

- **LUT usage**: Typically maps to priority encoder primitives
- **Efficiency**: Good utilization for most FPGA architectures
- **Scaling**: Linear increase in resources with WIDTH

- **Critical paths:** May need pipeline registers for large WIDTH
- **Clock frequency:** Generally achieves high fmax
- **Constraints:** Standard combinational timing constraints apply

35.8 Related Modules

- **find_last_set:** MSB-first search variant
- **encoder:** Basic priority encoder
- **encoder_priority_enable:** Enhanced priority encoder with enable

35.9 Testing

35.9.1 Test Scenarios

- **Single bit:** Each bit position individually
- **Multiple bits:** Various combinations
- **All zeros:** Edge case handling
- **All ones:** Maximum case behavior

35.9.2 Coverage Metrics

- **Bit coverage:** Each input bit position
- **Pattern coverage:** Representative bit combinations
- **Boundary cases:** Min/max index values

35.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

36 find_last_set

36.1 Overview

Finds the index of the most significant bit (MSB) that's set to '1' in the input vector — the mirror image of find_first_set, favoring the higher-indexed bits.

36.2 Parameters

Parameter	Default	Description
WIDTH	32	Width of input data vector — the module's only parameter

36.3 Ports

Port	Direction	Width	Description
data	Input	WIDTH	Input data vector to search
index	Output	$\$clog2(WIDTH)$	Index of the last (highest) set bit

36.4 Functional Description

36.4.1 Search Strategy

- **Direction:** MSB to LSB (right to left, descending index)
- **Priority:** Higher indices have higher priority
- **First match:** Stops at the first '1' bit encountered from the MSB
- **Found flag:** Prevents multiple assignments

36.4.2 Truth Table Examples (WIDTH=8)

data[7:0]	index[2:0]	Description
10000000	111	Bit 7 is last set
01000000	110	Bit 6 is last set
00100000	101	Bit 5 is last set
00010000	100	Bit 4 is last set
00001000	011	Bit 3 is last set

data[7:0]	index[2:0]	Description
00000100	010	Bit 2 is last set
00000010	001	Bit 1 is last set
00000001	000	Bit 0 is last set
00000000	000	No bits set - default
10000001	111	Multiple bits - MSB wins
11111111	111	All bits set - bit 7 wins

36.4.3 Key Characteristics

- **MSB priority:** Multiple bits set? You get the highest index
- **Zero default:** Returns 0 when no bits are set (same as find_first_set)
- **Deterministic:** Same input always produces same output
- **Combinational:** Immediate response to input changes

36.4.4 Core Algorithm

```

localparam int N = $clog2(WIDTH);
logic w_found;

always_comb begin
    index = {N{1'b0}}; // Default value if no bit is set
    w_found = 1'b0;

    for (int i = WIDTH - 1; i >= 0; i--) begin
        if (data[i] && !w_found) begin
            index = i[N-1:0]; // Ensure correct bit width
            w_found = 1'b1;
        end
    end
end

```

36.4.5 Reverse Loop Implementation

```

for (int i = WIDTH - 1; i >= 0; i--) begin

```

- **MSB-first:** Starts from the highest bit index
- **Decrementing:** Walks toward the LSB

- **Early termination:** Stops at the first match found

36.4.6 Width Parameterization

Same auto-sizing as `find_first_set`: - **Output width:** $\lceil \log_2(\text{WIDTH}) \rceil$ bits - **Examples:** -
 WIDTH=8 → index[2:0] (3 bits) - WIDTH=16 → index[3:0] (4 bits) - WIDTH=32 → index[4:0] (5 bits)

36.4.7 Found Flag Logic

```
if (data[i] && !w_found) begin
    index = i[N-1:0];
    w_found = 1'b1;
end
```

- **Single assignment:** Prevents overwriting the highest found bit
- **Priority enforcement:** Only the first match (highest index) is captured

36.5 Timing Characteristics

36.5.1 Propagation Delay

- **Worst case:** Multiple LUT delays (LSB only set)
- **Average case:** Depends on typical bit patterns

36.5.2 Critical Path Analysis

MSB check → MSB-1 check → ... → LSB check → Output

- **Path length:** Linear with WIDTH in the worst case
- **Typical case:** Shorter path when higher bits are set
- **Optimization:** Synthesis tools create efficient implementations

36.6 Usage Examples

36.6.1 Interrupt Priority Handling

```
find_last_set #(.WIDTH(8)) irq_priority (
    .data(interrupt_pending),
    .index(highest_priority_irq)
);
```

Service the highest-priority interrupt first — preemptive interrupt handling.

36.6.2 Leading Zero Count

```
// Find position of leading 1, then calculate leading zeros
find_last_set #(.WIDTH(32)) leading_one (
    .data(input_value),
    .index(msb_position)
);
```

```
assign leading_zero_count = (input_value == 0) ? 32 : (31 -
msb_position);
```

36.6.3 Bit Width Calculation

```
find_last_set #(.WIDTH(64)) width_calc (
    .data(value),
    .index(msb_pos)
);
```

```
assign required_bits = msb_pos + 1; // Minimum bits needed to
represent value
```

36.6.4 Cache Line Selection

```
find_last_set #(.WIDTH(16)) cache_priority (
    .data(cache_line_priorities),
    .index(selected_cache_line)
);
```

Select the highest priority cache line for replacement — LRU policy implementation.

36.6.5 Hierarchical Priority Encoding

```
// For very wide inputs, use hierarchical approach
find_last_set #(.WIDTH(8)) level0_fls [7:0] (
    .data(data_chunks), // 8 chunks of 8 bits each
    .index(chunk_indices)
);

find_last_set #(.WIDTH(8)) level1_fls (
    .data(chunk_valid), // Which chunks have set bits
    .index(active_chunk)
);

// Combine results
assign final_index = {active_chunk, chunk_indices[active_chunk]};
```

36.6.6 Conditional Priority Selection

```
// Select between different priority schemes
assign priority_vector = use_high_priority ? high_pri_requests :
```

```

        use_fair_mode ? fair_requests :
        normal_requests;

find_last_set #(.WIDTH(N)) flexible_priority (
    .data(priority_vector),
    .index(selected_request)
);

36.6.7 Dynamic Width Processing
// Process variable-width data
logic [MAX_WIDTH-1:0] padded_data;
assign padded_data = {(MAX_WIDTH-actual_width){1'b0}}, input_data};

find_last_set #(.WIDTH(MAX_WIDTH)) variable_fls (
    .data(padded_data),
    .index(raw_index)
);

// Adjust result for actual width
assign adjusted_index = (raw_index >= actual_width) ? 0 : raw_index;

```

36.7 Design Notes

36.7.1 Algorithm Comparison

Find last set vs. find first set:

Aspect	Find Last Set	Find First Set
Search Direction	MSB → LSB	LSB → MSB
Priority	Higher index wins	Lower index wins
Loop Direction	i-- (decrementing)	i++ (incrementing)
Use Case	Priority systems	Fair allocation
Default Output	0 (same)	0 (same)

Typical use case patterns:

Application Type	Preferred Algorithm
Interrupt Controllers	Find Last Set (priority)
Round-Robin Arbiters	Find First Set (fairness)
Error Reporting	Find Last Set (severity)
Resource Allocation	Find First Set (efficiency)
Bit Manipulation	Either (depends on need)

36.7.2 Zero Input Handling

Both modules return 0 for zero input:

```
// data = 8'b00000000
// Both find_first_set and find_last_set return 3'b000
```

Your application may need explicit zero detection:

```
assign valid_result = |data; // OR-reduce to detect any bits set
assign result_index = valid_result ? index : INVALID_INDEX;
```

36.7.3 Performance Scaling

- **Small WIDTH (<16):** Excellent performance
- **Medium WIDTH (16-64):** Good performance, may need pipelining
- **Large WIDTH (>64):** Consider hierarchical implementation

36.7.4 Synthesis Optimization

Modern synthesis tools recognize these patterns: - **Priority encoder primitives:** Often map to dedicated hardware - **LUT optimization:** Efficient resource utilization - **Timing optimization:** Automatic critical path optimization

36.8 Related Modules

- **find_first_set:** LSB-priority search
- **encoder_priority_enable:** Enhanced priority encoder
- **leading_one_trailing_one:** Position detection for Johnson counters

36.9 Testing

36.9.1 Comprehensive Test Coverage

```
// Test single bit at each position
for (int i = 0; i < WIDTH; i++) begin
    test_vector = 1 << i;
    expected_result = i;
    // Verify find_last_set returns i
end
```

```
// Test multiple bits with known MSB
test_vector = 8'b10101010;
expected_result = 7; // MSB position
```

```
// Test edge cases  
test_vector = 8'b00000000; // All zeros  
expected_result = 0; // Default
```

36.9.2 Random Testing

```
// Random bit patterns  
repeat (1000) begin  
    random_vector = $random();  
    expected = calculate_msb_position(random_vector);  
    // Verify against reference function  
end
```

36.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

37 hex_to_7seg

37.1 Overview

Converts 4-bit hexadecimal values (0-F) to 7-segment display patterns for common anode displays — the thing you reach for when you want human-readable output for debugging, user interfaces, and embedded systems.

37.2 Parameters

This module has no parameters — fixed functionality for standard 7-segment displays.

37.3 Ports

Port	Direction	Width	Description
hex	Input	4	4-bit hexadecimal input (0x0 to 0xF)
seg	Output	7	7-segment display pattern

37.4 Functional Description

37.4.1 Segment Naming Convention

```
aaa
f   b
f   b
ggg
e   c
e   c
ddd
```

37.4.2 Bit Assignment

- **seg[6:0] = {g, f, e, d, c, b, a}**
- **Common Anode:** 0 = segment ON, 1 = segment OFF
- **Active Low:** LED segments illuminate when driven low

37.4.3 Character Patterns

Complete truth table:

Hex	Display	seg[6:0]	Segments ON	Pattern
0x0	0	1000000	abcdef	Classic 0
0x1	1	1111001	bc	Right side
0x2	2	0100100	abged	Snake pattern
0x3	3	0110000	abgcd	Right-heavy
0x4	4	0011001	fgbc	Left + right
0x5	5	0010010	afgcd	S-curve
0x6	6	0000010	afgdce	Almost all
0x7	7	1111000	abc	Top + right
0x8	8	0000000	all	All segments
0x9	9	0011000	abgcf	Top + middle + right, no tail
0xA	A	0001000	abcefg	Letter A
0xB	b	0000011	fgedc	Lowercase b
0xC	C	1000110	afed	Letter C
0xD	d	0100001	bgedc	Lowercase d
0xE	E	0000110	afged	Letter E
0xF	F	0001110	afge	Letter F

Visual representation:

```

###  #  ###  ###  #  #  ###  ###  ###  ###  ###
#  #  #  #  #  #  #  #  #  #  #  #  #  #
#  #  #  ###  ###  ###  ###  ###  #  ###  ###
#  #  #  #  #  #  #  #  #  #  #  #  #
###  #  ###  ###  #  ###  ###  #  ###  #
0    1    2    3    4    5    6    7    8    9

```

```

###  ###  ##  ###  ###  ###
#  #  #  #  #  #  #

```

```

###  ###  #  #  #  ###  ###

```

```

# # #   # #   #   # #   #
# # ###   ##   ###   ###   #
  A      b      C      d      E      F

```

37.4.4 Case Statement Structure

```

always_comb begin
  casez (hex)
    4'h0: seg = 7'b1000000; // 0: abcdef (not g)
    4'h1: seg = 7'b1111001; // 1: bc
    4'h2: seg = 7'b0100100; // 2: abged
    4'h3: seg = 7'b0110000; // 3: abgcd
    4'h4: seg = 7'b0011001; // 4: fgbc
    4'h5: seg = 7'b0010010; // 5: afgcd
    4'h6: seg = 7'b0000010; // 6: afgdce
    4'h7: seg = 7'b1111000; // 7: abc
    4'h8: seg = 7'b0000000; // 8: all segments
    4'h9: seg = 7'b0011000; // 9: abgcf
    4'ha: seg = 7'b0001000; // A: abcefg (not d)
    4'hb: seg = 7'b0000011; // b: fgedc (not ab)
    4'hc: seg = 7'b1000110; // C: afed (not bcd)
    4'hd: seg = 7'b0100001; // d: bgedc (not af)
    4'he: seg = 7'b0000110; // E: afged (not bc)
    4'hf: seg = 7'b0001110; // F: afge (not bcd)
    default: seg = 7'bxxxxxxx; // Invalid - undefined output
  endcase
end

```

37.5 Timing Characteristics

- **Combinational logic:** The case statement maps to a LUT
- **No clock dependency:** Immediate response to input changes

On display update rate: - **Electronic response:** Immediate (propagation delay) - **Visual response:** Limited by LED rise/fall times (~microseconds) - **Human perception:** >30Hz refresh rate for flicker-free display

37.6 Usage Examples

37.6.1 Debug Displays

```

// Display processor status
hex_to_7seg status_display (
    .hex(processor_status[3:0]),
    .seg(status_segments)
);

```

37.6.2 Multi-Digit Displays

```
// 4-digit hex display
hex_to_7seg digit0 (.hex(value[3:0]), .seg(seg0));
hex_to_7seg digit1 (.hex(value[7:4]), .seg(seg1));
hex_to_7seg digit2 (.hex(value[11:8]), .seg(seg2));
hex_to_7seg digit3 (.hex(value[15:12]), .seg(seg3));
```

37.6.3 Memory/Register Viewers

```
// Display memory contents
hex_to_7seg mem_display (
    .hex(memory_data[address]),
    .seg(memory_segments)
);
```

37.6.4 Error Code Display

```
// Show error codes
hex_to_7seg error_display (
    .hex(error_code),
    .seg(error_segments)
);
```

37.7 Design Notes

37.7.1 Design Choices

Lowercase vs. uppercase: - **A**: Uppercase (more readable) - **b, d**: Lowercase (distinguishable from 8, 0) - **C**: Uppercase (standard representation) - **E, F**: Uppercase (clear representation)

Common anode configuration: - **Active low outputs**: 0 = LED on, 1 = LED off - **Current sourcing**: Display provides +V, controller sinks current - **Logic inversion**: Many controllers expect active-high, requiring external inversion

37.7.2 Display Technology Integration

Common anode connection:

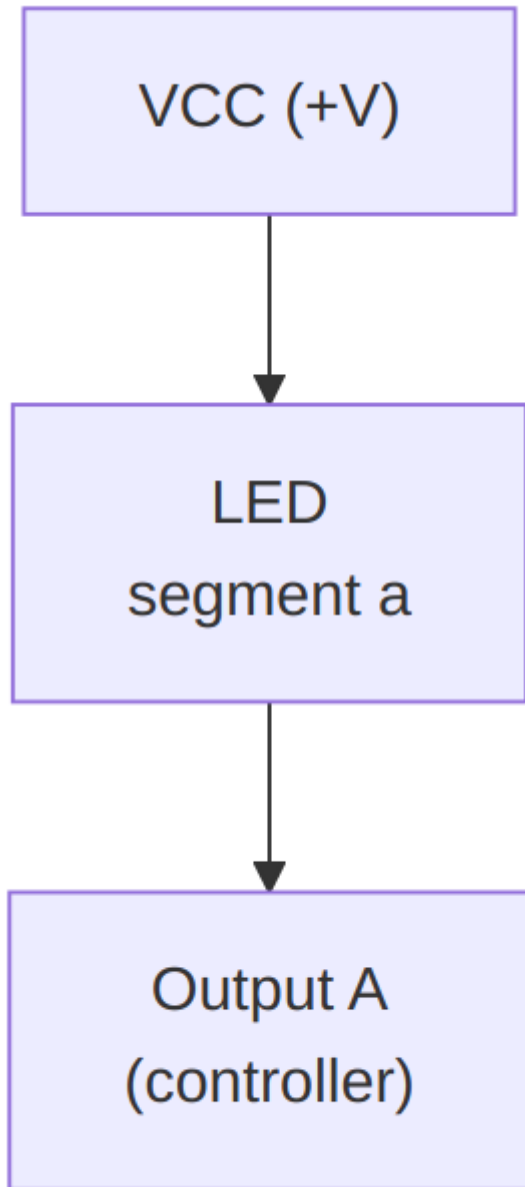


Diagram 1

Running common cathode instead? Invert all outputs:

```
// For common cathode displays:  
assign seg_common_cathode = ~seg;
```

Current limiting:

Controller → [Resistor] → LED Segment → Common Anode (+V)

Typical resistor values: 220Ω - 1kΩ depending on LED specifications.

37.7.3 Design Variations

Extended character set — some implementations add extra characters:

```
// Extended characters (implementation dependent)
4'h10: seg = 7'b1111111; // Blank/space
4'h11: seg = 7'b0111111; // Dash/minus
4'h12: seg = 7'b1110111; // Underscore
```

Alternative fonts for better readability:

```
// Alternative 'b' (uppercase-like)
4'hb: seg = 7'b0000111; // More symmetric b
```

Brightness control:

```
// PWM brightness control
logic [6:0] seg_raw;
hex_to_7seg decoder (.hex(hex_in), .seg(seg_raw));

// Apply PWM for brightness
assign seg_out = brightness_enable ? seg_raw : 7'b1111111;
```

37.7.4 Multiplexed Display Support

Time-division multiplexing:

```
// 4-digit multiplexed display
logic [1:0] digit_select;
logic [3:0] current_hex;

// Select current digit data
always_comb begin
    case (digit_select)
        2'b00: current_hex = value[3:0]; // Digit 0
        2'b01: current_hex = value[7:4]; // Digit 1
        2'b10: current_hex = value[11:8]; // Digit 2
        2'b11: current_hex = value[15:12]; // Digit 3
    endcase
end

hex_to_7seg display_decoder (
    .hex(current_hex),
    .seg(segments)
);

// Digit enable signals (one active at a time)
assign digit_enable = ~(4'b0001 << digit_select);
```


Refresh rate calculation:

```
// For 4-digit display at 1kHz total refresh:  
// Each digit displayed for 250µs  
// Switching frequency: 4kHz
```

```
localparam REFRESH_DIVIDER = CLOCK_FREQ / 4000;
```

37.7.5 Common Issues and Debugging

Segment mapping errors: - **Bit order**: Ensure {g,f,e,d,c,b,a} ordering matches hardware - **Active level**: Verify common anode vs. common cathode - **Pin assignment**: Check physical connections match bit assignments

Display quality issues: - **Ghost segments**: Check current limiting and multiplexing timing - **Dim display**: Verify current levels and refresh rates - **Flicker**: Ensure adequate refresh rate (>30Hz visible, >60Hz preferred)

Character readability: - **Ambiguous characters**: 6 vs b, 8 vs B, 0 vs O - **Context**: Use surrounding digits/letters for disambiguation - **Alternative fonts**: Consider modified patterns for better distinction

37.8 Related Modules

- **BCD to 7-segment**: For decimal-only displays
- **ASCII to 7-segment**: Extended character set support
- **Display multiplexer**: Time-division multiplexing controller
- **PWM brightness controller**: Dynamic brightness adjustment

37.9 Testing

37.9.1 Exhaustive Testing

```
// Test all 16 hex values  
for (int i = 0; i < 16; i++) begin  
    hex_input = i[3:0];  
    #1;  
    expected_segments = reference_table[i];  
    assert(seg_output == expected_segments);  
end
```

37.9.2 Visual Verification

- **Physical testing**: Connect to actual 7-segment display

- **Simulation:** Use testbench with visual output
- **Reference comparison:** Compare with known-good implementations

37.9.3 Pattern Validation

```
// Verify no segments are permanently on/off  
logic [6:0] all_segments_used = '0;  
for (int i = 0; i < 16; i++) begin  
    all_segments_used |= ~reference_table[i]; // Accumulate active  
segments  
end  
assert(all_segments_used == 7'b1111111); // All segments used  
somewhere
```

37.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

38 icg

38.1 Overview

An Integrated Clock Gating (ICG) cell: a latch-based clock gate that lets you selectively kill clock signals to cut dynamic power consumption. It's one of the fundamental power management moves in modern ASIC and FPGA designs, and for good reason.

38.1.1 Module Declaration

```
module icg(  
    input logic en,  
    input logic clk,  
    output logic gclk  
);
```

38.2 Parameters

This module has no parameters.

38.3 Ports

Port	Direction	Width	Description
en	Input	1	Enable signal - when high, allows clock to pass through
clk	Input	1	Input clock signal to be gated
gclk	Output	1	Gated clock output - clk when en=1, low when en=0

38.4 Functional Description

38.4.1 Clock Gating Operation

The ICG does clock gating the safe way:

1. **Latches Enable Signal:** Uses a level-sensitive latch, transparent while `clk` is low, to capture the enable signal
2. **Prevents Glitches:** Ensures the enable signal is stable during clock high periods

3. **Generates Gated Clock:** ANDs the latched enable with the input clock

38.4.2 Implementation Details

```

logic en_out;

// Latch-based approach - enable is latched on clock low
always_latch begin
    if (!clk) begin
        en_out = en;
    end
end

// Generate gated clock
assign gclk = en_out && clk;

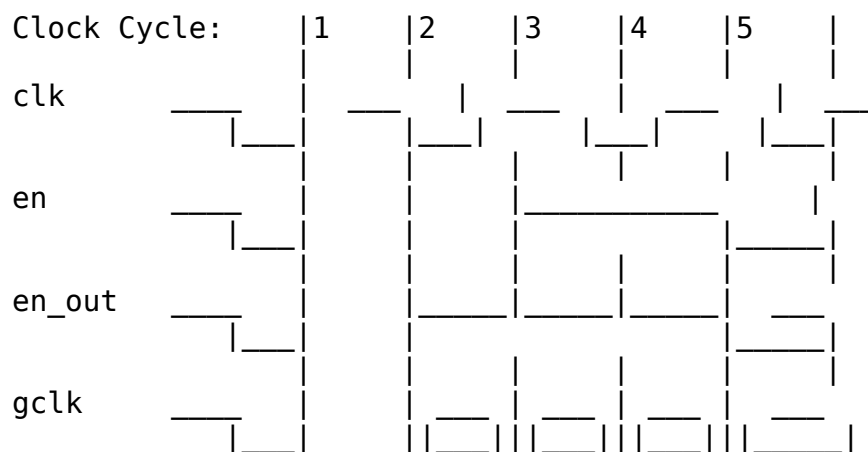
```

38.5 Timing Characteristics

38.5.1 Critical Timing Requirements

Characteristic	Description
Setup Time	en must be stable before clk rising edge
Hold Time	en must remain stable after clk rising edge
Propagation Delay	Delay from clk to gclk through AND gate
Enable Latching	en is latched when clk is low

38.5.2 Timing Diagram



38.6 Usage Examples

38.6.1 Basic Register Bank Gating

```
// Clock gating for a register bank
logic bank_enable;
logic gated_clk;

icg u_bank_icg (
    .en    (bank_enable),
    .clk   (sys_clk),
    .gclk  (gated_clk)
);

// Register bank using gated clock
always_ff @(posedge gated_clk or negedge rst_n) begin
    if (!rst_n) begin
        register_bank <= '0;
    end else begin
        register_bank <= next_register_values;
    end
end
```

38.6.2 CPU Pipeline Stage Gating

```
// Pipeline stage clock gating
logic pipe_stage_valid;
logic stage_clk;

icg u_pipe_icg (
    .en    (pipe_stage_valid),
    .clk   (cpu_clk),
    .gclk  (stage_clk)
);

// Pipeline stage registers
always_ff @(posedge stage_clk or negedge rst_n) begin
    if (!rst_n) begin
        pipe_stage_regs <= '0;
    end else begin
        pipe_stage_regs <= pipe_stage_inputs;
    end
end
```

38.6.3 Memory Interface Gating

```
// Memory controller clock gating
logic mem_access_active;
logic mem_clk;
```

```
icg u_mem_icg (
    .en    (mem_access_active),
    .clk   (system_clk),
    .gclk  (mem_clk)
);
```

```
assign mem_access_active = mem_read_req || mem_write_req || mem_busy;
```

38.7 Design Notes

38.7.1 Power Optimization Benefits

The ICG cuts dynamic power on three fronts:

- **Clock Tree Power:** Eliminates switching activity in downstream clock networks
- **Register Power:** Prevents unnecessary flip-flop transitions when data is not changing
- **Combinational Power:** Reduces spurious switching in logic fed by gated registers

Clock gating removes switching on the gated clock branch while a block is idle. The actual saving is **design- and technology-specific** — it depends on the activity factor, gating ratio, process node, and clock-tree structure — so no fixed percentage applies here. Quantify it from synthesis/power analysis of your own design rather than a generic figure. (Earlier revisions of this page quoted 20-40% / 30-60% / 10-30% numbers that had no source or stated conditions.)

38.7.2 Safety Requirements

- **Glitch-Free Operation:** Latch-based design prevents glitches on gated clock
- **Setup/Hold Timing:** Enable signal must meet timing requirements relative to clock
- **Clock Domain:** All logic using gclk must be in the same clock domain

38.7.3 Enable Signal Guidelines

```
// Good: Enable changes during clock low period
always_ff @(posedge clk) begin
    if (rst_n) begin
        enable_reg <= new_enable_value; // Changes on clock edge
    end
end
```

end

```
// Caution: Ensure enable is stable during clock high  
assign icg_enable = enable_reg && additional_condition;
```

38.7.4 Implementation Alternatives

Many ASIC libraries provide optimized ICG cells:

```
// Using library ICG cell (example)  
CLOCK_GATE_ICG u_lib_icg (  
    .CLK    (clk),  
    .EN     (en),  
    .GCLK   (gclk)  
);
```

For FPGA targets, use dedicated clock control primitives:

```
// Xilinx BUFGCE example  
BUFGCE u_bufgce (  
    .I    (clk),  
    .CE   (en),  
    .O    (gclk)  
);
```

38.7.5 Synthesis Guidelines

Technology mapping: - **ASIC**: Use library-specific ICG cells for optimal area/power/timing - **FPGA**: Map to dedicated clock control primitives (BUFGCE, etc.) - **Timing Closure**: Account for additional setup/hold requirements

Clock tree synthesis:

```
# Example synthesis constraints for ICG  
set_case_analysis 0 [get_pins u_icg/en] ;# For power analysis  
set_clock_gating_check -setup 0.1 [get_pins u_icg/gclk]  
set_clock_gating_check -hold 0.1 [get_pins u_icg/gclk]
```

38.7.6 Design Integration

Where you put the ICG in the clock tree matters:

```
Main Clock → ICG → Local Clock Tree → Registers  
              ↑  
          Enable Logic
```

Power domain considerations: - **Retention**: Ensure enable logic remains powered during clock gating - **Isolation**: Properly isolate gated domains in multi-voltage designs - **Power Sequencing**: Coordinate with power management units

The `icg` module is a fundamental building block of power-efficient digital design: significant power savings through controlled clock gating, without giving up design safety or functionality.

38.8 Related Modules

- `clock_gate_ctrl`: Enhanced clock gating controller with additional features
- `clock_divider`: Clock frequency division functionality
- `reset_sync`: Reset synchronization for gated clock domains

38.9 Testing

38.9.1 Functional Tests

- **Enable/Disable**: Verify correct gating behavior for enable transitions
- **Clock Integrity**: Ensure gated clock maintains proper duty cycle when enabled
- **Reset Behavior**: Verify proper operation during reset sequences

38.9.2 Timing Tests

```
// Timing verification example
initial begin
    // Test enable setup/hold timing
    @(negedge clk);
    en = 1'b1; // Enable during clock low
    @(posedge clk);
    // Verify gclk follows clk

    @(negedge clk);
    en = 1'b0; // Disable during clock low
    @(posedge clk);
    // Verify gclk remains low
end
```

38.9.3 Power Analysis

- **Activity Monitoring**: Track switching activity reduction in gated domains
- **Power Estimation**: Measure power savings using EDA power analysis tools
- **Efficiency Metrics**: Calculate gating efficiency (% time clock is disabled)

38.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

39 leading_one_trailing_one

39.1 Overview

Finds both ends of your set bits at once: the position of the most significant bit (leading one) and the least significant bit (trailing one) in an input data vector. You get index outputs, one-hot vector representations of both positions, and status flags for the edge cases.

- Finds the highest set bit position (leading one)
- Finds the lowest set bit position (trailing one)
- Provides one-hot vector outputs for both positions
- Includes validity and edge case detection
- Parameterizable width with automatic index sizing

39.2 Parameters

Parameter	Default	Description
WIDTH	8	Width of input data vector — the module's only parameter

39.3 Ports

Port	Direction	Width	Description
data	Input	WIDTH	Input data vector to analyze
leadingone	Output	$\$clog2(WIDTH)$	Index of the most significant set bit
leadingone_vector	Output	WIDTH	One-hot vector with leading one position marked
trailingone	Output	$\$clog2(WIDTH)$	Index of the least significant set bit
trailingone_vector	Output	WIDTH	One-hot vector with trailing one position marked
all_zeros	Output	1	High when input data is

Port	Direction	Width	Description
all_ones	Output	1	all zeros High when input data is all ones
valid	Output	1	High when input data contains at least one set bit

39.4 Functional Description

39.4.1 Submodules

Internally it's two specialized finder modules working opposite ends:

1. **find_last_set**: Locates the position of the most significant set bit
 - Connected to leadingone output
 - Implements priority encoding from MSB to LSB
2. **find_first_set**: Locates the position of the least significant set bit
 - Connected to trailingone output
 - Implements priority encoding from LSB to MSB

39.4.2 One-Hot Vector Generation

```

always_comb begin
    leadingone_vector = '0;
    trailingone_vector = '0;

    // Only set vector bits if there is valid data
    if (!data) begin
        if (int'(leadingone) < WIDTH) begin
            leadingone_vector[leadingone] = 1'b1;
        end

        if (int'(trailingone) < WIDTH) begin
            trailingone_vector[trailingone] = 1'b1;
        end
    end
end

```

39.4.3 Status Flag Logic

- **all_ones:** Uses reduction AND (&data) to detect when all bits are set
- **all_zeros:** Uses reduction OR negation (~(|data)) to detect when no bits are set
- **valid:** Uses reduction OR (|data) to indicate presence of at least one set bit

39.4.4 Special Implementation Notes

1. **Index Width Calculation:** Uses `$clog2(WIDTH)` to automatically size index outputs for any input width
2. **Bounds Checking:** Includes explicit bounds checking (`int'(leadingone) < WIDTH`) to prevent out-of-range indexing
3. **Zero Input Handling:** When input is all zeros:
 - One-hot vectors remain all zeros
 - Index values are deterministically **0** —
`find_first_set/find_last_set` initialize `index = '0` and never reassign it when no bit is set (the RTL header documents “0 if all zeros”). They are not X/undefined.
 - `valid` flag indicates invalid state (use it to qualify the indices)
4. **Type Casting:** Uses `int'()` casting for index comparisons to ensure proper integer arithmetic

39.5 Timing Characteristics

This module is **purely combinational** – it contains no `always_ff` and no latch, so it holds no state and adds no clock cycles. Its outputs settle a propagation delay after its inputs, and it introduces no latency into a pipeline that instantiates it.

Timing closure is therefore a question of the surrounding logic’s slack, not of this module’s cycle count. No synthesis figures are quoted; none have been measured.

39.6 Usage Examples

39.6.1 8-bit Example

```
Input:  data = 8'b00101000
Output: leadingone = 3'd5
        leadingone_vector = 8'b00100000
        trailingone = 3'd3
        trailingone_vector = 8'b00001000
        all_zeroes = 1'b0
        all_ones = 1'b0
        valid = 1'b1
```

39.6.2 Edge Cases

```
Input:  data = 8'b00000000
Output: leadingone = 3'd0    (deterministic, not X; qualify with valid)
        leadingone_vector = 8'b00000000
        trailingone = 3'd0    (deterministic, not X; qualify with
valid)
        trailingone_vector = 8'b00000000
        all_zeroes = 1'b1
        all_ones = 1'b0
        valid = 1'b0
```

```
Input:  data = 8'b11111111
Output: leadingone = 3'd7
        leadingone_vector = 8'b10000000
        trailingone = 3'd0
        trailingone_vector = 8'b00000001
        all_zeroes = 1'b0
        all_ones = 1'b1
        valid = 1'b1
```

39.7 Design Notes

39.7.1 Applications

- Priority arbitration systems
- Bit manipulation operations
- Data compression algorithms
- Network packet processing
- Cache management systems

39.8 Related Modules

- **find_last_set**: Locates the most significant set bit; drives the leadingone output
- **find_first_set**: Locates the least significant set bit; drives the trailingone output

39.9 Testing

val/common/test_leading_one_trailing_one.py exercises this module. It collects 2 parameter cases at the default REG_LEVEL.

```
source env_python
pytest val/common/test_leading_one_trailing_one.py -v
```

39.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

40 pwm

40.1 Overview

Generates multiple independent pulse width modulation signals with configurable duty cycles, periods, and repeat counts. Each channel runs as its own state machine with precise timing control and completion detection.

- Multiple independent PWM channels (parameterizable)
- Individual duty cycle, period, and repeat count control per channel
- Edge-triggered start control with automatic edge detection
- Completion status reporting per channel
- Support for continuous operation (infinite repeat)
- Precise timing with configurable counter width

40.2 Parameters

Parameter	Default	Description
WIDTH	8	Bit width of counters and timing parameters
CHANNELS	4	Number of independent PWM channels

40.3 Ports

Port	Direction	Width	Description
clk	Input	1	System clock
rst_n	Input	1	Active-low asynchronous reset
sync_rst_n	Input	1	Active-low synchronous reset (functional — all state/counter/edge-detect blocks reset on ! sync_rst_n). Must be

Port	Direction	Width	Description
			driven; leaving it unconnected gives X in sim / spurious resets in hardware.
start	Input	CHANNELS	Start trigger for each channel (edge-triggered)
duty	Input	CHANNELS* WIDTH	Concatenated duty cycle values for all channels
period	Input	CHANNELS* WIDTH	Concatenated period values for all channels
repeat_count	Input	CHANNELS* WIDTH	Concatenated repeat counts (0 = infinite)
done	Output	CHANNELS	Completion status for each channel
pwm_out	Output	CHANNELS	PWM output signals for each channel

40.4 Functional Description

40.4.1 State Machine

Each PWM channel runs a 3-state finite state machine:

```
typedef enum logic [1:0] {
    IDLE = 2'b00,      // Waiting for start trigger
    RUNNING = 2'b01,   // Actively generating PWM
    DONE = 2'b10       // Completed all repeats
} state_t;
```

40.4.2 State Transitions

IDLE → RUNNING: - **Condition:** w_start_edge && local_period > 0 - **Action:** Reset counters and begin PWM generation

RUNNING → DONE: - **Condition:** w_period_complete && w_all_repeats_done - **Action:** Stop PWM generation and assert done flag

DONE → RUNNING: - **Condition:** w_start_edge - **Action:** Restart PWM generation with reset counters

Any State → IDLE: - **Condition:** System reset - **Action:** Initialize all registers

40.4.3 Per-Channel Parameter Extraction

```
localparam int EndIdx = (i + 1) * WIDTH - 1;
assign local_duty = duty[EndIdx-WIDTH];
assign local_period = period[EndIdx-WIDTH];
assign local_repeat = repeat_count[EndIdx-WIDTH];
```

40.4.4 Edge Detection for Start Signal

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) r_start_prev <= 1'b0;
    else r_start_prev <= start[i];
end
assign w_start_edge = start[i] && !r_start_prev;
```

40.4.5 PWM Output Generation Logic

```
always_comb begin
    case (r_state)
        RUNNING: begin
            if (local_duty == 0) begin
                // 0% duty cycle
                pwm_out[i] = 1'b0;
            end else if (local_duty >= local_period) begin
                // 100% duty cycle (duty >= period)
                pwm_out[i] = 1'b1;
            end else begin
                // Normal case: PWM high for first 'duty' cycles
                pwm_out[i] = (r_count < local_duty);
            end
        end
        default: begin
            pwm_out[i] = 1'b0;
        end
    endcase
end
```

40.4.6 Counter Management

Main counter (period timing): - Increments each clock cycle during RUNNING state - Resets to 0 when reaching local_period - 1 - Controls PWM frequency

Repeat counter: - Increments each time a period completes - Compared against repeat_count to determine completion - Special case: repeat_count == 0 means infinite operation

40.4.7 Critical Control Signals

Period completion detection:

```
assign w_period_complete = (r_count == local_period - 1) && (r_state == RUNNING);
```

All repeats done detection:

```
assign w_all_repeats_done = (local_repeat == 0) ? 1'b0 : // 0 means  
infinite  
                                (r_repeat_value >= local_repeat - 1'b1);
```

The `- 1'b1` is load-bearing. The check fires in the same cycle that `r_repeat_value` increments at a period boundary, so a bare `>= local_repeat` emits `local_repeat + 1` periods — one too many. With `local_repeat = 1`: the corrected form is done at the first boundary (`0 >= 0`), while the bare form needs a second (`0 >= 1` false, then `1 >= 1`). The `local_repeat == 0` branch also keeps the `- 1` from underflowing, since 0 means infinite.

40.4.8 Special Implementation Notes

1. Edge-Triggered Start Control

- Prevents continuous restart while start signal is held high
- Enables precise control of PWM initiation
- Each channel has independent edge detection

2. Duty Cycle Edge Cases

- **0% Duty:** Output permanently low during RUNNING state
- **100% Duty:** Output permanently high when `duty >= period`
- **Normal Operation:** High for first duty clock cycles of each period

3. Infinite Repeat Mode

- `repeat_count = 0` enables continuous operation
- PWM runs indefinitely. A start pulse while RUNNING is **ignored** — the FSM leaves RUNNING only on `w_period_complete` && `w_all_repeats_done`, which never fires in continuous mode — so a running channel cannot be restarted or stopped by start. Use reset, or a non-zero `repeat_count`, to end it.
- Useful for continuous motor control or LED dimming

4. Simultaneous Multi-Channel Operation

- All channels operate independently
- No inter-channel dependencies or conflicts

- Scalable to any number of channels via parameters

5. Reset Behavior

- Two resets: `rst_n` (asynchronous) and `sync_rst_n` (synchronous). Either one returns the channel FSM to IDLE and clears counters.
- Asynchronous reset immediately stops all PWM outputs
- All internal counters and state machines reset to IDLE
- Clean startup guaranteed after reset deassertion

40.5 Timing Characteristics

40.5.1 Single Period Example (WIDTH=8, duty=3, period=8)

```
Clock:  0 1 2 3 4 5 6 7 0 1 2 3 4 5 6 7 ...
Count:  0 1 2 3 4 5 6 7 0 1 2 3 4 5 6 7 ...
PWM:    1 1 1 0 0 0 0 0 1 1 1 0 0 0 0 0 ...
```

40.5.2 Multi-Repeat Example (repeat_count=2)

```
Period: 1      2      DONE
PWM:    _____
```

40.6 Usage Examples

40.6.1 Motor Speed Control

```
// 50% duty cycle, 1kHz frequency (assuming 1MHz clock)
// WIDTH >= 10 needed for these values (default 8 truncates silently)
duty = 500;
period = 1000;
repeat_count = 0; // Continuous operation
```

40.6.2 LED Dimming Sequence

```
// Fade effect with 10 brightness levels
duty = brightness_level * 10;
period = 100;
repeat_count = fade_duration;
```

40.6.3 Servo Motor Control

```
// 20ms period with 1-2ms pulse width
duty = servo_position; // 1000-2000 for 1-2ms
period = 20000; // needs WIDTH >= 15 -- at the default 8, 20000
```

```
truncates to 32           // 20ms at 1MHz clock
repeat_count = 1;         // Single pulse
```

40.7 Design Notes

40.7.1 Applications

- Motor speed control
- LED brightness control
- Servo motor positioning
- Audio signal generation
- Power supply switching
- Heating element control

40.8 Related Modules

Read out of the RTL, not curated: these are the modules this one instantiates and the modules that instantiate it.

Instantiated by: - arbiter_rr_pwm_monbus - arbiter_wrr_pwm_monbus

40.9 Testing

val/common/test_pwm.py exercises this module. It collects 2 parameter cases at the default REG_LEVEL.

```
source env_python
pytest val/common/test_pwm.py -v
```

40.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

41 rtl-common Quick Start Guide

Version: 1.0 Last Updated: 2025-09-30 Status: Stable, Production-Ready

41.1 Overview

The Common RTL Library is **48 reusable building blocks** for FPGA and ASIC designs. Every module is technology-agnostic, fully parameterizable, and verified with comprehensive CocoTB test suites. This guide is the fast path in.

Quick Stats: - **48 modules** across 10 categories (arithmetic split out to `rtl/math/`, clock-crossing to `rtl/cdc/`) - **Technology agnostic** (FPGA/ASIC portable) - **Well documented** (inline + external docs) - **Every module has a CocoTB test** under `val/common/`

41.2 Module Categories

Counts live in one place — the [count table in index.md](#). Here's the browse-oriented view of what's in each category:

Category	Examples
Clock/Reset	<code>clock_divider</code> , <code>clock_gate_ctrl</code> , <code>clock_pulse</code>
Counters	<code>counter</code> , <code>counter_bin</code> , <code>counter_bin_load</code> , <code>counter_freq_invariant</code> , <code>counter_load_clear</code> , <code>counter_ring</code>
Data Integrity	<code>dataint_crc</code> , <code>dataint_ecc_hamming</code> , <code>dataint_parity</code> , <code>dataint_checksum</code>
Conversion & Encoding	<code>bin_to_bcd</code> , <code>decoder</code> , <code>encoder</code> , <code>encoder_priority_enable</code> , <code>hex_to_7seg</code>
Bit Ops & Search	<code>count_leading_zeros</code> , <code>find_first_set</code> , <code>find_last_set</code> , <code>leading_one_trailing_one</code>
Shifters & LFSRs	<code>shifter_barrel</code> , <code>shifter_lfsr</code> , <code>shifter_lfsr_fibonacci</code> , <code>shifter_lfsr_galois</code>
Arbiters	<code>arbiter_round_robin</code> , <code>arbiter_round_robin_weighted</code> ,

Category	Examples
FIFOs	arbiter_priority_encoder fifo_sync, fifo_control (fifo_async moved to rtl/cdc/)
CAM	cam_tag
Miscellaneous	pwm, sort

Arithmetic (math_*) split out into rtl/math/; its docs are the [rtl-math](#) book.

Full per-module docs: [docs/markdown/rtl-common/](#)

41.3 Quick Start

41.3.1 1. Browse Available Modules

List all modules

```
ls rtl/common/*.sv
```

List by category

```
ls rtl/common/counter*.sv
```

```
ls rtl/common/arbiter*.sv
```

```
ls rtl/common/dataint*.sv
```

Search for functionality

```
find rtl/common/ -name "*.sv" | xargs grep "fifo\|counter\|arbiter"
```

41.3.22. Check Module Interface

View module header and parameters

```
head -50 rtl/common/counter_bin.sv
```

Extract module definition

```
grep "^module\|parameter\|input\|output" rtl/common/counter_bin.sv
```

41.3.33. Review Test Examples

Check if test exists

```
ls val/common/test_counter_bin.py
```

View test for usage examples

```
cat val/common/test_counter_bin.py
```

41.3.44. Integrate into Your Design

// Example: Instantiate binary counter

```
counter_bin #(
    .WIDTH(11),           // 10 count bits + the wrap flag in the MSB
    .MAX  (1000)
) u_my_counter (
    .clk           (my_clk),
    .rst_n         (my_rst_n),
    .enable        (count_enable),
    .counter_bin_curr (count_value),
    .counter_bin_next (count_value_next)
);
```

41.4 Common Use Cases

41.4.1 Use Case 1: Simple Event Counter

Need: Count events, wrapping at a maximum

Module: counter_bin.sv — a FIFO-pointer counter. The low WIDTH-1 bits count 0..MAX-1; on wrap they clear and the MSB toggles, so a read/write pointer pair can tell full from empty. There is no overflow output.

```
counter_bin #(
    .WIDTH(9),           // 8 count bits + the wrap flag
    .MAX  (255)
) u_event_counter (
    .clk           (clk),
    .rst_n         (rst_n),
    .enable        (event_detected),
    .counter_bin_curr (event_count),
    .counter_bin_next (event_count_next)
);
```

41.4.2 Use Case 2: Timeout Timer

Need: A time base that stays correct when the clock frequency changes

Module: counter_freq_invariant.sv — a microsecond tick generator, not a timeout timer. It has no timeout parameter and no timeout output; build the timeout yourself by counting tick.

```
counter_freq_invariant #(
    .COUNTER_WIDTH  (16),
    .MIN_FREQ_MHZ   (5),
    .MAX_FREQ_MHZ   (220),
```

```

        .NUM_FREQ_ENTRIES(16)
    ) u_us_tick (
        .clk            (clk),
        .rst_n          (rst_n),
        .sync_reset_n   (sync_clear_n),
        .freq_sel       (freq_sel),      // Picks the prescaler for the
actual clock
        .o_counter      (microseconds),
        .tick           (us_tick)       // One pulse per microsecond
    );

// Timeout = count us_tick pulses and compare against your own
threshold.

```

41.4.3 Use Case 3: Multi-Master Arbitration

Need: Fair arbitration between 4 bus masters

Module: arbiter_round_robin.sv

```

arbiter_round_robin #(
    .CLIENTS      (4),    // 4 requesters -- N is the DERIVED
$clog2(CLIENTS)
    .WAIT_GNT_ACK(0)     // 1 = hold the grant until grant_ack
) u_bus_arbiter (
    .clk          (clk),
    .rst_n        (rst_n),
    .block_arb    (1'b0),
    .request       (master_requests[3:0]), // One bit per master
    .grant_ack     ('0),
    .grant_valid   (grant_valid),
    .grant         (master_grants[3:0]),   // One-hot grant
    .grant_id      (master_grant_idx),
    .last_grant    (last_grant)
);

```

41.4.4 Use Case 4: CRC-32 (Ethernet)

Need: Calculate Ethernet CRC-32 for packet data

Module: dataint_crc.sv

```

dataint_crc #(
    .DATA_WIDTH(32),
    .CRC_WIDTH (32),
    .REFIN     (1),

```



```

        .REFOUT      (1)
    ) u_crc32 (
        .POLY          (32'h04C11DB7), // CRC-32 Ethernet -- a PORT,
not a param
        .POLY_INIT      (32'hFFFFFFFF),
        .XOROUT          (32'hFFFFFFFF),
        .clk             (clk),
        .rst_n           (rst_n),
        .load_crc_start  (crc_restart),
        .load_from_cascade(data_valid),
        .cascade_sel     (4'b1000), // One-hot over DATA_WIDTH/8
bytes
        .data            (packet_data),
        .crc              (crc_result)
    );

// The polynomial, init and xorout are WIRED, so the CRC can be
// retuned at run
// time. 250 configurations are exercised by the test suite.

```

41.4.5 Use Case 5: Memory ECC (Single Error Correction)

Need: Protect memory with error correction

Modules: dataint_ecc_hamming_encode_secdec.sv +
dataint_ecc_hamming_decode_secdec.sv

```

// Encoder (write path) -- purely combinational
dataint_ecc_hamming_encode_secdec #(
    .WIDTH(64) // parameter is WIDTH, not DATA_WIDTH
) u_ecc_encoder (
    .data          (mem_write_data),
    .encoded_data  (mem_data_with_ecc) // one codeword: data + parity
+ SECDED bit
);

// Decoder (read path) -- CLOCKED, unlike the encoder
dataint_ecc_hamming_decode_secdec #(
    .WIDTH(64)
) u_ecc_decoder (
    .clk           (clk),
    .rst_n         (rst_n),
    .enable        (1'b1),
    .hamming_data  (mem_read_data_with_ecc),
    .data          (corrected_data),
    .error_detected (single_bit_error), // Correctable

```

```

        .double_error_detected (double_bit_error)    // Detectable only
    );

```

41.4.6 Use Case 6: Clock Domain Crossing (CDC)

Need: Safely cross a signal from one clock domain to another

Module: `cdc_synchronizer.sv` (for data) or `sync_pulse.sv` – both in `rtl/cdc/` (for pulses).
 There is no `sync_2ff` module — earlier revisions of this guide named one, but it has never existed in the tree.

```

// Option 1: Synchronize multi-bit data (rtl/cdc/cdc_synchronizer.sv)
// NOTE: a plain multi-flop synchronizer is only safe for signals
whose bits
// can be sampled independently -- a single-bit flag, or a quasi-
static value
// held stable across the crossing. For a multi-bit value that changes
as a
// unit, use Gray coding (counter_bingray) or a handshake/async FIFO
instead.
cdc_synchronizer #(
    .WIDTH(8),
    .FLOP_COUNT(3)
) u_data_sync (
    .clk          (dst_clk),
    .rst_n        (dst_rst_n),
    .async_in     (src_data),    // From source clock domain
    .sync_out     (dst_data)     // In destination clock domain
);

// Option 2: Synchronize single pulse
sync_pulse u_pulse_sync (
    .i_src_clk    (src_clk),
    .i_src_rst_n  (src_rst_n),
    .i_pulse      (src_pulse),   // Single-cycle pulse
    .i_dst_clk    (dst_clk),
    .i_dst_rst_n  (dst_rst_n),
    .o_pulse      (dst_pulse)    // Single-cycle pulse in dst domain
);

```

41.5 Module Selection Guide

41.5.1 “I need a counter...”

Requirement	Module	Notes
Simple up counter	counter_load_clear.sv	Plain counting to a value (counter_bin is a FIFO-POINTER counter – its MSB is a wrap flag, not a count bit)
With load/clear	counter_load_clear.sv	Explicit control
Time-based timeout	counter_freq_invariant.sv	Frequency-independent
Gray code	counter_bingray.sv (in rtl/cdc/)	For CDC
Ring counter	counter_ring.sv	Circular/sequential
Johnson counter	counter_johnson.sv (in rtl/cdc/)	2N states with N FFs

41.5.2 “I need an arbiter...”

Requirement	Module	Notes
Fair round-robin	arbiter_round_robin.sv	Most versatile
Weighted QoS	arbiter_round_robin_weighted.sv	Assign priorities
Fixed priority	arbiter_priority_encoder.sv	Lowest index wins
Minimal logic	arbiter_round_robin_simple.sv	Smallest area

41.5.3 “I need CRC...”

Standard	Module	Configuration
Any CRC standard	dataint_crc.sv	Drive the POLY input port (a PORT, not a parameter)
CRC-32 (Ethernet)	dataint_crc.sv	POLY port = 32'h04C11DB7
CRC-16-CCITT	dataint_crc.sv	POLY port = 16'h1021

Standard	Module	Configuration
CRC-8	<code>dataint_crc.sv</code>	POLY port = 8'h07
Custom	<code>dataint_crc_xor_shift.sv</code>	Build custom

The validated CRC configurations (250 standards) are the `crc_parameters` table in `bin/TBClasses/common/crc_testing.py`, which drives `val/common/test_dataint_crc.py`.

41.5.4 "I need a FIFO..."

For production designs: → Use `rtl/amba/gaxi/gaxi_fifo_sync.sv` (solid, well-tested)

For learning/simple cases: → Check `rtl/common/` for basic FIFO examples

41.6 Integration Checklist

When you integrate one of these modules into your design:

- ☐ **Search first** - Verify no better alternative exists
- ☐ **Read module header** - Understand parameters and constraints
- ☐ **Check parameter ranges** - Ensure your values are valid
- ☐ **Review test** - See `val/common/test_{module}.py` for examples
- ☐ **Match reset convention** - the port is `rst_n`, active-low async (`clock_gate_ctrl` uses `aresetn`); no `rtl/common` module exposes `i_rst_n`
- ☐ **Verify port widths** - Match parameter-dependent widths
- ☐ **Lint your design** - Run `verilator --lint-only` on top level
- ☐ **Test integration** - Create simple testbench
- ☐ **Check timing** - Verify no timing violations

41.7 Testing Your Integration

41.7.1 Option 1: Quick Functional Test

Run existing module test to verify baseline
`pytest val/common/test_{module}.py -v`

```
# Example:
pytest val/common/test_counter_bin.py -v
```

41.7.2 Option 2: Create Your Own Test

```
# val/your_subsystem/test_your_design.py
import cocotb
from cocotb.clock import Clock
from cocotb.triggers import RisingEdge

@cocotb.test()
async def test_your_design(dut):
    """Test your design that uses common modules"""

    # Start clock
    clock = Clock(dut.clk, 10, units="ns")
    cocotb.start_soon(clock.start())

    # Reset
    dut.rst_n.value = 0
    await RisingEdge(dut.clk)
    await RisingEdge(dut.clk)
    dut.rst_n.value = 1

    # Your test logic here
    # ...
```

41.7.3 Option 3: Waveform Debug

```
# Run test with VCD waveform dump
pytest val/common/test_counter_bin.py -v --vcd=waves.vcd

# View waveforms
gtkwave waves.vcd

# Or use saved waveform config
gtkwave val/common/GTKW/counter_bin.gtkw
```

41.8 Common Pitfalls and How to Avoid Them

These four show up in design reviews again and again. All of them are cheap to avoid once you know what to look for.

41.8.1 Pitfall 1: Wrong Reset Polarity

Wrong:

```
counter_bin u_counter (  
    .rst_n(my_positive_reset), // ERROR: Inverted!  
    // ...  
);
```

Right:

```
counter_bin u_counter (  
    .rst_n(~my_positive_reset), // Invert if you have positive reset  
    // OR better: use active-low reset throughout design  
    .rst_n(my_rst_n),  
    // ...  
);
```

41.8.2 Pitfall 2: Parameter Mismatch

Wrong:

```
counter_bin #(  
    .WIDTH(16)  
    // Forgot MAX - uses the default of 10  
) u_counter (  
    .counter_bin_curr(count[7:0]) // ERROR: Width mismatch!  
);
```

Right:

```
counter_bin #(  
    .WIDTH(8), // Match output width  
    .MAX (200)  
) u_counter (  
    .counter_bin_curr(count[7:0]) // Correct width  
);
```

41.8.3 Pitfall 3: Clock Domain Crossing Without Sync

Wrong:

```
// Signal crosses from clk_a to clk_b domain  
always_ff @(posedge clk_b)  
    r_data <= signal_from_clk_a; // ERROR: Metastability!
```

Right:

```
// Use proper synchronizer (rtl/cdc/cdc_synchronizer.sv)  
cdc_synchronizer #(.WIDTH(8), .FLOP_COUNT(3)) u_sync (  
    .clk      (clk_b),  
    .rst_n    (rst_n_b),
```

```

        .async_in (signal_from_clk_a),
        .sync_out (signal_in_clk_b)
    );

```

41.8.4 Pitfall 4: Creating New Module Without Searching

Wrong:

```

// "I need a counter, let me write one..."
module my_new_counter (...);
    // Reinventing the wheel
endmodule

```

Right:

```

# "I need a counter, does one exist?"
ls rtl/common/counter*.sv
# → Found counter_bin.sv, counter_load_clear.sv, ...
# → Use existing module!

```

41.9 Performance Considerations

41.9.1 Area Optimization

Small area: - Use arbiter_round_robin_simple.sv instead of full arbiter_round_robin.sv
 - Minimize WIDTH parameters - Use FFs instead of RAMs for small storage (<16 entries)

Large area (more features): - Use full-featured modules - Use RAM-based FIFOs for large buffers

41.9.2 Timing Optimization

Critical path reduction: - Enable output pipelining where available - Break long combinational chains - Use registered versions of modules

Example:

```

arbiter_round_robin #(
    .CLIENTS(16)           // N is derived ($clog2(CLIENTS)); do not
    override it
) u_arbiter (
    // ...
);

```

41.9.3 Power Optimization

Clock gating: - Use clock_gate_ctrl.sv to gate inactive blocks - Gate counter enables when not counting

Example:

```

clock_gate_ctrl #(
    .IDLE_CNTR_WIDTH(4)
) u_gate (
    .clk_in          (clk),
    .aresetn         (aresetn),           // NOTE: this module uses
aresetn
    .cfg_cg_enable   (cg_enable),
    .cfg_cg_idle_count(4'd8),             // Cycles idle before gating
    .wakeup          (block_active),
    .clk_out         (clk_gated),
    .gating          (clk_is_gated)
);

// Use clk_gated for power-sensitive logic

```

41.10 Getting Help

41.10.1 Documentation

- **This file** - Quick start and common use cases
- docs/markdown/rtl-common/index.md - Detailed specifications and module catalog
- rtl/common/CLAUDE.md - AI assistance guide for this subsystem
- /CLAUDE.md - Repository-wide guidance
- /PRD.md - Master project requirements

41.10.2 Examples

- **Tests:** val/common/test_*.py - Working examples for every module
- **Usage:** Search rtl/amba/ and projects/components/ for integration examples

41.10.3 Commands

```

# Find usage examples of a module
grep -r "counter_bin" rtl/amba/ projects/components/

# See how tests use the module
cat val/common/test_counter_bin.py

```



```
# Check module parameters
grep "parameter" rtl/common/counter_bin.sv
```

41.10.4 For Claude Code Users

See `rtl/common/CLAUDE.md` for AI-specific guidance including: - Module search strategies - Common integration patterns - Debugging tips - Anti-patterns to avoid

41.11 Quick Command Reference

```
# List all common modules
ls rtl/common/*.sv
```

```
# Search for specific functionality
find rtl/common/ -name "*.sv" | xargs grep -i "keyword"
```

```
# Run test for a module
pytest val/common/test_{module}.py -v
```

```
# Run all common tests
pytest val/common/ -v
```

```
# Lint a module
verilator --lint-only rtl/common/{module}.sv
```

```
# View waveforms
gtkwave waves.vcd
```

```
# Count lines of code
wc -l rtl/common/*.sv
```

```
# Find instantiations of a module
grep -r "module_name" rtl/
```

41.12 What's Next?

41.12.1 Explore Other Subsystems

- **rtl/amba/** - AMBA protocol infrastructure (AXI, APB, AXIS monitors)
- **projects/components/dma-ip/rapids/** - Rapid AXI Programmable In-band Descriptor System (example accelerator)

- **bin/TBClasses/** - Verification infrastructure

41.12.2 Create Your Own Modules

1. Follow the patterns in the existing modules
2. Use consistent naming (`category_function.sv`)
3. Add comprehensive header comments
4. Create a test in `val/common/`
5. Update this quickstart and the count table in `index.md` if you add a category

41.12.3 Contribute

- Report issues
 - Suggest improvements
 - Share integration examples
 - Document lessons learned
-

Questions? Check `docs/markdown/rtl-common/index.md` for detailed specifications or `rtl/common/CLAUDE.md` for AI assistance guidance.

Version: 1.0 **Last Updated:** 2025-09-30 **Maintained By:** RTL Design Sherpa Project

42 reset_sync

42.1 Overview

The `reset_sync` module is a parameterized reset synchronizer with asynchronous assertion and synchronous deassertion. That combination is the whole point: reset needs to hit immediately when it fires, but it has to come *off* on a clock edge. Release it asynchronously and every flop downstream gets a setup/hold gamble. This is the standard circuit for proper reset distribution, and it exists to keep reset release from going metastable or violating timing.

- Asynchronous reset assertion (immediate)
- Synchronous reset deassertion (clocked)
- Parameterizable synchronization depth
- Metastability resolution
- Standard reset synchronizer topology
- Clean reset release timing

42.2 Parameters

`reset_sync` parameters

Parameter	Type	Default	Description
N	int	3	Number of synchronization stages. Must be ≥ 2 ; an elaboration <code>\$error</code> rejects anything smaller.
KEEP_ATTRS	bit	1'b1	Attach vendor CDC attributes (<code>ASYNC_REG</code> , <code>SHREG_EXTRACT</code> , <code>altera_attribute</code> , <code>syn_preserve</code>) to the chain so tools recognize it and do not extract it into an SRL.
IN_ACTIVE_LOW	bit	1'b1	Polarity of the <code>rst_n</code> input. 1 = active-low

Parameter	Type	Default	Description
OUT_ACTIVE_LOW	bit	1'b1	(default), 0 = active-high. Polarity of the sync_rst_n output. 1 = active-low (default), 0 = active-high.
ASYNC_ASSERT	bit	1'b1	1 = asynchronous assert with synchronous deassert (FPGA best practice). 0 = fully synchronous style, where assertion is also clocked.

Polarity without renaming ports. IN_ACTIVE_LOW and OUT_ACTIVE_LOW let the module sit in an active-high reset domain while the ports stay named rst_n and sync_rst_n. The input gets normalized to an internal active-high form on the way in, and the requested polarity gets applied on the way out:

```
wire rst_in_h = IN_ACTIVE_LOW ? ~rst_n : rst_n;
...
sync_rst_n = OUT_ACTIVE_LOW ? ~sync_rst_h : sync_rst_h;
```

The names stick around for compatibility, which means a reset_sync with OUT_ACTIVE_LOW = 0 drives an active-**high** reset out of a port still called sync_rst_n. Check the parameter, not the name.

42.3 Ports

42.3.1 Inputs

Port	Width	Description
clk	1	System clock for synchronization
rst_n	1	Asynchronous input reset (active-low)

42.3.2 Outputs

Port	Width	Description
sync_rst_n	1	Synchronized reset output (active-low)

42.4 Functional Description

42.4.1 The Reset Problem

Reset release is where synchronous designs get hurt. Mishandle it and you're looking at: -

Metastability: When reset is released asynchronously relative to clock - **Timing Violations:**

Setup/hold violations during reset release - **Race Conditions:** Different flip-flops releasing from reset at different times

42.4.2 Asynchronous Assert, Synchronous Deassert

The behavior you actually want: - **Assert:** Immediate (asynchronous) when reset input becomes active - **Deassert:** Synchronized to clock edge to ensure clean timing

42.4.3 Synchronization Register Chain

```
logic [N-1:0] r_sync_reg = '0;
```

An N-stage shift register provides the synchronization delay. Here's the part that trips everyone up: the chain carries the **active-high** internal form of the reset — a 1 in the chain means “reset asserted”. That's the opposite of what the port names suggest, and it's the single most common source of confusion when people read this module.

42.4.4 Reset Logic

```
// ASYNC_ASSERT = 1 (default): async assert, sync deassert
always_ff @(posedge clk or posedge rst_in_h) begin
    if (rst_in_h) r_sync_reg <= '1;           // hold asserted
    through chain
    else          r_sync_reg <= {r_sync_reg[N-2:0], 1'b0};
end
```

While reset is asserted, the chain is held at all ones. Once it releases, zeros shift in from the LSB, and after N clock edges the MSB has cleared and the reset is considered released.

With ASYNC_ASSERT = 0 the sensitivity list drops rst_in_h and assertion gets sampled on the clock edge like any other synchronous load:

```
always_ff @(posedge clk) begin
    if (rst_in_h) r_sync_reg <= '1;
    else          r_sync_reg <= {r_sync_reg[N-2:0], 1'b0};
end
```

That style can't assert reset while the clock is stopped — which is exactly why the asynchronous-assert variant is the default.

42.4.5 Output Assignment

```
wire sync_rst_h = r_sync_reg[N-1];
always_comb begin
```

```

    sync_rst_n = OUT_ACTIVE_LOW ? ~sync_rst_h : sync_rst_h;
end

```

The MSB of the chain is the active-high synchronized reset; the output stage applies the requested polarity.

The RTL body is written out four times inside a generate, once per combination of ASYNC_ASSERT and KEEP_ATTRS, because the vendor attributes have to be attached at the declaration. All four variants are logically identical apart from the attributes and the sensitivity list. Yes, it's repetitive. That's the attribute syntax's fault, not the author's.

42.5 Timing Characteristics

42.5.1 Reset Assertion (Asynchronous)

```

rst_n: _____
sync_rst_n: _____
               ^
               Immediate assertion

```

42.5.2 Reset Deassertion (Synchronous, N=3)

```

clk:      _ _ _ _ _
rst_n:    _ _ _ _ _
r_sync_reg: 111 111 111 110 100 000 000 000 000
sync_rst_n: _____
               ^
               Synchronized deassertion
               (N clock cycles after rst_n release)

```

42.5.3 State Sequence for N=3

Default polarities (IN_ACTIVE_LOW = 1, OUT_ACTIVE_LOW = 1), so rst_n = 0 means reset asserted and sync_rst_n = 0 means reset asserted downstream. Remember the chain holds the active-high form, so 111 is "asserted".

reset_sync state sequence for N=3 at default polarities

Clock	rst_n	rst_in_h	r_sync_reg	sync_rst_n	Description
-	0	1	111	0	Reset asserted, chain forced to all ones
1	1	0	110	0	Reset released;

Clock	rst_n	rst_in_h	r_sync_reg	sync_rst_n	Description
					first 0 shifted in
2	1	0	100	0	Second 0 shifted in
3	1	0	000	1	MSB clears; reset released downstream
4+	1	0	000	1	Stable, out of reset

The output releases on the third clock edge after rst_n goes high — those N cycles are the metastability margin the chain exists to buy you.

42.6 Usage Examples

42.6.1 Choosing N Value

42.6.1.1 N=2 (Minimum)

```
reset_sync #(.N(2)) rst_sync_inst (
    .clk(sys_clk),
    .rst_n(external_rst_n),
    .sync_rst_n(internal_rst_n)
);
```

- **Use when:** Fast reset release required
- **Risk:** Minimal metastability margin
- **Applications:** High-speed designs with good clock quality

42.6.1.2 N=3 (Recommended)

```
reset_sync #(.N(3)) rst_sync_inst (
    .clk(sys_clk),
    .rst_n(external_rst_n),
    .sync_rst_n(internal_rst_n)
);
```

- **Use when:** Standard applications
- **Benefit:** Good balance of speed vs. metastability margin
- **Applications:** Most digital designs

42.6.1.3 $N \geq 4$ (Conservative)

```
reset_sync #(.N(4)) rst_sync_inst (
    .clk(sys_clk),
    .rst_n(external_rst_n),
    .sync_rst_n(internal_rst_n)
);
```

- **Use when:** Critical applications, poor clock quality
- **Benefit:** Maximum metastability protection
- **Trade-off:** Slower reset release

42.6.2 Global Reset Distribution

// Synchronize external reset for internal use

```
reset_sync #(.N(3)) global_rst_sync (
    .clk(system_clock),
    .rst_n(external_reset_n),
    .sync_rst_n(global_reset_n)
);
```

// Use synchronized reset throughout design

```
always_ff @(posedge system_clock or negedge global_reset_n) begin
    if (!global_reset_n) begin
        // Reset state
    end else begin
        // Normal operation
    end
end
```

42.6.3 Clock Domain Crossing

// Separate reset synchronizers for each clock domain

```
reset_sync #(.N(3)) clk_a_rst_sync (
    .clk(clk_a),
    .rst_n(system_rst_n),
    .sync_rst_n(clk_a_rst_n)
);
```

```
reset_sync #(.N(3)) clk_b_rst_sync (
    .clk(clk_b),
```



```

        .rst_n(system_rst_n),
        .sync_rst_n(clk_b_rst_n)
    );

```

42.6.4 Power-On Reset

```

// Synchronize power-on reset
reset_sync #(.N(3)) por_sync (
    .clk(main_clk),
    .rst_n(power_on_rst_n),
    .sync_rst_n(system_rst_n)
);

```

42.6.5 Reset Controller Integration

```

module reset_controller (
    input  logic sys_clk,
    input  logic external_rst_n,
    input  logic watchdog_rst_n,
    input  logic sw_rst,
    output logic sync_rst_n
);

    logic combined_rst_n;

    // Combine all reset sources
    assign combined_rst_n = external_rst_n & watchdog_rst_n & ~sw_rst;

    // Synchronize the combined reset
    reset_sync #(.N(3)) rst_sync_inst (
        .clk(sys_clk),
        .rst_n(combined_rst_n),
        .sync_rst_n(sync_rst_n)
    );

endmodule

```

42.7 Design Notes

42.7.1 1. Why the chain looks inverted

The chain stores the **active-high** form of the reset, while the ports are named for the active-low one. Read the dataflow in order and the confusion evaporates:

- `rst_in_h` is the input normalized to active-high
- while `rst_in_h` is 1, the chain is forced to all ones (reset asserted)

- when `rst_in_h` falls, zeros shift in from the LSB
- after N edges the MSB is 0, and the output stage inverts it back to the requested polarity

So during reset the register holds 1s and after release it drains to 0s — the opposite of what the port names alone would tell you.

42.7.22. Metastability Resolution

The N stages buy you multiple clock cycles for metastability to resolve: - Each flip-flop stage gets a chance to resolve potential metastability - Probability of metastability propagation: $(MTBF)^{-N}$ - N=2 typically sufficient, N=3 provides extra margin

42.7.33. Initialization

```
logic [N-1:0] r_sync_reg = '0;
```

The declaration initializer gives FPGA configuration a defined starting value. Note which value: 0, the *released* state. So on an FPGA the design comes out of configuration with reset already deasserted unless `rst_n` is asserted. On an ASIC the initializer is ignored, and the first asynchronous assertion establishes the state.

42.7.44. Vendor attributes

With `KEEP_ATTRS = 1` (the default) the chain carries `ASYNC_REG = "TRUE"` and `SHREG_EXTRACT = "N0"` for Xilinx, `altera_attribute` forcing synchronizer identification for Quartus, and `syn_preserve` for Synplify. These do two jobs: they stop the tool from packing the chain into an SRL primitive — which would quietly destroy your metastability margin — and they let CDC reports recognize it as a synchronizer. Set `KEEP_ATTRS = 0` only if your flow objects to the attributes.

42.7.55. Parameterization Benefits

Configurable N lets you tune for different requirements: - N=2: Minimum permitted, fast reset release - N=3: Standard choice, good metastability margin - N>=4: Conservative, slower release in exchange for maximum metastability protection

42.7.6 Reset Tree Planning

```
// Plan reset distribution hierarchy
// Level 1: Global reset synchronizer
// Level 2: Domain-specific reset synchronizers
// Level 3: Module-level reset distribution
```

42.7.7 Clock Quality Requirements

- **Clean Clock:** Stable, low-jitter clock required
- **Clock Frequency:** Must be running during reset release

- **Clock Gating:** Avoid gating clock used for reset synchronization

42.7.8 Reset Source Management

```
// Multiple reset sources need careful combination
wire combined_reset = por_n & external_rst_n & watchdog_rst_n &
~soft_reset;
```

42.7.9 Incorrect Sensitivity List

```
// WRONG: Missing async reset in sensitivity list
always_ff @(posedge clk) begin
    if (!rst_n) r_sync_reg <= '0;
    else r_sync_reg <= {r_sync_reg[N-2:0], 1'b1};
end
```

```
// CORRECT: Include async reset
always_ff @(posedge clk or negedge rst_n) begin
    // ... correct implementation
end
```

42.7.10 Wrong Reset Polarity

```
// Ensure consistent reset polarity throughout design
// This module uses active-low reset (rst_n)
```

42.7.11 Clock Domain Issues

```
// Each clock domain needs its own reset synchronizer
// Don't share synchronized reset across clock domains
```

42.8 Related Modules

- **Positive-edge reset:** For active-high reset systems
- **Multi-domain reset:** For complex clock domain systems
- **Reset pulse generator:** For generating reset pulses
- **Reset debouncer:** For mechanical switch inputs

Integration points worth knowing about:

- Clock domain crossing modules
- PLL/clock management units
- Power management controllers
- System controllers and reset managers

42.9 Testing

42.9.1 Test Coverage

Test coverage for the reset_sync module lives in: - **Testbench Class:**

bin/TBClasses/reset_sync_tb.py - **Test Runner:** val/common/test_reset_sync.py

42.9.2 Test Scenarios

- Reset assertion during various clock phases
- Reset deassertion timing verification
- Multiple reset assertion/deassertion cycles
- Clock jitter during reset release
- Power-on behavior

42.9.3 Testbench Scenarios

Basic Reset Synchronization - Verifies N-cycle synchronization delay **Immediate Reset Assertion** - Verifies asynchronous assertion behavior **Multiple Reset Cycles** - Tests repeated reset/release sequences **Reset Glitch Filtering** - Validates recovery from short reset pulses

42.9.4 Assertions

```
// Verify synchronous deassertion
property sync_deassert;
    @(posedge clk)
        $rose(rst_n) |-> ##N $rose(sync_rst_n);
endproperty
```

```
// Verify asynchronous assertion
property async_assert;
    $fell(rst_n) |-> $fell(sync_rst_n);
endproperty
```

42.9.5 Coverage Points

- All reset assertion scenarios
- Reset release with different clock phases
- Multiple consecutive reset cycles
- Parameter variation coverage (different N values)

42.9.6 Running Tests

Run all reset_sync tests (4 configurations: N=2,3,4,5)

```
pytest val/common/test_reset_sync.py -v
```

Run specific configuration

```
pytest val/common/test_reset_sync.py::test_reset_sync[2-min] -v
```

Run with waveform generation

```
pytest val/common/test_reset_sync.py -v -s
```

42.9.7 Test Results

All 4 parameter configurations passing (100% success rate): - N=2 (min) - PASSED - N=3 (typical) - PASSED - N=4 (safe) - PASSED - N=5 (max) - PASSED

Bug Discovery: the test suite caught a critical RTL bug on the initial run — an inverted reset polarity. Exactly the kind of bug that sails through a smoke test and ruins someone's bring-up. Thorough verification earns its keep.

42.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

43 reverse_vector

43.1 Overview

The `reverse_vector` module flips the bit order of an input vector end for end: MSB swaps with LSB, second MSB with second LSB, and so on down the line. It's a purely combinational operation — no clock, no state, just wires with an opinion about ordering.

- Combinational bit reversal operation
- Parameterizable vector width
- Zero propagation delay (combinational logic only)
- No clock or reset required

43.2 Parameters

Parameter	Type	Default	Description
WIDTH	—	32	Width of the input/output vectors

43.3 Ports

43.3.1 Inputs

Port	Width	Description
vector_in	WIDTH	Input vector to be reversed

43.3.2 Outputs

Port	Width	Description
vector_rev	WIDTH	Bit-reversed output vector

43.4 Functional Description

43.4.1 Bit Reversal Logic

```
always_comb begin
    for (integer i = 0; i < WIDTH; i++) begin
        vector_rev[(WIDTH-1)-i] = vector_in[i];
    end
end
```

end
end

A simple for-loop maps each input bit to its mirrored output position: - Input bit 0 → Output bit (WIDTH-1) - Input bit 1 → Output bit (WIDTH-2) - Input bit i → Output bit (WIDTH-1-i) - Input bit (WIDTH-1) → Output bit 0

43.4.28-bit Vector Reversal

Input: `vector_in = 8'b10110001`
Output: `vector_rev = 8'b10001101`

Bit mapping:

```
vector_in[7] = 1 → vector_rev[0] = 1
vector_in[6] = 0 → vector_rev[1] = 0
vector_in[5] = 1 → vector_rev[2] = 1
vector_in[4] = 1 → vector_rev[3] = 1
vector_in[3] = 0 → vector_rev[4] = 0
vector_in[2] = 0 → vector_rev[5] = 0
vector_in[1] = 0 → vector_rev[6] = 0
vector_in[0] = 1 → vector_rev[7] = 1
```

43.4.34-bit Vector Reversal

Input: `vector_in = 4'b1011`
Output: `vector_rev = 4'b1101`

Bit mapping:

```
vector_in[3] = 1 → vector_rev[0] = 1
vector_in[2] = 0 → vector_rev[1] = 0
vector_in[1] = 1 → vector_rev[2] = 1
vector_in[0] = 1 → vector_rev[3] = 1
```

43.5 Timing Characteristics

There is no clock and no state here, so timing in the usual sense doesn't apply. The output follows the input after gate and wire delay, and that's the whole story:

- **Propagation Delay:** Minimal (wire delay only), < 1ns typical
- **Critical Path:** Direct wire connections
- **Setup/Hold:** Not applicable (combinational)
- **Maximum Frequency:** Unlimited by this module — limited by surrounding logic
- **Throughput:** One operation per clock cycle (if clocked)

43.6 Usage Examples

43.6.1 Endianness Conversion

```
// Convert between big-endian and little-endian bit ordering  
little_endian_data = reverse_vector(big_endian_data);
```

43.6.2 Serial Communication

```
// Reverse bit order for serial transmission protocols  
// that send LSB first instead of MSB first  
serial_data = reverse_vector(parallel_data);
```

43.6.3 Cryptographic Operations

```
// Bit permutation operations in encryption algorithms  
permuted_key = reverse_vector(encryption_key);
```

43.6.4 Signal Processing

```
// Bit-reversed addressing for FFT algorithms  
reversed_address = reverse_vector(sequential_address);
```

43.6.5 Data Format Conversion

```
// Convert between different data representation formats  
converted_data = reverse_vector(input_format);
```

43.6.6 Mirror Pattern Generation

```
// Create mirror patterns for display or test applications  
mirror_pattern = reverse_vector(original_pattern);
```

43.6.7 Instantiation with Different Widths

```
// 16-bit reversal  
reverse_vector #(.WIDTH(16)) rev16 (  
    .vector_in(data_16bit),  
    .vector_rev(reversed_16bit)  
);
```

```
// 64-bit reversal  
reverse_vector #(.WIDTH(64)) rev64 (  
    .vector_in(data_64bit),  
    .vector_rev(reversed_64bit)  
);
```

43.6.8 Pipeline Integration

```
// Use in data processing pipeline  
always_ff @(posedge clk) begin  
    stage1_data <= input_data;  
    stage2_data <= reverse_vector_inst.vector_rev;  
end
```



```
    stage3_data <= process(stage2_data);  
end
```

43.6.9 Conditional Reversal

```
// Conditional bit reversal based on control signal  
assign output_data = reverse_enable ?  
    reverse_vector_inst.vector_rev :  
    vector_in;
```

43.7 Design Notes

43.7.1 1. Combinational Operation

- No clock signal required
- No sequential logic elements
- Output changes immediately when input changes
- Zero propagation delay (limited only by gate delays)

43.7.22. Parameterizable Width

The module works with any vector width specified by the WIDTH parameter: - Automatically scales to any desired width - Synthesis tools optimize for the specific width - No hardcoded bit indices

43.7.33. For-Loop Synthesis

Synthesis unrolls the for-loop completely, leaving one wire assignment per bit position. What you get: - Parallel bit assignments (no sequential operation) - Optimal timing characteristics - Efficient hardware implementation

43.7.44. Bidirectional Property

Reverse twice and you're back where you started:

```
original_vector == reverse(reverse(original_vector))
```

43.7.5 Power Consumption

- **Static Power:** Minimal (no storage elements)
- **Dynamic Power:** Only during input transitions
- **Switching Activity:** Proportional to input bit toggles

43.7.6 Area Utilization

- **Logic Elements:** Zero (pure routing)

- **Routing Resources:** WIDTH wire connections
- **Memory:** None required
- **Scalability:** Linear with WIDTH

43.7.7 Hardware Implementation

What synthesis actually builds: - Direct wire connections from input bits to output bits - No logic gates required (just routing) - Minimal area overhead - Excellent timing characteristics

43.7.8 Optimization

```
// Synthesis tools will optimize this to simple wire assignments:
assign vector_rev[WIDTH-1] = vector_in[0];
assign vector_rev[WIDTH-2] = vector_in[1];
// ...
assign vector_rev[1] = vector_in[WIDTH-2];
assign vector_rev[0] = vector_in[WIDTH-1];
```

43.7.9 Performance Metrics

Area: - **Gate Count:** 0 (wire-only implementation) - **Equivalent Gates:** WIDTH (for routing complexity) - **Memory Bits:** 0

Power: - **Leakage:** Negligible - **Dynamic:** Proportional to toggle rate - **Peak Power:** During simultaneous bit transitions

43.7.10 Common Pitfalls

Simulation warnings:

```
// Use 'integer' instead of 'int' for compatibility
for (integer i = 0; i < WIDTH; i++) begin
```

Synthesis issues:

```
// Ensure WIDTH is properly parameterized
localparam int W = WIDTH; // Local parameter for clarity
```

Timing closure:

```
// Add pipeline stage if needed for timing
always_ff @(posedge clk) begin
    vector_rev_reg <= vector_rev;
end
```

43.8 Related Modules

43.8.1 Byte Reversal

If what you actually need is byte-level endianness conversion, a separate module is the better tool:

```
// Reverse byte order instead of bit order
assign byte_reversed = {data[7:0], data[15:8], data[23:16],
data[31:24]};
```

43.8.2 Configurable Reversal

An enhanced version could add an enable:

```
assign vector_out = enable ? vector_rev : vector_in;
```

43.8.3 Multi-Width Reversal

Support multiple data widths in a single module:

```
generate
  case (WIDTH)
    8: assign vector_rev = {vector_in[0], vector_in[1], ...};
    16: assign vector_rev = {vector_in[0], vector_in[1], ...};
    32: assign vector_rev = {vector_in[0], vector_in[1], ...};
  endcase
endgenerate
```

43.9 Testing

43.9.1 Test Cases

```
// Test all zeros
vector_in = '0;
assert(vector_rev == '0);

// Test all ones
vector_in = '1;
assert(vector_rev == '1);

// Test alternating pattern
vector_in = 8'b10101010;
assert(vector_rev == 8'b01010101);

// Test double reversal
temp = reverse_vector_inst1.vector_rev;
assert(reverse_vector_inst2.vector_rev == vector_in);
```

43.9.2 Coverage Points

- All bit positions exercised
- Boundary conditions (all 0s, all 1s)
- Random patterns
- Double reversal verification

43.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

44 shifter_barrel

44.1 Overview

The `shifter_barrel` module is a combinational barrel shifter: any shift amount, any supported mode, done in a single clock cycle. It covers logical left/right shifts with and without wrap-around, arithmetic right shifts, and a no-shift pass-through, all selected by a 3-bit control signal.

- Single-cycle shift operations for any shift amount
- Multiple shift modes: logical, arithmetic, and rotational
- Configurable shift amount up to data width
- Wrap-around (rotation) capabilities
- Purely combinational logic for maximum speed

44.2 Parameters

Parameter	Type	Default	Description
WIDTH	—	8	Width of the data bus

44.3 Ports

44.3.1 Inputs

Port	Width	Description
data	WIDTH	Input data to be shifted
ctrl	3	Control signal for shift operation mode
shift_amount	$\$clog2(WIDTH)+1$	Number of positions to shift

44.3.2 Outputs

Port	Width	Description
data_out	WIDTH	Shifted output data

44.4 Functional Description

44.4.1 Control Signal Encoding

The 3-bit ctrl signal determines the shift operation:

ctrl	Operation	Description
3'b000	No shift	Output equals input
3'b001	Logical Right Shift (no wrap)	Fill with zeros from left
3'b010	Arithmetic Right Shift	Preserve sign bit (MSB)
3'b011	Logical Right Shift (wrap)	Rotate right
3'b100	Logical Left Shift (no wrap)	Fill with zeros from right
3'b110	Logical Left Shift (wrap)	Rotate left
Others	No shift	Default case

44.4.2 Shift Amount Modulation

```
logic [$clog2(WIDTH)-1:0] w_shift_amount_trunc, w_shift_amount_mod;
assign w_shift_amount_trunc = shift_amount[$clog2(WIDTH)-1:0];
assign w_shift_amount_mod = (32'(w_shift_amount_trunc) >= WIDTH)
    ? $clog2(WIDTH)'(32'(w_shift_amount_trunc) -
WIDTH)
    : w_shift_amount_trunc;
```

shift_amount_mod folds the amount into [0, WIDTH) for the wrap (rotate) paths — **only they use it**. The fold is truncate-to-\$clog2(WIDTH)-bits then one conditional subtract of WIDTH: plain truncation is modulo a power of two, which for non-power-of-2 widths could reach past the rotate arrays (an out-of-bounds X before 2026-08-21; exact for all widths now). The three no-wrap paths take the RAW amount, with IEEE saturation for amounts at or beyond the width: logical right/left by $\geq \text{WIDTH}$ gives 0, arithmetic right gives all-sign-bits. (Before 2026-08-20 the no-wrap paths carried a `shift_amount_mod == 0` passthrough guard, so shifting by exactly WIDTH returned the *input* — and the arithmetic path shifted by the modded amount, wrong for every amount $\geq \text{WIDTH}$. Found by a doc review round, sim-confirmed, fixed with directed WIDTH-boundary tests and an un-tautologized formal model that fails on the old RTL.)

44.4.3 Wrap-Around Implementation

```
logic [WIDTH*2-1:0] w_data_double;
assign w_data_double = {data, data};
```

```

// Generate lookup values for rotating shifts
genvar i;
generate
  for (i = 0; i < WIDTH; i++) begin : gen_unrolled_shifts
    assign w_array_rs[i] = w_data_double[WIDTH-1+i:i]; //
    // Right shift
    assign w_array_ls[i] = w_data_double[WIDTH*2-1-i:WIDTH-i]; //
    // Left shift
  end
endgenerate

```

44.4.4 Pre-computed Rotation Arrays

Two lookup arrays do the heavy lifting for rotation: - **w_array_rs[WIDTH]**: Pre-computed right rotation results for all possible shift amounts - **w_array_ls[WIDTH]**: Pre-computed left rotation results for all possible shift amounts

Pre-computing means there's no barrel-shifting network to evaluate at runtime — any rotation amount is a constant-time array lookup.

44.4.5 Main Shift Logic

```

always_comb begin
  case (ctrl)
    3'b000: // No shift
      data_out = data;

    3'b001: // Logical Right Shift (no wrap)
      data_out = data >> shift_amount;

    3'b010: // Arithmetic Right Shift (preserve sign)
      // if/else, NOT a ternary: a mixed-signedness ternary
      // degrades >>> to a logical shift
      if (shift_amount >= ($clog2(WIDTH)+1)'(WIDTH))
        data_out = {WIDTH{data[WIDTH-1]}};
      else
        data_out = $signed(data) >>> shift_amount;

    3'b011: // Logical Right Shift with wrap
      data_out = w_array_rs[w_shift_amount_mod];

    3'b100: // Logical Left Shift (no wrap)
      data_out = data << shift_amount;

    3'b110: // Logical Left Shift with wrap
      data_out = w_array_ls[w_shift_amount_mod];
  endcase
end

```

```
        default:
            data_out = data;
    endcase
end
```

44.5 Timing Characteristics

44.5.1 8-bit Examples (WIDTH=8)

44.5.1.1 *Logical Right Shift (no wrap)*

Input: data = 8'b11010110, shift_amount = 3, ctrl = 3'b001
Output: 8'b00011010

44.5.1.2 *Arithmetic Right Shift*

Input: data = 8'b11010110, shift_amount = 3, ctrl = 3'b010
Output: 8'b11111010 (sign extended)

44.5.1.3 *Right Rotation*

Input: data = 8'b11010110, shift_amount = 3, ctrl = 3'b011
Output: 8'b11011010 (bits wrap around)

44.5.1.4 *Logical Left Shift (no wrap)*

Input: data = 8'b11010110, shift_amount = 3, ctrl = 3'b100
Output: 8'b10110000

44.5.1.5 *Left Rotation*

Input: data = 8'b11010110, shift_amount = 3, ctrl = 3'b110
Output: 8'b10110110 (bits wrap around)

44.6 Usage Examples

Barrel shifters show up anywhere bits need to move fast. Typical homes for this one:

- ALU implementations
- Cryptographic operations
- Signal processing
- Bit manipulation engines
- Network packet processing
- Data alignment circuits

44.7 Design Notes

44.7.1 1. Combinational Design

- All operations complete in zero clock cycles
- No state machines or sequential logic
- Suitable for high-frequency operations

44.7.2.2. Arithmetic Right Shift

The arithmetic shift uses SystemVerilog's `>>>` operator with a `$signed()` cast, so sign extension behaves correctly for two's complement numbers.

44.7.3.4. Double-Width Concatenation

The `{data, data}` concatenation is the oldest trick in the book: paste the data next to itself and a rotation becomes a plain slice out of a $2\times$ -wide window. Any rotation amount, same cost.

44.7.4.5. Generate Block Optimization

The generate block pre-computes every possible rotation at compile time, so runtime is just a lookup. You spend a little area, you get back maximum speed.

44.7.5 Area Considerations

- **Lookup Arrays:** Require $\text{WIDTH} \times \text{WIDTH}$ bits of combinational logic
- **Multiplexers:** Large mux structures for final output selection
- **Total:** Approximately $O(\text{WIDTH}^2)$ area complexity

44.7.6 Speed Characteristics

- **Propagation Delay:** Constant regardless of shift amount
- **Critical Path:** Through lookup array and final output mux
- **Frequency:** Limited by combinational delay, not shift complexity

44.8 Related Modules

Read out of the RTL, not curated: these are the modules this one instantiates and the modules that instantiate it.

Instantiated by: -math_bf16_adder -math_ieee754_2008_fp32_adder

44.9 Testing

`val/common/test_shifter_barrel.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/common/test_shifter_barrel.py -v
```

44.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

45 shifter_beat_pack

Module: shifter_beat_pack.sv **Location:** rtl/common/ **Status:** Production Ready

45.1 Overview

shifter_beat_pack is a bit-granular packing/aligning shifter. It accepts fixed-width `CHUNK_BITS` entries on a push handshake, accumulates them in a multi-chunk staging register, and drains **configurable-width “beats”** out the low end on a pop handshake. The beat width is a runtime input (bytes minus one), so the same instance can emit different beat sizes across bursts.

All of the shift/load/mux logic lives inside this one module so that callers — an aligner that needs to repack DFI cycles into DRAM beats, say — don’t have to invent their own shift-and-compensate scheme. Push and pop can occur in the same cycle; the next-state logic applies the pop first and the push second, and a single non-blocking assignment commits the result, so no internal last-assignment race is possible.

This module exists for a problem that shows up constantly: repacking a stream of one fixed width into beats of another (often runtime-selected) width. Data arrives one whole chunk at a time and leaves as beats whose width is chosen at run time; the module holds the partial-beat residue between cycles and shifts it down as beats drain, so the caller sees clean valid/ready handshakes on both sides.

Use Cases:

- Packing DFI cycles into DRAM beats in a memory-controller aligner
- Width adaptation where ingress chunk width and egress beat width differ and the beat width is runtime-configurable
- Any packer that must hold partial-beat residue while new fixed-width data lands

Key Benefit: Centralizes the shift, load, and residue-compensation logic behind two simple handshakes, so callers get runtime-variable beat repacking without hand-rolling their own barrel shifter.

Feature summary:

- **Bit-granular staging:** A `DEPTH_CHUNKS × CHUNK_BITS` register packs chunks and drains beats at bit resolution
- **Runtime beat width:** `cfg_beat_bytes_m1` selects the beat size (bytes – 1) per burst without re-elaboration

- **Independent ingress / egress sizing:** Chunk width and beat cap are separate parameters
- **Same-cycle push + pop:** Pop-then-push ordering with one NBA eliminates internal races
- **Elaboration-time contract checks:** \$error guards enforce `DEPTH_CHUNKS >= 2` and `MAX_BEAT_BITS < STORAGE_BITS`
- **Status outputs:** `empty` and `count_bits_o` expose occupancy to the wrapper

45.2 Parameters

Parameter	Type	Default	Description
CHUNK_BITS	int	128	Width of one push entry (ingress granularity)
MAX_BEAT_BYTES	int	16	Maximum beat width in bytes; caps what reaches <code>pop_data</code>
CFG_BITS	int	8	Width of the runtime <code>cfg_beat_bytes_m1</code> field. 8 bits gives <code>beat_bytes</code> $\in$ 1..256; 4 bits gives 1..16, etc.
DEPTH_CHUNKS	int	2	Staging-register depth in chunks. Default 2 = “current + prefetched-next”. Must be ≥ 2 . Larger depths absorb more before draining but widen the pop-side barrel shifter (timing cost).
MAX_BEAT_BITS	int	$\text{MAX_BEAT_BYTES} * 8$	Derived (do not override): maximum beat width in bits
STORAGE_BITS	int	$\text{DEPTH_CHUNKS} * \text{CHUNK_BITS}$	Derived (do not override): total staging-

Parameter	Type	Default	Description
COUNT_BITS	int	$\$clog2(STORAGE_BITS + 1)$	register width Derived (do not override): width of the bit-occupancy counter
IDX_BITS	int	$\$clog2(STORAGE_BITS)$	Derived (do not override): index width for <code>r_data</code> part-selects

45.3 Ports

Functionally the ports form a clock/reset pair, a runtime configuration input, a push (ingress) handshake, a pop (egress) handshake, and two status outputs. Grouped by direction:

45.3.1 Inputs

Port	Width	Description
<code>clk</code>	1	Clock
<code>rst_n</code>	1	Active-low reset (via <code>reset_defs.svh</code> macros)
<code>cfg_beat_bytes_m1</code>	CFG_BITS	Beat width in bytes – 1 (0 → 1 byte). Held stable per burst.
<code>push_valid</code>	1	New chunk valid (push/ingress handshake)
<code>push_data</code>	CHUNK_BITS	The chunk to pack
<code>pop_ready</code>	1	Consumer accepts a beat (pop/egress handshake)

45.3.2 Outputs

Port	Width	Description
<code>push_ready</code>	1	Room for another whole chunk (post-push count $\leq STORAGE_BITS$)
<code>pop_valid</code>	1	At least one whole beat is available
<code>pop_data</code>	MAX_BEAT_BITS	Low MAX_BEAT_BITS of the staging register; consumer reads only the low $(cfg_beat_bytes_m1+1)*8$ bits
<code>empty</code>	1	Staging register holds no bits (<code>r_count</code>

Port	Width	Description
		<code>== 0)</code>
<code>count_bits_o</code>	<code>COUNT_BITS</code>	Current occupancy in bits

45.4 Functional Description

45.4.1 Runtime Beat Width

The runtime beat width in bits comes from the byte-minus-one config field, widened by 4 bits so the counter math can represent the largest beat any `CFG_BITS`-wide value can encode:

```
localparam int BEAT_BITS_W = CFG_BITS + 4;
assign w_beat_bits = ({..., cfg_beat_bytes_m1} + 1) << 3; //
(bytes_m1 + 1) * 8
```

`MAX_BEAT_BYTES` caps what actually appears on `pop_data`; `w_beat_bits` is used only for the occupancy accounting.

45.4.2 Handshake and Status

The combinational handshakes are driven directly from the bit-occupancy counter `r_count`:

- `empty = (r_count == 0)`
- `push_ready =` there is room for another whole chunk, i.e. `r_count <= STORAGE_BITS - CHUNK_BITS`
- `pop_valid =` the register holds at least one whole beat, i.e. `r_count >= w_beat_bits` and non-zero
- `pop_data =` the low `MAX_BEAT_BITS` of `r_data` (a plain slice, well-defined because a beat is always strictly narrower than the staging window)

When the configured beat is narrower than `MAX_BEAT_BITS`, the upper bits of `pop_data` are stale bytes belonging to the next beat. That's fine — the consumer already knows the width from `cfg` and reads only the low valid bits.

45.4.3 Next-State: Pop First, Push Second

Each cycle the combinational block computes next-state `w_v_data` / `w_v_count` in two ordered steps:

1. **Pop** (when `pop_valid && pop_ready`): shift the register down by `w_beat_bits` and subtract that many bits from the count, draining the low beat.

2. **Push** (when push_valid && push_ready): land the new chunk at [w_v_count +: CHUNK_BITS] using the *post-pop* occupancy, so the chunk sits right above the still-valid bits whether or not a pop fired, then add CHUNK_BITS to the count.

```
if (pop_valid && pop_ready) begin
    w_v_data  = w_v_data >> w_beat_bits;
    w_v_count = w_v_count - COUNT_BITS'(w_beat_bits);
end
if (push_valid && push_ready) begin
    w_v_data[w_v_count[IDX_BITS-1:0] +: CHUNK_BITS] = push_data;
    w_v_count = w_v_count + COUNT_BITS'(CHUNK_BITS);
end
```

One non-blocking assignment then writes r_data / r_count from w_v_data / w_v_count, so same-cycle push + pop has no internal last-assignment race.

45.4.4 Elaboration-Time Contract

Two \$error guards enforce the sizing rules that keep forward progress guaranteed:

- DEPTH_CHUNKS >= 2: a single-chunk register cannot hold partial-beat residue while a new chunk lands.
- MAX_BEAT_BITS < STORAGE_BITS: a beat must be strictly narrower than the staging window.

45.4.5 Sizing Guidance

Pick CHUNK_BITS and MAX_BEAT_BYTES so any runtime beat width fits in the 2N (or deeper) storage, i.e. $\text{cfg_beat_bytes} \times 8 \leq \text{DEPTH_CHUNKS} \times \text{CHUNK_BITS}$. Ingress chunk width and egress beat cap are independent, so callers can right-size both.

45.5 Timing Characteristics

This module is **sequential**: it contains clocked logic (via always_ff or the repository's ALWAYS_FF_RST macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

45.6 Usage Examples

```
// Repack 128-bit DFI cycles into runtime-sized DRAM beats (up to 16
bytes).
shifter_beat_pack #(
    .CHUNK_BITS      (128),
    .MAX_BEAT_BYTES  (16),
    .CFG_BITS        (8),
    .DEPTH_CHUNKS    (2)
) u_beat_pack (
    .clk              (clk),
    .rst_n            (rst_n),
    .cfg_beat_bytes_m1 (beat_bytes - 8'd1),    // e.g. 8 bytes -> 7

    // ingress: one 128-bit chunk per handshake
    .push_valid       (dfi_valid),
    .push_ready       (dfi_ready),
    .push_data        (dfi_cycle),

    // egress: one beat per handshake
    .pop_valid        (beat_valid),
    .pop_ready        (beat_ready),
    .pop_data         (beat_data),              // read low
    (beat_bytes*8) bits

    // status
    .empty            (pack_empty),
    .count_bits_o     (pack_bits)
);
```

45.7 Design Notes

- **Bit occupancy, not entry count.** `r_count` tracks bits, not entries, because chunks land whole but beats drain at a runtime-variable bit width.
- **Pop-then-push ordering is deliberate.** Draining first frees the low bits so the incoming chunk lands at the correct post-pop position, which is what makes same-cycle push + pop safe.
- **pop_data upper bits may be stale.** Only the low $(\text{cfg_beat_bytes_m1}+1)*8$ bits are meaningful; the consumer must mask to the configured width.

- **Depth vs. timing.** Increasing DEPTH_CHUNKS lets the packer absorb more data before draining but widens the pop-side barrel shifter, degrading timing.
- **Reset macros.** The module uses the project `reset_defs.svh` ALWAYS_FF_RST / RST_ASSERTED macros for the registered state.

45.8 Related Modules

- **Used by:** memory-controller aligners that repack DFI cycles into DRAM beats
- **Uses:** nothing beyond the `reset_defs.svh` reset macros — the module is self-contained
- **See also:** [fifo_sync](#) - Whole-entry synchronous FIFO (contrast: this module repacks at bit granularity)

45.9 Testing

`val/common/test_shifter_beat_pack.py` exercises this module. It collects 12 parameter cases at the default REG_LEVEL.

```
source env_python
pytest val/common/test_shifter_beat_pack.py -v
```

45.10 References

45.10.1 Source Code

- `rtl/common/shifter_beat_pack.sv`

45.10.2 Documentation

- `docs/markdown/rtl-common/index.md`

Last Updated: 2026-07-15

45.11 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

46 shifter_lfsr_fibonacci

46.1 Overview

The `shifter_lfsr_fibonacci` module is a Fibonacci Linear Feedback Shift Register — the textbook form, where all the feedback funnels into a single point at the most significant bit. It's the complement to the Galois LFSR, and the two come with genuinely different implementation trade-offs.

- Fibonacci (external XOR) LFSR architecture
- Configurable tap positions and polynomial
- Right-shift operation with feedback to MSB
- Seed loading and cycle detection
- Non-zero state enforcement
- Parameterizable width and tap configuration

46.2 Parameters

Parameter	Type	Default	Description
WIDTH	—	8	Width of the LFSR register
TAP_INDEX_WIDTH	—	12	Width of each tap index
TAP_COUNT	—	4	Number of feedback taps
TIW	—	—	Shorthand for TAP_INDEX_WIDTH

46.3 Ports

46.3.1 Inputs

Port	Width	Description
clk	1	System clock
rst_n	1	Active-low

Port	Width	Description
enable	1	asynchronous reset Enable LFSR operation
seed_load	1	Load seed value into LFSR
seed_data	WIDTH	Seed value for LFSR initialization
taps	TAP_COUNT*TIW	Concatenated tap positions

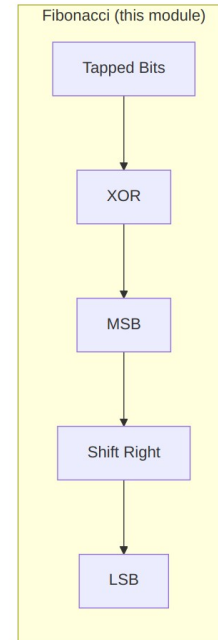
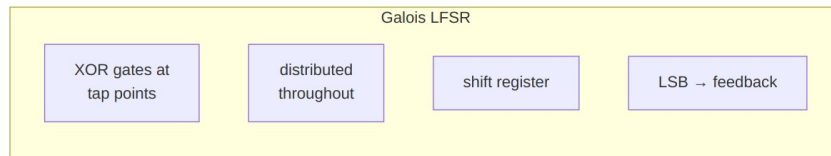
46.3.2 Outputs

Port	Width	Description
lfsr_out	WIDTH	Current LFSR value
lfsr_done	1	High when LFSR returns to seed value

46.4 Functional Description

46.4.1 Fibonacci LFSR Characteristics

- **Feedback Location:** Single XOR gate feeding MSB
- **Data Path:** Simple right-shift with external feedback
- **XOR Gate Count:** One multi-input XOR gate
- **Critical Path:** Through feedback XOR to MSB



Fibonacci vs Galois at a Glance

46.4.2 Tap Processing (Same as Standard LFSR)

```
// Split concatenated tap positions
always_comb begin
    for (int i = 0; i < TAP_COUNT; i++)
        w_tap_positions[i] = taps[i*TIW+:TIW];
end

// Convert to bit mask
always_comb begin
    w_taps = 'b0;
    for (int i = 0; i < TAP_COUNT; i++)
        if (w_tap_positions[i] > 0)
            w_taps[w_tap_positions[i]-1'b1] = 1'b1;
end
```

46.4.3 Fibonacci Feedback Calculation

```
assign w_feedback = ^(r_lfsr & w_taps);
```

Key Difference: XOR (^) here, not the XNOR (~^) the standard LFSR module uses.

46.4.4 Right-Shift with MSB Feedback

```
always_ff @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        r_lfsr <= 'b0; // initialization to all 0's
    end else begin
```

```

    if (enable) begin
        if (seed_load) begin
            r_lfsr <= seed_data; // Load seed
        end else if (!r_lfsr) begin // Only shift if we have non-
zero value
            // Fibonacci LFSR: Shift right, feedback to MSB
            r_lfsr <= {w_feedback, r_lfsr[WIDTH-1:1]};
        end
    end
end
end
end

```

46.5 Timing Characteristics

46.5.1 4-bit Fibonacci LFSR Example

Configuration: - WIDTH = 4 - Polynomial: $x^4 + x^3 + 1$, which **this module encodes as taps [4,1]** — see the tap-direction note under “Polynomial Examples” below - Seed: 4'b1001

46.5.2 Sequence Generation

Tap position p selects bit $p-1$, feedback is the XOR of the tapped bits, and the register shifts right with the feedback entering the MSB ($\{w_feedback, r_lfsr[WIDTH-1:1]\}$):

Cycle	LFSR	Tap bits (4,1)	Feedback	Next LFSR
0	1001	1,1	$1^1=0$	0100
1	0100	0,0	$0^0=0$	0010
2	0010	0,0	$0^0=0$	0001
3	0001	0,1	$0^1=1$	1000
4	1000	1,0	$1^0=1$	1100
5	1100	1,0	$1^0=1$	1110
...				

The sequence runs the full period of 15 and returns to the seed, at which point `lfsr_done` asserts. Reset loads all zeros, and the `|r_lfsr` guard means the register cannot leave zero on its own — a `seed_load` is required to start it. Getting the taps wrong costs the full period, and how it fails depends on the seed: sweeping all 15 non-zero seeds at WIDTH=4 with [4,3], three walk to zero and freeze under the `|r_lfsr` guard (seeds 1..3), three revisit their seed so `lfsr_done` DOES assert (seeds 6, 11, 13), and the remaining nine settle into a cycle that never revisits the seed.

46.6 Usage Examples

- Pseudo-random sequence generation (same maximal-length guarantees as the Galois form; the STATE SEQUENCES differ – see the Usage Note and the Galois page)
- CRC calculation (with appropriate polynomial)

- Data scrambling and encryption
- Test pattern generation
- Spread spectrum communications
- Error detection and correction

46.7 Design Notes

46.7.1 1. XOR vs XNOR Feedback

- **Fibonacci LFSR:** Uses XOR (^) for feedback calculation
- **Standard LFSR:** Uses XNOR (~^) for feedback calculation
- Both produce maximal-length sequences with appropriate polynomials

46.7.22. Non-Zero State Protection

```
end else if (|r_lfsr) begin // Only shift if we have non-zero value
```

This guard stops the LFSR from shifting when it's sitting at all zeros — a state it could never leave on its own. The sequence would lock at zero permanently.

46.7.33. Right-Shift Architecture

```
r_lfsr <= {w_feedback, r_lfsr[WIDTH-1:1]};
```

- Feedback enters at MSB position
- Data shifts right (towards LSB)
- LSB data is discarded each cycle

46.7.44. Polynomial Compatibility

Your tap positions need to correspond to a primitive polynomial appropriate for Fibonacci LFSR implementation. The polynomial form is:

$$P(x) = x^n + x^{(\text{tap1})} + x^{(\text{tap2})} + \dots + x^{(\text{tapk})} + 1$$

46.7.55. Reset to All Zeros

Unlike some LFSR implementations that reset to all ones, this module resets to all zeros and relies on seed loading for proper initialization.

46.7.6 Advantages of Fibonacci LFSR

- **Simpler Data Path:** Only one shift register with external feedback

- **Easy to Understand:** Classical LFSR textbook implementation
- **Single XOR Gate:** All feedback logic concentrated in one location

46.7.7 Disadvantages of Fibonacci LFSR

- **Fan-out:** Feedback XOR gate may have high fan-out for many taps
- **Critical Path:** All tapped bits must route to single XOR gate
- **Timing:** May be slower than distributed Galois implementation

46.7.8 When to Use Fibonacci LFSR

- **Educational Purposes:** Easier to understand and debug
- **Few Taps:** When polynomial has only 2-3 taps
- **Area Constraints:** When minimizing XOR gate count is critical
- **Legacy Compatibility:** When interfacing with existing Fibonacci LFSR systems

46.7.9 Polynomial Examples for Fibonacci LFSR

The tap numbers below are specific to this module's shift direction. The tap column published for Galois LFSRs – and the table in the `shifter_lfsr.sv` header, which is XNOR feedback with a *left* shift – encodes the same polynomials differently. Get it wrong and you lose the full period, and the failure is quiet either way. Sweeping all 15 non-zero seeds on this RTL at WIDTH=4 with taps [4,3]: three seeds (1, 2, 3 – including 4'b0001, which reaches zero in **one** step) walk to zero and freeze under the `|r_lfsr` guard; three (6, 11, 13) DO revisit their seed, so `lfsr_done` asserts for them; the remaining nine settle into a cycle that never revisits the seed and never assert it. [4,1] runs the full period of 15 from every seed.

Width	Polynomial	Tap Positions	Period
3	x^3+x^2+1	[3,1]	7
4	x^4+x^3+1	[4,1]	15
5	x^5+x^3+1	[4,1]	31
8	$x^8+x^6+x^5+x^4+1$	[7,6,5,1]	255
16	$x^{16}+x^{15}+x^{13}+x^4+1$	[16,14,5,1]	65535

Width	Polynomial	Tap Positions	Period
24	$x^{24} + x^{23} + x^{22} + x^{17} + 1$	[24,23,18,1]	16777215
32	$x^{32} + x^{22} + x^2 + x + 1$	[23,3,2,1]	4294967295

The polynomials themselves are the standard primitive set; only the tap *encoding* differs from the published one. The full 168-width table in this module's RTL header (`rtl/common/shifter_lfsr_fibonacci.sv`) is already converted – use it directly.

46.7.10 Why the Encoding Differs

This module computes $fb = \text{^(lfsr \& taps)}$ and shifts it into the MSB, so a tap at position t contributes $x^{(t-1)}$ and the characteristic polynomial is:

$$x^{\text{WIDTH}} + \text{SUM over taps of } x^{(\text{tap}-1)}$$

For a published polynomial $x^n + x^a + x^b + 1$, the taps here are $[a+1, b+1, 1]$. Two consequences worth remembering:

- **Tap 1 is always present** – it supplies the constant term. A tap set without it cannot be maximal, and typically locks at zero.
- **n itself is not a tap** – the register width supplies the leading term.

To convert any row of the standard table: drop the leading n , add 1 to every remaining number, then append 1.

46.7.11 Usage Note

The same polynomial works for both Fibonacci and Galois LFSRs, and both produce maximal-length sequences of the same period – but the *tap numbers you pass in* differ, and the state orderings differ. `shifter_lfsr_galois.sv` takes the exponents directly ($[n, a, b]$); this module takes $[a+1, b+1, 1]$.

46.8 Related Modules

- [shifter_lfsr_galois](#) — the distributed-XOR Galois form. Same polynomials and period, but the tap numbers are **not** interchangeable with this module's, and the state orderings differ.
- [shifter_lfsr](#) — the generic XNOR-feedback LFSR; left-shift, different tap encoding again (see the warning above).

46.9 Testing

`val/common/test_shifter_lfsr_fibonacci.py` exercises this module. It collects 6 parameter cases at the default `REG_LEVEL`.


```
source env_python
pytest val/common/test_shifter_lfsr_fibonacci.py -v
```

46.10 **Navigation**

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

47 shifter_lfsr_galois

47.1 Overview

The `shifter_lfsr_galois` module is a Galois Linear Feedback Shift Register: instead of one feedback XOR at the end of the chain, the XOR gates are distributed across the tap positions inside the register. That buys you better timing and higher operating frequencies than the Fibonacci form — which is usually why you're reaching for it.

- Galois (internal XOR) LFSR architecture
- Distributed feedback XOR gates at tap positions
- Right-shift operation with LSB feedback
- Superior timing characteristics for high-speed operation
- Configurable tap positions and polynomial
- Seed loading and cycle detection

47.2 Parameters

Parameter	Type	Default	Description
WIDTH	—	8	Width of the LFSR register
TAP_INDEX_WIDTH	—	12	Width of each tap index
TAP_COUNT	—	4	Number of feedback taps
TIW	—	—	Shorthand for TAP_INDEX_WIDTH

47.3 Ports

47.3.1 Inputs

Port	Width	Description
clk	1	System clock
rst_n	1	Active-low

Port	Width	Description
enable	1	asynchronous reset Enable LFSR operation
seed_load	1	Load seed value into LFSR
seed_data	WIDTH	Seed value for LFSR initialization
taps	TAP_COUNT*TIW	Concatenated tap positions

47.3.2 Outputs

Port	Width	Description
lfsr_out	WIDTH	Current LFSR value
lfsr_done	1	High when LFSR returns to seed value

47.4 Functional Description

47.4.1 Key Architectural Differences

Fibonacci LFSR:

MSB ← [XOR of all taps] ← LSB
 ↓
 Shift Right
 ↓
 LSB

Galois LFSR:

MSB → [XOR] → [XOR] → [XOR] → LSB
 ↑ ↑ ↑ ↓
 tap1 tap2 tap3 feedback
 ↑

47.4.2 Advantages of Galois Architecture

- **Distributed Logic:** XOR gates spread throughout the shift register
- **Better Timing:** Shorter critical paths between flip-flops
- **Higher Frequency:** Can operate at higher clock speeds
- **Localized Routing:** Reduced interconnect complexity

47.4.3 Tap Position Processing

```
// Split concatenated tap positions
always_comb begin
    for (int i = 0; i < TAP_COUNT; i++) begin
        w_tap_positions[i] = taps[i*TIW+:TIW];
    end
end
```

47.4.4 LSB Feedback Generation

```
assign w_feedback = r_lfsr[0];
```

The feedback is just the LSB (bit 0) of the current LFSR state. That's it.

47.4.5 Galois Next State Calculation

```
always_comb begin
    // Start with right shift (include 0 in MSB)
    next_lfsr = {1'b0, r_lfsr[WIDTH-1:1]};

    // Apply Galois feedback taps if LSB is 1
    if (w_feedback) begin
        for (int j = 0; j < TAP_COUNT; j++) begin
            if (w_tap_positions[j] > 0 && w_tap_positions[j] <= WIDTH)
                // Apply XOR to the tap positions
                next_lfsr[w_tap_positions[j]-1] =
next_lfsr[w_tap_positions[j]-1] ^ 1'b1;
            end
        end
    end
end
```

47.4.6 Sequential Update

```
always_ff @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        r_lfsr <= {WIDTH{1'b1}}; // initialization to all 1's
    end else if (enable) begin
        if (seed_load) begin
            r_lfsr <= seed_data;
        end else begin
            r_lfsr <= next_lfsr;
        end
    end
end
```

47.5 Testing

val/common/test_shifter_lfsr_galois.py exercises the module, and val/common/test_shifter_lfsr_galois_sequence.py checks the generated sequence against a reference model.

```
source env_python
pytest val/common/test_shifter_lfsr_galois.py
val/common/test_shifter_lfsr_galois_sequence.py -v
```

47.6 Timing Characteristics

47.6.1 4-bit Galois LFSR Example

Configuration: - WIDTH = 4 - Polynomial: $x^4 + x^3 + 1$ (taps at positions 4,3) - Seed: 4'b1001

47.6.2 Sequence Generation

Galois right-shift: shift right by 1; if the shifted-out LSB was 1, XOR the tap mask 4'b1100 (bits 3 and 2 — positions 4,3, 1-indexed) into the shifted value.

Cycle	LFSR	LSB	after >>1	XOR 1100?	Next LFSR
0	1001	1	0100	yes	1000
1	1000	0	0100	no	0100
2	0100	0	0010	no	0010
3	0010	0	0001	no	0001
4	0001	1	0000	yes	1100
...					

47.7 Usage Examples

47.7.1 High-Speed Systems

```
// Use Galois LFSR for high-frequency applications
// Clock rates > 500MHz where timing is critical
```

47.7.2 Communications

```
// Spread spectrum systems requiring fast sequence generation
// Real-time scrambling/descrambling applications
```

47.7.3 Cryptography

```
// Stream cipher implementations
// Key sequence generation
```

47.7.4 Testing

```
// High-speed BIST pattern generation
// Fast pseudo-random test vectors
```

47.8 Design Notes

47.8.11. Reset to All Ones

```
r_lfsr <= {WIDTH{1'b1}}; // initialization to all 1's
```

Where the Fibonacci version resets to all zeros, this one resets to all ones — a valid non-zero state, so it can run immediately.

A zero seed locks this module, and nothing stops you loading one. Unlike `shifter_lfsr_fibonacci.sv`, which shifts only while `r_lfsr` holds, this module has no zero guard: with `r_lfsr = 0` the feedback bit `r_lfsr[0]` is 0, no taps are applied, and `next_lfsr = {1'b0, r_lfsr[WIDTH-1:1]}` is zero again. The register stays at zero for good — and because `lfsr_done` is the equality `lfsr_out == seed_data`, a zero seed also parks `lfsr_done` high forever instead of pulsing once per period. Drive `seed_data` non-zero on `seed_load`; the reset value is safe, only a loaded seed can do this.

47.8.22. Combinational Next State Logic

The next LFSR state is calculated combinatorially in the `always_comb` block, then registered in the `always_ff` block. Clean separation: timing concerns stay out of the logic, and the logic stays easy to read.

47.8.33. Conditional XOR Application

```
if (w_feedback) begin
    // Only apply tap XORs when LSB is 1
end
```

The tap XORs fire only when the LSB feedback bit is 1 — that conditional inversion is the Galois behavior.

47.8.44. Bounds Checking

```
if (w_tap_positions[j] > 0 && w_tap_positions[j] <= WIDTH) begin
```

Explicit bounds checking prevents out-of-range array access and handles unused tap positions (value 0).

47.8.55. Right-Shift with Distributed Feedback

The basic operation is a right shift, but with XOR gates inserted at tap positions that conditionally invert bits based on the LSB feedback.

47.8.6 Critical Path Analysis

Fibonacci LFSR Critical Path:

All tapped bits → Multi-input XOR → MSB flip-flop

Galois LFSR Critical Path:
Previous bit → XOR gate → Current bit flip-flop

47.8.7 Timing Advantages

- **Shorter Logic Depth:** Each bit only sees local XOR gate
- **Parallel Processing:** All XOR operations occur simultaneously
- **Reduced Fan-out:** No single XOR gate with multiple inputs
- **Better Scaling:** Critical path doesn't grow with tap count

47.8.8 Tap Position Mapping

For a polynomial $P(x) = x^n + x^a + x^b + \dots + 1$ - Galois LFSR taps: [n, a, b, ...] – the exponents, used directly - The same polynomial is maximal for both Fibonacci and Galois, but **the tap numbers you pass in are not interchangeable**. This module XORs the mask into the post-shift value, so taps are the exponents; `shifter_lfsr_fibonacci.sv` XORs the tapped bits into the MSB, which shifts the encoding to [a+1, b+1, 1]. Passing Galois taps to the Fibonacci module costs the full period, but what you see depends on the seed: measured at WIDTH=4 with taps [4, 3] fed to the FIBONACCI module, three of the 15 non-zero seeds walk to zero and freeze (1..3), three revisit their seed so `lfsr_done` still asserts (6, 11, 13), and the remaining nine cycle without ever revisiting. (This module itself with [4, 3] is maximal: all 15 seeds run the full period – enumerated 2026-08-22.) - Different sequences produced but same mathematical properties

47.8.9 Example Polynomials

Width	Polynomial	Galois Taps	Max Period
3	$x^3 + x^2 + 1$	[3,2]	7
4	$x^4 + x^3 + 1$	[4,3]	15
5	$x^5 + x^3 + 1$	[5,3]	31
8	$x^8 + x^6 + x^5 + x^4 + 1$	[8,6,5,4]	255
12	$x^{12} + x^6 + x^4 + x + 1$	[12,6,4,1]	4095
16	$x^{16} + x^{15} + x^{13} + x^4 + 1$	[16,15,13,4]	65535
24	$x^{24} + x^{23} + x^{22} + x^{17} + 1$	[24,23,22,17]	16777215
32	$x^{32} + x^{22} + x^2 + x + 1$	[32,22,2,1]	$2^{32}-1$

Widths 3 through 16 were confirmed by simulation against this RTL; each yields period $2^n - 1$. The full 168-width table is in the module's RTL header (`rtl/common/shifter_lfsr_galois.sv`).

47.8.10 Galois LFSR Advantages

- **Speed:** Higher maximum clock frequency
- **Timing:** Better setup/hold characteristics
- **Scalability:** Performance doesn't degrade with tap count
- **Power:** Potentially lower dynamic power

47.8.11 Galois LFSR Disadvantages

- **Complexity:** More complex combinational logic
- **Area:** May require more logic gates for many taps
- **Debug:** Harder to trace intermediate states
- **Understanding:** Less intuitive than classical Fibonacci

47.8.12 When to Choose Galois over Fibonacci

Use Galois LFSR when: - High clock frequency is required (>100MHz) - Timing closure is challenging - Multiple taps are needed (>3 taps) - Power consumption is critical - Synthesis tools favor distributed logic

Use Fibonacci LFSR when: - Educational/demonstration purposes - Legacy system compatibility required - Very few taps (2 taps or less) - Area is more critical than speed - Debugging capabilities are important

47.9 Related Modules

- [shifter_lfsr_fibonacci](#) — the single-XOR Fibonacci form. Do **not** pass this page's tap numbers to it; the encoding differs (see Tap Position Mapping above).
- [shifter_lfsr](#) — the generic XNOR-feedback LFSR.

47.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

48 shifter_lfsr

48.1 Overview

The `shifter_lfsr` module is the generic Linear Feedback Shift Register: configurable tap positions for pseudo-random number generation, support for various LFSR polynomials, and cycle completion detection so you know exactly when the deterministic sequence has wrapped.

- Configurable tap positions for different polynomial implementations
- Seed loading capability for sequence initialization
- Cycle completion detection (sequence wrap-around)
- Enable control for conditional operation
- Support for various LFSR widths and tap configurations

48.2 Parameters

Parameter	Type	Default	Description
WIDTH	—	8	Width of the LFSR register
TAP_INDEX_WIDTH	—	12	Width of each tap index
TAP_COUNT	—	4	Number of feedback taps
TIW	—	—	Shorthand for TAP_INDEX_WIDTH

48.3 Ports

48.3.1 Inputs

Port	Width	Description
clk	1	System clock
rst_n	1	Active-low asynchronous reset
enable	1	Enable LFSR

Port	Width	Description
seed_load	1	operation Load seed value into LFSR
seed_data	WIDTH	Seed value for LFSR initialization
taps	TAP_COUNT*TIW	Concatenated tap positions

48.3.2 Outputs

Port	Width	Description
lfsr_out	WIDTH	Current LFSR value
lfsr_done	1	High when LFSR returns to seed value

48.4 Functional Description

48.4.1 Feedback Polynomial

The LFSR implements a linear feedback shift register based on a primitive polynomial. The feedback is calculated by XORing bits at specified tap positions.

48.4.2 Tap Position Processing

// Split concatenated tap positions into separate groups

```
always_comb begin
    for (int i = 0; i < TAP_COUNT; i++)
        w_tap_positions[i] = taps[i*TIW+:TIW];
end
```

// Convert tap positions to bit mask

```
always_comb begin
    w_taps = 'b0;
    for (int i = 0; i < TAP_COUNT; i++)
        if (w_tap_positions[i] > 0)
            w_taps[w_tap_positions[i]-1'b1] = 1'b1;
end
```

48.4.3 Feedback Calculation

```
assign w_feedback = ~(r_lfsr[WIDTH-1:0] & w_taps[WIDTH-1:0]);
```

The feedback uses XNOR reduction (inverted XOR) of tapped bits, which is characteristic of maximal-length LFSR sequences.

48.4.4 Shift Register Operation

```
always_ff @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        r_lfsr <= 'b0;
    end else begin
        if (enable) begin
            if (seed_load)
                r_lfsr <= seed_data;
            else
                r_lfsr <= {r_lfsr[WIDTH-2:0], w_feedback};
        end
    end
end
```

48.5 Timing Characteristics

48.5.1 Seed Loading and Operation

Clock: _ _ _ _ _
enable: _ _ _ _ _
seed_load: _ _ _ _ _
seed_data: =====< SEED >=====

lfsr_out:	00000000	SEED	VAL1	VAL2	VAL3
lfsr_done:	_____	_____	_____	_____	_____

48.5.2 Sequence Example (3-bit LFSR, taps=[3,2])

Feedback is $\sim^{\wedge}(\text{lfsr} \& \text{taps})$ (XNOR reduction) with taps = 3'b110, and the next state is {lfsr[1:0], feedback}. Full period is 7 (all states except the all-ones lockout):

Step	LFSR	lfsr & 110	Feedback	Next
0	001	000	1	011
1	011	010	0	110
2	110	110	1	101
3	101	100	0	010
4	010	010	0	100
5	100	100	0	000
6	000	000	1	001 (wraps to start; period 7)

48.6 Usage Examples

48.6.1 Pseudo-Random Number Generation

```
// Initialize with non-zero seed
seed_data = 8'hA5;
seed_load = 1'b1; // Load seed
// Then enable normal operation
```

```
enable = 1'b1;  
seed_load = 1'b0;  
// lfsr_out provides pseudo-random sequence
```

48.6.2 Built-In Self-Test (BIST)

```
// Use lfsr_done to detect test completion  
if (lfsr_done) begin  
    // Full test sequence completed  
    test_complete = 1'b1;  
end
```

48.6.3 Scrambling/Descrambling

```
// XOR data with LFSR output for scrambling  
scrambled_data = input_data ^ lfsr_out;
```

48.6.4 Common Use Cases

- Pseudo-random number generation
- Data scrambling and encryption
- Built-in self-test pattern generation
- CRC calculation
- Spread spectrum communications
- Test vector generation
- Noise generation for audio applications

48.7 Design Notes

48.7.1 1. Tap Position Encoding

- Tap positions are 1-indexed (position 1 = bit 0)
- Zero tap position is ignored (allows unused taps)
- Multiple tap positions concatenated into single bus

48.7.2.2. XNOR Feedback

- Uses XNOR ($\sim^{\wedge}$) instead of XOR for feedback
- Produces maximal-length sequences for primitive polynomials

- Different from some LFSR implementations that use XOR

48.7.33. Cycle Detection

```
assign lfsr_done = (lfsr_out == seed_data) ? 1'b1 : 1'b0;
```

Detects when LFSR returns to its initial seed value, indicating completion of full pseudo-random sequence.

48.7.44. Left-Shift Architecture

The implementation uses left-shift with feedback to LSB, which is equivalent to traditional right-shift LFSRs but with different bit ordering.

48.7.55. Zero State Handling

- Reset initializes LFSR to all zeros
- All-zeros is a **normal** state for this module, not a lockout. Feedback is XNOR ($\sim(r_lfsr \& w_taps)$), so an all-zeros register produces $\sim^0 = 1$ and advances on its own – the 3-bit example in this page shows exactly that (000 follows 100, then wraps to 001)
- The lockout state for XNOR feedback is **all-ones**, as stated elsewhere on this page. The all-zeros-avoidance rule belongs to XOR LFSRs such as `shifter_lfsr_fibonacci`, which carries an explicit `|r_lfsr guard`
- Seed loading is therefore optional: it selects where in the sequence you start, but the reset state runs correctly without it

48.7.6 LFSR Polynomial Reference

The module supports various standard LFSR polynomials. Common configurations (XNOR taps):

Width	Tap Positions	Max Period
3	[3,2]	7
4	[4,3]	15
5	[5,3]	31
8	[8,6,5,4]	255
16	[16,15,13,4]	65535
32	[32,22,2,1]	$2^{32}-1$

48.7.7 Example Tap Configuration for 8-bit LFSR

```
// For WIDTH=8, polynomial  $x^8 + x^6 + x^5 + x^4 + 1$   
// Taps at positions 8,6,5,4 (1-indexed)  
parameter TAP_COUNT = 4;  
parameter TIW = 4; // 4 bits enough for tap positions up to 8  
  
// Concatenated tap positions: {8,6,5,4}  
wire [TAP_COUNT*TIW-1:0] taps = {4'd8, 4'd6, 4'd5, 4'd4};
```

48.8 Related Modules

- [shifter_lfsr_fibonacci](#) — Fibonacci (external-XOR) form; carries the |r_lfsr zero guard this module doesn't need.
- [shifter_lfsr_galois](#) — Galois (distributed-XOR) form for higher clock frequencies.

48.9 Testing

val/common/test_shifter_lfsr.py exercises this module. It collects 6 parameter cases at the default REG_LEVEL.

```
source env_python  
pytest val/common/test_shifter_lfsr.py -v
```

48.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

49 shifter_universal

49.1 Overview

The `shifter_universal` module is the general-purpose shift register: left shift, right shift, parallel load, and hold, with both serial and parallel data in and out. If your design needs bits moved sideways in any direction, this is the part you reach for.

- Four distinct operations: hold, right shift, left shift, parallel load
- Bidirectional serial data input/output
- Parallel data load capability
- Serial data capture during shift operations
- Synchronous operation with asynchronous reset
- Configurable width

49.2 Parameters

Parameter	Type	Default	Description
WIDTH	—	4	Width of the shift register. Must be ≥ 2 — at WIDTH=1 the shift expressions become illegal part-selects (<code>o_pdata[WIDTH-1:1]</code> -> <code>[0:1]</code> , <code>o_pdata[WIDTH-2:0]</code> -> <code>[-1:0]</code>).

49.3 Ports

49.3.1 Inputs

Port	Width	Description
clk	1	System clock
rst_n	1	Active-low asynchronous reset
select	2	Operation select control

Port	Width	Description
i_pdata	WIDTH	Parallel data input
i_sdata_lt	1	Serial data input (left side)
i_sdata_rt	1	Serial data input (right side)

49.3.2 Outputs

Port	Width	Description
o_pdata	WIDTH	Parallel data output
o_sdata_lt	1	Serial data output (left side)
o_sdata_rt	1	Serial data output (right side)

49.4 Functional Description

49.4.1 Operation Control

The 2-bit select signal controls the shifter operation:

select	Operation	Description
2'b00	Hold	Maintain current state
2'b01	Right Shift	Shift data right; new bit enters from i_sdata_rt
2'b10	Left Shift	Shift data left; new bit enters from i_sdata_lt
2'b11	Parallel Load	Load parallel input data

49.4.2 Combinational Logic for Next State

```

always_comb begin
    // Default values - hold current state
    w_pdata = o_pdata;
    w_sdata_lt = 1'b0;
    w_sdata_rt = 1'b0;

    casez (select)
        2'b00: begin // Hold
            w_pdata = o_pdata;

```



```

        w_sdata_lt = 1'b0;
        w_sdata_rt = 1'b0;
    end

    2'b01: begin // Right Shift
        w_pdata = {i_sdata_rt, o_pdata[WIDTH-1:1]};
        w_sdata_lt = 1'b0;
        w_sdata_rt = o_pdata[0];
    end

    2'b10: begin // Left Shift
        w_pdata = {o_pdata[WIDTH-2:0], i_sdata_lt};
        w_sdata_lt = o_pdata[WIDTH-1];
        w_sdata_rt = 1'b0;
    end

    2'b11: begin // Parallel Load
        w_pdata = i_pdata;
        w_sdata_lt = 1'b0;
        w_sdata_rt = 1'b0;
    end
endcase
end

```

49.4.3 Sequential Logic for State Update

```

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        o_pdata    <= '0;
        o_sdata_lt <= 1'b0;
        o_sdata_rt <= 1'b0;
    end else begin
        o_pdata    <= w_pdata;
        o_sdata_lt <= w_sdata_lt;
        o_sdata_rt <= w_sdata_rt;
    end
end

```

49.4.4 Hold Operation (select = 2'b00)

- **Function:** Maintains current register state
- **Data Flow:** No data movement
- **Serial Outputs:** Both remain low (no shift occurring)
- **Use Case:** Pause operation while maintaining data integrity

49.4.5 Right Shift Operation (select = 2'b01)

- **Function:** Shifts data toward LSB
- **Data Flow:** $i_sdata_rt \rightarrow \text{MSB}, \text{LSB} \rightarrow o_sdata_rt$
- **Serial Input:** New data enters from i_sdata_rt
- **Serial Output:** Shifted-out data exits via o_sdata_rt
- **Use Case:** Serial-to-parallel conversion, delay lines

49.4.6 Left Shift Operation (select = 2'b10)

- **Function:** Shifts data toward MSB
- **Data Flow:** $i_sdata_lt \rightarrow \text{LSB}, \text{MSB} \rightarrow o_sdata_lt$
- **Serial Input:** New data enters from i_sdata_lt
- **Serial Output:** Shifted-out data exits via o_sdata_lt
- **Use Case:** Multiplication by 2, bit-serial processing

49.4.7 Parallel Load Operation (select = 2'b11)

- **Function:** Loads entire parallel word
- **Data Flow:** $i_pdata \rightarrow o_pdata$
- **Serial Outputs:** Both remain low (no shift occurring)
- **Use Case:** Initialization, data injection

49.5 Related Modules

Nothing in the tree instantiates this module and it instantiates nothing: it is a leaf, used directly by whatever design needs it. Its nearest neighbours in `rtl/common/` are:

- `shifter_barrel`
- `shifter_beat_pack`
- `shifter_lfsr`
- `shifter_lfsr_fibonacci`
- `shifter_lfsr_galois`

49.6 Timing Characteristics

49.6.1 Right Shift Operation Example (WIDTH=4)

The first column is the reset state, before any edge; every later column is the value *after* that cycle's edge. o_sdata_rt is registered, so the bit it shows in a column is the LSB that the *previous* column's o_pdata ejected.

Clock:	—	—	—	—	—	—
select:	----	01	01	01	01	01
i_sdata_rt:	----	1	0	1	1	0
o_pdata:	0000	1000	0100	1010	1101	0110
o_sdata_rt:	--	0	0	0	0	1

The single 1 on o_sdata_rt is the LSB of 1101, ejected by the fifth shift and visible in the column that shift produces – not alongside 1101 itself.

49.6.2 Parallel Load followed by Left Shift

Clock:	—	—	—	—
select:	11	10	10	10
i_pdata:	1010	----	----	----
i_sdata_lt:	----	1	0	1
o_pdata:	1010	0101	1010	0101
o_sdata_lt:	----	1	0	1

49.7 Usage Examples

49.7.1 Serial Communication Interface

```
// Receive shift register for UART
// Right shift to collect incoming serial data
select = 2'b01;
i_sdata_rt = uart_rx;
// When 8 bits received, parallel data available
received_byte = o_pdata;
```

49.7.2 Multiplication/Division by Powers of 2

```
// Multiply by 2: left shift
select = 2'b10;
i_sdata_lt = 1'b0; // Fill with zeros

// Divide by 2: right shift
select = 2'b01;
i_sdata_rt = 1'b0; // Fill with zeros (unsigned)
// or i_sdata_rt = o_pdata[WIDTH-1] for arithmetic shift
```

49.7.3 Data Delay Line

```
// Create N-cycle delay using N shifters
// Right shift with data input
select = 2'b01;
i_sdata_rt = data_input;
delayed_data = o_sdata_rt; // N cycles later
```

49.7.4 Pattern Generator

```
// Load pattern then circulate
// 1. Load initial pattern
select = 2'b11;
i_pdata = initial_pattern;

// 2. Circulate pattern
select = 2'b01; // Right shift
i_sdata_rt = o_pdata[0]; // Combinational tap -- NOT o_sdata_rt
```

Feed back `o_pdata[0]`, not `o_sdata_rt`. Here's the trap: the serial outputs are registered (`o_sdata_rt <= w_sdata_rt`), so routing `o_sdata_rt` back returns the ejected bit one shift too late and the pattern does not rotate – it runs a `WIDTH+1`-long cycle. With `WIDTH=4` loaded to `1011`, the registered feedback gives `1011 -> 0101 -> 1010 -> 1101 -> 0110 -> 1011` (period 5, and `1110` and `0111` never appear), whereas the combinational tap gives the true rotation `1011 -> 1101 -> 1110 -> 0111 -> 1011`.

49.7.5 Scan Chain for Testing

```
// JTAG-style scan chain
// Load test data
select = 2'b11;
i_pdata = test_vector;

// Shift out for observation
select = 2'b01;
i_sdata_rt = next_scan_in;
scan_out = o_sdata_rt;
```

49.7.6 Common Use Cases

- Serial-to-parallel conversion
- Parallel-to-serial conversion
- Data delay and timing adjustment
- Arithmetic operations ($\times 2$, $\div 2$)
- Pattern generation and circulation

- Scan chain testing
- FIFO buffer implementation
- Digital signal processing pipelines

49.8 Design Notes

49.8.1 1. Bidirectional Serial Interface

The module provides separate serial inputs and outputs for both directions: - **Left side:** i_sdata_lt (input), o_sdata_lt (output) - **Right side:** i_sdata_rt (input), o_sdata_rt (output)

That's what lets you chain multiple shifters or hook into bidirectional serial systems.

49.8.22. Clean Serial Output Management

Serial outputs are explicitly driven to zero when not actively shifting in that direction — no glitches, no ambiguous states.

49.8.33. Asynchronous Reset

Reset immediately clears all outputs to zero, providing deterministic startup conditions.

49.8.44. Combinational/Sequential Separation

The design cleanly separates: - **Combinational logic:** Calculates next state - **Sequential logic:** Registers the state on clock edge

That split improves timing predictability and synthesis results.

49.8.5 Chaining Multiple Shifters

```
// Connect shifters in series
shifter1.i_sdata_rt = serial_input;
shifter2.i_sdata_rt = shifter1.o_sdata_rt;
shifter3.i_sdata_rt = shifter2.o_sdata_rt;
```

49.8.6 Bidirectional Operation

```
// Support both shift directions
if (shift_direction == RIGHT) begin
    select = 2'b01;
    i_sdata_rt = serial_data;
end else begin
    select = 2'b10;
    i_sdata_lt = serial_data;
end
```

49.8.7 Control Logic Integration

```
// State machine control
case (state)
    LOAD: select = 2'b11;
    SHIFT_RIGHT: select = 2'b01;
    SHIFT_LEFT: select = 2'b10;
    HOLD: select = 2'b00;
endcase
```

49.9 Testing

val/common/test_shifter_universal.py exercises this module. It collects 1 parameter cases at the default REG_LEVEL.

```
source env_python
pytest val/common/test_shifter_universal.py -v
```

49.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)

50 sort

50.1 Overview

The Sort module is a pipelined hardware sorting engine built on the odd-even sort algorithm, sorting arrays of configurable size in descending order. It's aimed at high-throughput work: new data accepted every clock cycle, with predictable latency equal to the number of elements being sorted.

50.1.1 Module Declaration

```
module sort #(
    parameter int NUM_VALS = 5,    // Number of values to sort
    parameter int SIZE      = 16   // Size of each value in bits
) (
    input logic clk,              // Clock signal
    input logic rst_n,           // Active-low reset
    input logic [NUM_VALS*SIZE-1:0] data, // Packed input data
    input logic valid_in,        // Start sorting when
    asserted
    output logic [NUM_VALS*SIZE-1:0] sorted, // Packed sorted
    output
    output logic done            // Asserted when
    sorting complete
);
```

50.2 Parameters

Parameter	Type	Default	Description
NUM_VALS	int	5	Number of values in the array to be sorted. Determines pipeline depth. Range 2-16 (RTL header documents the supported range as 2 to 16).
SIZE	int	16	Bit width of each individual value. All values must be unsigned integers. Range ≥ 1 (no upper bound enforced).

50.3 Ports

50.3.1 Inputs

Port	Width	Description
clk	1	System clock signal. All operations are synchronous to the rising edge.
rst_n	1	Active-low asynchronous reset. Clears all pipeline stages while asserted (low) ; normal operation is rst_n high.
data	NUM_VALS*SIZE	Packed input array. Elements are packed as data[i*SIZE +: SIZE] for element i.
valid_in	1	Control signal. When asserted (high) for one cycle, starts the sorting process for the current data input.

50.3.2 Outputs

Port	Width	Description
sorted	NUM_VALS*SIZE	Packed output array sorted in descending order. Same packing format as input.
done	1	Status signal. Asserted (high) when the sorting operation is complete and sorted output is valid.

50.4 Functional Description

50.4.1 Data Format

Input/Output Packing: Arrays are packed into a single wide bus where: - Element 0 occupies bits [SIZE-1:0] - Element 1 occupies bits [2*SIZE-1:SIZE] - Element i occupies bits [(i+1)*SIZE-1:i*SIZE]

Sort Order: Output is sorted in **descending order** (largest to smallest).

50.4.2 Algorithm: Odd-Even Sort

The module implements the **Odd-Even Sort** (also known as **Brick Sort**) algorithm — a particularly good fit for pipelined hardware, as you'll see below.

Odd-Even Sort works by alternating between two types of comparison passes:

1. **Odd Passes:** Compare and swap adjacent pairs at positions (0,1), (2,3), (4,5), etc.
2. **Even Passes:** Compare and swap adjacent pairs at positions (1,2), (3,4), (5,6), etc.

The algorithm requires exactly NUM_VALS passes to guarantee complete sorting, regardless of input data.

50.4.3 Example Execution

For input array [5, 3, 8, 1, 9] (5 elements):

Initial: [5, 3, 8, 1, 9]

Pass 1 (Odd): Compare (0,1), (2,3), (4,X)
 [5, 3] → [5, 3] (no swap)
 [8, 1] → [8, 1] (no swap)
 [9, X] → [9, X] (no comparison)
 Result: [5, 3, 8, 1, 9]

Pass 2 (Even): Compare (1,2), (3,4)
 [3, 8] → [8, 3] (swap)
 [1, 9] → [9, 1] (swap)
 Result: [5, 8, 3, 9, 1]

Pass 3 (Odd): Compare (0,1), (2,3), (4,X)
 [5, 8] → [8, 5] (swap)
 [3, 9] → [9, 3] (swap)
 [1, X] → [1, X] (no comparison)
 Result: [8, 5, 9, 3, 1]

Pass 4 (Even): Compare (1,2), (3,4)
 [5, 9] → [9, 5] (swap)
 [3, 1] → [3, 1] (no swap)
 Result: [8, 9, 5, 3, 1]

Pass 5 (Odd): Compare (0,1), (2,3), (4,X)
 [8, 9] → [9, 8] (swap)
 [5, 3] → [5, 3] (no swap)
 [1, X] → [1, X] (no comparison)
 Final: [9, 8, 5, 3, 1] -- sorted

50.4.4 Algorithm Comparison

The “Space (SW)” column is the classical **in-place software** space bound; “HW Area” is the cost of a fully-pipelined hardware network of that algorithm. For **this** module the network is $O(n^2)$: $\sim n/2$

compare-swaps per stage $\times$ n flopped stages (the RTL header: “ $O(\text{NUM_VALS}^2)$ comparators, $O(\text{NUM_VALS}^2 \cdot \text{SIZE})$ registers”). Do not read $O(1)$ as the hardware area.

Algori thm	Time Complexit y	Space (SW)	HW Area (this style)	Hardware Efficiency	Pipeline Stages	Best Use Case
Odd- Even Sort	$O(n^2)$	$O(1)$	$O(n^2)$	Excellent	n	Small arrays, predicta ble timing
Bubbl e Sort	$O(n^2)$	$O(1)$	$O(n^2)$	Poor	Variable	Sequenti al processo rs only
Bitoni c Sort	$O(n \log^2 n)$	$O(1)$	$O(n \log^2 n)$	Very Good	$\log^2 n$	Power- of-2 sizes, larger arrays
Merg e Sort	$O(n \log n)$	$O(n)$	$O(n \log n)$	Good	$\log n$	Large arrays, general purpose
Insert ion Sort	$O(n^2)$	$O(1)$	$O(n^2)$	Good	Variable	Nearly sorted data

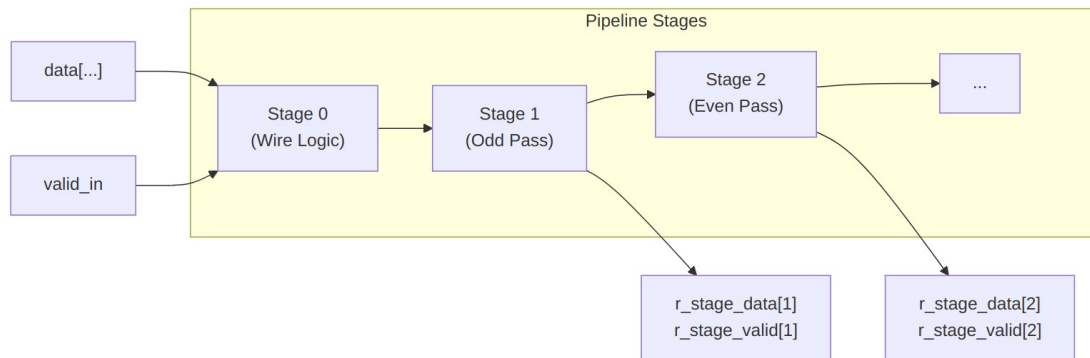
Why Odd-Even Sort for Hardware?

1. **Regular Structure:** Each pipeline stage is identical, making synthesis predictable
2. **Parallel Operations:** All comparisons in each pass happen simultaneously
3. **Fixed Latency:** Always takes exactly NUM_VALS cycles regardless of input data
4. **Simple Control:** No complex indexing or variable loop bounds
5. **Scalable:** Pipeline depth scales linearly with array size

50.4.5 Pipeline Structure

The module implements a NUM_VALS-stage pipeline where: - **Stage 0**: Input capture (combinational) - **Stages 1 to NUM_VALS**: Pipeline registers with compare-swap logic

Stage: 0 -----> 1 -----> 2 -----> 3 -----> ... -----> NUM_VALS
 Input Odd Pass Even Pass Odd Pass ... Final Output
 (Comb) (FF) (FF) (FF) (FF)



Signal Flow

50.4.6 Naming Conventions

Naming is strict, and it pays off when you're reading waveforms:

- **Wire signals** (combinational): Prefixed with w_
 - w_values[stage][element]: Unpacked arrays for easy manipulation
 - w_next_values[stage][element]: Computed next values
 - w_stage_data_0: Input stage data wire
 - w_stage_valid_0: Input stage valid wire
- **Flopped signals** (sequential): Prefixed with r_
 - r_stage_data[stage]: Pipeline data registers
 - r_stage_valid[stage]: Pipeline valid registers

50.4.7 Compare-Swap Logic

Each pipeline stage implements compare-swap operations based on the stage number:

```
// Determine if this is an odd or even pass
localparam bit IS_ODD_PASS = ((stage-1) % 2) == 0;
```

```

always_comb begin
    // Default: pass through unchanged
    for (int i = 0; i < NUM_VALS; i++) begin
        w_next_values[stage][i] = w_values[stage-1][i];
    end

    if (IS_ODD_PASS) begin
        // Odd pass: compare (0,1), (2,3), (4,5), etc.
        for (int i = 0; i < NUM_VALS-1; i += 2) begin
            if (w_values[stage-1][i] < w_values[stage-1][i+1]) begin
                w_next_values[stage][i] = w_values[stage-1][i+1];
                w_next_values[stage][i+1] = w_values[stage-1][i];
            end
        end
    end else begin
        // Even pass: compare (1,2), (3,4), (5,6), etc.
        for (int i = 1; i < NUM_VALS-1; i += 2) begin
            if (w_values[stage-1][i] < w_values[stage-1][i+1]) begin
                w_next_values[stage][i] = w_values[stage-1][i+1];
                w_next_values[stage][i+1] = w_values[stage-1][i];
            end
        end
    end
end
end

```

50.4.8 Reset Behavior

- **Asynchronous Reset:** All pipeline stages are cleared while `rst_n` is asserted (low)
- **Data Reset:** All `r_stage_data` registers are cleared to zero
- **Valid Reset:** All `r_stage_valid` registers are cleared to zero
- **Output:** Both sorted and done outputs go to zero during reset

50.4.9 Control Flow

The Sort module does **not** use explicit Finite State Machines (FSMs). Control is just the pipeline itself — a **pipeline-based control scheme**:

1. **Input Stage (Combinational):**
 - Accepts new data when `valid_in` is asserted
 - No state storage at input stage
2. **Pipeline Progression (Sequential):**

- Valid signal propagates through pipeline automatically
- Each stage processes data independently
- No centralized control or state machine needed

3. Output Indication:

- done signal is simply the valid signal from the final pipeline stage
- No complex output state management required

50.5 Related Modules

Nothing in the tree instantiates this module and it instantiates nothing: it is a leaf, used directly by whatever design needs it. Its nearest neighbours in rtl/common/ are:

- (none in this category)

50.6 Timing Characteristics

50.6.1 Timing Characteristics

- **Latency:** NUM_VALS clock cycles from valid_in assertion to done assertion
- **Throughput:** 1 array per clock cycle (once pipeline is full)
- **Critical Path:** One compare-swap operation — a SIZE-bit magnitude comparator feeding a 2:1 mux, i.e. several logic levels (consistent with the ~1.5-2.0 ns figures in the synthesis table below, not “1-2 gate delays”)
- **Pipeline Depth:** NUM_VALS stages

50.6.2 Valid Signal Flow

Cycle:	1	2	3	4	5	6	
valid_in:	1	0	0	0	0	0	
r_stage_valid[1]:	-		1	0	0	0	0
r_stage_valid[2]:	-	-		1	0	0	0
r_stage_valid[3]:	-	-	-		1	0	0
r_stage_valid[4]:	-	-	-	-		1	0
r_stage_valid[5]:	-	-	-	-	-		1
done:	0	0	0	0	0	1	

50.7 Usage Examples

50.7.1 Typical Use Cases

1. **Packet Processing:** Sorting priority fields or timestamps
2. **Signal Processing:** Median filtering, statistical operations
3. **Graphics:** Z-buffer sorting, polygon vertex ordering
4. **Control Systems:** Sorting sensor readings or control parameters

50.7.2 Integration Considerations

1. **Clock Domain:** Ensure `clk` is stable and meets timing requirements
2. **Reset Strategy:** Include proper reset sequencing in system design
3. **Data Formatting:** Pack input arrays correctly according to element ordering
4. **Flow Control:** Monitor done signal for output validity
5. **Resource Planning:** Consider pipeline depth impact on latency-sensitive applications

50.7.3 Performance Optimization

1. **Pipeline Depth:** Use smallest practical `NUM_VALS` for lowest latency
2. **Data Width:** Use smallest practical `SIZE` for best area efficiency
3. **Clock Frequency:** Critical path is typically one comparison operation
4. **Resource Sharing:** Multiple instances can share input/output buses if properly arbitrated

50.8 Design Notes

50.8.1 1. Parameterizable Design

- **Flexible Array Size:** `NUM_VALS` supported range is 2 to 16 (per the RTL header)

- **Configurable Data Width:** SIZE sets the bits per element. The RTL applies no validation to it – any SIZE ≥ 1 elaborates; wide values simply cost a wider comparator on the critical path
- **Automatic Pipeline Scaling:** Pipeline depth automatically adjusts to NUM_VALS

50.8.22. High Throughput Pipeline

- **Back-to-Back Operations:** New data can be accepted every cycle
- **Overlapped Processing:** Multiple arrays can be in the pipeline simultaneously
- **Predictable Performance:** Fixed latency regardless of data patterns

50.8.33. Synthesis-Friendly Design

- **Regular Structure:** All pipeline stages are nearly identical
- **Local Connectivity:** No long combinational paths across stages
- **Inference-Friendly:** Uses standard SystemVerilog constructs for reliable synthesis

50.8.44. Packed Data Handling

- **Packed Buses:** Efficient use of bus resources
- **Unpacked Internal Arrays:** Easy element-wise manipulation
- **Automatic Packing/Unpacking:** Transparent data format conversion

50.8.5 Synthesis Results

Typical synthesis results for common configurations:

Configuration	Pipeline Stages	Critical Path	Area (LUTs)	Frequency
3×8-bit	3	~1.2ns	~150	>800 MHz
5×16-bit	5	~1.5ns	~400	>650 MHz
8×32-bit	8	~2.0ns	~1200	>500 MHz

Results vary by target FPGA family and synthesis tool settings

50.8.6 Future Enhancements

Potential improvements for future versions:

1. **Bidirectional Sorting:** Parameter to select ascending vs descending order
 2. **Early Termination:** Detect when array is already sorted and terminate early
 3. **Variable Width Elements:** Support for mixed-width data elements
 4. **Priority Encoding:** Include secondary sort keys for equal elements
 5. **Power Optimization:** Clock gating for unused pipeline stages
-

This documentation describes the Sort module implementation. For the latest updates and additional examples, refer to the source code and testbench files.

50.9 Testing

The module comes with a comprehensive CocoTB testbench that includes:

50.9.1 Test Levels

1. **Basic Tests:** Core functionality verification
2. **Random Tests:** Extensive random data patterns
3. **Boundary Tests:** Edge cases and limit conditions
4. **Pipeline Tests:** Throughput and overlapped operation verification
5. **Reset Tests:** Proper reset behavior validation

50.9.2 Test Coverage

- **Functional:** All possible input combinations for small arrays
- **Parametric:** Multiple NUM_VALS and SIZE combinations
- **Temporal:** Pipeline timing and valid signal propagation
- **Stress:** Back-to-back operations and maximum throughput

50.10 Navigation

- [← Back to rtl-common Index](#)
- [← Back to Main Documentation Index](#)